IA pour les réseaux
De la question
à la solution

Modèles, algorithmes et expériences

Pratique de l’IA et reprise de formation

Jean-Pierre Morard

Édition 1.0 - collection par public

ID : LGIR-COL-4-1.0-20261014

Généré le 14 octobre 2026

CC BY-NC-ND 4.0 · Partage avec attribution

Sommaire

Les numéros renvoient à la pagination imprimée, qui commence après la couverture.

Section	Page
Une question avant un modèle	2
Licence de partage	3
Passerelle — Science des données et ingénierie réseau	4
Traité transversal — Intelligence artificielle pour les réseaux : architectures, cas d’usage et preuve	54
Étude de cas — Comprendre sb3_trainer_project	181
Laboratoires	192
Parcours IA pour les réseaux en 12 semaines	201
Repères pour vérifier une solution d’IA	202
Glossaire	203
Périmètre des cas présentés	208
Pour aller plus loin — Bibliographie	209
Index thématique	211
Références citées	213

Une question avant un modèle

Choisir un chemin, anticiper une charge, détecter une anomalie : chaque problème demande des données, une méthode de référence et une manière de juger le résultat. Un bon modèle ne suffit pas à construire une bonne solution. Il faut aussi décider où il intervient et ce qui vérifie ses sorties.

Ce livre s’adresse aux ingénieurs, aux praticiens de la science des données et aux professionnels qui reprennent une formation scientifique. La lecture de tableaux et de graphiques fournit le point de départ. Les probabilités, les vecteurs et les notions d’apprentissage sont introduits avant les cas qui les emploient. Les laboratoires demandent Python et les dépendances indiquées dans leurs fiches.

Suivez d’abord la passerelle vers la science des données. Elle relie les observations, les calculs et les premiers modèles. Le panorama de l’apprentissage permet ensuite de choisir les notions à approfondir, avant les cas réseau, les compositions d’algorithmes et les agents. L’étude d’application et les laboratoires donnent un terrain d’essai.

À chaque étape, comparez la solution à une référence simple, expliquez ce qu’elle gagne et examinez ce qu’elle perd. Le guide mathématique accompagne les notations et les calculs. L’IA s’apprend aussi dans un parcours professionnel : les reprises de bases font partie du travail.

Ce volume prolonge les *Grandes Idées des Réseaux* : apprendre une décision ne fait disparaître ni les contraintes physiques ni les pannes. Le volume d’ingénierie explique ces contraintes et les essais. Le guide mathématique aide à lire les calculs ; le récit montre comment ces domaines se sont rejoints dans mon travail.

Licence de partage

© 2026 **Jean-Pierre Morard** — *Les Grandes Idées des Réseaux*.

Les ressources et travaux pratiques de la collection sont accessibles en ligne.

Les textes et schémas originaux dont l’auteur détient les droits sont mis à disposition sous la licence **Creative Commons Attribution — Pas d’utilisation commerciale — Pas de diffusion de versions modifiées 4.0 International** (**CC BY-NC-ND 4.0**).

Vous pouvez les copier et les partager, sur papier ou au format numérique, en créditant **Jean-Pierre Morard**, en conservant les mentions de droits et en joignant le lien vers la licence. Cette autorisation exclut l’usage commercial et la diffusion de versions adaptées ou modifiées.

Consulter la licence CC BY-NC-ND 4.0.

Les citations, documents d’archives, sujets de concours et autres éléments attribués à des tiers conservent leurs propres droits et ne sont pas couverts par cette licence. Les logiciels mentionnés conservent également leurs licences respectives.

Passerelle — Science des données et ingénierie réseau

Cette passerelle donne aux deux métiers un vocabulaire commun. Commencer par les encadrés **À retenir**, le dictionnaire commun, le contrat de collaboration et le mini-guide de l'étude. Pour une première lecture, partez des mesures disponibles et de la décision à prendre. Les familles d'apprentissage et les modèles spécialisés viendront ensuite. Le fil conducteur est simple : observer, décider, mesurer l'effet sur le réseau, puis vérifier le résultat.

Les repères mathématiques se trouvent à la fin de cette passerelle. Vous pouvez y revenir lorsqu'une notation ou un calcul vous arrête.

1. Repères réseau pour le spécialiste des données

Repère de première lecture : la vue des grandes bandes et des acteurs explique en quelques pages qui normalise, qui autorise et qui exploite un réseau radio.

Avant les paquets : information, signal, support, liaison et réseau

Une **information** est ce que l'on veut faire comprendre. Un **signal** est sa représentation physique observable. Le **support** est le milieu traversé : conducteur, fibre ou espace libre. Une **liaison** organise la transmission entre deux extrémités ; un **réseau** coordonne plusieurs terminaux, liaisons, relais et ressources.

Le paquet n'est donc pas « ce qui voyage » dans toute télécommunication. C'est une unité d'organisation choisie par certaines architectures numériques, dont les réseaux IP. Sur chaque liaison, même un paquet doit finalement être représenté par des symboles puis par un signal physique.

Paquet, trame, flux et session

Un **paquet** est un bloc de données accompagné d'informations permettant son acheminement. Une **trame** est l'unité employée sur un lien local, par exemple Ethernet ou Wi-Fi.

Le paquet porte l'acheminement de bout en bout. La trame est adaptée, retirée puis recréée à chaque changement de lien.

Un **flux** regroupe des paquets qui appartiennent au même échange selon une règle donnée. Une **session** ajoute un état de conversation : identité, progression, clés ou contexte applicatif.

Ces mots ne sont pas interchangeables :

```
application
  -> session
  -> un ou plusieurs flux
  -> paquets IP
  -> trames sur chaque lien
```

Lien, chemin et route

Un **lien** relie directement deux points voisins. Un **chemin** est une suite de liens. Une **route** est une décision ou une information permettant de choisir le prochain passage vers une destination.

L'action choisie par le modèle ne commande qu'une partie du système. La capacité disponible, l'état des files, les décisions inter-domaines, l'évolution radio et la durée de validité du chemin restent des variables d'environnement à observer et à borner séparément.

Les quatre mesures à ne pas confondre

Mesure	Question simple
débit utile	combien de données nouvelles arrivent par seconde ?
latence	combien de temps faut-il pour arriver ?
gigue	ce délai varie-t-il ?

Mesure	Question simple
perte	quelle part n’arrive pas à temps ou pas du tout ?

Un modèle peut améliorer le débit moyen tout en dégradant la latence des pires cas. Il faut donc regarder plusieurs mesures et leurs **percentiles** : par exemple, le 99e percentile est une valeur que 99 mesures sur 100 ne dépassent pas.

File et congestion

Une **file** apparaît lorsqu’une ressource reçoit du travail plus vite qu’elle ne peut le traiter. La **congestion** n’est pas seulement une perte de paquets : c’est un état où la demande et les rafales consomment la capacité et la marge.

Ajouter des retransmissions peut alors aggraver le problème. Une récompense d’apprentissage qui valorise seulement les paquets envoyés peut apprendre ce comportement nocif.

Plans de données, de contrôle et de gestion

- **Plan de données** : transfère les paquets.
- **Plan de contrôle** : calcule ou distribue les décisions de transfert.
- **Plan de gestion** : configure, observe et audite.

Un modèle peut appartenir à l’un de ces plans ou seulement les conseiller. Cette position détermine son délai de décision, son autorité et les dégâts possibles en cas d’erreur.

Réseau fixe, mobile et ad hoc

- **Réseau fixe** : les principaux liens changent peu pendant une session.
- **Réseau mobile avec infrastructure** : le terminal change de point d’attachement, mais un opérateur organise la mobilité.
- **Réseau ad hoc** : les nœuds peuvent aussi servir de relais et reconstruire eux-mêmes la connectivité.

La même politique d’apprentissage ne se généralise pas automatiquement entre ces trois régimes.

Accès, cœur et interconnexion

Le **réseau d’accès** (*access network*) attache le terminal. La **collecte** (*aggregation* ou *backhaul*) agrège les liaisons. Le **cœur de réseau** (*core network*) applique les fonctions communes de transport, d’identité, de session, de mobilité ou de politique. Le **bord** (*network edge*) choisit ensuite une sortie vers un service interne, un réseau privé ou une interconnexion avec un autre opérateur.

```
accès -> collecte -> cœur -> bord -> interconnexion -> service
```

Cette chaîne est un modèle de diagnostic, non une promesse de centralisation. Les fonctions du cœur et les sorties peuvent être distribuées près de l’accès. Voir la big picture et l’architecture détaillée du cœur.

Les données racontent-elles ce qui se passe ?

Un **biais** est ici une déformation systématique de ce que les données nous permettent de voir. Il peut venir du choix des personnes observées, de la manière de mesurer ou des pratiques de déclaration. C’est une autre question que le terme précis de « biais » dans la décomposition mathématique biais–variance.

Reconnaissance faciale : réussir ici ne garantit pas de réussir ailleurs

Imaginons un modèle entraîné surtout sur des visages recueillis dans une région du monde. Un bon résultat sur des photographies semblables ne garantit pas la même fiabilité auprès de populations peu représentées dans cet apprentissage. Les erreurs peuvent augmenter : rejeter la bonne personne ou confondre deux personnes. Une région reste diverse ; elle ne définit pas un type unique de visage.

Le NIST a constaté en 2019 des écarts de taux d'erreur entre groupes démographiques, variables selon les algorithmes et le type de comparaison. Cette étude ne démontre pas à elle seule que l'origine géographique des données d'entraînement explique chaque écart. La qualité des images, le modèle et le seuil de décision comptent également. (National Institute of Standards and Technology 2019)

On peut alors demander : **qui manque dans les données, et pour qui mesurons-nous les erreurs ?** Il faut éprouver le système sur les personnes et dans les conditions auxquelles il sera destiné. Une moyenne satisfaisante peut masquer des échecs concentrés sur une partie des utilisateurs.

Une usine d'aérosols : optimiser une production déjà optimisée

Retour d'expérience de l'auteur. Lors de mon projet de fin d'études, je devais concevoir un algorithme de type système expert pour ordonnancer une ligne de production dans une fabrique d'aérosols. L'objectif était de réduire les changements d'encres sur les machines d'impression offset, en choisissant mieux l'ordre de fabrication.

Le résultat pratique s'est révélé décevant. Les ouvriers optimisaient déjà l'enchaînement des fabrications, mais ces améliorations n'apparaissaient pas dans le suivi de production. Dans le fonctionnement que j'avais découvert, le temps ainsi gagné leur permettait de prolonger la pause café, sans que ce gain soit déclaré. Le suivi donnait donc une image de la production différente de celle que leur savoir-faire rendait possible.

Avec le recul, il aurait fallu commencer par comprendre ces pratiques avec les opérateurs et fiabiliser le suivi. Je cherchais un gain sur une situation de référence qui ne décrivait pas correctement le travail réel. Ce souvenir personnel rapporte un échec instructif ; il ne mesure pas un gain industriel reproductible.

Deux biais, une même exigence : regarder la situation de référence

Dans le premier exemple, certaines situations sont insuffisamment représentées. Dans le second, elles existent mais sont mal décrites par les enregistrements. Le système expert repose sur des règles explicites ; il n'a pas besoin d'apprendre sur des exemples pour être trompé par une description inexacte des temps, des contraintes ou de l'ordre réel.

L'erreur serait donc de penser qu'un algorithme plus sophistiqué suffit. Avant d'optimiser, il faut confronter les enregistrements au terrain, reconnaître les améliorations déjà réalisées et comprendre ce qui encourage ou décourage une déclaration fidèle. Ajouter des capteurs peut aider ; la coopération avec les personnes qui font le travail reste essentielle.

Le transfert aux réseaux est immédiat : un historique d'incidents qui omet les interventions informelles peut faire croire à un rétablissement spontané. Un automate entraîné ou réglé sur cet historique risque alors de négliger l'action humaine dont dépendait le résultat.

À vous de chercher. Un relevé annonce dix minutes de changement d'encre ; l'observation montre que les opérateurs n'en prennent déjà que six. L'algorithme propose huit minutes. Quel gain afficherait une comparaison au relevé ? Que conclurait une comparaison au travail réel ? Cet exemple est fictif : il sert à choisir la bonne référence avant de calculer un gain.

2. Repères de science des données pour l'ingénieur réseau

Donnée, variable et cible

Une observation brute devient une **variable d'entrée**, souvent appelée *feature*, après sélection et transformation. Une **cible** ou *label* est la réponse connue que le modèle doit apprendre à prédire.

Un **modèle** est une règle de calcul qui transforme les entrées en une sortie. Ses **paramètres** sont les coefficients ajustables de cette règle. Une **régression** prédit une valeur numérique ; une **classification** attribue une classe.

Exemple :

```
entrées : qualité radio, charge, vitesse, succès et échecs antérieurs
cible   : débit utile obtenu avec une modulation et un codage donnés
```

Une corrélation entre entrée et cible ne prouve pas un lien de cause à effet.

Entraînement et inférence

L'**entraînement** ajuste les paramètres du modèle sur des données. L'**inférence** utilise ensuite le modèle pour produire une estimation ou une décision.

Le réseau de production voit l'inférence, mais dépend aussi :

- des données utilisées pour entraîner ;
- du code et de la version du modèle ;
- de la manière dont les entrées sont calculées ;
- du domaine dans lequel le modèle a été validé.

Généralisation

Un modèle mémorisant les exemples d'entraînement peut échouer sur une nouvelle situation. On sépare donc :

- données d'entraînement ;
- données de validation pour choisir le modèle ;
- données de test gardées pour l'évaluation finale.

Perte, métrique et objectif de service

La **fonction de perte** (*loss*) est la quantité que l'algorithme minimise pendant l'entraînement : erreur quadratique pour une régression, entropie croisée pour de nombreuses classifications, ou fonction spécialisée. Elle doit guider le calcul, mais ne suffit pas à dire si le résultat est utile.

Une **métrique d'évaluation** mesure le comportement sur des données non utilisées pour ajuster le modèle : MAE, rappel, F1, violation de capacité ou temps de décision. L'**objectif de service** porte sur les incidents graves, le budget de fausses alertes, l'**accord de niveau de service** (**SLA**, *Service Level Agreement*) ou le repli sûr. Perte, métrique et objectif peuvent être liés sans être identiques : réduire l'entropie croisée ne garantit ni moins de pannes manquées ni un meilleur service.

La **MAE** (*Mean Absolute Error*, erreur absolue moyenne) moyenne les écarts entre prédictions et mesures en ignorant leur signe. Des écarts de +2 et -6 ms donnent une MAE de $(2+6)/2=4$ ms : ils ne se compensent pas. Cette métrique garde l'unité de la grandeur prédite.

Paramètres, hyperparamètres et choix du modèle

Les **paramètres** sont appris à partir des données d'entraînement : coefficients d'une régression ou poids d'un réseau neuronal. Les **hyperparamètres** sont choisis autour de cet apprentissage : intensité λ d'une régularisation ridge, profondeur d'un arbre, taux d'apprentissage, taille d'une fenêtre temporelle ou architecture. Un seuil de décision n'est pas appris automatiquement par toutes les méthodes ; lorsqu'il est réglé, il doit l'être sur la validation, jamais sur le test final.

La **validation croisée** estime le comportement d'un modèle sur plusieurs groupes de données. Un **pli** est un groupe gardé hors de l'entraînement pendant un tour de validation.

Exemple fictif en cinq plis. Sur 600 observations, réservons 100 pour le test final. Les 500 restantes forment cinq groupes de 100. Au premier tour, un modèle est ajusté sur quatre groupes, soit 400 observations, puis évalué sur le cinquième. On recommence avec un nouveau modèle à chaque tour, en changeant le groupe de validation : chacun des cinq groupes est évalué une fois. La moyenne des cinq scores aide à comparer les choix de modèle ou d'hyperparamètres. Par exemple, les scores 0,80, 0,84, 0,82, 0,78 et 0,86 donnent une moyenne de 0,82.

Après ces choix, on peut réentraîner le modèle sur les 500 observations, puis l'évaluer sur les 100 du test final. Ce test garde son rôle parce qu'il n'a servi ni à entraîner ni à choisir le modèle. À chaque tour, normalisation et sélection de variables sont ajustées sur les quatre groupes d'entraînement.

La répartition classique suppose des observations comparables. Pour des données réseau, un mélange aléatoire est souvent trop optimiste. On préfère des blocs temporels, des groupes par site ou équipement, ou un régime complet laissé de côté. Les hyperparamètres et le point de sauvegarde sont choisis sur ces validations ; le test reste scellé jusqu'à une mesure finale, répétée sur plusieurs graines si l'entraînement est aléatoire.

Normaliser : les unités ne doivent pas décider à notre place

Deux équipements présentent un écart de délai de 20 millisecondes et un écart de charge de 10 points de pourcentage. La somme des carrés de ces nombres donne $20^2+10^2=500$. Si le délai est exprimé en secondes, le même calcul devient $0,020^2+10^2=100,0004$. Le phénomène n'a pas changé, mais le délai pèse désormais beaucoup moins.

Les unités changent le poids des nombres bruts.

Deux observations : écart de délai de 20 ms et écart de charge de 10 points.

Unité du délai	Écart de délai	Écart de charge	Carrés bruts additionnés
Millisecondes	20	10	400 + 100 = 500
Secondes	0,020	10	0,0004 + 100

Les mesures restent identiques ; seul le poids relatif des deux variables change.

Mêmes mesures, deux unités : la somme des carrés change.

La **normalisation** rassemble plusieurs façons de mettre les variables à une échelle adaptée. Le **centrage-réduction**, ou standardisation, soustrait une moyenne μ, puis divise par un écart-type σ non nul :

$$\tilde{x}=\frac{x-\mu}{\sigma}.$$

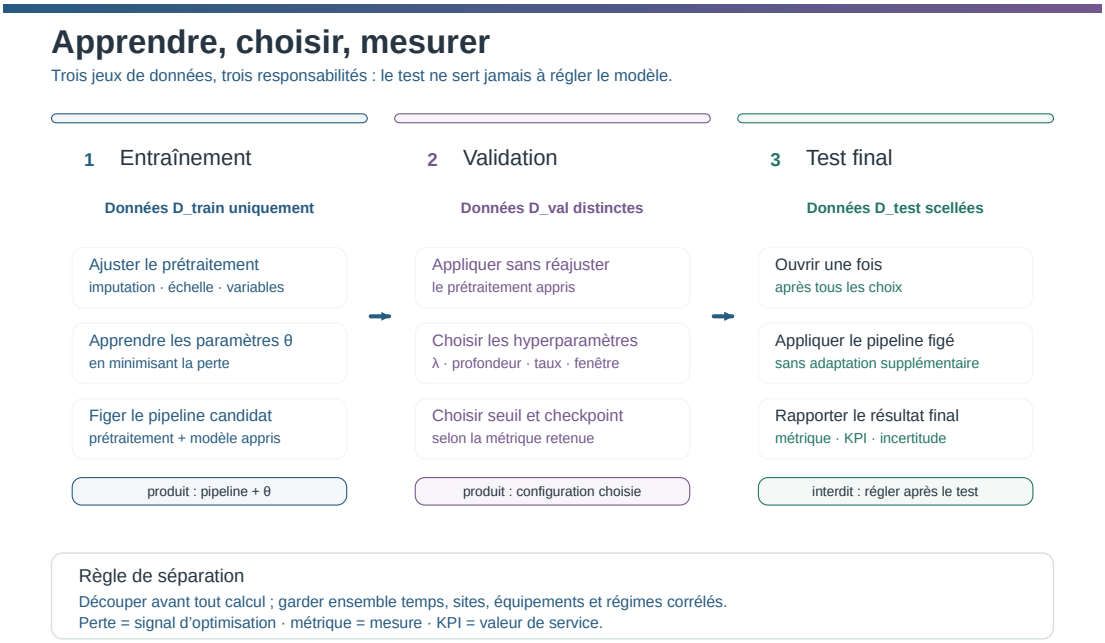
Le tilde sur x repère la valeur transformée. Pour les quatre délais $(10,10,30,30)$ ms, la moyenne vaut 20 ms. La moyenne des quatre carrés des écarts vaut 100 ms² ; sa racine donne l'écart-type, 10 ms. Les valeurs transformées sont donc $(-1,-1,1,1)$. En secondes, moyenne et écart-type deviennent 0,020 s et 0,010 s ; les valeurs transformées restent identiques. L'unité s'annule dans le rapport. Une variable constante demande un traitement adapté, puisqu'on ne peut pas diviser par zéro (Scikit-learn developers 2026).

Cette opération évite qu'une unité domine une distance, une pénalité ou certains calculs de gradient. Elle ne décide pas de l'importance métier des variables ; des poids peuvent rester nécessaires. Ces statistiques se calculent sur les seules données d'entraînement. La section suivante explique comment éviter de les calculer sur les données de test. Le guide mathématique propose le calcul complet et une question pour le refaire.

Prétraiter et construire les variables sans fuite

Le découpage doit précéder tout calcul dépendant des données. Imputation, normalisation, encodage, sélection de variables et réduction de dimension sont ajustés sur l'entraînement seulement, puis appliqués sans réajustement à la validation et au test. Les mêmes règles valent pour l'**ingénierie des variables** (*feature engineering*) : taux d'occupation, retards, quantiles glissants, degrés du graphe ou agrégats de voisinage doivent être calculables avec l'information réellement disponible au moment de la décision.

Une **fuite de données** se produit lorsqu'une information du futur, du site tenu à l'écart ou du test entre indirectement dans l'entraînement. Ajuster un normaliseur avant le découpage, construire une fenêtre avec des mesures futures ou choisir une configuration après avoir regardé le test suffit à fausser le résultat. Regrouper prétraitement et modèle dans une chaîne de traitement figée rend cette frontière vérifiable et rejouable.



Cycle sans fuite entre entraînement, validation et test, distinguant perte, métrique, paramètres et hyperparamètres

Classification rare : matrice de confusion, seuil et coût

Dans la détection de panne, d'intrusion ou d'anomalie, la classe importante est souvent rare. Sur 1 000 fenêtres contenant 10 incidents, prédire « normal » partout donne une **exactitude** (*accuracy*) de 99 %, mais un rappel nul : aucun incident n'est détecté. L'exactitude seule masque donc l'échec.

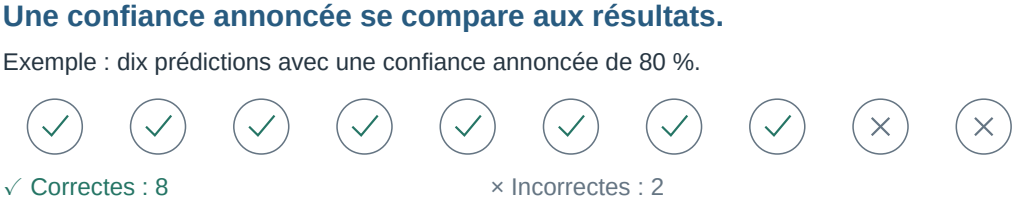
La matrice de confusion sépare quatre cas : vrai positif (VP), faux positif (FP), vrai négatif (VN) et faux négatif (FN). La **précision**, calculée comme **VP / (VP + FP)**, répond à « parmi les alertes, combien sont justes ? ». Le **rappel**, calculé comme **VP / (VP + FN)**, répond à « parmi les incidents, combien sont détectés ? ». Le score **F1 = 2 × P × R / (P + R)**, où P désigne la précision et R le rappel, résume leur moyenne harmonique, mais ne remplace pas le coût métier d'une panne manquée ou d'une alerte inutile.

Ces fractions supposent un dénominateur non nul. Sans aucune alerte, $VP+FP=0$: la précision est indéfinie. Sans incident réel, $VP+FN=0$: le rappel est indéfini. Pour calculer F1 directement à partir des effectifs, on peut écrire $F1=2VP/(2VP+FP+FN)$ lorsque ce dénominateur est non nul. Dans notre exemple sans aucune alerte, $VP=FP=0$ et $FN=10$: F1 vaut zéro. Un logiciel peut remplacer un indicateur indéfini par zéro ou une autre valeur conventionnelle ; cette convention doit accompagner le résultat.

Le **seuil de décision** transforme un score continu en action. L'abaisser capte généralement davantage d'incidents, au prix de plus d'alertes ; le relever réduit la charge opérateur, au risque d'augmenter les omissions. Il se choisit sur la validation selon le coût des erreurs, la capacité de traitement et les contraintes de sûreté. Pour une classe positive rare, une courbe précision–rappel décrit souvent mieux ce compromis qu'une valeur d'exactitude. Si le score est interprété comme une probabilité, sa calibration doit être évaluée séparément.

Que signifie une confiance de 80 % ?

Supposons qu'un modèle classe dix observations et annonce une confiance de 80 % pour chaque prédiction. Si huit prédictions sont correctes et deux incorrectes, la fréquence de succès vaut $8/10=80\,\%$. C'est cette correspondance que l'on cherche à vérifier sur de nombreux cas comparables.



Dix prédictions annoncées à 80 % : huit sont correctes.

La **calibration** compare la confiance annoncée à la fréquence observée. Si, sur un grand ensemble de prédictions annoncées autour de 80 %, seules 60 % sont correctes, le modèle est trop confiant dans cette plage. Le petit groupe du dessin illustre le calcul ; dix cas ne suffisent pas à valider la calibration. On examine plusieurs plages de confiance et des données indépendantes de l'entraînement (Guo et al. 2017).

L'exactitude compte les prédictions correctes ; la calibration vérifie la confiance annoncée. Même bien calibré, un modèle peut se tromper sur un cas particulier. On ajuste sa calibration sur des données réservées, puis on l'évalue sur un test distinct. Si les conditions changent, cette correspondance est à réexaminer.

Comment le score pédagogique du laboratoire est construit

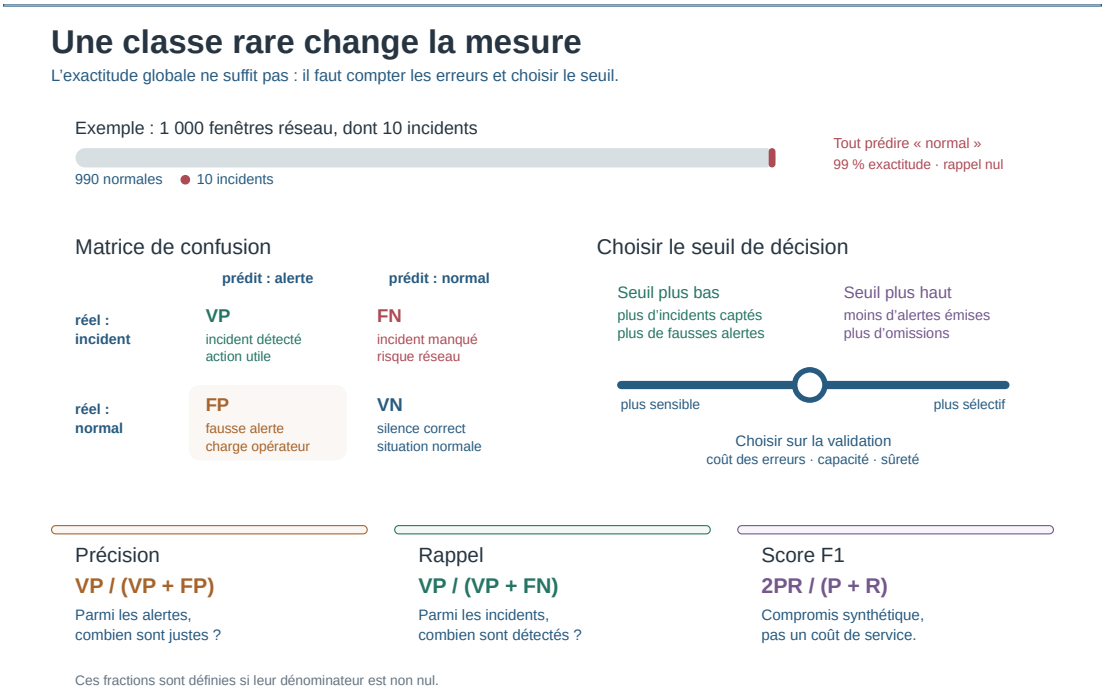
Le laboratoire L23 emploie volontairement un modèle à une seule variable afin de séparer l'apprentissage du score et le choix du seuil. Sur l'entraînement seulement, il calcule les deux centroïdes μ_0 et μ_1, moyennes de la variable respectivement pour les exemples normaux et les incidents. Il estime ensuite une échelle brute s_{brut} par la racine de la moyenne des résidus au carré autour du centroïde de chaque classe. Le code retient $s = max(s_{brut}, 10^{-9})$, avec ce plancher exprimé dans l'unité de la variable, pour éviter une division par zéro.

Pour une nouvelle valeur x, le milieu $m = (\mu_0 + \mu_1)/2$ sépare les deux centroïdes et $d \in \{-1, +1\}$ indique dans quel sens se trouve la classe incident. Le score est :

$$z(x) = d\,\frac{x - m}{s}, \quad score(x) = \frac{1}{1 + \exp(-z(x))}.$$

Le code borne z entre -30 et $+30$ avant l'exponentielle uniquement pour éviter un débordement numérique. Cette transformation sigmoïde fournit un score monotone entre 0 et 1 ; ce n'est ni une régression logistique ajustée ni la preuve d'une probabilité calibrée. Le modèle apprend μ_0, μ_1 et s sur l'entraînement. La validation choisit ensuite le seuil qui minimise $C_{FP}\,FP + C_{FN}\,FN$, avec le rappel comme départage en cas de coût égal. Le test scellé ne participe à aucun de ces calculs.

C_{FP} est le coût d'une fausse alerte et C_{FN}, celui d'un incident manqué. Par défaut dans le laboratoire, ces coûts valent respectivement 1 et 12 : deux fausses alertes et un incident manqué donnent donc $2 \times 1 + 1 \times 12 = 14$.



Classification rare : exactitude trompeuse, matrice de confusion et effet du seuil sur alertes et omissions

Le laboratoire L23 transforme cette lecture en protocole exécutable : le seuil est choisi sur la validation, le jeu de test reste scellé jusqu'à la décision finale, et les empreintes des trois partitions sont conservées avec les métriques.

Les modes d'apprentissage : plusieurs axes, pas une seule liste

Repère taxonomique — ne pas confondre niveau de description et catégorie. EUROCAE WG-114 et SAE G-34 ont publié une taxonomie commune destinée à clarifier le vocabulaire des techniques d'IA employées en aéronautique (EUROCAE et SAE International 2024). Pour la grille éditoriale ci-dessous, nous distinguons finalité opérationnelle, rôle système, régime d'apprentissage et famille de modèles : ces axes répondent à des questions différentes. Cette adaptation aux réseaux ne reproduit pas le document normatif.

Niveau ou axe	Question à laquelle il répond	Exemples	Confusion à éviter
Finalité opérationnelle	Quel effet réseau recherche-t-on ?	prévoir une congestion, détecter une anomalie, proposer une route	la finalité n'est pas un algorithme
Rôle et autorité dans le système	Que peut faire la sortie du modèle ?	informer, recommander, contraindre, commander	l'autonomie n'est pas un mode d'apprentissage
Constituant IA	Quel élément apprend ou infère ?	estimateur, classifieur, politique, moniteur	le constituant n'est pas le système complet
Régime d'apprentissage	D'où vient le signal d'adaptation ?	supervisé, auto-supervisé, renforcement, apprentissage en ligne	le régime n'est pas une famille de modèles
Famille de modèles	Quelle structure représente la fonction ?	arbre, réseau neuronal, GNN, transformeur	l'architecture ne détermine pas à elle seule l'usage
Phase du cycle de vie	À quel moment porte l'activité ?	données, entraînement, vérification, intégration, exploitation	une activité d'assurance n'est pas une capacité IA

Ces axes se croisent sans se confondre. Une même famille de modèles peut servir plusieurs objectifs, apprendre selon différents régimes et recevoir des droits d'action très différents.

La trilogie « supervisé, non supervisé, renforcement » est utile pour commencer, mais insuffisante pour choisir ou auditer une IA réseau. Elle mélange parfois la source du signal d'apprentissage, la manière d'interagir avec le système, le moment où le modèle est mis à jour et le lieu où résident les données.

Un système réel se décrit donc par **plusieurs axes orthogonaux** :

Axe	Question à poser	Exemples
forme de sortie	que produit le modèle ?	classe, valeur, score, rang, embedding, action
signal d'apprentissage	d'où vient l'information qui corrige le modèle ?	labels, données masquées, récompense
interaction et décision	apprend-on une prédiction ou les conséquences d'actions ?	prédiction indépendante, imitation, bandit, RL
régime de mise à jour	quand les paramètres changent-ils ?	batch, en ligne, continu
transfert	comment réutilise-t-on un savoir déjà appris ?	ajustement fin, adaptation de domaine, distillation
distribution	où sont données, calcul et agrégation ?	centralisé, fédéré, apprentissage fractionné
autorité opérationnelle	jusqu'où la sortie peut-elle agir ?	observer, prévoir, recommander, planifier, exécuter

Un GNN ou un Transformer est une **architecture**. PPO est un **algorithme de renforcement**. LoRA est une **technique d'adaptation de paramètres**. Le fédéré est un **régime de coordination des données et mises à jour**. Un **modèle de fondation** (*foundation model*) décrit un **préentraînement large puis une réutilisation**. Ces termes répondent à des questions différentes et peuvent se combiner.

Avant les modes — Quelle forme prend la sortie ?

Le **mode d'apprentissage** ne détermine pas la **forme de sortie**. Une classification peut être supervisée, faiblement supervisée ou effectuée *zero-shot* par un modèle de fondation. Un embedding peut être appris de façon auto-supervisée (*self-supervised*) puis alimenter une régression. Une action peut être imitée, apprise par RL ou calculée par un solveur sans IA.

Dans les exemples radio, un **MCS** (*Modulation and Coding Scheme*) est un profil qui associe une modulation et un taux de codage.

Forme de sortie	Ce que le système renvoie	Exemple réseau
classification	une classe, plusieurs labels ou leurs probabilités	incident « fibre », « alimentation » ou « configuration »
régression / estimation	une valeur numérique ou un vecteur	débit utile, latence, durée avant panne
notation (*rating*) / scoring	un score ordinal, de risque ou de qualité, pas nécessairement une grandeur physique	santé d'un lien sur 100, risque de panne
classement (*ranking*)	une liste ordonnée, sans imposer une distance absolue entre rangs	routes, alarmes ou interventions candidates
distribution / intervalle	plusieurs valeurs plausibles et une incertitude	quantiles de délai ou ensemble de MCS possibles
prévision	une valeur ou trajectoire future pour un horizon explicite	matrice de trafic dans 5, 30 et 60 minutes
réduction / embedding	un vecteur compact ou structuré destiné à une tâche ultérieure	résumer une fenêtre de télémétrie ou un graphe
clustering	un identifiant de groupe découvert sans classe imposée	familles de comportements ou de sites

Forme de sortie	Ce que le système renvoie	Exemple réseau
anomalie / détection	un score d'écart et parfois une localisation ou un segment	rupture de profil sur un lien ou un flux
reconstruction / imputation	une donnée restaurée, complétée ou débruitée	télémétrie manquante, canal ou média reconstruit
prédiction structurée (*structured prediction*)	une séquence, un graphe, une carte ou une matrice liée à l'entrée	matrice de trafic ou liens probables
génération	un nouvel objet : texte, média, topologie, scénario ou configuration candidate	diagnostic ou scénario de test
politique / action / allocation	une action, une distribution d'actions ou un calendrier de ressources	choisir route, MCS, placement, admission ou abandon

Trois distinctions évitent beaucoup de malentendus :

- **Régression** : estime une quantité ayant une unité ou une interprétation numérique ; un **score** peut seulement ordonner le risque ;
- **Rating et ranking** : un rating attribue une note à chaque objet ; un **ranking** promet surtout un ordre relatif ;
- **Réduction de dimension et embedding** : produisent une représentation, pas encore une décision métier. Sa qualité se juge aussi par ce que la tâche suivante réussit ou perd.

Pour une même liaison, les sorties peuvent donc être :

```
classification : « congestionnée »
régression     : latence prévue = 37 ms, intervalle [31 ; 48]
rating         : risque = 82 / 100
ranking        : chemin B > chemin A > chemin C
embedding      : [0,18 ; -0,72 ; 0,05 ; ...]
action         : marquer ECN, servir la file voix, différer ou abandonner
```

`ECN` (*Explicit Congestion Notification*, notification explicite de congestion) marque un paquet plutôt que de l'abandonner, pour signaler une congestion naissante sans perte.

La métrique doit correspondre à la sortie. Une classification exige matrice de confusion, précision, rappel et score F1 par classe plutôt qu'une exactitude globale (*accuracy*) isolée. Si les scores produits par le classifieur sont utilisés comme probabilités ou pour fixer un seuil, leur calibration doit aussi être testée.

Une régression ponctuelle se juge notamment par erreur absolue ou quadratique ; une prévision ajoute l'erreur par horizon. Si le modèle prétend fournir un intervalle ou une distribution, il faut alors mesurer quantiles, calibration et couverture.

Un ranking peut utiliser le gain cumulé actualisé normalisé (NDCG), le rang réciproque moyen (MRR) ou le rappel au rang k, mais une route bien classée doit encore être réalisable.

Embeddings et clusters demandent stabilité, conservation des voisinages et utilité sur une tâche aval. Prédiction structurée, génération et action se jugent aussi par validité globale, respect des contraintes, effet de service, équité et possibilité de repli — pas par vraisemblance apparente.

Ces formes peuvent s'enchaîner. Un GNN construit un embedding, une tête produit des scores, un solveur ou un environnement en déduit une allocation faisable, puis un contrôleur applique la décision. Il faut donc séparer :

```
sortie brute du modèle
 -> post-traitement ou décodage
 -> décision calculée
 -> action réellement appliquée
 -> contraintes effectivement garanties
```

Dans le mode PPO par arêtes de `sb3_trainer_project`, la politique échantillonne une action continue. Stable-Baselines3, bibliothèque d'apprentissage par renforcement, borne chaque composante dans `[0, 1]`, l'intervalle déclaré par l'environnement. Celui-ci l'écrête encore, la transforme en poids d'arêtes dans `[edge_action_min_scale, 1]`, sélectionne les chemins, déduit les fractions servies, puis applique capacités et contention. La sortie brute n'est donc pas une route complète et ne prouve à elle seule ni route faisable ni débit livré.

Axe 1 — D'où vient le signal d'apprentissage ?

Mode	Signal disponible	Exemple réseau	Piège principal
supervisé	chaque exemple possède une cible	prédire le débit obtenu à partir de SINR (*Signal-to-Interference-plus-Noise Ratio*, rapport signal sur interférence plus bruit), charge et MCS	labels coûteux, biaisés ou issus du futur
semi-supervisé	peu d'exemples labellisés et beaucoup sans label	classifier des incidents avec quelques tickets fiables	propager des pseudo-labels faux
supervision faible (*weak supervision*)	labels bruités, incomplets ou produits par des règles	déduire une panne probable depuis alarmes et heuristiques	apprendre les erreurs de la règle
auto-supervisé	la donnée fabrique sa propre cible, par masquage, ordre ou contraste	masquer une métrique ou une arête puis la reconstruire	réussir le prétexte sans aider la tâche finale
non supervisé	aucune cible explicite n'est fournie	regrouper des profils de trafic ou estimer une densité	interpréter un groupe comme une cause

Dans l'**auto-supervisé**, cacher une portion de télémétrie puis apprendre à la prédire fournit un signal massif sans annotation humaine ; la représentation peut ensuite être adaptée à une tâche supervisée ou décisionnelle. Le **non supervisé** recherche une structure, une densité ou une compression sans cible externe. La frontière mathématique n'est pas absolue ; la question pratique reste : « quelle cible optimise-t-on, et qui l'a construite ? »

Une anomalie est seulement un écart au profil appris. Elle n'est pas automatiquement une attaque, une panne ni même un événement indésirable.

Acquisition active des labels. En **apprentissage actif** (*active learning*), le modèle choisit les exemples pour lesquels demander une annotation à l'opérateur. Il réduit potentiellement le coût de labellisation, mais peut concentrer l'attention sur les cas qu'il juge déjà intéressants et laisser invisibles des familles de pannes. Ce mécanisme concerne d'abord **qui choisit les labels à acquérir**, pas la cadence à laquelle les poids sont mis à jour.

Justesse et dispersion d'un instrument

Avant d'étudier l'erreur d'un modèle, regardons celle des mesures qui l'alimentent. On répète la mesure d'une même grandeur stable, avec la même procédure, le même instrument et le même opérateur, au même endroit et pendant une courte période. Ces conditions permettent d'étudier la **répétabilité** de l'instrument.

Deux questions se séparent. Les indications sont-elles proches les unes des autres ? Leur moyenne est-elle proche de la référence ? La première porte sur la **dispersion** : une faible dispersion correspond à une bonne **fidélité de mesure**, appelée ici « précision » au sens courant du schéma des cibles. La seconde porte sur la **justesse** : un décalage systématique dégrade la justesse, même si les indications se répètent très bien. Le **biais de mesure** est une estimation de ce décalage. Le Vocabulaire international de métrologie distingue ces notions de l'**exactitude** ; justesse et fidélité ne sont donc pas deux noms pour une même propriété (Joint Committee for Guides in Metrology 2012).

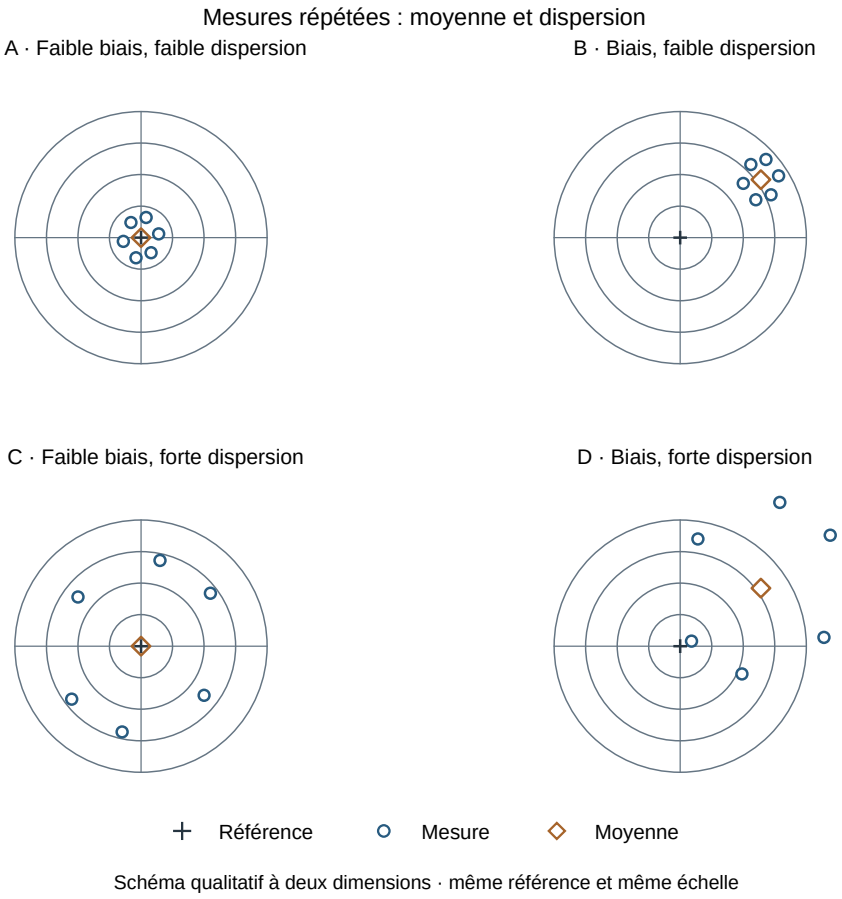


Schéma qualitatif à deux dimensions · même référence et même échelle

Quatre cas de mesures répétées : moyenne proche ou éloignée de la référence, avec faible ou forte dispersion. Le losange indique la moyenne des points dessinés ; les cibles sont un schéma qualitatif, pas un résultat expérimental.

Exemple en millisecondes. Un délai de référence stable vaut $r = 10\,ms$. Pour ce calcul pédagogique, on néglige l'incertitude de cette référence. Trois indications de l'instrument A valent $10,9$, $11,0$ et $11,1\,ms$; celles de B valent $9,0$, $10,0$ et $11,0\,ms$.

Notons z_i les $n = 3$ indications, m leur moyenne, $\hat{b}$ le biais estimé et s l'écart-type de l'échantillon. m, $\hat{b}$ et s s'expriment en millisecondes :

$$m = \frac{1}{n}\sum_{i=1}^{n} z_i,\ \hat{b} = m - r,\ s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(z_i - m)^2}.$$

Pour A, $m_A = (10,9 + 11,0 + 11,1)/3 = 11,0\,ms$. Ainsi, $\hat{b}_A = +1,0\,ms$, tandis que $s_A = \sqrt{(0,1^2 + 0 + 0,1^2)/2} = 0,1\,ms$. Pour B, $m_B = 10,0\,ms$, donc $\hat{b}_B = 0$, mais $s_B = \sqrt{(1^2 + 0 + 1^2)/2} = 1,0\,ms$. A répète mieux ses indications ; B est mieux centré dans cet exemple. Trois valeurs ne suffisent cependant pas à établir la justesse d'un instrument : une moyenne proche de la référence peut survenir par hasard.

Un **étalonnage** établit une relation entre les indications et des références avec leurs incertitudes, puis utilise cette relation pour obtenir un résultat de mesure à partir d'une indication, dans des conditions et un domaine donnés. Appliquer à A une correction de $-1\,ms$ pourrait corriger ce biais local, sans réduire sa dispersion. Cette correction ne prouve pas la validité à un autre délai, une autre température ou après une dérive. L'étalonnage n'est pas l'ajustage de l'instrument et ne supprime pas toute incertitude (Joint Committee for Guides in Metrology 2012).

Dans la section suivante, la variance concerne le changement de **prédiction quand le jeu d'apprentissage change** ; ici, la dispersion concerne les **indications répétées de l'instrument**. Un biais de mesure peut affecter les données d'un modèle, mais les deux expériences restent distinctes. Enfin, pour un classifieur, VP et FP comptent les vrais et faux

positifs. Lorsque $VP+FP>0$, la métrique anglaise *precision*, $VP/(VP+FP)$, désigne la proportion de vrais positifs parmi les résultats annoncés positifs. Elle ne mesure pas la répétabilité d'un capteur.

D'où vient l'erreur ? Biais, variance et bruit irréductible

Lire les moyennes probabilistes. L'**espérance** $E[Z]$ est la moyenne des valeurs possibles de Z, pondérées par leurs probabilités. Un débit de 10 Mbit/s avec une probabilité de 0,8, et de zéro sinon, a une espérance de 8 Mbit/s ; ce n'est pas un débit garanti à chaque transfert. La **variance** est la moyenne des carrés des écarts à cette moyenne ; l'**écart-type** est sa racine carrée et retrouve l'unité de la grandeur. Ici, E_D moyenne sur les différents jeux d'apprentissage possibles. $E[\varepsilon/x]=0$ signifie que, pour l'entrée x fixée, le bruit est nul en moyenne.

Pour une régression évaluée par l'erreur quadratique, supposons qu'à une entrée fixée x, la cible vérifie $Y=f(x)+\varepsilon$, avec $E[\varepsilon/x]=0$. Si $\hat{f}_D$ est le modèle appris sur un jeu d'entraînement aléatoire D, supposons aussi que le bruit de cette **cible de test** soit indépendant de D à x fixé. En notant $\sigma_\varepsilon^2(x)$ sa variance, supposée finie, on obtient :

$$E_{D,\varepsilon}\left[(Y-\hat{f}_D(x))^2\right]=B^2(x)+V(x)+\sigma_\varepsilon^2(x),$$

avec

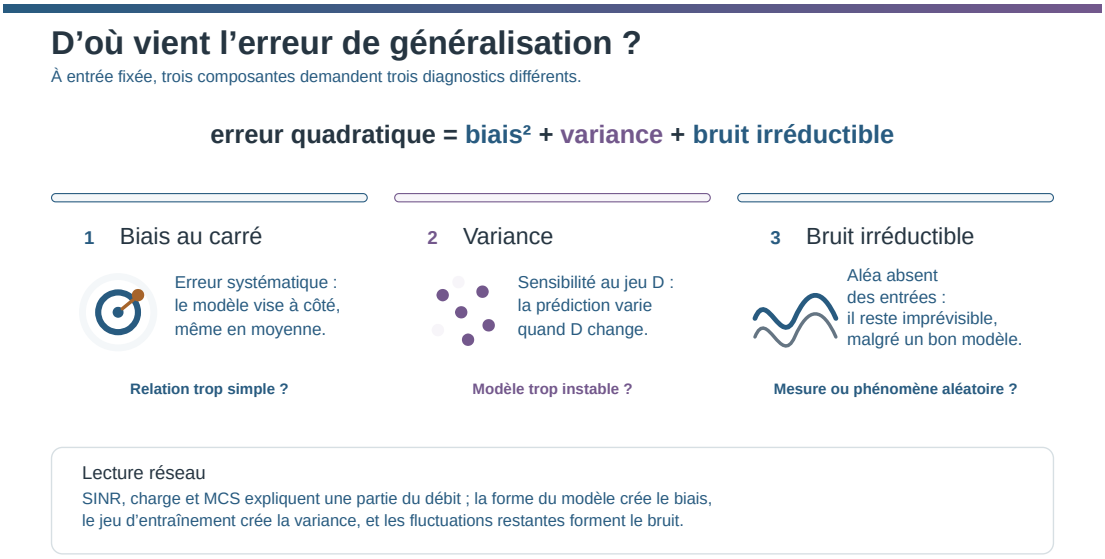
$$B(x)=E_D[\hat{f}_D(x)]-f(x)$$

et

$$V(x)=E_D\left[\left(\hat{f}_D(x)-E_D[\hat{f}_D(x)]\right)^2\right].$$

Les trois termes ont des causes différentes :

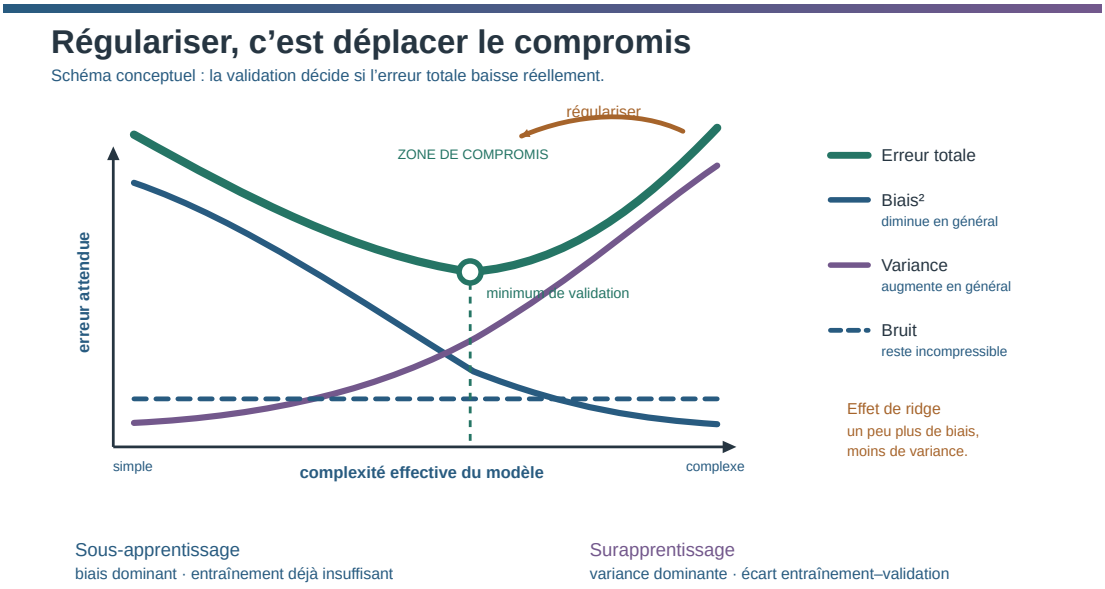
- **Biais au carré** : mesure l'erreur systématique du modèle, par exemple une relation trop simple pour représenter le phénomène ;
- **Variance** : mesure combien la prédiction change lorsque le jeu d'entraînement change ;
- **Bruit irréductible** : représente les fluctuations de la cible qui ne peuvent pas être déduites des variables disponibles. Il peut dépendre de x et ne doit pas être confondu avec la variance totale de Y.



Décomposition de l'erreur quadratique de généralisation en biais au carré, variance et bruit irréductible

Un **sous-apprentissage** se manifeste souvent par un biais dominant : même les données d'entraînement sont mal expliquées. Un **surapprentissage** se manifeste souvent par une variance dominante : l'erreur d'entraînement est faible,

mais l’erreur sur des situations nouvelles augmente. La **régularisation**, comme la régression ridge, accepte alors parfois un peu plus de biais pour réduire davantage la variance ; elle cherche à diminuer l’erreur totale de généralisation, pas un terme isolé.



Compromis conceptuel entre biais, variance, bruit et complexité, avec l’effet de la régularisation

Dans un réseau, SINR, charge et MCS peuvent être fortement corrélés. Les coefficients d’une régression non régularisée deviennent alors instables d’un échantillon à l’autre. Une pénalisation ridge peut les stabiliser, mais son bénéfice doit être vérifié sur des périodes, sites et régimes non vus : si le biais ajouté coûte plus que la variance supprimée, une référence simple peut rester meilleure.

*Source conceptuelle : Ricco Rakotomalala, **Régression régularisée — Ridge, Lasso, Elasticnet**, Université Lumière Lyon 2, 2018, p. 9–11. Formulation et exemple réseau adaptés pour cet ouvrage.*

Méthodes d’ensemble — Bagging, boosting et stacking

Une méthode d’**ensemble** combine plusieurs modèles. Elle ne définit ni la forme de sortie, ni l’autorité reçue :

- **Bagging** : l’agrégation bootstrap entraîne indépendamment plusieurs estimateurs sur des échantillons tirés avec remise, puis moyenne leurs régressions ou fait voter leurs classifications. Il réduit surtout la variance d’un modèle instable ; les forêts aléatoires en sont une famille ;
- **Boosting** : construit des modèles successifs qui se concentrent sur les erreurs ou résidus précédents. Il peut réduire le biais, mais amplifier bruit, labels erronés ou poids extrêmes ;
- **Stacking** : entraîne un méta-modèle à combiner les sorties de plusieurs modèles. Ses entrées d’entraînement doivent être produites hors du pli ayant servi à ajuster chaque modèle de base, sinon la fuite rend le résultat artificiellement bon.

Pour prévoir une panne de lien, un bagging d’arbres peut être plus stable qu’un arbre isolé. Si tous les arbres apprennent toutefois les mêmes sites, saisons et firmwares, leur vote unanime peut rester faux sur un nouveau domaine. La dispersion de l’ensemble fournit un indice d’incertitude, pas une calibration ni une garantie. Les découpages d’évaluation doivent respecter temps, sites et causes communes.

Axe 2 — Prédire, imiter ou apprendre les conséquences d’une action

Apprentissage supervisé. Le modèle approxime une relation entre entrée et cible à partir d’exemples. Il convient à la prévision de trafic, la classification d’alarme ou l’estimation de qualité lorsque des exemples représentatifs existent. Un solveur d’optimisation peut aussi générer des solutions servant de labels : le modèle apprend alors à **imiter le solveur**, sans hériter de sa preuve d’optimalité.

Apprentissage par imitation (*imitation learning*). Une politique reproduit les actions d'un expert humain, d'un contrôleur classique ou d'un solveur. Le *behavior cloning* est proche d'une classification supervisée des actions. Il échoue souvent lorsque sa propre petite erreur conduit à un état absent des démonstrations. Des méthodes interactives ou adversariales peuvent réduire ce décalage, sans transformer une démonstration imparfaite en vérité.

Bandit contextuel. À chaque décision, le modèle observe un contexte, choisit une action et reçoit un résultat principalement attribuable à cette action. Il est adapté au choix d'une variante, d'un canal ou d'un profil lorsqu'il n'est pas nécessaire de modéliser une longue chaîne de conséquences. Si le choix remplit une file, épuise une batterie ou modifie les futurs transferts de connexion, le problème ne se réduit plus à un simple bandit.

Explorer sans confondre score et probabilité

Les bandits constituent une famille de méthodes publiques. Une stratégie gloutonne choisit la meilleure estimation courante ; une stratégie dite ε-gloutonne réserve une part des choix à l'exploration. Une méthode à borne supérieure de confiance privilégie un compromis entre résultat estimé et incertitude. Ce sont des alternatives, pas les étapes successives d'une même architecture.

Le symbole $\square^\top$ **transpose** un vecteur ou une matrice : il échange lignes et colonnes. Pour un vecteur colonne x de composantes 2 et 3, $x^\top \theta = 2\theta_1 + 3\theta_2$ est une somme pondérée. Le produit $x\,x^\top$ forme au contraire un tableau dont les lignes sont $(4, 6)$ et $(6, 9)$: il conserve les produits entre composantes.

Dans le modèle linéaire disjoint de **LinUCB**, chaque action a possède ses propres statistiques A_a et b_a. Pour les observations recueillies après avoir choisi cette action, on écrit :

$$A_a = \lambda I + \sum_{i : a_i = a} x_i x_i^\top, b_a = \sum_{i : a_i = a} r_i x_i, \hat{\theta}_a = A_a^{-1} b_a.$$

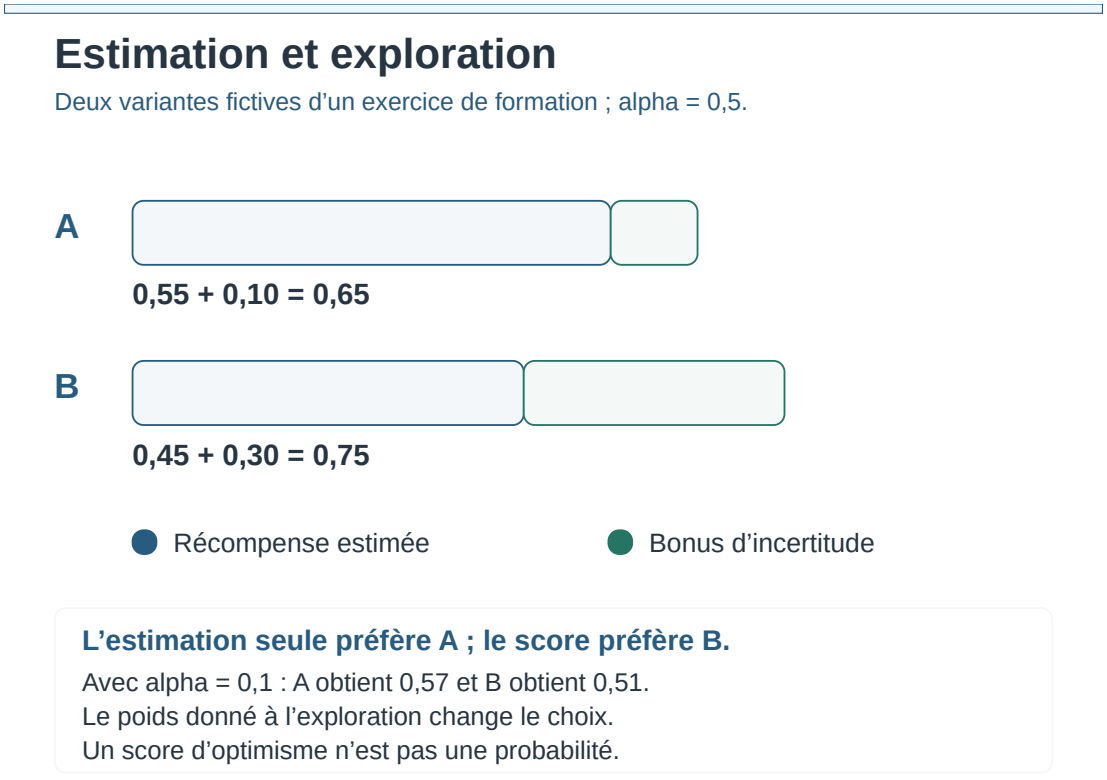
$$U_a(x) = x^\top \hat{\theta}_a + \alpha \sqrt{x^\top A_a^{-1} x}.$$

Le vecteur x décrit le contexte ; r_i est la récompense observée. I est la matrice identité et $\lambda > 0$ stabilise l'estimation ridge. Le produit extérieur $x_i x_i^\top$ décrit les directions déjà observées. Le premier terme du score exploite les résultats ; le second favorise les options moins documentées. Le paramètre α règle ce compromis. La notation A_a^{-1} n'impose pas de calculer explicitement un inverse : la résolution d'un système linéaire peut suffire (Li et al. 2010).

Exemple synthétique indépendant. Un laboratoire compare deux variantes d'un exercice de formation. Supposons les estimations et incertitudes suivantes, pour un contexte fixé et $\alpha = 0,5$:

Variante	Récompense estimée	Incertitude	Bonus	Score
A	0,55	0,20	0,10	0,65
B	0,45	0,60	0,30	0,75

L'estimation seule préfère A ; le score avec exploration préfère B. Avec $\alpha = 0,1$, les scores deviennent 0,57 et 0,51 : A repasse devant. Ces valeurs sont inventées pour le calcul. Elles ne sont ni des paramètres d'application ni des résultats expérimentaux.



Deux composantes d'un score d'exploration : estimation et bonus d'incertitude. Exemple public de bandit, indépendant de tout produit.

Un tel score peut dépasser 1 : ce n'est pas une probabilité de succès. La récompense doit être définie avant l'expérience ; un clic, une réponse correcte et un temps gagné ne mesurent pas la même chose. Le résultat des actions non choisies reste généralement inconnu. Une évaluation hors ligne doit tenir compte de la manière dont les observations ont été collectées : réutiliser un journal arbitraire ne garantit pas une comparaison sans biais. Enfin, l'exploration reste limitée aux actions autorisées par le protocole d'essai.

Pourquoi une prédiction ne suffit plus : l'idée du RL

L'**apprentissage par renforcement** (*reinforcement learning*, **RL**) devient pertinent lorsque l'action présente modifie les situations et les choix futurs. Il ne cherche plus seulement à répondre « que va-t-il se passer ? », mais « quelle suite de décisions produit le meilleur résultat à long terme ? ».

Le mot « récompense » désigne ici un signal numérique défini pour l'agent. Le parallèle avec l'humain a ses limites : je ne pense pas qu'on obtienne le meilleur d'une personne en la réduisant à un automate qui réagit au bâton ou à la carotte. Pour approfondir cette question, je vous invite à consulter la hiérarchie des besoins de Maslow, souvent représentée par une pyramide : besoins physiologiques, sécurité, appartenance, estime et accomplissement de soi (Maslow 1943). C'est une grille de réflexion ; Maslow lui-même reconnaissait des exceptions à l'ordre proposé.

Prenons deux relais entre une source et une destination :

- le relais A a maintenant 5 ms de délai, mais sa file est presque pleine ;
- le relais B a 12 ms de délai, mais sa file est vide.

Envoyer encore tous les paquets vers A donne une bonne récompense immédiate. Quelques pas plus tard, sa file déborde, des paquets sont abandonnés, le transport retransmet et le délai augmente. Choisir B pour une partie du trafic peut être moins bon maintenant et meilleur sur l'épisode complet.

```
prédiction : « si je choisis A maintenant, délai estimé = 5 ms »
RL         : « si je choisis souvent A, quelle file et quel service
               obtiendrai-je ensuite ? »
```

Une prévision parfaite du prochain délai ne donne donc pas automatiquement la meilleure politique. Il faut encore représenter les transitions, les effets retardés, l'incertitude et le compromis entre objectifs.

À retenir au premier passage.

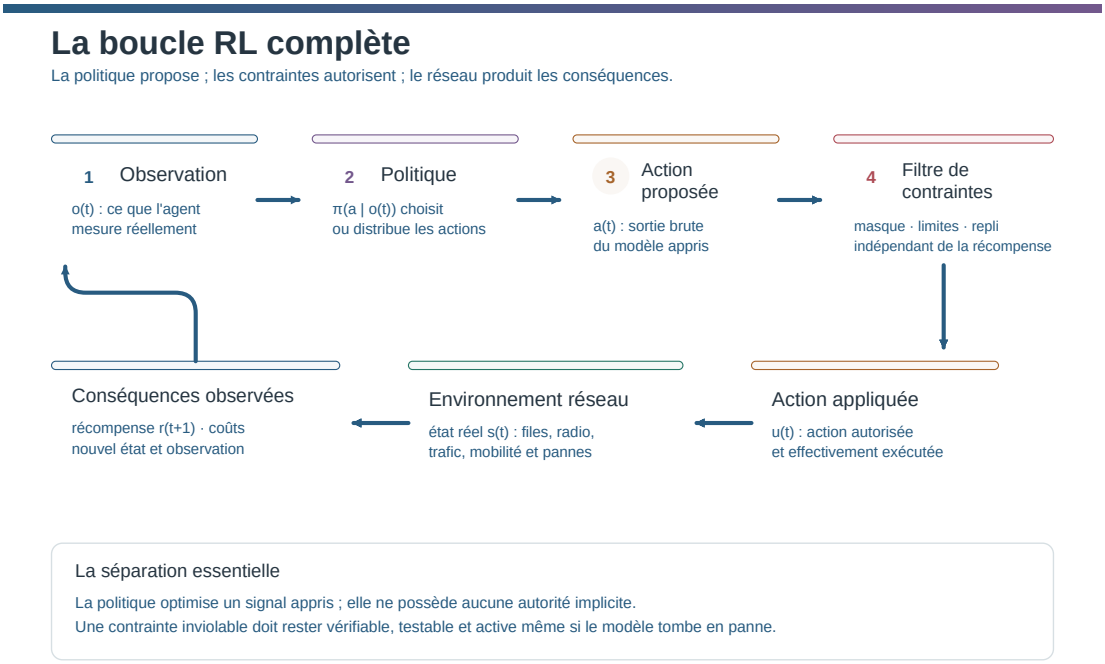
Le RL apprend par les conséquences d'actions successives. L'agent observe, agit, reçoit un signal de récompense et recommence. Il est utile seulement si l'action change assez le futur pour justifier cette complexité.

Parcours conseillé.

Pour une première lecture, suivre l'exemple, le schéma de boucle et le mini-calcul à trois pas, puis aller directement à « Quand ne pas employer le RL ? ». Les passages signalés comme approfondissements — MDP/POMDP, V, Q, avantage, erreur TD, GAE et carte des familles — forment une deuxième lecture utile pour comprendre PPO et auditer un code. L'étude de cas Gymnasium/PPO utilise Gymnasium, une bibliothèque qui organise les échanges entre l'agent et son environnement. Cette étude transforme ensuite ces notions en contrats d'implémentation.

La boucle RL : état, observation, action et transition

À chaque pas t, la politique ne fait que proposer une action. Le filtre de contraintes conserve l'autorité, l'environnement réseau produit les conséquences, puis une nouvelle observation referme la boucle.



Boucle RL séparant observation, politique, action proposée, filtre de contraintes, action appliquée, environnement réseau, récompense et coûts

Les termes ont des rôles précis :

Terme	Sens	Exemple réseau
état s_t	description suffisante du monde pour déterminer la distribution du futur	toutes les files, canaux, positions et flux actifs
observation o_t	partie mesurée ou construite que reçoit l'agent	files locales, SINR retardé, topologie connue
action a_t	choix de la politique	poids de liens, MCS, admission ou allocation

Terme	Sens	Exemple réseau
transition	évolution du monde après l'action	arrivées, service des files, mobilité, panne
récompense r_{t+1}	signal scalaire utilisé pour apprendre	débit livré moins pénalités de délai et perte
coût	mesure séparée à borner	puissance, violation de SLA, utilisateur non servi
politique π	règle qui associe une distribution d'actions à une observation	probabilité ou paramètres d'allocation
épisode	suite de pas entre initialisation et fin	scénario de 60 s ou jusqu'à la fin d'une mission
horizon	durée physique dont les conséquences comptent	100 pas de 100 ms, soit 10 s

Comprendre une chaîne de Markov

La machine à états décrit des transitions régies par des événements. Une **chaîne de Markov** décrit une évolution aléatoire, ou **stochastique** : le prochain état est décrit par des probabilités. À chaque pas discret t, le système possède un état S_t. L'**hypothèse de Markov** signifie que l'état courant résume le passé utile :

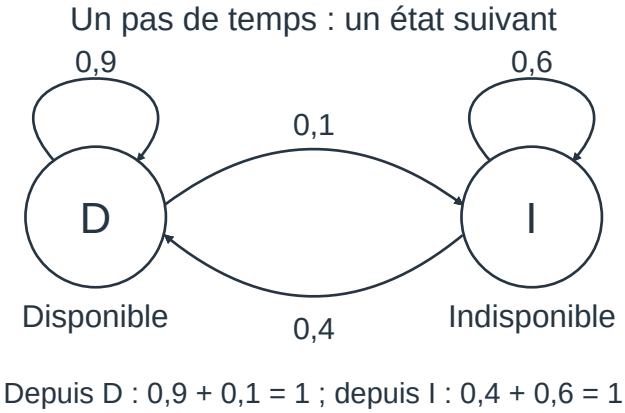
$$P(S_{t+1}=j \mid S_t=i, S_{t-1}, \ldots, S_0) = P(S_{t+1}=j \mid S_t=i).$$

On lit : « la probabilité du prochain état j, sachant l'état courant i, ne change pas lorsqu'on ajoute les états antérieurs ». Les états successifs peuvent pourtant être dépendants : une liaison en panne a souvent davantage de chances de rester en panne.

Prenons une liaison **disponible (D)** ou **indisponible (I)**. Si les probabilités restent fixes, sa **matrice de transition** peut être :

$$P = \begin{pmatrix} 0,9 & 0,1 \\ 0,4 & 0,6 \end{pmatrix}.$$

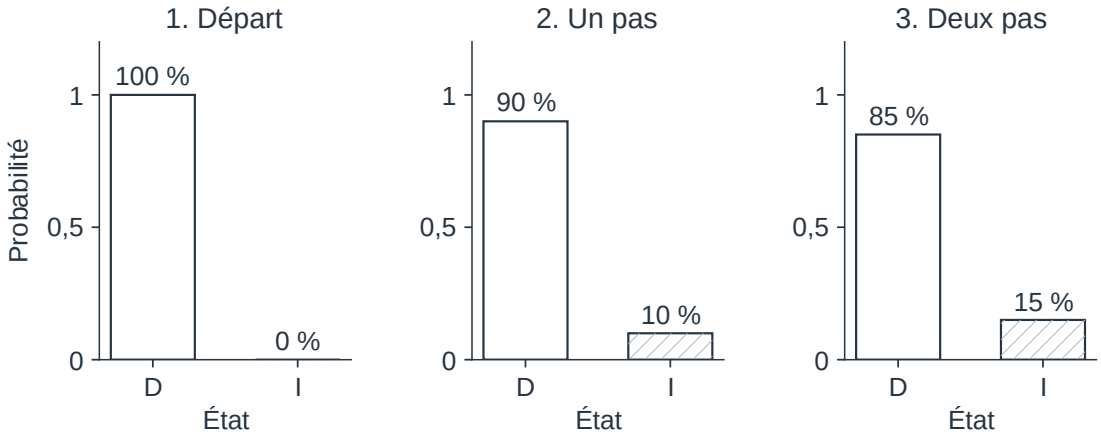
Les lignes donnent l'état courant et les colonnes le prochain état, dans l'ordre D, I. Depuis D, la liaison reste disponible avec probabilité 90 % ; depuis I, elle redevient disponible avec probabilité 40 %. Chaque coefficient est positif ou nul ; **chaque ligne totalise 1**.



Chaîne de Markov à deux états : à chaque pas, les probabilités partant d'un même état somment à 1. La flèche en boucle représente le maintien dans cet état.

Une **distribution sur les états** est une liste de leurs probabilités. Avec la convention d'un vecteur ligne, elle évolue selon $\pi_{t+1} = \pi_t P$. Si la liaison est initialement disponible avec certitude, $\pi_0 = (1,0)$: on obtient $\pi_1 = (0,9, 0,1)$, puis

$\pi_2 = (0,85 , 0,15)$. La probabilité d'être disponible au deuxième pas vaut $0,9 \times 0,9 + 0,1 \times 0,4 = 0,85$: on additionne les deux chemins possibles, rester disponible ou tomber en panne puis revenir.



D : disponible I : indisponible Même matrice à chaque pas

Les probabilités de disponibilité évoluent : départ certain, un pas, puis deux pas. Chaque panneau totalise 100 % ; il montre une distribution, pas les états successifs d'une liaison observée.

Une distribution **stationnaire** vérifie $\pi = \pi P$: ses probabilités restent les mêmes après un pas, même si la liaison change d'état. Notons d_t la probabilité d'être disponible au pas t. Ici :

$$d_{t+1} = 0,9\, d_t + 0,4\,(1 - d_t) = 0,4 + 0,5\, d_t .$$

À l'équilibre, $d = 0,4 + 0,5\, d$, donc $d = 0,8$ et $\pi = (0,8 , 0,2)$. Comme $d_{t+1} - 0,8 = 0,5\,(d_t - 0,8)$, l'écart est divisé par deux à chaque pas : cette chaîne converge vers cette distribution.

Une distribution stationnaire ne garantit pas toujours la convergence. Une chaîne qui alterne obligatoirement D puis I possède la distribution stationnaire $(0,5 , 0,5)$. Mais, depuis un départ certain en D, ses distributions alternent entre $(1,0)$ et $(0,1)$.

Dans cet exemple, l'état est connu. Un **modèle de Markov caché (HMM)** infère des états non directement observés à partir de mesures. Un **processus de décision markovien (MDP)** ajoute des actions : le choix de l'action peut modifier les probabilités de transition et les récompenses.

> **Deuxième lecture — MDP et observation partielle.**
>
> Le tableau et la boucle suffisent pour comprendre le mini-calcul. Les paragraphes suivants précisent les hypothèses nécessaires pour construire ou auditer un environnement réel.

Un **processus de décision markovien** (*Markov decision process*, **MDP**) suppose que l'état courant contient l'information nécessaire pour modéliser la distribution du prochain état et de la récompense, une fois l'action connue. « Markov » ne veut pas dire que le réseau n'a pas de passé ; cela signifie que le passé utile a été résumé dans l'état.

L'agent voit souvent seulement une observation incomplète : une file distante est cachée, le CQI (*Channel Quality Indicator*, indicateur de qualité de canal) est retardé, l'intention d'un autre domaine est inconnue. On parle alors de **processus de décision markovien partiellement observable** (*partially observable MDP*, **POMDP**). Un historique, un filtre d'état ou un réseau récurrent peut aider à inférer ce qui manque ; il ne transforme pas une mesure absente en certitude.

La durée du pas est une décision de modélisation. À 1 ms, le canal évolue mais l'orchestrateur n'a pas le temps d'agir ; à 10 s, plusieurs phénomènes rapides sont fusionnés. État, action et récompense doivent donc correspondre à la même échelle de contrôle.

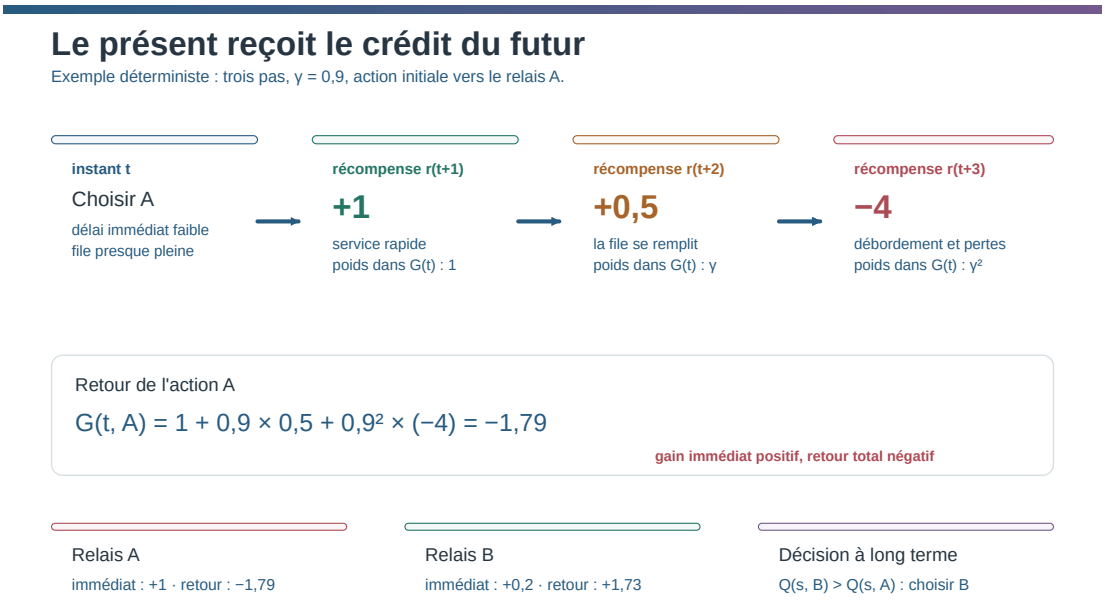
Une fin naturelle — objectif atteint, batterie épuisée, service perdu — est une **terminaison** (*termination*). Une limite externe, par exemple un scénario arrêté après 1 000 pas, est une **troncature** (*truncation*). Cette distinction change l'estimation de la valeur future : à une troncature, le monde n'est pas nécessairement terminal.

Approfondissement — Relier une action présente à ses effets futurs

La récompense immédiate ne suffit pas. Le **retour** (*return*) depuis le pas t additionne les récompenses futures avec un facteur d'actualisation γ, le **discount factor** :

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots$$

Avec $\gamma = 0$, seul le prochain résultat compte. Lorsque γ se rapproche de 1, des effets plus lointains influencent davantage la décision. Mais $\gamma = 0{,}99$ n'a pas le même sens physique pour un pas de 1 ms et un pas de 10 s. L'horizon, la durée du pas, les échéances de service et la terminaison doivent être interprétés ensemble.



Attribution temporelle du crédit sur trois pas : une récompense immédiate positive vers le relais A conduit à un retour actualisé négatif après le débordement de sa file

Trois fonctions organisent ce raisonnement pour une politique π :

- $V^\pi(s)$, la **valeur d'état** (*state value*), estime le retour attendu depuis l'état s en suivant ensuite π ;
- $Q^\pi(s,a)$, la **valeur d'action** (*action value*), estime le retour attendu si l'on choisit d'abord a, puis suit π ;
- $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$, l'**avantage** (*advantage*), mesure si l'action est meilleure ou pire que ce que la politique obtenait en moyenne dans cet état.

Attribuer une perte retardée aux décisions qui l'ont préparée est le problème d'**attribution temporelle du crédit** (*temporal credit assignment*).

Bellman : décomposer le futur en un pas

Le retour se décompose en deux termes :

$$G_t = r_{t+1} + \gamma G_{t+1}$$

La prochaine récompense est suivie du reste du parcours actualisé. Pour une politique π fixée, l'**équation de Bellman** exprime cette même idée en moyenne :

$$V^{\pi}(s) = E_{\pi}\left[r_{t+1} + \gamma V^{\pi}(S_{t+1}) / S_t = s\right].$$

On lit : « la valeur de l'état s est la moyenne de la prochaine récompense plus la valeur actualisée de l'état suivant ». E désigne cette moyenne ; l'indice π précise que les actions suivent la politique ; la barre $/\, S_t = s$ signifie que l'on part de l'état s. La moyenne tient compte des actions choisies et de l'incertitude des transitions et récompenses.

Pour rechercher la meilleure politique, l'**équation d'optimalité de Bellman** utilise la meilleure valeur d'action disponible au pas suivant :

$$Q^{*}(s,a) = E\left[r_{t+1} + \gamma \max_{a' \in A(S_{t+1})} Q^{*}(S_{t+1}, a') / S_t = s, A_t = a\right].$$

L'astérisque « * » signifie « valeur optimale » ; $A_t = a$ fixe la première action ; a' désigne une action du pas suivant ; A est l'ensemble des actions autorisées. max retient la plus grande valeur. L'environnement reste aléatoire : maximiser ne supprime pas la moyenne. À la terminaison, la valeur future vaut zéro. On suppose ici des récompenses bornées et $0 \leq \gamma < 1$, afin que le retour infini reste fini.

La cible du DQN applique cette décomposition à une transition observée, avec un réseau qui estime Q^{*}. Cette cible échantillonnée n'est pas la valeur optimale déjà connue. Pour approfondir : Sutton et Barto, chapitre 3 (Sutton et Barto 2018).

Exemple guidé — Deux relais, trois pas

Considérons un mini-simulateur déterministe. L'état contient les deux files ; l'observation les révèle sans retard ; l'action envoie le prochain lot vers A ou B. La récompense combine service livré et pertes. Après ce premier choix, les deux pas suivants suivent une politique fixée π, puis l'épisode se termine. Pour $\gamma = 0,9$, les deux trajectoires sont :

Action initiale	r_{t+1}	r_{t+2}	r_{t+3}	Retour G_t
relais A, rapide mais presque plein	$+1$	$+0,5$	-4	$-1,79$
relais B, plus lent mais disponible	$+0,2$	$+0,8$	$+1$	$+1,73$

Les calculs complets, vérifiables terme par terme, sont :

$$G_t(A) = 1 + 0,9 \times 0,5 + 0,9^2 \times (-4) = -1,79$$

$$G_t(B) = 0,2 + 0,9 \times 0,8 + 0,9^2 \times 1 = 1,73$$

Avec B, les deux derniers pas valent $0,8 + 0,9 \times 1 = 1,7$. Bellman permet donc de relire le même calcul : $0,2 + 0,9 \times 1,7 = 1,73$. On remonte le parcours d'un pas à l'autre, sans oublier les conséquences lointaines.

Dans ce monde déterministe, $Q^{\pi}(s,A)=-1,79$ et $Q^{\pi}(s,B)=1,73$. Si la politique choisit initialement A et B avec la même probabilité, sa valeur vaut $V^{\pi}(s)=(-1,79+1,73)/2=-0,03$. Les avantages sont donc $A^{\pi}(s,A)=-1,76$ et $A^{\pi}(s,B)=+1,76$: l'apprentissage doit diminuer la probabilité de A et augmenter celle de B.

Attendre la fin de tous les épisodes donne une cible complète mais souvent très variable. Le **bootstrap** utilise une estimation actuelle de la valeur future avant la fin. Une erreur de différence temporelle (*temporal-difference error*, erreur TD) simple est :

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Elle compare ce qui était attendu à une récompense plus une nouvelle estimation. Dans le mini-simulateur, supposons qu'avant la mise à jour le critique prévoie $V(s_t)=0,5$ et, après l'action A, $V(s_{t+1})=-2$. Alors $\delta_t = 1+0,9\times(-2)-0,5=-1,3$. L'erreur est négative malgré la récompense immédiate $+1$, car la nouvelle estimation révèle la dégradation future.

La **GAE** (*Generalized Advantage Estimation*) combine plusieurs de ces erreurs avec un paramètre λ afin d'arbitrer biais et variance. Ce calcul n'ajoute aucune information au simulateur ; il rend l'apprentissage de l'avantage plus exploitable.

Dans une méthode **acteur–critique** (*actor–critic*) :

```
acteur   : politique qui choisit ou distribue les actions
critique : estimateur de V ou Q qui aide à juger ces actions
```

Le critique facilite l'entraînement. Lors de l'inférence, c'est normalement l'acteur qui produit l'action ; le critique n'est pas un contrôleur de sûreté.

Explorer sans confondre apprentissage et autorisation

Si la politique choisit toujours l'action déjà jugée meilleure, elle **exploite** son savoir mais peut ignorer une meilleure possibilité. Si elle essaie une action incertaine, elle **explore** et collecte de l'information au risque d'un résultat inférieur. Ce compromis **exploration–exploitation** est central.

Une politique stochastique associe des probabilités aux actions. Un bonus d'**entropie** peut éviter qu'elle ne se fige trop tôt. Pour des actions continues, on ajoute souvent un bruit structuré ou l'on échantillonne une distribution paramétrée. Cette diversité d'entraînement n'est pas une permission de tester n'importe quoi sur le réseau vivant.

```
simulateur : exploration large dans une enveloppe physique
laboratoire: actions autorisées et arrêt d'urgence
mode d'observation sans action : décisions calculées mais non appliquées
production : politique figée ou adaptation bornée, surveillance et repli
```

Les actions impossibles ou interdites sont retirées par un **masque d'actions** (*action mask*) ou un filtre indépendant. Puissance, spectre, isolement, débit minimal et règles de sécurité sont des contraintes, pas de simples récompenses négatives. Le mécanisme de repli doit fonctionner sans le modèle appris.

Trois mécanismes suffisent pour construire une première carte :

```
méthode de valeur     : estimer quelles actions auront le meilleur retour
gradient de politique : ajuster directement la probabilité des actions
acteur-critique       : combiner une politique et un estimateur de valeur
```

PPO appartient à la troisième famille. Les autres noms ci-dessous précisent comment les données sont réutilisées, si un modèle du monde existe, comment le risque est représenté ou combien d'agents décident.

Pour voir ces notions dans une chaîne exécutable, lire ensuite « Transformer le routage en environnement Gymnasium », puis « PPO, GNN, génétique et ILP ».

Approfondissement — Une carte des principales familles de RL

Les familles suivantes répondent à des questions différentes et peuvent se combiner :

Famille	Idée	Quand elle apparaît
Q-learning / DQN (*Deep Q-Network*)	apprendre $Q(s, a)$, puis choisir une action de grande valeur	espace d'actions discret et dénombrable
gradient de politique / REINFORCE	ajuster directement la probabilité des actions selon le retour	politique stochastique, action structurée
acteur–critique / A2C (*Advantage Actor-Critic*) / PPO	entraîner ensemble une politique et un estimateur de valeur	actions discrètes ou continues
SAC (*Soft Actor-Critic*) / TD3 (*Twin Delayed Deep Deterministic Policy Gradient*)	acteur–critique off-policy avec réutilisation d'expériences	contrôle continu et interactions coûteuses
RL avec modèle	connaître ou apprendre les transitions, puis simuler ou planifier	modèle exploitable, contrôle prédictif
RL hors ligne / CQL (*Conservative Q-Learning*) / IQL (*Implicit Q-Learning*)	apprendre depuis un journal fixe en limitant l'extrapolation	nouvelle exploration interdite
RL hiérarchique	choisir une intention lente puis des actions rapides	orchestration et contrôle à plusieurs échelles
RL distributionnel ou sensible au risque	modéliser la distribution du retour ou ses mauvais quantiles	pertes rares et queues de distribution critiques
MARL	plusieurs agents apprennent ou agissent ensemble	contrôle distribué et information locale
RL contraint / *safe RL*	optimiser avec coûts, boucliers de sûreté et supervision	actions susceptibles de violer une limite

Cette table n'est pas un classement. DQN est un exemple de méthode de valeur ; PPO, SAC et TD3 sont des acteurs–critiques ayant des compromis différents. « Profond » signifie que des réseaux neuronaux approximent politique ou valeurs ; cela ne définit pas un nouveau problème décisionnel.

Deux axes traversent cette carte :

- **RL sans modèle** (*model-free*) : comme PPO, apprend politique ou valeur sans apprendre explicitement les transitions ;
- **RL avec modèle** (*model-based*) : connaît ou apprend un modèle des transitions, puis l'utilise pour prévoir, simuler ou planifier.

Et, séparément :

- **Méthode on-policy** : apprend surtout depuis des trajectoires produites par sa politique courante ou très récente ; PPO est principalement on-policy ;
- **Méthode off-policy** : peut réutiliser des trajectoires issues d'autres politiques, souvent dans un tampon de rejeu (*replay buffer*).

Approfondissement — Lire une mise à jour DQN minimale

Le laboratoire L21 utilise une observation normalisée $x \in \mathbb{R}^7$, c'est-à-dire une liste de sept nombres réels : les deux longueurs de file, l'arrivée courante, les deux capacités de service et les deux indicateurs de disponibilité. Son réseau q_θ enchaîne une couche linéaire de 7 à 32 neurones, une activation **ReLU**, qui conserve les valeurs positives et remplace les valeurs négatives par zéro, une seconde couche de 32 neurones avec ReLU, puis deux sorties. La sortie $q_\theta(x)_a$ estime la valeur de l'action a. Il n'y a pas de *softmax* : les deux nombres ne sont ni des probabilités ni tenus de sommer à un.

Pendant la collecte, une politique ε-greedy choisit une action aléatoire autorisée avec probabilité ε, sinon l'action autorisée de plus grande valeur. ε décroît progressivement afin de passer de l'exploration à l'exploitation. Chaque transition $(x_i, a_i, r_i, x_i', t_i)$ est placée dans le tampon de rejeu ; $t_i = 1$ signifie ici une vraie terminaison, pas une troncature temporelle. Échantillonner un mini-lot casse une partie de la corrélation entre pas successifs et permet de réutiliser les observations.

Pour chaque transition du mini-lot, `gather` sélectionne dans les deux sorties du réseau en ligne la valeur de l'action réellement appliquée :

$$q_i = q_\theta(x_i)_{a_i}.$$

La cible de Bellman est calculée par un second réseau, de paramètres $\hat{\theta}$, sans propager de gradient à travers ce calcul :

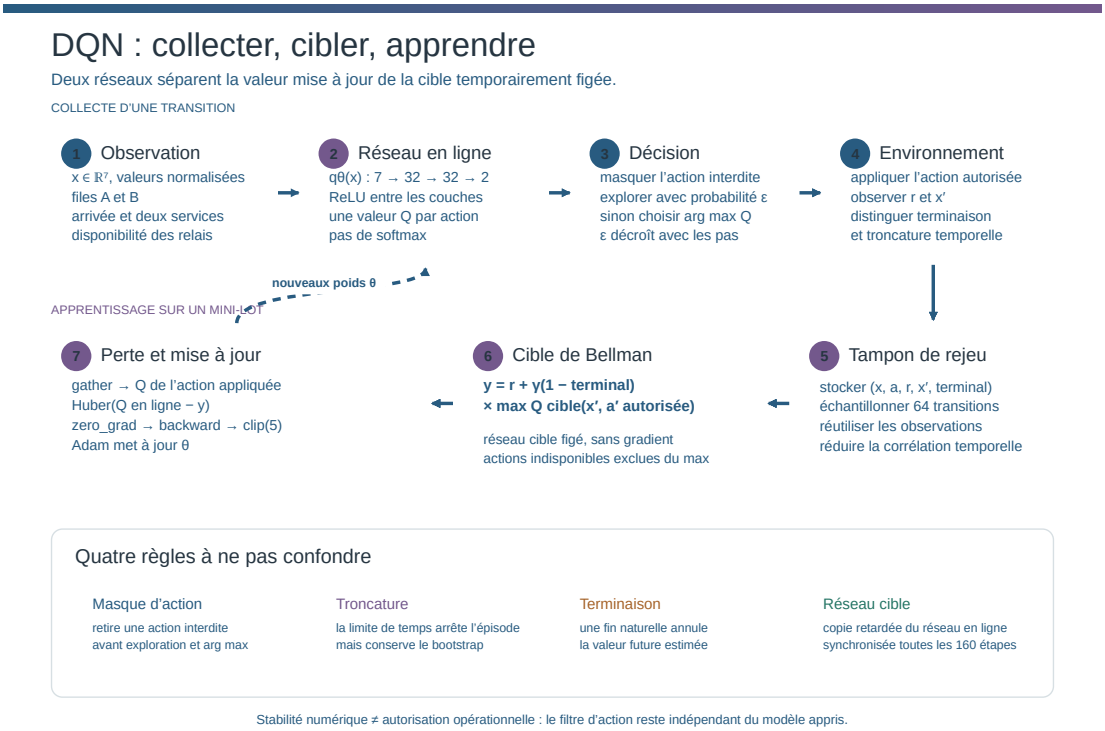
$$y_i = r_i + \gamma(1 - t_i) \max_{a' \in A(x_i')} q_{\hat{\theta}}(x_i')_{a'}.$$

L'ensemble $A(x_i')$ contient seulement les actions autorisées par le masque du prochain état. Le facteur $1 - t_i$ supprime le bootstrap après une terminaison naturelle. Après une simple limite de temps, $t_i = 0$ et la valeur future reste donc présente. Le réseau cible est une copie retardée du réseau en ligne : le maintenir fixe entre deux synchronisations évite que la prédiction et sa cible ne se déplacent ensemble à chaque mise à jour.

Le code minimise une perte de Huber, appelée `smooth_l1_loss` dans PyTorch, une bibliothèque de calcul pour les réseaux de neurones. Pour l'erreur $e_i = q_i - y_i$, elle est quadratique près de zéro et linéaire pour une grande erreur :

$$\ell(e_i) = \begin{cases} 1/2\, e_i^2, & |e_i| \le 1, \\ |e_i| - 1/2, & |e_i| > 1. \end{cases}$$

Pour les petites erreurs TD, son gradient varie avec l'erreur ; pour les grandes, il est borné. Dans la séquence PyTorch, `zero_grad` efface les gradients accumulés, `backward` applique la règle de chaîne, `clip_grad_norm_` borne leur norme à 5, puis l'optimiseur Adam modifie les poids du réseau en ligne. Toutes les 160 étapes, ces poids sont copiés vers le réseau cible. Ce sont des mécanismes de stabilisation de l'apprentissage, pas des garanties de sûreté opérationnelle.



Boucle DQN complète : collecte d'une transition, rejeu, cible de Bellman, perte de Huber et distinction entre masque d'action, troncature et terminaison

RL hors ligne (*offline RL*) ajoute une contrainte différente : le journal de trajectoires est figé et aucune nouvelle interaction ne corrige une action mal couverte. Une politique peut attribuer une valeur fictive à une route jamais tentée. Les méthodes conservatrices cherchent à limiter cette extrapolation, sans inventer la conséquence manquante.

Ces termes ne sont pas synonymes :

```
batch       : les paramètres sont mis à jour par lots
off-policy  : les données peuvent venir d'une autre politique
hors ligne  : aucune nouvelle interaction n'enrichit le journal
```

Un entraînement batch peut collecter de nouveaux épisodes entre deux mises à jour. Une méthode off-policy peut continuer à interagir. Une politique hors ligne peut être déployée ensuite pour une inférence en temps réel.

RL interactif en simulateur ne signifie pas apprentissage sur le réseau vivant : l'agent peut agir en ligne vis-à-vis de son environnement simulé, puis être figé avant déploiement. À l'inverse, un **RL en ligne en production** continue à collecter des conséquences et à modifier la politique pendant l'exploitation. L'exploration devient alors un acte opérationnel nécessitant une **enveloppe de contraintes impératives** — l'ensemble des actions toujours autorisées —, une limitation du rythme, un déploiement canari (*canary*), une surveillance et un retour arrière.

RL contraint et *safe RL*. Une limite réglementaire, une séparation de trafic ou une latence inviolable doit être appliquée comme contrainte, filtre d'action ou mécanisme indépendant. Une pénalité dans la récompense peut être acceptée par l'agent si le gain estimé paraît supérieur. Le *safe RL* étudie aussi robustesse, vérification et **bouclier de sûreté** (*safety shield*), un filtre indépendant qui bloque ou remplace une action interdite, ainsi que surveillance externe et reprise. Aucun algorithme ne rend seul la boucle déployée sûre.

RL multi-agent, ou **MARL**. Plusieurs agents rendent l'environnement non stationnaire les uns pour les autres. L'entraînement peut être centralisé avec une vue globale, puis l'exécution décentralisée avec des observations locales. Il faut attribuer le crédit collectif, coordonner les actions et les communications, traiter des politiques hétérogènes et tester la disparition ou le comportement fautif d'un agent.

Quand ne pas employer le RL ?

Si chaque décision a un effet presque immédiat, un bandit peut suffire. Si une cible fiable existe, une prédiction supervisée suivie d'un solveur est souvent plus simple. Si le modèle physique et les contraintes sont connus, contrôle prédictif, théorie des files, programmation mathématique ou heuristique peuvent offrir davantage d'interprétabilité et de garanties.

Avant l'entraînement, le contrat de l'expérience doit préciser :

1. ce qui constitue l'état réel et ce que l'agent observe réellement ;
2. l'action brute, sa transformation et l'action finalement appliquée ;
3. la durée du pas, l'horizon, la terminaison et la troncature ;
4. chaque terme de récompense, son unité, son échelle et les coûts séparés ;
5. les contraintes indépendantes et la politique de repli ;
6. la méthode de référence non apprise qui résout déjà une partie du problème.

Sans ce contrat, PPO ne corrige pas une mauvaise formulation : il l'optimise.

Auto-évaluation — Formuler la boucle

1. Une prévision donne le prochain délai de chaque relais. Quelle condition supplémentaire peut justifier le RL ?
2. La file distante existe dans le simulateur, mais arrive dans la télémétrie avec deux pas de retard. Relève-t-elle de l'état ou de l'observation ?
3. Une violation de SLA reçoit une pénalité de -100. Est-elle devenue impossible ?
4. Dans l'exemple à trois pas, pourquoi A est-il mauvais malgré $r_{t+1} = +1$?
5. Quelle référence faut-il construire avant d'entraîner PPO ?

Réponses brèves. 1. Les actions doivent modifier assez les situations et choix futurs pour que les conséquences retardées comptent. 2. Elle appartient à l'état réel, mais pas à l'observation courante de l'agent. 3. Non : une contrainte inviolable exige un masque, un filtre ou un mécanisme indépendant. 4. Son retour actualisé vaut $-1,79$ à cause du débordement futur. 5. Une règle, une heuristique, un protocole ou un solveur non appris, évalué sur les mêmes scénarios et métriques.

> **Passer des formules à l'expérience.** Le laboratoire L20 fait calculer retour, Q, avantage et erreur TD sur le choix de deux relais. Le laboratoire L21 entraîne ensuite un DQN minimal et le compare à deux références. Le laboratoire L22 transpose enfin la méthode à la surface opérateur PPO-GNN/ILP, avec contrat de preuve et scénarios tenus à l'écart.
>
> Le laboratoire L24 revient au banc minimal pour vérifier si une pénalité de perte peut corriger un compromis défavorable sans masquer un résultat négatif. Le laboratoire L25 gèle ensuite le candidat retenu et mesure sa robustesse sous décalage de distribution. Le laboratoire L26 teste enfin un ensemble qui se replie sur l'heuristique lorsque ses cinq modèles ne s'accordent pas. Le laboratoire L27 ajoute un moniteur de support observable, calibré uniquement sur le régime nominal.

Axe 3 — Quand le modèle change-t-il ?

Régime	Ce qui change	Intérêt réseau	Risque
mise à jour par lots (*batch*)	paramètres modifiés après une collecte finie, avec possibilité de recollecter ensuite	calcul stable, audit par version	modèle inchangé entre deux lots
RL hors ligne (*offline RL*)	apprentissage sur un journal définitivement figé, sans nouvelle interaction	pas d'exploration du réseau vivant	couverture et extrapolation hors support
en flux	données arrivent continuellement, modèle pas forcément modifié	détection et inférence temps réel	confondre flux d'entrée et apprentissage

Régime	Ce qui change	Intérêt réseau	Risque
apprentissage en ligne	paramètres mis à jour au fil des observations	adaptation rapide	contamination et instabilité
apprentissage continu (*continual learning*)	nouvelles tâches ou distributions sans oublier les anciennes	nouveaux sites, firmwares ou bandes	oubli catastrophique (*catastrophic forgetting*)
adaptation au moment du test	statistiques ou paramètres adaptés pendant l'inférence, souvent sans labels	suivre un décalage local récent	dérive, effondrement ou adaptation à une attaque

Inférence en ligne ne signifie donc pas **apprentissage en ligne**. Un modèle peut décider toutes les millisecondes tout en conservant des poids figés. L'**apprentissage continu** ajoute une autre exigence : intégrer le nouveau sans détruire les compétences validées sur l'ancien. Il faut rejouer des cas historiques, conserver des modèles de référence et tester explicitement l'oubli.

L'**adaptation au moment du test**, ou *test-time adaptation*, utilise les données rencontrées pendant l'inférence pour ajuster statistiques, représentations ou paramètres. Elle peut corriger un décalage de domaine sans labels immédiats, mais aussi s'effondrer sous un flux corrélé, apprendre une perturbation hostile ou oublier le domaine validé. Elle exige détection, limites d'amplitude, état réinitialisable et comparaison à un modèle figé.

Axe 4 — Réutiliser et adapter un modèle

- **Préentraînement puis ajustement fin** (*fine-tuning*) : apprendre une représentation sur un grand corpus, puis ajuster tout ou partie des poids pour une tâche réseau.
- **Adaptation économe en paramètres** (*parameter-efficient fine-tuning*, **PEFT**) : des adaptateurs ou **LoRA** apprennent une petite modification tandis que l'essentiel du modèle reste figé. Cela réduit mémoire et volume à distribuer, pas le besoin de validation.
- **Apprentissage par transfert** (*transfer learning*) : réutiliser des représentations ou poids d'une source vers une cible liée.
- **Apprentissage multitâche** (*multi-task learning*) : partager une représentation entre plusieurs objectifs, par exemple prévoir charge et perte ; une tâche dominante peut toutefois dégrader les autres.
- **Adaptation de domaine** (*domain adaptation*) : utiliser des données du domaine cible pendant l'adaptation ; la **généralisation de domaine** (*domain generalization*) vise au contraire un domaine nouveau non vu pendant cette adaptation.
- **Méta-apprentissage** (*meta-learning*) : apprendre une procédure qui s'adapte avec peu d'exemples. **MAML** (*Model-Agnostic Meta-Learning*) cherche notamment une initialisation des paramètres depuis laquelle quelques mises à jour par descente de gradient suffisent sur une nouvelle tâche. En réseau, on peut entraîner sur plusieurs sites, topologies ou régimes radio, puis adapter le modèle à un nouveau site avec peu de traces. Son optimisation imbriquée, dite bi-niveau, est coûteuse ; la qualité dépend de la distribution des tâches d'entraînement et une adaptation rapide peut aussi apprendre très vite un bruit, une attaque ou une politique dangereuse.

  MAML n'est donc ni une politique de repli, ni une garantie de sûreté, ni automatiquement de l'apprentissage en ligne. « Few-shot » décrit la faible quantité d'exemples d'adaptation ; « zero-shot » décrit l'absence d'exemple spécifique, pas une garantie de correction.
- **Distillation de connaissances** (*knowledge distillation*) : entraîner un petit modèle à reproduire un modèle plus coûteux. Elle peut réduire délai et énergie en périphérie du réseau, tout en transmettant certains biais du professeur.

Le **prompting**, la recherche documentaire augmentée (*RAG*) ou l'appel d'outils modifient le contexte d'un modèle de fondation sans nécessairement modifier ses poids. Ce sont des mécanismes d'usage ou d'ancrage, pas à eux seuls un nouvel entraînement.

Axe 5 — Où résident les données et le calcul ?

L'**apprentissage fédéré** (*federated learning*) envoie le calcul vers plusieurs participants puis agrège des mises à jour, au lieu de centraliser systématiquement les données brutes. Il peut répondre à une contrainte de souveraineté ou de volume, mais les clients possèdent souvent des données non identiquement distribuées, des capacités différentes et des disponibilités intermittentes. Les mises à jour peuvent aussi divulguer de l'information ou être empoisonnées : « fédéré » ne signifie ni privé, ni robuste, ni digne de confiance.

Un entraînement **distribué** répartit le calcul d'un même apprentissage. Un entraînement **décentralisé** évite un agrégateur unique et échange avec des pairs. Un entraînement **fédéré** organise des participants conservant leurs données et souvent leur autonomie. Ces ensembles se recouvrent, mais ne sont pas synonymes.

L'**apprentissage fractionné** (*split learning*) place des morceaux d'un modèle de part et d'autre d'une liaison et échange des activations ou gradients. Il déplace charge, latence et surface de fuite ; il ne supprime pas la dépendance réseau. Centralisé, fédéré, fractionné et partagé entre périphérie et nuage décrivent ainsi la **topologie de l'entraînement**, pas la source du signal : chacun peut être supervisé ou auto-supervisé.

Axe 6 — Quelle autorité la sortie reçoit-elle ?

Le même modèle peut avoir des risques très différents selon qu'il :

1. **observe** et enrichit une télémétrie ;
2. **prévoit** une charge ou une qualité future ;
3. **recommande** une action à un opérateur ;
4. **planifie** une suite d'actions encore soumise à validation ;
5. **agit** dans une enveloppe bornée ;
6. **réconcilie** continuellement un état de production.

Il faut associer à chaque niveau une échelle de temps, une portée, une autorité, des contraintes impératives, une preuve attendue et un repli indépendant. Une précision de 99 % peut suffire pour classer des tickets et rester inacceptable pour modifier automatiquement 10 000 équipements. La question « quel mode d'apprentissage ? » reste incomplète sans « quel droit d'agir ? ».

Modèles génératifs et modèles de fondation

Un modèle **discriminatif** estime typiquement une classe, une valeur ou une décision. Un modèle **génératif** apprend une distribution lui permettant de produire texte, configuration, voix, image, topologie ou scénario. Cette distinction porte sur l'objectif et la sortie, pas sur le niveau de supervision.

Un **modèle de fondation** — LLM, modèle multimodal ou futur modèle de fondation pour graphes — est préentraîné sur des données nombreuses puis adapté ou conditionné pour plusieurs tâches. Dans un réseau, il peut résumer des alarmes, aider à interroger la télémétrie, proposer une configuration ou générer des scénarios de test. Sa réponse plausible n'est ni une mesure, ni une configuration valide, ni une autorisation d'agir. Schéma, contraintes, simulation, vérification et approbation restent séparés.

Le même problème de routage vu par six configurations

Supposons qu'il faille répartir des demandes sur un graphe mobile :

1. **supervisé** : un MILP calcule des allocations sur des scénarios connus ; le modèle apprend à les approximer ;
2. **auto-supervisé** : le GNN reconstruit des liens ou métriques masqués pour préentraîner ses embeddings ;
3. **imitation** : la politique reproduit les décisions d'un opérateur ou d'un algorithme classique ;
4. **bandit contextuel** : elle choisit un lien pour un gain court, seulement si cette action n'altère pas significativement le futur ;
5. **RL hors ligne** : elle apprend sur des trajectoires enregistrées, sous la limite des actions couvertes par ce journal ;

6. **RL interactif on-policy dans un simulateur** : elle explore des conséquences longues sans toucher la production, puis affronte le décalage simulation-réalité.

Le simulateur ou **jumeau numérique** (*digital twin*) est l'**environnement**, pas un mode d'apprentissage. Le PPO du projet y collecte des rollouts principalement on-policy ; l'inférence exportée peut rester figée. Un agent par nœud ajoute la dimension multi-agent. Adapter les poids en production exige de distinguer réentraînement batch périodique, apprentissage en ligne, apprentissage continu et adaptation au moment du test. Garder les données sur les sites peut permettre un entraînement fédéré. Ces étiquettes sont complémentaires.

Choisir le mode le plus simple qui répond au problème

Besoin	Premier candidat	Pourquoi
prévoir charge ou qualité avec historiques labellisés	supervisé	cible et erreur directement mesurables
apprendre une représentation depuis beaucoup de télémétrie brute	auto-supervisé	labels produits à partir des données
découvrir des profils inconnus	non supervisé	exploration de structure
reproduire un solveur fiable en temps borné	imitation supervisée	référence explicite et comparable
arbitrer une action à effet surtout immédiat	bandit contextuel	complexité séquentielle limitée
optimiser des conséquences retardées	RL	état futur affecté par l'action
éviter l'exploration du réseau vivant	imitation ou RL hors ligne	apprentissage sur preuves enregistrées
adapter plusieurs sites sans centraliser le brut	fédéré, si justifié	contrainte de localisation des données

Avant un modèle complexe, comparer une règle, un ordonnanceur classique, un solveur ou une politique heuristique. Un problème bien spécifié peut relever de l'optimisation, du contrôle ou des files d'attente sans nécessiter d'apprentissage.

Questions à examiner avec l'expert réseau

1. Le signal optimisé correspond-il au **service utile**, ou seulement à un proxy facile à mesurer ?
2. Les décisions ont-elles un effet retardé qui invalide un bandit ou une classification statique ?
3. Quelles actions et topologies sont absentes des données hors ligne ?
4. Qu'est-ce qui change réellement en production : entrées, mémoire, paramètres, règles ou seulement contexte de la requête ?
5. Quelle contrainte est dure, quel objectif est négociable et qui possède l'autorité finale ?
6. Comment sont testés dérive, panne corrélée, observation partielle, adversaire, oubli et transfert vers un autre équipement ?
7. Quelle politique sûre fonctionne si modèle, agrégateur fédéré ou jumeau numérique devient indisponible ?

Une récompense élevée n'est pas automatiquement un service sûr. Un modèle préentraîné n'est pas automatiquement transférable. Une mise à jour rapide n'est pas automatiquement une adaptation saine.

Cette lecture par cycle de vie et responsabilités est cohérente avec l'architecture ML pour les réseaux de l'UIT-T Y.3172 et avec l'architecture ETSI ENI, sans présumer qu'un cadre normalisé impose un algorithme particulier.

Du texte aux vecteurs : corpus, comptage et TF-IDF

Un journal d'événements contient des mots, des codes et des identifiants. Pour retrouver des incidents semblables, il faut choisir une représentation numérique de ces observations. Un **document** est ici une unité de texte délimitée : une ligne de journal, un ticket ou un passage de procédure. Un **corpus** est l'ensemble des documents étudiés. Ce choix de découpage compte : une ligne isolée peut perdre l'explication donnée par la ligne précédente.

Construire un vocabulaire et une matrice

Prenons trois messages fictifs, volontairement simplifiés :

```
A : lien radio perdu
B : lien radio rétabli
C : route IP perdue
```

Pour cet exemple seulement, mettons les lettres en minuscules et ramenons « perdue » à « perdu ». Le **vocabulaire** est la liste ordonnée des termes retenus. Chaque document devient un **vecteur**, c'est-à-dire une liste de nombres dont chaque position correspond toujours au même terme :

Document	lien	radio	perdu	rétabli	route	ip
A	1	1	1	0	0	0
B	1	1	0	1	0	0
C	0	0	1	0	1	1

Le tableau est une **matrice documents-termes** de trois lignes et six colonnes : les lignes sont les observations, les colonnes sont les variables d'entrée. La dimension d'un vecteur est ici six, même si le document n'a que trois mots. Dans un grand vocabulaire, la plupart des cases sont nulles ; une représentation **creuse** stocke surtout les valeurs non nulles et leurs positions. Le **sac de mots** conserve ainsi les occurrences, mais oublie l'ordre des mots (scikit-learn developers 2026g).

Pour comprendre ce tableau et les opérations qu'on peut lui appliquer : les matrices avec ParaMaths.

Sur de vrais journaux, les choix de préparation doivent être explicites : conserver les codes d'erreur et les négations, décider du traitement des adresses et horodatages, documenter les mots exclus. Supprimer « non » peut transformer « lien non disponible » en « lien disponible ». Une normalisation utile pour rechercher des documents peut donc détruire une distinction opérationnelle essentielle.

Pondérer les termes avec le TF-IDF

Un terme fréquent dans un document peut le caractériser ; un terme présent partout distingue peu les documents. Le **TF-IDF** combine la fréquence du terme (*term frequency*, TF) et sa rareté dans le corpus (*inverse document frequency*, IDF). Fixons une convention simple, sans lissage ni normalisation :

$$w(t,d) = c(t,d)\ln\left(\frac{N}{df(t)}\right).$$

Ici, $c(t,d)$ compte les occurrences du terme t dans le document d, N est le nombre total de documents et $df(t)$ le nombre de documents contenant ce terme au moins une fois. Le symbole $\ln$ désigne le logarithme naturel. Nous ne calculons cette expression que pour les termes du vocabulaire, donc avec $df(t) > 0$.

Dans notre corpus, $N = 3$. « radio » apparaît dans deux documents : son IDF vaut $\ln(3/2)$, soit environ 0,405. « rétabli » n'apparaît que dans un document : son IDF vaut $\ln(3)$, soit environ 1,099. Le vecteur de B devient, dans le même ordre de colonnes :

```
B : [0,405 ; 0,405 ; 0 ; 1,099 ; 0 ; 0]
```

Un mot présent dans tous les documents aurait ici un poids nul. Les logiciels emploient aussi des variantes : lissage pour éviter certaines valeurs extrêmes, TF logarithmique, puis normalisation de la longueur du vecteur. Il faut annoncer la convention avant de comparer les nombres. La documentation de scikit-learn détaille ces variantes (scikit-learn developers 2026g).

Le vocabulaire et les poids IDF sont **ajustés sur le corpus de référence**. Pour évaluer un classifieur, cet ajustement se fait sur la partie entraînement, sans utiliser les futurs documents de test. Pour un moteur de recherche, l'index décrit les

documents disponibles à une date donnée ; une requête est transformée dans ce même espace. Un terme inconnu n'a pas de colonne dans ce vocabulaire figé.

Comparer des textes sans confondre ressemblance et diagnostic

La **similarité cosinus** compare l'orientation de deux vecteurs non nuls : elle additionne les produits des coordonnées correspondantes, puis divise par le produit des longueurs des deux vecteurs. Pour nos comptages, A et B ont deux termes communs et une similarité de 2/3 ; A et C n'en ont qu'un et une similarité de 1/3. Cette mesure retrouve surtout les mêmes termes. Le score doit être défini séparément pour un vecteur nul.

Or A décrit une perte et B un rétablissement. Leur proximité lexicale ne prouve donc pas un état réseau identique. Le TF-IDF ne connaît pas spontanément les synonymes, les causes des incidents ni la portée d'une négation. Des groupes de mots consécutifs, appelés **n-grammes**, peuvent préserver quelques relations locales ; ils ne remplacent pas la vérification du message complet.

Exercice de contrôle. Ajouter « lien radio non rétabli » au corpus. Quelles colonnes et quels poids changent si l'on reconstruit le vocabulaire ? Que perd-on si « non » est supprimé ? La réponse attendue distingue le recalcul des statistiques du corpus de l'interprétation de l'état du lien.

Embedding : représenter un objet par des coordonnées apprises

Un modèle ne manipule directement ni un mot, ni un routeur, ni un graphe. Il manipule des nombres. Un **embedding**, que l'on peut traduire par **représentation vectorielle apprise**, est un vecteur latent produit à partir des variables observables d'un objet :

```
objet réel ou symbolique
  -> variables observables explicites
  -> encodeur appris
  -> embedding latent [0,18 ; -0,72 ; 0,05 ; ...]
  -> calcul du modèle
```

Les variables explicites peuvent être la capacité, le délai ou la charge. Chaque coordonnée de l'embedding, elle, n'a généralement pas une signification humaine isolée. C'est leur combinaison qui place les objets dans un espace où le modèle peut exploiter proximités, différences et relations. Deux objets utiles de manière similaire pour la tâche peuvent y recevoir des représentations proches, sans être identiques dans le monde réel.

Exemple avec un grand modèle de langage (LLM) de type Transformer. Le texte est d'abord découpé en unités appelées **jetons** (*tokens*). À chaque identifiant de jeton correspond un vecteur initial appris. Les couches d'attention combinent ensuite ce vecteur avec le contexte. Ainsi, « port » dans « port Ethernet » et dans « port maritime » part d'une unité écrite semblable, mais reçoit progressivement une représentation contextuelle différente. Cette représentation aide notamment le modèle à estimer la suite du texte ; elle n'est ni une définition de dictionnaire ni la preuve que le modèle comprend le port comme un humain.

```
LLM : tokens -> vecteurs initiaux -> contexte entre tokens
              -> représentations contextualisées -> prédiction
```

Exemple avec un réseau neuronal de graphe (GNN). Les objets sont cette fois des nœuds et des arêtes. Un nœud peut porter une position ou une charge ; une arête, une capacité, un délai ou un état actif. À chaque couche, le GNN combine les variables d'un élément avec des messages provenant de son voisinage. Il peut produire :

- **Embedding de nœud** : par exemple pour résumer un relais et ses voisins ;
- **Embedding d'arête** : par exemple pour noter une liaison dans son contexte ;
- **Embedding de graphe** : par exemple pour classer une topologie complète.

```
GNN : nœuds + arêtes -> échanges avec les voisins
                     -> embeddings contextualisés -> prédiction ou action
```

Deux relais peuvent avoir la même capacité propre. Si le premier est entouré de liaisons congestionnées et le second de liaisons disponibles, leurs embeddings de nœud peuvent devenir différents après échange avec les voisins. Le contexte appris complète les variables locales ; il ne crée pas une mesure qui n'était présente nulle part dans l'observation.

La ressemblance avec le LLM est conceptuelle : dans les deux cas, une représentation initiale est transformée par le contexte. Le contexte d'un Transformer de langage vient principalement des autres tokens ; celui d'un GNN vient des voisins reliés par le graphe. Le GNN reçoit explicitement cette structure ; le Transformer reçoit une séquence et construit des interactions par attention.

Un embedding est une **représentation orientée par les données, l'architecture et l'objectif d'apprentissage** : il peut compresser l'entrée, augmenter sa dimension ou réorganiser l'information. Il peut perdre une information absente, masquée, hors contexte, agrégée ou ignorée par l'objectif ; oublier une panne non observée, rapprocher des situations distinctes pour l'exploitant ou changer après réentraînement. La proximité entre vecteurs n'établit ni causalité, ni sûreté, ni route réalisable. Demander alors :

1. quels objets et quelles variables ont été encodés ;
2. quel contexte le modèle a pu agréger ;
3. pour quelle tâche la représentation a été apprise ;
4. quelle information utile a pu être perdue ;
5. sur quels cas indépendants l'embedding a été évalué.

Quatre façons d'apprendre à partir d'un graphe

Le passage de messages est une famille de méthodes, parmi plusieurs façons de représenter un graphe :

Approche	Calcul appris	Limite à examiner
Plongements directs de nœuds, dits *shallow embeddings*	DeepWalk apprend à partir de marches aléatoires ; node2vec modifie la façon d'explorer le voisinage	un vecteur appris pour un nœud donné ne fournit pas automatiquement celui d'un nouveau nœud
Autoencodeur de graphe	un encodeur produit des vecteurs ; un décodeur reconstruit des liens ou des attributs	reconstruire des liens observés ne prouve pas leur cause ni leur disponibilité future
Réseau à passage de messages, ou MPNN	des transformations partagées combinent les informations des voisins	la portée dépend des couches ; agréger peut effacer des différences utiles
Régularisation par le graphe	une pénalité favorise la ressemblance de prédictions ou de représentations de nœuds reliés	elle suppose une similarité pertinente entre voisins, souvent appelée homophilie ; deux relais voisins peuvent pourtant avoir des rôles différents

Dans la troisième famille, un **GCN** (*Graph Convolutional Network*) combine des représentations voisines avec une normalisation liée au graphe. **GraphSAGE** (*Graph Sample and Aggregate*) apprend une fonction d'agrégation à partir d'un voisinage échantillonné ; elle peut être appliquée à de nouveaux nœuds disposant des attributs requis. Un **GAT** (*Graph Attention Network*) apprend des poids d'attention sur les voisins. Ces architectures changent le calcul de la représentation, pas les lois du réseau observé. Leurs noms ne garantissent pas une généralisation à toute nouvelle topologie (Perozzi et al. 2014; Grover et Leskovec 2016; Kipf et Welling 2017; Hamilton et al. 2017; Veličković et al. 2018).

Pour un graphe simple non orienté, un **laplacien non normalisé** s'écrit $L = D - A$: A indique les voisins et D place leur nombre sur la diagonale. Appliqué aux valeurs des nœuds, il compare chaque valeur à ses voisines ; d'autres conventions normalisent ce calcul.

Les méthodes **spectrales** décrivent certains calculs à partir des modes propres du laplacien du graphe ; les méthodes **spatiales** les expriment par des opérations sur les voisinages. Ces lectures peuvent se rejoindre, comme dans certaines formulations des GCN. La **génération de graphes** est une tâche supplémentaire : produire des nœuds et des liens plausibles ne garantit pas une topologie réalisable.

Pour approfondir, la synthèse de Hamilton sur l'apprentissage de représentations de graphes relie ces familles. Dans le livre, les expériences PPO-GNN utilisent le passage de messages décrit avec le routage ; elles ne comparent pas expérimentalement toutes les familles de cette table.

Lire un modèle de graphe : entrées, apprentissage et portée

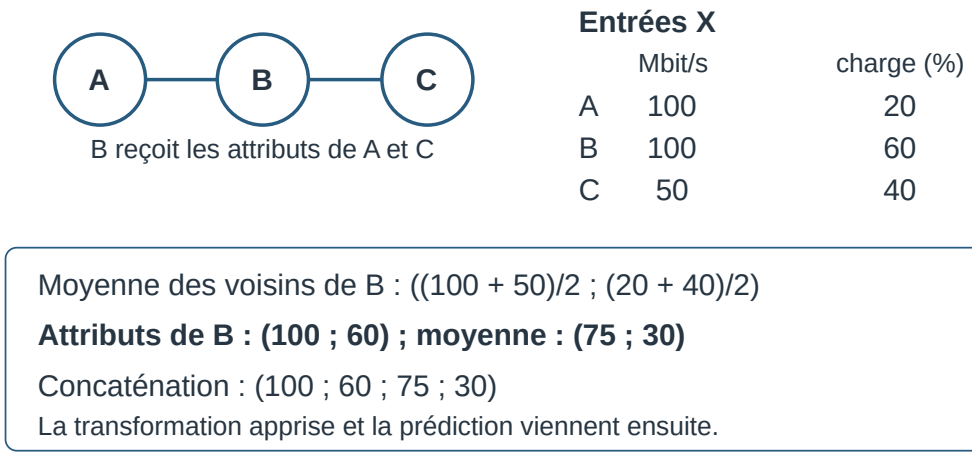
Les quatre familles précédentes décrivent des mécanismes. Pour comparer deux modèles, on peut aussi examiner **ce qu'ils reçoivent, ce qui guide l'apprentissage et quelles parties du graphe peuvent interagir**. Ces questions se croisent ; elles ne forment pas un classement où chaque méthode occuperait une seule case.

Structure et variables d'entrée. Notons N le nombre de nœuds. La **matrice d'adjacence** A, de taille $N \times N$, indique les liaisons : dans un graphe simple non orienté, $A_{ij}=1$ si les nœuds i et j sont reliés, et 0 sinon. Un graphe pondéré peut placer une valeur de liaison dans cette matrice ; les attributs détaillés des arêtes peuvent aussi être conservés séparément. La matrice X, de taille $N \times d$, rassemble les entrées des nœuds : une ligne par nœud et d colonnes par ligne.

Le choix $X=I_N$ utilise la matrice identité : chaque nœud reçoit une ligne avec un seul 1, à sa propre position. Cela code une **identité**, pas une capacité, une position ou une charge mesurée. Si le modèle apprend un vecteur libre par identifiant, il représente les nœuds connus pendant l'entraînement : cet usage est **transductif**. Un nouveau nœud n'a pas automatiquement de vecteur appris ; ajouter son identifiant ne suffit pas à le produire.

Avec des **attributs disponibles sur les nouveaux nœuds**, une fonction partagée qui les transforme et agrège leur voisinage peut au contraire produire une représentation sans apprendre une nouvelle ligne de paramètres pour chaque identité. C'est le principe **inductif** illustré par GraphSAGE. La seule notation $X \neq I_N$ ne garantit toutefois ni cette propriété ni la qualité sur une topologie nouvelle : le calcul doit être applicable, et les mêmes variables doivent rester disponibles avec une signification comparable (Hamilton et al. 2017).

Un calcul intermédiaire concret. Prenons trois relais reliés en chaîne, $A — B — C$. Dans cet exemple illustratif, chaque ligne de X contient une capacité en Mbit/s et une charge en pourcentage : $x_A=(100\,;20)$, $x_B=(100\,;60)$, $x_C=(50\,;40)$. Pour B, la moyenne des attributs des voisins A et C vaut $(75\,;30)$. La concaténer à ses propres attributs donne $(100\,;60\,;75\,;30)$, avant une transformation apprise. Ce vecteur n'est encore ni une décision ni une prédiction. Dans un modèle réel, la préparation des unités et le traitement des valeurs manquantes font partie du contrat d'entrée.



Exemple illustratif sur trois relais : attributs des nœuds, moyenne des voisins de B et concaténation avant transformation apprise

Signal d’apprentissage. Une architecture recevant des étiquettes métier, par exemple « relais congestionné », relève du supervisé. Si seuls certains nœuds sont étiquetés, tandis que la structure et les entrées des autres participent au calcul, on parle souvent de semi-supervisé. Reconstruire des liens masqués ou prédire un contexte obtenu par des marches dans le graphe utilise un signal construit à partir des données, sans étiquettes métier pour ces cibles : l’objectif relève du non supervisé ou de l’auto-supervisé, selon sa construction. L’absence de ces étiquettes n’efface ni l’objectif, ni les choix de données, ni les hypothèses (Perozzi et al. 2014; Kipf et Welling 2017).

Portée et formulation du calcul. Un passage de messages limité aux voisins immédiats fait circuler l’information d’un saut par couche. Après k couches, un nœud peut ainsi dépendre de nœuds situés à k sauts, si aucune opération ne lui apporte un contexte plus large. Un calcul global peut, lui, faire dépendre une sortie de l’ensemble du graphe. Sa portée et ses besoins en calcul ou en mémoire sont à examiner sur la taille visée.

La lecture **spectrale** définit des filtres à partir du laplacien, une matrice construite avec les liaisons et les degrés des nœuds. Ses vecteurs propres décrivent des modes de variation sur le graphe ; leur base complète peut faire intervenir des informations réparties sur tout le graphe. La lecture **spatiale** décrit le calcul par les relations entre nœuds. Ces axes ne se confondent pas avec « global » et « local » : un filtre spectral polynomial peut être limité à un voisinage. Les GCN relient une approximation spectrale à un calcul spatial utilisant une matrice d’adjacence normalisée. « Spectral » ne détermine donc à lui seul ni la portée ni le coût (Defferrard et al. 2016; Kipf et Welling 2017).

Un exemple de diffusion sur trois nœuds montre comment des échanges locaux atténuent les écarts.

Géométrie de la représentation. La structure irrégulière d’un graphe n’impose pas l’espace de ses embeddings. On utilise souvent des vecteurs euclidiens ; des espaces hyperboliques, notamment le modèle de Poincaré, ont aussi été étudiés pour représenter des hiérarchies. Géométrie et distance s’évaluent selon les relations à préserver. Une proximité dans cet espace n’est pas pour autant une distance physique ou un chemin réseau (Nickel et Kiela 2017).

Tokens, contexte et génération : trois étapes distinctes

Le sac de mots fournit un point de comparaison utile, mais un modèle de langage ne commence pas nécessairement par ce tableau TF-IDF. Son **tokenizer** découpe le texte en unités, ou **jetons** (*tokens*), selon un vocabulaire et des règles propres au modèle. Un jeton peut représenter un mot, une partie de mot ou un élément de ponctuation. Une adresse IP et un nom d’interface peuvent donc occuper plusieurs jetons ; compter les mots ne suffit pas à compter les jetons.

Un identifiant de jeton est un numéro de case, pas une quantité : deux numéros consécutifs n’ont aucune raison de désigner des sens proches. La **table d’embeddings** associe à chaque identifiant un vecteur appris. À titre illustratif, 20 jetons et des vecteurs de dimension 8 produisent un tableau de 20 lignes et 8 colonnes avant les couches contextuelles. Ces dimensions sont fictives ; elles ne décrivent aucun des modèles cités plus loin.

```
texte -> découpage en jetons -> identifiants -> vecteurs initiaux
      -> couches contextuelles -> scores sur les prochains jetons
      -> choix d’un jeton -> continuation de la séquence
```

Pour une génération autorégressive, les scores sont convertis en une distribution de probabilités sur les sorties possibles. Une règle de décodage choisit ensuite le jeton suivant. Les étapes se répètent jusqu’à une condition d’arrêt. Le contexte et l’ordre comptent ; les mécanismes d’attention sont détaillés dans la section sur les architectures neuronales (Vaswani et al. 2017). Une forte probabilité de continuation ne constitue pas une probabilité que le diagnostic réseau soit vrai.

Le **contexte** rassemble l’information fournie au modèle pour une exécution : instructions, question, extraits et résultats d’outils. Sa taille est bornée. Ajouter une procédure modifie l’entrée de l’inférence sans réentraîner les poids. Un résumé peut perdre une négation, une version ou un horodatage déterminant : le document source doit rester accessible.

Enfin, le vecteur d’un jeton, les états contextuels d’une séquence et le vecteur d’un document utilisé pour la recherche sont trois objets différents. Pour indexer des tickets, on choisit un encodeur et une méthode de représentation adaptés, puis on évalue la recherche. Les coordonnées issues de deux modèles ne sont pas interchangeables, même si elles ont la même dimension.

Dérive et hors distribution (*out of distribution*, OOD)

La **dérive** signifie que les données ou la relation à prédire changent avec le temps. Une observation **hors distribution** est suffisamment différente du domaine validé pour rendre la confiance incertaine.

Dans un réseau, une nouvelle topologie, une météo rare, un firmware différent ou une panne corrélée peut produire ce cas. Le modèle doit pouvoir s'abstenir et laisser une politique classique sûre reprendre la main.

Méthode de référence et étude d'ablation (*ablation study*)

Une **méthode de référence**, souvent appelée *baseline*, est une méthode simple servant de comparaison : plus court chemin, tour de rôle, seuil ou règle métier. Sur un graphe à poids non négatifs, Dijkstra fournit une telle référence (merci à Edsger Dijkstra, 1959 ; article original).

Une **ablation** retire un composant ou une entrée pour mesurer ce qu'il apporte réellement. Sans méthode de référence ni ablation, on ne sait pas si le modèle bat une règle simple ni pourquoi.

Reproductibilité

Un résultat dépend des données, du code, des bibliothèques, des paramètres, du matériel et de l'aléatoire. Un **générateur pseudoaléatoire** produit par calcul une suite qui imite le hasard. Une **graine**, aussi appelée **germe** ou *seed*, fixe son état initial : à algorithme et ordre d'appels identiques, on retrouve la même suite. Cela ne garantit pas à lui seul un calcul identique sur tout le matériel et les bibliothèques.

La preuve utile conserve :

- versions ;
- configuration ;
- données ou leur empreinte ;
- métriques ;
- artefacts ;
- environnement ;
- commande de rejeu.

Choisir une variante de l'ACP pour les réseaux

Dossier de consultation. Les calculs suivants prolongent la question de la représentation des observations. Vous pouvez rejoindre directement le dictionnaire commun et revenir à cette étude pour choisir une décomposition.

Un **KPI** (*Key Performance Indicator*), ou indicateur clé de performance, est une mesure choisie pour suivre une performance, comme le débit livré ou la latence.

L'**analyse en composantes principales (ACP)** remplace plusieurs variables par quelques combinaisons linéaires : des axes sur lesquels on projette chaque observation. Les premiers axes retiennent le plus de variance ; réduire leur nombre conserve certaines variations et en écarte d'autres. L'introduction à la réduction de dimension en donne la lecture intuitive.

L'analyse en composantes principales (ACP, ou *principal component analysis*, PCA) ne désigne pas une recette unique. La variante la plus pertinente dépend de la structure des données et de l'objectif opérationnel :

Données ou objectif	Variante d'ACP la plus adaptée
Détecter des pannes, des interférences ou de la congestion	**ACP robuste (RPCA)**
Analyser des séries temporelles de KPI	**ACP dynamique (DPCA)**
Prendre en compte la topologie des cellules ou des sites	**ACP régularisée par graphe**

Données ou objectif	Variante d'ACP la plus adaptée
Traiter des données site × KPI × temps × fréquence	**ACP tensorielle**
Modéliser des relations non linéaires	**ACP à noyau (*kernel PCA*)**
Obtenir des combinaisons de KPI interprétables	**ACP parcimonieuse (*sparse PCA*)**
Traiter des échantillons I/Q ou un CSI complexe	**ACP complexe / transformée de Karhunen–Loève**

Pourquoi l'ACP robuste est particulièrement pertinente

En fonctionnement normal, les KPI d'un réseau radio sont fortement corrélés : **RSRP** (puissance reçue sur les signaux de référence), **RSRQ** (indicateur de qualité de réception) et SINR ; débit et utilisation des **PRB** (blocs de ressources radio) ; BLER et retransmissions ; latence et pertes de paquets ; échecs de transfert de connexion et appels interrompus. Le comportement nominal est donc approximativement de faible rang, tandis que les défauts sont souvent rares et localisés. On peut l'écrire :

Modèle faible rang + anomalies parcimonieuses : $X = L + S + E$.

où L représente le comportement réseau normal et corrélé, S les anomalies parcimonieuses — cellule en panne, interférence ou congestion soudaine — et E le bruit de mesure. La RPCA vise à limiter la déformation du sous-espace nominal par des anomalies de grande amplitude. Cette séparation n'est pas garantie par la seule rareté des incidents : elle suppose notamment que le faible rang ne soit pas lui-même concentré sur quelques coefficients et que les anomalies ne se confondent pas avec une composante de faible rang. Un changement de régime ou un défaut aligné avec les modes nominaux peut rester ambigu.

Recommandation pour la supervision au niveau cellule

Une architecture conceptuelle particulièrement adaptée est :

Recommandation : ACP robuste, dynamique et régularisée par graphe.

L'ACP dynamique représente l'évolution temporelle, l'ACP robuste sépare le comportement nominal des défauts, et la régularisation par graphe impose une cohérence entre cellules voisines ou interférentes. Cette combinaison est une orientation de conception, pas un algorithme universel prêt à déployer : ses hyperparamètres et ses seuils doivent être validés sur des incidents réellement annotés.

Pour deux variables réelles, la **covariance** mesure comment leurs écarts à la moyenne varient ensemble : positive, elles tendent à varier dans le même sens ; négative, dans des sens opposés. Une covariance nulle ne prouve pas leur indépendance. Une matrice de covariance rassemble ces mesures pour toutes les paires de variables.

Pour le traitement du canal radio ou du CSI, l'ACP complexe appliquée à la matrice de covariance du canal est plus naturelle. Ses composantes principales correspondent aux modes propres dominants de propagation ou d'espace, ce qui la rend utile pour le MIMO, la formation de faisceaux et la compression du canal.

Enfin, il faut standardiser les KPI et, de préférence, ajuster des modèles distincts selon les régimes — heure chargée ou nuit, intérieur ou extérieur, technologie et bande de fréquences. Sans cette précaution, l'ACP risque surtout de redécouvrir le niveau de trafic, plutôt que les véritables défauts.

Vocabulaire linéaire minimal pour lire le notebook

Soit $K \in \mathbb{R}^{n \times p}$ la table brute de n observations et p KPI. Le centrage retranche à chaque colonne sa moyenne. Comme les KPI n'ont pas la même unité, le notebook les standardise aussi :

$$X_{tj} = \frac{K_{tj} - \acute{K}_j}{s_j},$$

où s_j est l'écart-type du KPI j. Dans une tâche prédictive, moyenne et écart-type doivent être ajustés sur l'entraînement seulement ; ici, le notebook réalise une analyse descriptive d'un jeu synthétique unique.

Des directions sont **linéairement indépendantes** si aucune ne se reconstruit comme somme pondérée des autres. **Orthonormales** signifie perpendiculaires entre elles et de longueur 1. Le **rang** est le nombre de directions linéairement indépendantes portées par une matrice.

SVD : fondement numérique de l'ACP

La décomposition en valeurs singulières (*singular value decomposition*, SVD) est particulièrement pertinente en télécommunications radio. Elle constitue aussi le fondement numérique naturel de l'ACP. Pour une matrice de KPI centrée X contenant n observations :

Décomposition SVD :

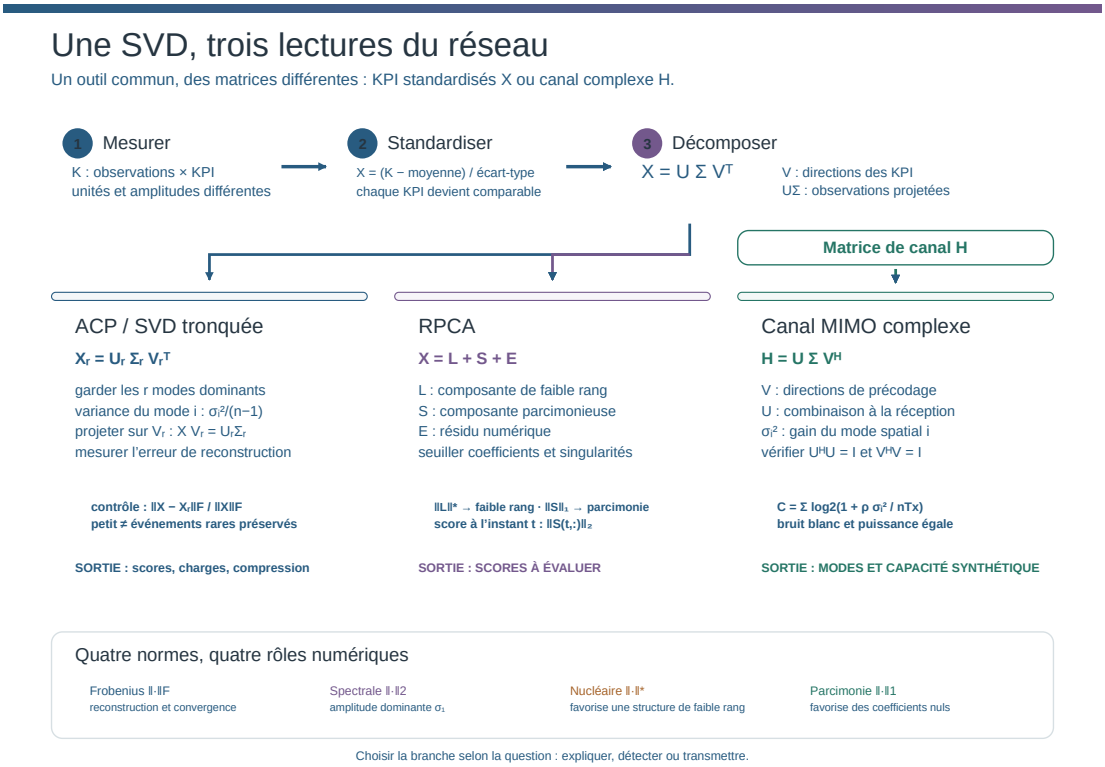
$$X = U \Sigma V^{\top}$$

Dans $X = U \Sigma V^{\top}$, les colonnes de U et V sont orthonormales et les valeurs singulières $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ occupent la diagonale de Σ. Garder les r premières donne la SVD tronquée $X_r = U_r \Sigma_r V_r^{\top}$, meilleure approximation de rang au plus r pour la norme de Frobenius (Eckart et Young, 1936). Le notebook mesure son erreur relative par

$$\frac{\|X - X_r\|_F}{\|X\|_F}, \|X\|_F = \sqrt{\sum_{t,j} |X_{tj}|^2}.$$

La norme spectrale $\|X\|_2 = \sigma_1$ mesure au contraire la plus forte amplification linéaire. Ces deux normes ont donc des rôles distincts dans le code : la première contrôle reconstruction et convergence, la seconde initialise l'algorithme robuste.

Les colonnes de V sont les directions principales, c'est-à-dire les combinaisons de KPI définies par l'ACP. La matrice $U \Sigma$ contient les observations projetées sur ces directions. La variance expliquée par la composante i vaut $\sigma_i^2/(n-1)$: les valeurs singulières ordonnent donc directement les modes dominants.



Carte de lecture de la SVD : KPI standardisés pour l'ACP tronquée et la RPCA, matrice de canal complexe H pour le MIMO, avec les normes utilisées dans le notebook

Du modèle RPCA au code numérique

La version *Principal Component Pursuit* décrite par Candès, Li, Ma et Wright, utilisée dans le notebook, part de $X = L + S$ et résout le problème convexe

$$\min_{L,S} \|L\|_* + \lambda \|S\|_1 \text{ sous la contrainte } X = L + S.$$

Un ensemble est **convexe** si le segment entre deux choix admissibles reste admissible. Une fonction est convexe si, sur ce segment, sa valeur ne dépasse pas l'interpolation entre ses valeurs aux extrémités : sa courbe reste sous la corde. Dans un problème convexe, un **minimum local**, au moins aussi bon que les choix suffisamment proches, est aussi **global**, au moins aussi bon que tous les choix admissibles. Cela ne garantit pas qu'il soit unique.

Norme nucléaire : favoriser un faible rang.

$$\|L\|_* = \sum_i \sigma_i(L)$$

Elle encourage un petit nombre de modes.

Norme des anomalies : favoriser la parcimonie.

$$\|S\|_1 = \sum_{t,j} |S_{tj}|$$

Elle encourage beaucoup de coefficients nuls, donc des anomalies parcimonieuses. Le choix usuel repris dans le code est :

$$\lambda = 1/\sqrt{max(n, p)}$$

Résidu numérique et bruit de mesure. Ici $E = X - L - S$ est exclusivement le résidu numérique de la contrainte d'égalité, pas une estimation du bruit de mesure. Le bruit dense injecté dans les KPI doit donc être absorbé par L ou S.

Un modèle qui sépare explicitement ce bruit utiliserait, par exemple, une contrainte $\|X - L - S\|_F \leq \delta$, avec un budget de bruit justifié : c'est une formulation de PCP stable distincte de celle exécutée dans ce TP.

Deux opérateurs expliquent les fonctions du notebook. Le seuillage doux agit coefficient par coefficient :

$$shrink_\tau(z) = sign(z)\,max(|z| - \tau, 0).$$

Le seuillage des valeurs singulières applique la même contraction au spectre :

$$D_\tau(M) = U\,diag\big(max(\sigma_i - \tau, 0)\big)V^\top, M = U\,\Sigma\,V^\top.$$

Avec un multiplicateur Y et un paramètre de pénalité μ, l'algorithme de Lagrangien augmenté étudié par Lin, Chen et Ma alterne alors :

$$L \leftarrow D_{1/\mu}(X - S + Y/\mu), S \leftarrow shrink_{\lambda/\mu}(X - L + Y/\mu),$$

$$E \leftarrow X - L - S, Y \leftarrow Y + \mu E.$$

Le notebook augmente progressivement μ et s'arrête lorsque $\|E\|_F / \|X\|_F$ passe sous la tolérance. Il transforme ensuite chaque ligne parcimonieuse en score d'anomalie $a_t = \|S_{t,:}\|_2$. Les étiquettes d'incident servent uniquement à calculer le rappel des plus grands scores après l'ajustement ; elles ne sont pas utilisées pour construire L ou S.

Le carnet publie aussi la fraction de coefficients non nuls de S et le taux de fausses alertes de la règle naïve « un coefficient non nul suffit ». Sur les données fixées du TP, environ 79 % des coefficients de S dépassent 10^{-6} : son support ne désigne donc pas les seuls incidents. Ce contre-exemple rend visible l'absorption du bruit par la composante pénalisée. Le rappel est mesuré au rang $k = 24$, nombre d'incidents connu par construction ; il décrit un classement, pas un détecteur à seuil calibré. Une évaluation de détection demanderait un seuil choisi sur une calibration séparée, puis précision, rappel et fausses alertes sur un test indépendant.

Canal radio MIMO

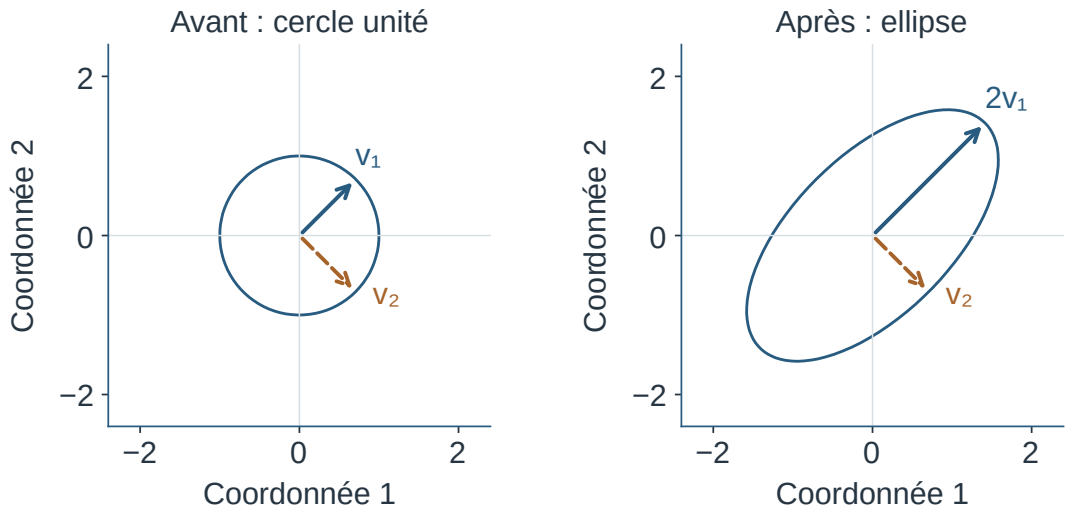
Pour un canal MIMO complexe, on applique directement la SVD complexe à la matrice de canal H :

Décomposition du canal MIMO : $H = U\,\Sigma\,V^H$.

Pour reprendre la notion de nombre complexe : les nombres complexes avec ParaMaths.

Les colonnes de V définissent les directions de précodage à l'émission, celles de U les directions de combinaison à la réception, et les valeurs diagonales de Σ la force de chaque flux spatial. La SVD transforme ainsi le canal MIMO, sous les hypothèses du modèle — canal linéaire connu et bruit gaussien spatialement blanc —, en sous-canaux spatiaux indépendants. Elle est fondamentale pour la formation de faisceaux, le multiplexage spatial, l'estimation du rang du canal et le calcul de capacité (Telatar, 1999, § 3.1).

Choisir des directions qui simplifient le mélange. Une matrice peut mélanger les coordonnées d'un vecteur. La SVD trouve des directions d'entrée et de sortie dans lesquelles ce mélange devient une amplification séparée de chaque composante. Pour $H = \begin{pmatrix} 1{,}5 & 0{,}5 \\ 0{,}5 & 1{,}5 \end{pmatrix}$, les directions $(1, 1)/\sqrt{2}$ et $(1, -1)/\sqrt{2}$ sont amplifiées respectivement par 2 et par 1 : une combinaison commune traverse mieux que leur différence. Le MIMO exploite ce principe pour choisir les combinaisons émises et reçues. Fourier choisit des directions fréquentielles fixées ; la SVD choisit des directions adaptées à la matrice étudiée. Cet exemple symétrique possède les mêmes directions à l'entrée et à la sortie ; ce n'est pas le cas général (Goodfellow et al. 2016b).



SVD géométrique : un cercle unité devient une ellipse, avec des gains de 2 et 1 dans deux directions orthogonales. Exemple réel symétrique ; les coordonnées normalisées ne sont pas les positions des antennes.

Dans le cas complexe, H^H est la transposée conjuguée de H. Les matrices U et V sont unitaires : $U^H U = I$ et $V^H V = I$. Le notebook vérifie ces deux identités ainsi que la reconstruction $H = (U\,\Sigma)\,V^H$ avec une erreur relative de Frobenius proche de zéro.

Pour relier les valeurs singulières à la capacité, il suppose un bruit spatial blanc normalisé, une connaissance parfaite du canal, une puissance répartie également entre les n_t antennes d'émission et aucune optimisation par *water-filling*. Un rapport signal sur bruit de ρ_{dB} décibels devient $\rho = 10^{\rho_{dB}/10}$. La capacité spectrale synthétique est alors

$$C = \sum_i \log_2\left(1 + \frac{\rho}{n_t}\sigma_i^2\right) = \log_2 det\left(I_{n_r} + \frac{\rho}{n_t} H\,H^H\right) \text{bit/s/Hz}.$$

L'égalité vient de ce que les valeurs propres non nulles de $H\,H^H$ sont σ_i^2. Le calcul `slogdet` du notebook évalue le logarithme du déterminant de façon plus stable que `log(det(...))` et vérifie numériquement que la somme par modes et la forme matricielle coïncident. Cette formule décrit les hypothèses synthétiques du TP ; elle ne constitue pas à elle seule un budget de liaison ou une prédiction de capacité terrain.

Variantes recommandées

Données ou objectif	Variante adaptée
Compresser le CSI et extraire les modes propres MIMO	**SVD complexe tronquée**
Traiter de très grands jeux de données réseau	**SVD aléatoire (*randomized SVD*)**
Mettre à jour un modèle avec une télémétrie continue	**SVD incrémentale**
Détecter des anomalies ou des pannes	**SVD robuste / RPCA**
Traiter des données cellule × KPI × temps × fréquence	**Décomposition tensorielle**, analogue multilinéaire de la SVD

En pratique, le choix peut se résumer ainsi :

MIMO ou CSI : SVD complexe.

KPI réseau : SVD/ACP robuste, ou SVD tronquée incrémentale.

La troncature doit être dimensionnée sur la variance utile et validée contre les événements rares : une compression trop agressive peut justement supprimer les signatures faibles que l'on cherche à détecter.

Le carnet SVD radio reproductible met ces calculs en pratique avec des KPI synthétiques, une RPCA et un canal MIMO complexe. Il adapte le carnet de réduction dimensionnelle original sans présenter les données simulées comme une validation RF.

> **Vocabulaire.** La SVD est une décomposition matricielle ; SVG est un format graphique vectoriel. L'ACP peut servir à aligner ou compresser des formes vectorielles, mais cet usage n'a pas de signification radio particulière.

3. Le dictionnaire commun

Réseau	Science des données / contrôle	Idée commune
télémétrie	observations / variables d'entrée (*features*)	ce que le système sait
topologie	graphe d'entrée	structure des relations
métrique de route	coût	préférence quantifiée
règle de transfert	action	décision appliquée
objectif de service	fonction objectif	classe les solutions préférables
contrainte dure	masque ou ensemble faisable	exclut les solutions interdites
politique réseau	préférences, règles et autorités	comportements permis et arbitrages
ordonnanceur (*scheduler*)	politique de décision	qui reçoit la ressource
file	travail en attente (*backlog*)	demande non encore servie
capacité	contrainte de ressource	limite physique ou contractuelle
convergence réseau	stabilisation	retour vers un état cohérent
jumeau numérique	simulateur / environnement	monde contrôlé pour tester
déploiement témoin (*canary*)	expérimentation limitée	réduire le risque du changement
méthode de référence (*baseline*)	comparateur expérimental	prouver l'apport du modèle
repli (*fallback*)	politique indépendante validée	comportement exécutable sans le modèle
dérive de configuration	dérive (*drift*)	écart entre attendu et observé

Deux mots demandent une attention particulière :

- **politique** signifie en réseau un ensemble de règles et préférences ; en apprentissage par renforcement, elle désigne la fonction qui choisit une action ;
- **état** peut désigner l'état réel du réseau, l'état mémorisé par un protocole ou l'observation donnée à un agent. Ces trois ensembles ne sont pas toujours identiques.

Une **méthode de référence** n'est pas automatiquement une **solution de repli**. Le plus court chemin peut constituer un comparateur simple, tout en étant un mauvais repli s'il ignore capacité, autorisations ou échéances. Une admission conservatrice associée à un ordonnanceur classique peut être un meilleur repli, parce qu'elle est indépendante du modèle, réellement déployable et validée dans les modes dégradés.

4. Comment les deux métiers peuvent travailler ensemble

Pour interpréter un fichier de métriques, le spécialiste des données a besoin de connaître :

- la couche mesurée : radio, liaison locale, IP, transport ou application ;
- la fraîcheur de la mesure ;

- l’action réellement possible ;
- les contraintes du protocole ;
- les modes de panne ;
- le coût d’une mauvaise décision.

Pour interpréter un score de précision, l’ingénieur réseau a besoin de connaître :

- le domaine d’entraînement ;
- la méthode de référence ;
- les pires cas ;
- l’incertitude ;
- le délai d’inférence ;
- la dérive ;
- le mécanisme d’abstention et de retour arrière (*rollback*).

Le contrat commun peut se résumer ainsi :

```
observer avec une fraîcheur connue
 -> décider dans une enveloppe autorisée
 -> appliquer avec une portée limitée
 -> mesurer l’effet utile
 -> revenir à une politique sûre si le doute augmente
```

> **Invariant — Le modèle ne remplace pas les protocoles. Une exécution réseau réussie ne prouve pas à elle seule que le modèle se généralisera.**
> La compréhension complète vient de la combinaison des deux disciplines, puis d’essais représentatifs et de preuves rejouables.

5. Mini-guide de lecture de `sb3_trainer_project`

L’étude de cas compare plusieurs façons de prendre des décisions de routage dans un monde simulé. Pour ne pas confondre les couches, la lire comme trois blocs :

```
MONDE
positions -> liaisons possibles -> graphe et demandes

DÉCISION
observation -> politique -> action -> allocation réalisable

PREUVE
effet simulé -> indicateurs -> comparaison -> artefacts rejouables
```

Pour suivre le déroulé sans auditer l’algorithme, les mots indispensables sont :

Terme	Sens simple dans le projet
étape	travail qui consomme des entrées et produit des sorties
DAG	graphe orienté sans cycle : une étape ne peut pas dépendre d’elle-même, même indirectement
vague	groupe d’étapes indépendantes pouvant s’exécuter en parallèle
environnement	simulateur avec lequel l’algorithme interagit
observation	information effectivement visible au décideur
action	choix que le décideur est autorisé à effectuer
politique	règle, heuristique ou modèle qui choisit l’action

Terme	Sens simple dans le projet
récompense	score qui résume ce que l’apprentissage cherche à favoriser
embedding	vecteur latent appris qui représente un objet pour une tâche
pooling	agrégation de plusieurs embeddings en un résumé, avec un risque de perte d’information
trajectoire d’expérience (*rollout*)	suite d’observations, d’actions et de résultats simulés
point de sauvegarde (*checkpoint*)	copie versionnée d’un modèle pendant ou après l’entraînement
KPI	indicateur clé de performance, par exemple débit livré ou latence
artefact	fichier produit et conservé : modèle, décision, métrique ou preuve
MLflow	outil qui relie paramètres, métriques, modèles et fichiers d’une expérience
MARL	apprentissage par renforcement avec plusieurs agents qui agissent dans le même environnement

Pour auditer le code RL et PPO, il faut en plus :

Terme RL	Sens simple
état	description interne du monde suffisante pour calculer son évolution
récompense immédiate	signal reçu après une transition
retour	somme actualisée des récompenses futures depuis un pas
facteur d’actualisation γ	poids donné aux conséquences futures
valeur V	retour attendu depuis un état ou une observation
valeur d’action Q	retour attendu après une action donnée
avantage $A = Q - V$	action jugée meilleure ou pire que l’attente courante
acteur	partie qui produit la distribution d’actions
critique	partie qui estime une valeur pour aider l’apprentissage

Le projet ne compare pas quatre objets situés au même niveau. Il combine modes d’apprentissage, architectures et solveurs. La matrice ci-dessous décrit les **familles de méthodes prises en charge**, dont certaines seulement appartiennent au DAG historique de 17 étapes :

Famille prise en charge	Signal ou recherche	Représentation / décision	DAG historique de 17 étapes
`fcas_routing_supervised_gnn`	supervision par allocations cibles	GNN à passage de messages, prédicteur de fractions	non
`fcas_routing_ppo` — surface canonique ; `fcas_routing_ppo_gnn` — branche de compatibilité	RL interactif on-policy, PPO	politique plate ou extracteur GNN selon configuration	oui, sous le nom `fcas_routing_ppo_gnn`
`uav_relay_queue_genetic`	recherche évolutionnaire	politique linéaire compacte, référence face au PPO	oui
`fcas_routing_ilp`	optimisation ILP/MILP sous contraintes	solveur, référence si le vrai solveur et les entrées sont présents	oui

Famille prise en charge	Signal ou recherche	Représentation / décision	DAG historique de 17 étapes
`fcas_routing_path_ac`	acteur–critique/gradient de politique à un pas sur des instantanés	score de chemins candidats avec masque de validité	non
`uav_active_mesh_ppo`	RL centralisé sur une action jointe	une politique PPO pour les mouvements des relais	oui ; « MARL-style », pas MARL décentralisé général

PPO décrit comment une politique est mise à jour. Un **GNN** décrit comment le graphe est représenté et traité. Un **algorithme génétique** recherche des paramètres sans gradient. Une **ILP/MILP** cherche une solution explicite sous contraintes. Un GNN peut donc être supervisé ou intégré à PPO ; « GNN contre PPO » n'est pas une comparaison de même nature.

Dans l'implémentation actuelle, le Path Actor-Critic (*acteur–critique de chemins*) traite chaque instantané séparément. L'acteur note des chemins ; le critique estime le résultat normalement attendu ; `reward - value` indique si l'action a fait mieux ou pire que cette attente. Comme il n'existe ici ni transition vers un prochain état ni **bootstrap temporel multi-pas** (*multi-step bootstrapping*), ce comportement est plus proche d'un bandit contextuel combinatoire entraîné par gradient de politique (*policy gradient*) que d'un RL séquentiel général.

La question commune aux deux métiers est toujours :

> Qu'a vu le décideur, qu'avait-il le droit de faire, quel effet le simulateur a-t-il calculé, et quelle preuve permet de rejouer cette conclusion ?

Une récompense élevée n'établit pas la déployabilité d'une route ; sa conformité à une métrique n'établit pas que le modèle est correctement entraîné et généralisable.

Repères mathématiques — du résultat à l'usage

Une même formule peut répondre à deux questions. **Pourquoi est-elle vraie, et sous quelles hypothèses ?** Les mathématiques fondamentales s'interrogent sur sa démonstration. **Quel phénomène représente-t-elle, avec quelle erreur et quel coût de calcul ?** Les mathématiques appliquées s'interrogent sur son usage. La frontière reste ouverte : les applications suscitent des théorèmes, et les mathématiques appliquées développent aussi leurs démonstrations.

La différence se ressent lorsqu'on passe d'une fonction idéale à un compteur, à un calcul embarqué, à un signal mesuré ou à un modèle appris sur des données.

Pour retrouver les notations à son rythme, le compagnon Lire les mathématiques de l'IA propose une progression illustrée et des exercices corrigés. Chaque symbole rejoint une opération que l'on peut refaire sur quelques nombres.

Si les premiers calculs résistent, commencez par les fractions, les puissances et les unités. Les intervalles précisent ensuite les valeurs autorisées ; les limites distinguent une somme finie de son comportement à long terme. Le guide reprend aussi les systèmes linéaires, l'approximation de Taylor, les contraintes d'optimisation, les matrices de graphe, le modulo et le dénombrement. Ces gestes permettent de lire les formules qui suivent en sachant ce que l'on calcule et sous quelles hypothèses.

Lire les notations avant les formules. Une **variable** représente une valeur qui peut changer ; une **fonction** associe une sortie à une entrée. Ainsi, $f(x)=x^2$ donne $f(3)=9$. Un **indice** repère un élément : x_2 est la deuxième composante ; un **exposant** indique une puissance : $x^2=x\times x$. Un **paramètre** règle la fonction et reste fixé lorsque l'on étudie sa variation par rapport à x. Dans $f(x)=w\,x+b$, w est la pente et b le décalage.

La dérivée : définir une variation, puis estimer un débit

Le calcul différentiel formalise les variations. Pour une fonction f, la dérivée en x est la limite

$$f'(x)=\lim_{h\to 0}\left(\frac{f(x+h)-f(x)}{h}\right),$$

lorsque cette limite existe. $h \to 0$ signifie que l'on prend des pas non nuls de plus en plus petits. La **limite** est la valeur vers laquelle le quotient se rapproche, si l'approche par chaque côté conduit à la même valeur. Le quotient se calcule pour $h \neq 0$, avant de chercher cette limite.

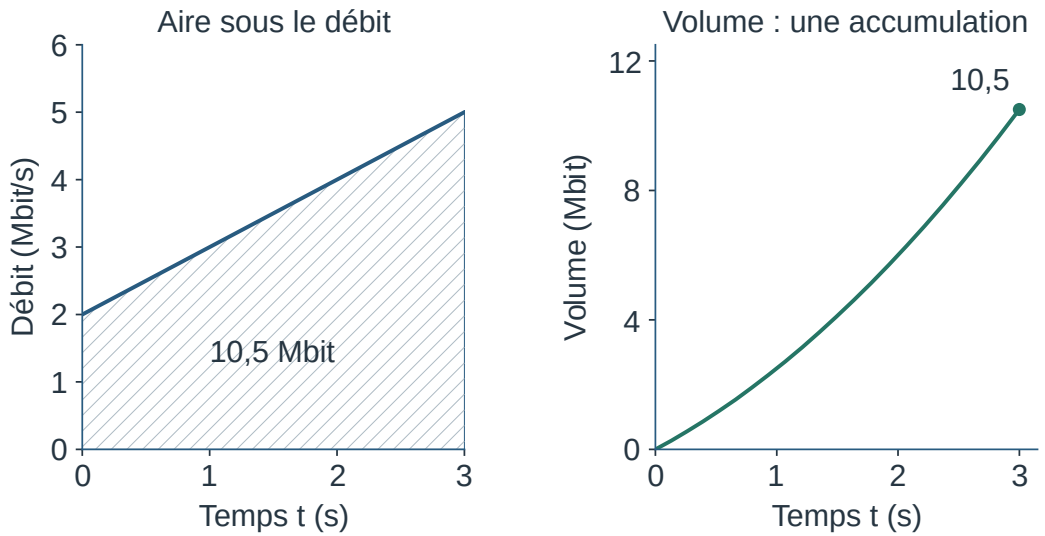
Avec $f(x) = x^2$, le quotient en $x = 2$ vaut $((2+h)^2 - 4)/h = 4 + h$ pour $h \neq 0$: sa limite est donc 4. Les pas $0,1$ et $0,01$ donnent respectivement $4,1$ et $4,01$. La démonstration établit une pente exacte ; chaque quotient à pas fini reste une approximation. Cette distinction découle de la définition de la dérivée. Pour $y = f(x)$, cette dérivée se note aussi dy/dx.

Pour voir comment une courbe révèle sa pente : les dérivées avec ParaMaths.

En exploitation réseau, un compteur qui augmente de **25 000 octets en 20 ms** indique un débit moyen de $25\,000 / 0,020 = 1\,250\,000$ octets/s, soit **10 Mbit/s**. Le compteur progresse par sauts à la réception des paquets : le quotient mesuré est une moyenne sur l'intervalle. Une dérivée décrit un modèle continu du trafic. Choisir l'intervalle d'observation devient alors essentiel : un intervalle long masque les pointes ; un intervalle court expose davantage les fluctuations et les erreurs de mesure.

De la pente à l'aire : retrouver le volume transmis. La dérivée passe d'un volume cumulé à un débit ; l'intégrale effectue le trajet inverse. Dans le modèle continu ci-dessous, le débit de charge utile vaut $r(t) = 2 + t$ Mbit/s, avec t en secondes entre 0 et 3. Le volume reçu depuis le départ est $V(t) = \int_0^t r(u)\,du = 2t + t^2/2$ Mbit. Le symbole $\int_0^t \square$ indique une accumulation entre le départ et l'instant t : chaque petit intervalle de durée du apporte un volume $r(u)\,du$. La lettre u parcourt ces instants intermédiaires.

À trois secondes, l'aire sous la courbe du débit vaut donc **10,5 Mbit**. La pente de la courbe du volume redonne le débit : ces deux graphiques décrivent le même transfert. Pour des paquets réels, le compteur progresse par sauts ; le modèle représente une évolution continue idéale (Jerison 2010).



Intégrale et dérivée : l'aire sous le débit donne le volume cumulé ; sa pente redonne le débit, dans un modèle continu idéal.

Pour prolonger ce lien entre variation et accumulation : les intégrales avec ParaMaths.

Les polynômes de Taylor : approcher avec une erreur maîtrisée

Autour d'un point a, le polynôme de Taylor de degré n (merci à Brook Taylor, 1715) utilise les dérivées de la fonction en ce point :

$$P_n(x)=\sum_{k=0}^{n}\frac{f^{(k)}(a)}{k\,!}(x-a)^k,\quad f(x)=P_n(x)+R_n(x).$$

Dans cette somme, k parcourt les entiers de 0 à n. $f^{(k)}$ désigne la dérivée prise k fois, avec $f^{(0)}=f$. La **factorielle** $k\,!$ est le produit des entiers de 1 à k : $3\,!=1\times2\times3=6$ et, par convention, $0\,!=1$.

Le reste R_n mesure l'erreur. Si la dérivée d'ordre $n+1$ est continue et bornée en valeur absolue par M entre a et x, alors $|R_n(x)|\le M\,|x-a|^{n+1}/(n+1)\,!$. Passer du polynôme fini à une **série entière infinie** représentant exactement f exige que ce reste tende vers zéro. Posséder des dérivées de tout ordre ne suffit pas à garantir cette égalité : c'est une question de convergence (théorème de Taylor et reste).

Pour e^x autour de zéro, $P_2(x)=1+x+x^2/2$. À $x=0,1$, il donne $1,105$, contre environ $1,105170918$: l'erreur absolue est inférieure à $0,000185$. Dans un calcul embarqué, le choix du degré dépend ainsi de la précision requise, de l'intervalle utilisé et du coût des opérations. Un résultat excellent près de zéro peut devenir insuffisant plus loin.

Les séries de Fourier : décomposer un signal périodique

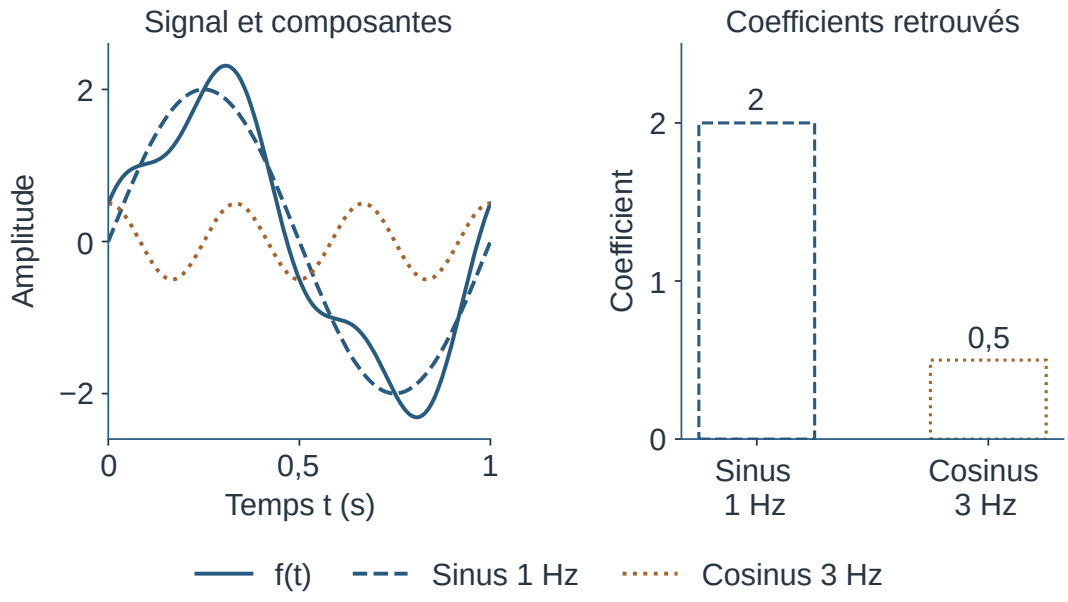
Une fonction périodique de période T possède, sous des conditions d'intégrabilité, des coefficients de Fourier. On étudie les sommes

$$S_N(t)=\frac{a_0}{2}+\sum_{n=1}^{N}\left[a_n\cos\left(\frac{2\pi n t}{T}\right)+b_n\sin\left(\frac{2\pi n t}{T}\right)\right].$$

Les fréquences n/T sont les **harmoniques**. Pour une fonction périodique continûment dérivable par morceaux, ces sommes convergent vers la fonction aux points continus et vers la moyenne des limites gauche et droite aux sauts. Le sens et les conditions de cette convergence relèvent de l'analyse (séries de Fourier).

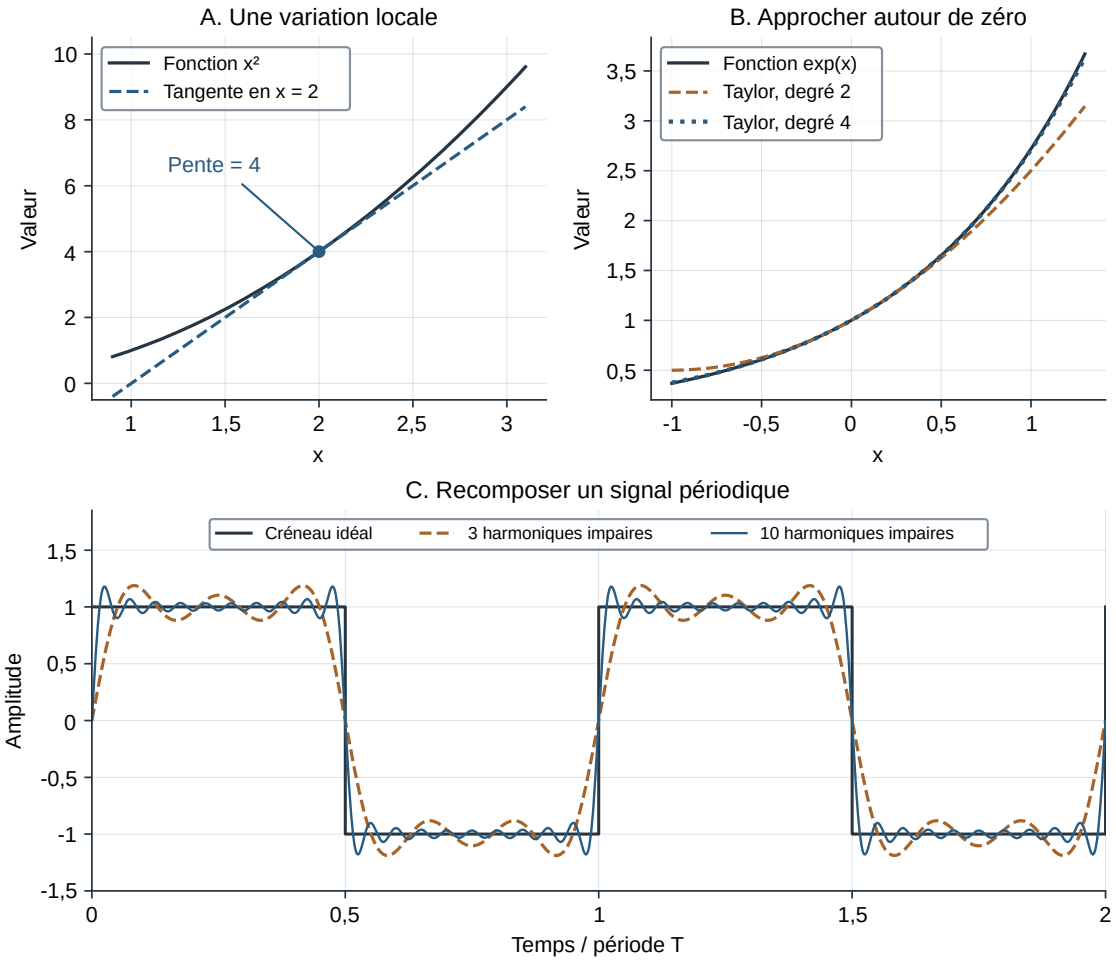
Un créneau alternant entre -1 et $+1$, positif sur la première demi-période, s'approche avec trois harmoniques impaires par $\frac{4}{\pi}\left(\sin\omega t+\frac{\sin 3\omega t}{3}+\frac{\sin 5\omega t}{5}\right)$, où $\omega=2\pi/T$. Davantage d'harmoniques affinent la représentation, mais des dépassements persistent près des sauts : c'est le **phénomène de Gibbs**. Pour transmettre ou filtrer un signal, on choisit donc les composantes conservées selon la bande passante et la déformation acceptable. Un calcul sur des échantillons finis apporte aussi ses limites de durée et de résolution.

Changer de coordonnées pour mieux lire un signal. Un vecteur se décrit par ses projections sur des axes. Fourier applique une idée comparable aux fonctions : les sinus et cosinus jouent le rôle de directions connues. Sur une période, multiplier le signal par une harmonique puis intégrer mesure sa contribution ; les autres harmoniques se compensent par orthogonalité. Ainsi, $f(t)=2\sin(2\pi t)+0,5\cos(6\pi t)$, de période une seconde, possède une composante sinus de coefficient 2 à 1 Hz et une composante cosinus de coefficient 0,5 à 3 Hz. Le signal n'a pas changé : sa description rend visibles ses fréquences. Une observation finie, bruitée ou tronquée peut toutefois empêcher cette séparation exacte (coefficients de Fourier).



Fourier comme projection : un signal périodique, ses deux composantes et leurs coefficients exacts sur une période. Ces coefficients ne sont ni une puissance ni une FFT mesurée.

Pour revoir les sinus et les cosinus : la trigonométrie avec ParaMaths.



Dérivée, approximation polynomiale et décomposition périodique : trois relations entre modèle exact et calcul fini

Un MLP à une couche cachée : approximation et apprentissage

Un **perceptron multicouche (MLP) à une seule couche cachée** et à sortie linéaire peut représenter une fonction sous la forme

$$\hat{f}_m(x) = b_0 + \sum_{j=1}^{m} a_j \sigma(w_j \cdot x + b_j).$$

Le **produit scalaire** $w_j \cdot x$ additionne les produits des composantes correspondantes : $(4,-1)\cdot(2,3) = 4\times 2 - 1\times 3 = 5$. Les **poids** w_j règlent la contribution des entrées ; le **biais** b_j décale la somme obtenue. L'**activation** σ transforme cette somme en réponse du neurone. Le coefficient a_j règle sa contribution à la sortie, et b_0 décale cette sortie.

Les m neurones cachés produisent chacun une réponse non linéaire ; la sortie en fait une somme pondérée. Avec la sigmoïde $\sigma(u) = 1/(1+e^{-u})$, le **théorème d'approximation universelle** établit que, pour toute fonction continue f sur le cube $[0,1]^d$ et toute précision $\varepsilon > 0$, il existe un nombre **fini** de neurones et des paramètres tels que

$$\sup_{x \in [0,1]^d} |f(x) - \hat{f}_m(x)| < \varepsilon.$$

$[0,1]^d$ contient les entrées dont les d composantes sont entre 0 et 1 ; ε est la tolérance choisie. Ici, sup décrit la plus grande erreur sur tout ce domaine : la formule demande qu'elle reste sous cette tolérance.

L'erreur est ainsi bornée sur tout le domaine, et pas seulement sur quelques exemples (Cybenko 1989). Parler d'« une infinité de neurones » évoque une limite : à mesure que l'on exige une erreur plus faible, on peut autoriser une largeur plus grande. Chaque approximation reste finie ; l'énoncé ne fixe pas une taille valable pour toutes les fonctions et toutes les précisions.

Le point de vue fondamental porte ici sur l'existence d'une représentation, ses hypothèses et le sens de la convergence. Comme avec les polynômes et les harmoniques, on combine des fonctions élémentaires. Dans un MLP, les poids w_j et les biais b_j ajustent aussi ces fonctions, au lieu de modifier seulement leurs coefficients dans une famille fixée.

Pour estimer un délai réseau à partir de la charge et du taux de pertes, le point de vue appliqué examine d'abord la continuité du modèle sur le domaine retenu. Il faut choisir une largeur, recueillir des mesures, apprendre les paramètres et vérifier les erreurs sur des situations nouvelles, sous les contraintes de mémoire et de temps de calcul. Le théorème ne garantit ni que l'entraînement trouvera les paramètres adéquats, ni que les données suffiront à généraliser (scikit-learn developers 2026q). **L'existence d'une bonne approximation et son apprentissage sont deux questions différentes.**

Dans ces quatre exemples, la démonstration précise ce que l'on peut conclure. Le travail appliqué ajoute une autre exigence : vérifier que les hypothèses, les unités, les données et l'erreur admise conviennent au problème réel.

Ressources mathématiques pour approfondir

Les exemples du livre introduisent les outils nécessaires à leurs applications : ils ne forment pas un cours complet de mathématiques. Le tableau suivant indique ce qui est déjà expliqué et ce que l'on peut travailler ensuite. Les ressources proposées donnent accès à des cours, et, selon le support, à des exercices ou à leurs corrigés.

Pour une entrée visuelle en français, je recommande aussi les animations de **Nicolas Masset** sur la chaîne **ParaMaths** (Masset, s. d.). Les liens placés près des notions renvoient aux vidéos correspondantes. Son site propose également des cours et exercices par chapitre.

Pour éveiller la curiosité avant les calculs, **Voyages au pays des maths, sur ARTE** propose des films courts en français sur les graphes, les probabilités ou les pavages. Ces derniers prolongent le problème des étiquettes chez AIRE : comment

couvrir une surface en respectant ses contraintes ? Après une vidéo, dessiner un petit cas et changer une hypothèse aide à passer de l'image au raisonnement (ARTE 2026).

Pour reprendre les outils de l'IA, **FIDLE, la formation du CNRS**, propose une entrée en français, puis des séquences sur les mathématiques et les gradients. En anglais, **StatQuest, de Josh Starmer**, illustre notamment Bayes, l'ACP, la validation croisée et la descente de gradient. Choisir la vidéo qui éclaire le calcul en cours suffit : il n'est pas nécessaire de suivre toutes les séries (CNRS 2026; Starmer 2026).

Domaine	Point de départ dans le livre	Pour développer
Analyse vectorielle	Dérivées partielles et gradients pour l'apprentissage ; azimut et élévation pour le pointage.	Champs, coordonnées curvilignes, intégrales multiples, flux et circulation, Green, divergence et Stokes : MIT 18.02SC, en anglais.
Équations différentielles	Une équation de relaxation du premier ordre, dont la solution est vérifiée dans l'exemple de PINN.	Séparation des variables, équations linéaires des premier et second ordres, puis systèmes différentiels : Exo7 L1 et L2.
Algèbre linéaire	Vecteurs, matrices, rang, normes, SVD, projections et matrices unitaires dans l'ACP et le MIMO.	Espaces vectoriels, applications linéaires, changements de base, déterminants, valeurs propres et diagonalisation : Exo7 L1 et L2.
Suites et séries	Taylor avec un reste contrôlé ; convergence des sommes de Fourier et phénomène de Gibbs.	Limites, critères de convergence, séries entières et rayon de convergence : Exo7 L1 et L2.
Intégrales	Une probabilité obtenue comme aire sous une densité.	Primitives, méthodes de calcul, intégrales impropres et dépendant d'un paramètre : Exo7 L1 et L2.
Probabilités	Densité, répartition, indépendance, modèles bayésiens, files et intervalles de confiance.	Dénombrement, calcul des probabilités, lois usuelles et théorème central limite : Rakotomalala, notes de cours — HAL.
Analyse complexe	Nombres complexes pour l'amplitude, la phase et le canal MIMO.	Théorie de Cauchy, intégration complexe, séries de Laurent et résidus : Harthong, Cours d'analyse — HAL, chapitres III et IV.
Fourier et Laplace	Décomposition d'un signal périodique en harmoniques.	Transformées continues, inversion, convolution et transformée de Laplace : Harthong — HAL, chapitre VII.
Espaces de Hilbert	Normes et projections dans les calculs matriciels en dimension finie.	Espaces de fonctions, complétude, bases orthonormées et opérateurs : Harthong — HAL, chapitres XI et XII.
Distributions au sens de Schwartz	Les densités de probabilité du livre concernent une autre notion de « distribution ».	Dirac, dérivées des discontinuités, convolution et Fourier des distributions : Harthong — HAL, chapitres IX et X.
Théorie des groupes	Conservation d'opérations illustrée par le chiffrement homomorphe.	Groupes, sous-groupes et morphismes : Exo7 L1. Pour les groupes et algèbres de Lie : MIT 18.745, en anglais, niveau avancé.
Exercices de mathématiques	Calculs guidés et laboratoires rattachés à des situations réseau.	Exercices et corrigés par thème : Exo7 L1 et L2 ; examens corrigés en calcul multivarié : MIT 18.02SC.

Des références que l'on peut retrouver. Les liens pointent vers des notices d'archive ou des pages de cours, plutôt que vers un fichier sur une page personnelle. Le titre et l'identifiant permettent de rechercher la ressource si le site change :

- **Exo7 — Cours et exercices de mathématiques**, niveaux L1 et L2, en français.
- **Denis Auroux, MIT OpenCourseWare 18.02SC — Multivariable Calculus**, session de 2010, en anglais.
- **Jacques Harthong — Cours d’analyse**, en français, notice **HAL cel-00519301, version 2**.
- **Ricco Rakotomalala — Probabilités et Statistique — Notes de cours**, 2025, en français, notice **HAL hal-05081503, version 1**.
- **Pavel Etingof, MIT OpenCourseWare 18.745 — Lie Groups and Lie Algebras I**, session de 2020, en anglais, niveau avancé.

Pour les signaux et l’IA, commencer par le calcul réel, l’algèbre linéaire et les probabilités ; poursuivre avec Fourier et les équations différentielles. Les opérateurs de champs, l’analyse complexe, Hilbert, les distributions et Lie constituent des prolongements selon le problème physique ou théorique étudié.

Traité transversal — Intelligence artificielle pour les réseaux : architectures, cas d'usage et preuve

Un réseau produit des observations, attend des décisions et impose des contraintes. L'IA peut aider à estimer ou à choisir ; la question est de savoir si cette aide améliore le service et comment reconnaître ses limites.

> **À retenir — Composer une solution.** S'il fallait retenir une idée sur les algorithmes d'IA, ce serait celle-ci : rendre un produit ou un système intelligent ne se résume pas à choisir l'algorithme réputé le meilleur. Il s'agit souvent de composer plusieurs algorithmes pour traiter les différentes parties d'un même problème. Même l'entraînement d'un modèle de machine learning demande une stratégie adaptée au contexte. Un arbre de décision ou un système expert à règles peut, par exemple, aider à choisir une méthode selon les données disponibles, les ressources ou les résultats obtenus.
>
> Dans le cas d'un grand modèle de langage (LLM), le post-entraînement — une étape qui suit son pré-entraînement — peut aussi recourir à l'apprentissage par renforcement. Un signal de récompense, fondé par exemple sur des préférences humaines, guide alors l'ajustement de ses réponses (Ouyang et al. 2022). La qualité de la solution dépend aussi de la manière dont ces éléments coopèrent.

Parallèle — Revisiter un outil familier. Je vous invite à regarder « Ca donne quoi de revisiter le binôme de Newton ? », de **Romain — L'élégance des maths**. Prenons l'expression $(a+b)^{1/3}$: elle désigne la racine cubique de $a+b$. Une puissance entière positive, comme $(a+b)^3$, se développe en une somme finie. Avec l'exposant $1/3$, on peut utiliser un développement en série : une somme dont on peut retenir les premiers termes pour approcher le résultat. Cette généralisation possède son domaine de validité, précisé dans le binôme généralisé du NIST.

J'en tire un parallèle avec la résolution d'un problème d'IA : comprendre ce qui change dans la question permet d'adapter les outils. Pour réduire des alertes réseau, par exemple, on commence par préciser le résultat attendu : les regrouper, les classer par urgence ou proposer une action. On peut ensuite combiner une règle métier, une représentation des mesures et un modèle appris. On compare cette solution à une méthode simple sur des situations nouvelles.

Dans les deux cas, la démarche relie formulation du problème, choix de la représentation, combinaison des méthodes et contrôle du résultat. La preuve mathématique établit une propriété sous ses hypothèses ; l'évaluation d'un modèle appris mesure ses résultats sur les données testées.

Votre question	Suite proposée
Que peut apporter l'IA à un réseau ?	grille de lecture → trois domaines → choisir l'outil
Comment juger un résultat ?	fiche commune de preuve → protocole de validation → banc d'essai
Comment organiser une réalisation ?	cas d'usage choisi → architecture et autorité correspondantes
Puis-je éprouver une idée maintenant ?	L23 : classification rare ; pour les fondations réseau, L02 : budget de latence

Grille de lecture

Dans le livre complet, l'Idée XII reprend cette grille dans « IA pour l'exploitation réseau », « Panorama des fonctions IA appliquées aux réseaux » et « Garde-fous pour une IA opératrice ».

Terminologie : le traité emploie « méthode de référence » (*baseline*), « repli » (*fallback*), « retour arrière » (*rollback*) et « déploiement canari » (*canary*).

« Mettre de l'IA dans le réseau » laisse ouvertes quatre questions :

1. **Quelles données ?**
2. **Quelle décision ?**

3. **Quel droit d’action ?**
4. **Quel repli sûr ?**

Les fonctions utiles se répartissent en quatre registres : **observation, prévision, recommandation, commande**. Avant de déléguer l’action, on suit une chaîne de validation proportionnée au risque :

```
observation -> recommandation -> simulation
            -> autorisation -> exécution bornée -> vérification
```

> **Autonomie et contrôle.** Plus une action est observable et réversible, plus il devient possible de déléguer son exécution tout en maîtrisant ses effets.

Risque	Exemple	Autorité
faible	enrichir une alerte	automatique
modéré	limiter un flux non critique	automatique borné
élevé	changer une politique BGP	validation forte
critique	modifier une racine de confiance	procédure dédiée

Une fiche commune de preuve

Une expérience raconte le même trajet : une question, des observations, une décision ou un mécanisme essayé, une comparaison et une conclusion que l’on peut discuter et rejouer. Conserver les entrées, les versions et les limites permet à un autre lecteur de reprendre ce trajet.

Pour transformer une idée en cas d’usage testable, les quatre questions du livre sont détaillées en dix contrats d’ingénierie :

Contrat	Question à rendre vérifiable
problème de service	quel effet utilisateur ou mission faut-il préserver ?
observation	quelles mesures, unités, horloges, incertitudes et valeurs manquantes ?
décision	quelle sortie exacte : score, prévision, classement ou action ?
autorité	quelles actions, sur quel périmètre, pendant combien de temps ?
référence	quelle règle, heuristique, optimisation ou procédure humaine faut-il battre ?
évaluation	quels scénarios indépendants, percentiles, coûts et intervalles d’incertitude ?
sûreté	quelles contraintes sont dures et appliquées hors de la récompense ?
repli	qui reprend la main, avec quel état, et comment éviter le bagottement ?
preuve	quelles empreintes cryptographiques, versions, entrées, sorties et commandes permettent le rejeu ?
exploitation	quelle latence, consommation, dérive, observabilité et procédure de retrait ?

Une estimation apprise possède un domaine d’entraînement et une incertitude. Si elle influence une action réseau, il faut préciser son droit d’action et vérifier un repli indépendant. Ce sont les particularités de l’IA que les dix contrats rendent visibles.

> **Test d’utilité.** Si une règle simple, un protocole éprouvé ou un solveur explicite satisfait le besoin avec de meilleures garanties, l’IA n’est pas justifiée par la seule complexité apparente du problème.

Repères de vocabulaire

Terme ou sigle	Sens dans ce guide
A1 / A1-ML	interface O-RAN du Non-RT RIC vers le Near-RT RIC ; A1-ML désigne le service de gestion de modèles ML, pas une autorité radio générique
ABR	*Adaptive Bitrate*, adaptation du débit d'un média segmenté
ACK / NACK	acquittement positif / négatif
AIS	*Automatic Identification System*, système d'identification maritime
AnLF / MTLF	fonctions logiques d'analyse / d'entraînement de modèle qu'un NWDAF 3GPP peut contenir
BBR	*Bottleneck Bandwidth and Round-trip propagation time*, contrôle de congestion fondé sur un modèle du chemin
BGP	*Border Gateway Protocol*, échange de routes entre domaines
BLER	*Block Error Rate*, taux d'erreur de blocs
déploiement témoin / observation sans action (*canary / shadow*)	déploiement sur un périmètre témoin / observation sans appliquer la décision
CLI	*Command-Line Interface*, interface en ligne de commande
CQI / SINR	indicateur de qualité de canal / rapport signal-interférence-plus-bruit
dataplane / goodput	plan de données qui transporte les paquets / débit utile effectivement livré
DSCP / ECN	marquage de classe de service / notification explicite de congestion
DTN / MANET	réseau tolérant aux délais / réseau mobile ad hoc
E2 / E2AP / E2SM	interface O-RAN entre Near-RT RIC et nœuds E2 / protocole applicatif / modèle de service propre à une fonction RAN
repli / retour arrière (*fallback / rollback*)	repli vers une politique sûre / retour à l'état antérieur
FEC / HARQ	correction d'erreurs anticipée / retransmission radio hybride
FRR	FRRouting, suite logicielle de protocoles de routage
GNN / MPNN	réseau neuronal de graphe / réseau à passage de messages
GNSS	système mondial de navigation par satellite
IA	intelligence artificielle
ILP / MILP	optimisation linéaire en nombres entiers, éventuellement mixte
KPI	*Key Performance Indicator*, indicateur clé de performance
LLM / RAG	grand modèle de langage / génération augmentée par recherche
LLR / MCS / MIMO	rapport de log-vraisemblance / modulation-codage / antennes multiples
MAC / PHY / RF	accès au médium / couche physique / radiofréquence
MDAF / MnS	fonction 3GPP d'analyse de données de gestion / service de management produit ou consommé

Terme ou sigle	Sens dans ce guide
netns / veth / tc	espace réseau Linux / paire d'interfaces virtuelles / contrôle de trafic Linux
Non-RT RIC / Near-RT RIC	contrôleurs intelligents O-RAN non temps réel / proche temps réel
NWDAF	*Network Data Analytics Function*, fonction d'analyse des données du cœur 5G 3GPP
OOD	*Out Of Distribution*, hors du domaine évalué
OSPF / OLSRv2	protocoles à état de liens ; OSPF pour un domaine IP administré, OLSRv2 optimisé pour les MANET
PACE	chaîne primaire, alternative, contingence, urgence
PPO / RL	*Proximal Policy Optimization* / apprentissage par renforcement
QoS / QoE / SLO	qualité de service / qualité d'expérience / objectif de service
R1	interface de services entre rApps et services du SMO, dont ceux du Non-RT RIC
RAN	*Radio Access Network*, réseau d'accès radio
rApp / xApp	application hébergée côté Non-RT RIC/SMO / côté Near-RT RIC
RCA	*Root Cause Analysis*, analyse de cause racine
RTT	temps aller-retour ; ce n'est pas un délai aller simple
SDO	organisme de normalisation ou de spécification (*Standards Development Organization*)
SMO	cadre O-RAN de gestion et d'orchestration des services
SWaP-C	taille, masse, puissance et coût
TCP / QUIC / UDP	transport fiable par flux / transport chiffré sur UDP, avec flux fiables et datagrammes optionnels non fiables / datagrammes sans garantie intrinsèque
TDD / FDD	duplexage temporel / fréquentiel
WCET	*Worst-Case Execution Time*, pire temps d'exécution d'une fonction, borné par analyse ou estimé par mesure ; à ne pas confondre avec WCETT
WCETT	*Weighted Cumulative Expected Transmission Time*, métrique de chemin combinant temps de transmission et diversité de canaux

Panorama des algorithmes et concepts d'apprentissage

Choisir un modèle exige de relier sortie attendue, structure des observations, signal d'apprentissage et coût de l'erreur. Les exemples suivants sont des pistes d'étude, pas des résultats expérimentaux. Ce panorama complète les axes de la section « Les modes d'apprentissage » par les mécanismes et leurs limites.

Des données au modèle : les repères nécessaires

Une **observation** rassemble les mesures d'un cas, par exemple la charge et le débit d'une liaison : ce sont ses **variables**. La **cible** est la réponse à apprendre, lorsqu'on la connaît. Le **modèle** transforme ces variables en une sortie : prévoir un délai est une **régression** ; choisir une catégorie d'incident est une **classification**. Ses **paramètres**, comme les coefficients d'une formule, sont ajustés pendant l'**entraînement**. L'**inférence** utilise ensuite le modèle pour une nouvelle observation.

On réserve des données à la **validation** pour choisir les réglages, puis un **jeu de test** pour l'évaluation finale. Ces réglages sont les **hyperparamètres** : on peut, par exemple, limiter le nombre de questions successives d'un arbre de décision. La **perte** mesure l'erreur que l'apprentissage cherche à réduire. Prévoir 32 ms pour un délai observé de 30 ms donne un écart de 2 ms ; une perte peut mesurer cet écart. Une **métrique** sert à évaluer les sorties. Si le modèle retient les particularités de l'entraînement mais réussit mal sur de nouveaux cas, il y a **surapprentissage**.

Une liste de variables forme un **vecteur** x ; un tableau d'observations forme une **matrice**. Les coefficients w pondèrent les variables. La transposée, indiquée par un T en exposant, permet ici d'écrire leur somme pondérée $w^{\top} x$. Avec $x=(2,3)$ et $w=(4,1)$, cette somme vaut $4 \times 2+1 \times 3=11$. Une constante b peut s'ajouter : c'est le **terme de biais**.

Si les notations sont devenues moins familières, le compagnon Lire les mathématiques de l'IA reprend les notations pas à pas, des premières sommes aux convolutions et à l'attention des Transformers. Comprendre et pratiquer l'IA se construit par ces étapes ; un doctorat n'est pas un préalable à cette démarche.

De la formule au programme. Les vidéos de **Machine Learnia** et leur dépôt de notebooks accompagnent les premiers essais en Python, en français. **Homemade Machine Learning**, d'Oleksii Trekhleb, montre en anglais le calcul derrière plusieurs modèles, avec des exemples interactifs. Partir d'une petite régression, changer une observation et comparer la courbe obtenue permet de relier les symboles au comportement du programme (Machine Learnia 2026; Trekhleb 2026).

Lire une carte du ML sans mélanger les catégories

Le paysage ML : huit questions qui se combinent

Ces cases ne sont ni des branches exclusives ni un classement de performance.

1 · Quel résultat cherche-t-on ?
Classification : une classe ou un risque
Régression : une valeur continue
Regroupement : des groupes à explorer
Représentation : des variables utiles

2 · Quel signal d'apprentissage ?
Supervisé : des cibles fournies
Non supervisé : la structure des données
Auto-supervisé : une cible construite
Semi-supervisé ; renforcement (retour)

3 · Quel algorithme de base ?
Régressions, Ridge/Lasso, KNN, SVM/SVR
Arbre ; K-means, hiérarchie, DBSCAN
Naive Bayes, réseau bayésien, GMM, HMM
Le choix dépend des données et de la tâche.

4 · Quelle représentation ?
PCA, t-SNE, UMAP : réduire la dimension
Autoencodeur ; plongements, Word2Vec
Apprentissage contrastif et profond
Architectures : MLP, CNN, RNN, transformeur

5 · Comment combiner les modèles ?
Bagging → forêt aléatoire
Boosting → AdaBoost, gradient boosting
XGBoost, LightGBM, CatBoost : outils
Stacking : apprendre la combinaison

6 · Comment réutiliser ou adapter ?
Transfert : réutiliser des paramètres
Actif : choisir les exemples à annoter
Méta-apprentissage : apprendre à adapter
Attention au décalage entre domaines.

7 · Comment entraîner ?
Descente de gradient ; SGD
Adam, RMSprop ; taux d'apprentissage
Régularisation : L1/L2, dropout
Arrêt anticipé ; budget de calcul

8 · Comment évaluer le modèle ?
Entraînement / validation / test
Validation croisée sans fuite de données
Matrice de confusion ; F1, RMSE, AUC
Exactitude ; biais–variance ; coût des erreurs

Exemple · Estimer le risque d'interruption d'une liaison réseau
Cible : interruption dans les 5 prochaines minutes.
Classification supervisée + arbres + bagging → forêt aléatoire.

Le paysage ML organisé en huit questions complémentaires ; une forêt aléatoire combine plusieurs réponses, sans appartenir à une catégorie unique.

Lire la carte en trois passages. Commencer par la sortie attendue et le signal d'apprentissage disponible (1–2), choisir des modèles et des représentations compatibles (3–6), puis définir l'entraînement et le protocole d'évaluation (7–8). Les catégories se recouvrent parce qu'elles répondent à des questions différentes.

Exemple filé — anticiper une interruption de liaison. À partir des mesures disponibles, on estime le risque qu'une liaison réseau soit déclarée indisponible dans les cinq prochaines minutes. Dans cet exemple pédagogique, l'événement est une alarme déclenchée après au moins dix secondes d'indisponibilité. Des historiques horodatés fournissent la cible binaire « interruption / pas d'interruption » : c'est une classification supervisée.

Premier repère. Quelle est la réponse attendue, et quelles mesures seront disponibles au moment de la prédiction ? Les comparaisons entre algorithmes seront reprises après leurs explications.

Question	Notions à distinguer
Quelle sortie ?	classification : une classe ou ses probabilités ; régression : une valeur continue ; regroupement : des groupes sans classes fournies
Quel signal ?	supervisé : couples entrée–cible ; non supervisé : structure des observations ; auto-supervisé : cible construite à partir des données ; renforcement : retour d'interaction
Quelle représentation ?	variables choisies à la main, projection linéaire, plongements appris, représentations profondes
Quelle famille ?	modèles linéaires, voisins, arbres, modèles probabilistes, réseaux neuronaux
Quelle combinaison ?	bagging, boosting, stacking ; ces méthodes peuvent servir à la classification comme à la régression
Comment entraîner et évaluer ?	optimisation, régularisation, séparation des données, validation et métriques ; ces étapes concernent plusieurs familles de modèles

Ainsi, un autoencodeur désigne une architecture et un objectif de reconstruction, tandis que « auto-supervisé » désigne l'origine de la cible. Une forêt aléatoire peut être à la fois supervisée et ensembliste. Un modèle probabiliste peut apprendre avec ou sans étiquettes. L'apprentissage profond (*deep learning*) compose plusieurs transformations apprises ; il ne forme pas une catégorie exclusive du supervisé, du non supervisé ou du renforcement.

Avant d'apprendre : identifier une parabole

Ce problème de courbe distingue la **forme du modèle** et ses **paramètres**. Les valeurs de x et $f(x)$ sont exactes et sans dimension. On choisit la famille $f(x) = a\,x^2 + b\,x + c$, dont les coefficients a, b et c sont inconnus. Une parabole exige $a \neq 0$.

Trois points dont les **abscisses sont distinctes** déterminent un unique polynôme de degré au plus deux : chaque point fournit une équation sur les trois coefficients. Si les points sont alignés, le coefficient a vaut zéro et la solution est une droite, ou une constante, plutôt qu'une parabole. Deux points seulement laissent en général plusieurs solutions. L'unicité repose sur la famille choisie : sans cette hypothèse, beaucoup d'autres courbes peuvent passer par les trois points.

Deux racines et une valeur. Supposons que la courbe passe par $(-2, 0)$, $(1, 0)$ et $(0, -1)$. Les deux premières valeurs donnent les racines -2 et 1, donc la forme factorisée est

$$f(x) = a(x+2)(x-1).$$

Il reste un seul coefficient à identifier. En remplaçant x par zéro, on obtient $f(0)=a\times 2\times(-1)=-2a$. La troisième valeur impose $-2a=-1$, d'où $a=1/2$. Ainsi,

$$f(x)=\frac{1}{2}(x+2)(x-1)=\frac{1}{2}x^2+\frac{1}{2}x-1.$$

On peut contrôler les trois informations : les facteurs s'annulent à $x=-2$ et à $x=1$, puis $f(0)=-1$.

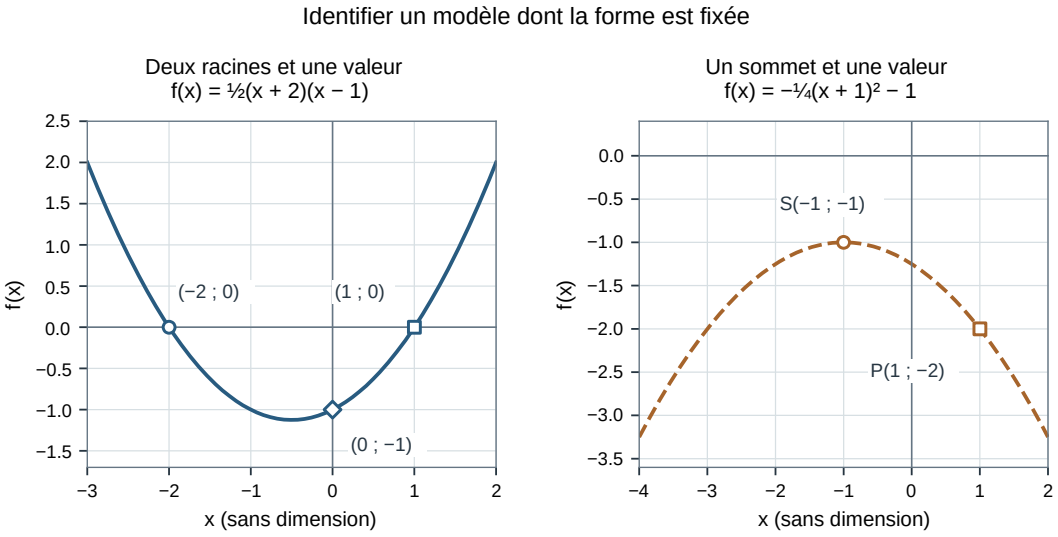
Un sommet et un point. Si le sommet a pour coordonnées $S(h,k)$, la forme devient $f(x)=a(x-h)^2+k$. Connaître le sommet fixe h et k, mais ne fixe pas a. Un point $P(x_0,y_0)$, avec $x_0\neq h$, donne alors

$$a=\frac{y_0-k}{(x_0-h)^2}.$$

Avec $S(-1,-1)$, on écrit $f(x)=a(x+1)^2-1$. Le point $P(1,-2)$ impose $a\times(1+1)^2-1=-2$, soit $4a=-1$, donc $a=-1/4$:

$$f(x)=-\frac{1}{4}(x+1)^2-1.$$

Le coefficient négatif ouvre la parabole vers le bas ; son sommet vaut bien $f(-1)=-1$, et $f(1)=-2$. Un point situé au sommet ne donne aucune information supplémentaire sur a. Si $x_0\neq h$ mais $y_0=k$, la formule donne $a=0$: les données ne définissent alors pas une parabole non dégénérée.



Les symboles marquent les informations fournies, les courbes la solution.

Deux paraboles identifiées par des informations exactes : deux racines et une troisième valeur à gauche ; un sommet et un point hors sommet à droite.

Ces deux calculs relèvent de l'algèbre, sans apprentissage automatique. **Connaître la forme** permet de chercher les paramètres ; **connaître déjà les paramètres** permet d'évaluer la fonction. Avec des mesures bruitées, passer exactement par chaque point peut reproduire le bruit. On recueille alors davantage d'observations et on ajuste les coefficients pour réduire une erreur globale : c'est une **régression**, plutôt qu'une interpolation exacte. Une famille plus souple se choisit selon les données et le phénomène ; l'incertitude sur une fonction ne justifie pas à elle seule un réseau profond.

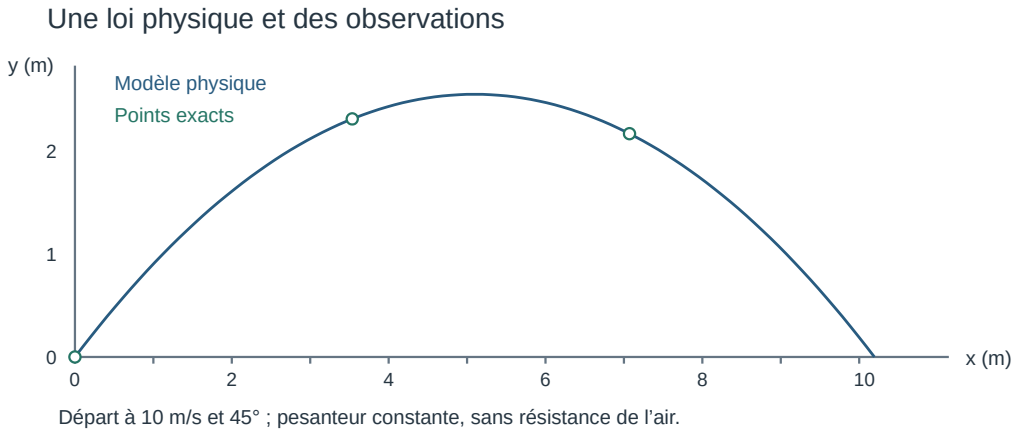
Une trajectoire : calculer une loi ou apprendre une approximation

Le vol balistique d'un missile illustre le choix entre un modèle physique et un modèle appris. Pour en refaire les calculs, prenons une **balle lancée**, avec une origine au point de départ, un axe horizontal x, un axe vertical y vers le haut et le temps t en secondes.

En négligeant la résistance de l'air et en supposant la pesanteur constante, la mécanique donne

$$x(t) = v_0 \cos\theta\, t\,,\; y(t) = v_0 \sin\theta\, t - \frac{1}{2} g\, t^2.$$

v_0 est la vitesse initiale en mètres par seconde ; θ, l'angle de départ ; $g = 9{,}81\ m/s^2$, l'accélération choisie pour cet exemple. Avec $v_0 = 10\ m/s$ et $\theta = 45^\circ$, les deux composantes initiales valent environ $7{,}07\ m/s$. À $t = 1$ seconde, on obtient $x \simeq 7{,}07$ m et $y \simeq 7{,}07 - 4{,}905 = 2{,}17$ m. Chaque terme de la seconde équation est bien une longueur.



Position d'une balle calculée par le modèle physique ; des observations illustratives permettent aussi d'ajuster une courbe. Les points sont des données pédagogiques, pas des mesures d'un système réel.

Un calcul **analytique** évalue cette formule. Un calcul **numérique** peut intégrer des équations physiques plus complexes par petits pas. Les deux emploient des lois explicites. Un apprentissage peut, lui, ajuster une fonction à des observations de position. Pour cette balle, une régression quadratique suffit à représenter $y(t) = a\,t^2 + b\,t + c$. Avec des données exactes aux instants 0, 0,5 et 1, on retrouve $a = -4{,}905$, $b \simeq 7{,}071$ et $c = 0$. Avec du bruit, on utilise davantage de points et on mesure l'erreur sur des observations conservées à part.

Ce sont **deux axes distincts** : loi physique ou fonction apprise, puis famille de modèle. Le *deep learning* est une famille du *machine learning*, pas le nom de tout calcul dont la loi est inconnue. Un arbre, une régression ou un réseau profond peuvent apprendre à partir de données. Une discontinuité n'impose pas un réseau profond.

La formule extrapole selon ses hypothèses ; le modèle appris doit faire ses preuves hors des données d'entraînement. Introduire la résistance de l'air rend la parabole précédente insuffisante. Une approche **hybride** peut apprendre une correction au modèle physique. Les PINN ajoutent, pendant l'apprentissage, une pénalité pour l'écart aux équations retenues. Cette pénalité aide à respecter la physique choisie ; elle ne constitue pas une preuve que toute prédiction sera exacte.

Prédire une valeur ou une classe

Régression linéaire : une première référence

La **régression linéaire** ajuste une combinaison des variables pour minimiser une erreur quadratique. Pour estimer un délai de transmission, on peut commencer par utiliser la charge, la taille des paquets et le nombre de sauts :

$$\hat{y} = b + w^\top x, J(w, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - b - w^\top x_i)^2.$$

x_i rassemble les mesures de l'observation i, y_i son délai mesuré et n le nombre d'observations. Les coefficients w et la constante b sont ajustés ; $w^\top x_i$ est leur somme pondérée. $\hat{y}$ est la prédiction. J est l'erreur quadratique moyenne que l'on cherche à réduire.

Elle fournit un modèle de comparaison simple à entraîner et à interpréter, mais extrapole selon la forme choisie et reste sensible aux observations extrêmes. « Linéaire » porte sur les coefficients : on peut ajouter des interactions ou des transformations de variables. L'indépendance statistique des variables explicatives n'est pas requise ; leur colinéarité peut rendre les coefficients instables. Les hypothèses sur les erreurs nécessaires aux intervalles statistiques ne se confondent pas avec le calcul d'une prédiction.

Ridge et Lasso : régulariser les coefficients

Avant ces pénalités, lisons les notations : $\| w \|_1$ additionne les valeurs absolues des coefficients et $\| w \|_2^2$ additionne leurs carrés. Le coefficient λ règle la force de la pénalité.

Ridge ajoute une pénalité $\lambda \| w \|_2^2$, **Lasso** une pénalité $\lambda \| w \|_1$. La première stabilise les coefficients ; la seconde peut en annuler. Standardiser les variables dans les données d'entraînement rend ces pénalités comparables. Leur intensité se choisit à l'aide du jeu de validation, avec le compromis biais–variance (scikit-learn developers 2026k).

La pénalité ridge possède aussi une lecture probabiliste, présentée après les distributions.

Lire « sachant » avant d'utiliser Bayes

Lire « sachant » avant d'utiliser Bayes. La barre $|$ indique les informations que l'on suppose connues. $P(B \mid A)$ est la proportion de cas B **parmi les cas A**, et non parmi toutes les observations. Prenons 1 000 transmissions fictives, avec A = « liaison saturée » et B = « paquet perdu » :

Situation	Paquet perdu (B)	Paquet reçu	Total
Liaison saturée (A)	20	80	100
Liaison non saturée	10	890	900
Total	30	970	1 000

Parmi les 100 transmissions en saturation, 20 ont perdu leur paquet : $P(B \mid A) = 20/100 = 20\,\%$. Parmi les 30 transmissions ayant perdu leur paquet, 20 étaient en saturation : $P(A \mid B) = 20/30 \approx 66{,}7\,\%$. Les mêmes données donnent deux réponses différentes parce que le groupe de référence change. La perte sur toutes les transmissions vaut, elle, $P(B) = 30/1\,000 = 3\,\%$. **Conditionner n'établit pas une cause** : ce tableau ne prouve pas, à lui seul, que la saturation a provoqué les pertes. La formule de Bayes relie ces deux lectures, lorsque $P(A) > 0$ et $P(B) > 0$:

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B)}.$$

Ici, $P(A) = 100/1\,000 = 0{,}1$, donc $P(A \mid B) = 0{,}2 \times 0{,}1 / 0{,}03 = 2/3$. On retrouve les 20 transmissions saturées parmi les 30 paquets perdus.

Régression bayésienne : représenter l'incertitude

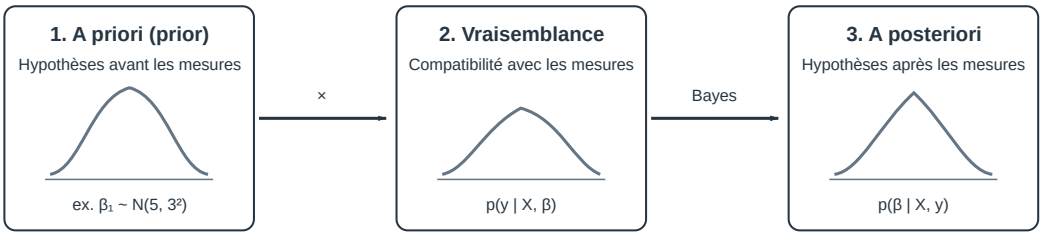
Régression bayésienne : apprendre une distribution de coefficients. Reprenons le délai y en fonction de la charge x : $y = \beta_0 + \beta_1 x + \varepsilon$. L'ordonnée à l'origine β_0 et la pente β_1 sont les coefficients ; ε représente les variations que la droite n'explique pas. X rassemble les entrées mesurées et y les délais observés.

Avant les mesures, une distribution **a priori (prior)** décrit les valeurs envisagées pour ces coefficients. La **vraisemblance** $p(y \mid X, \beta)$ indique dans quelle mesure chaque choix de coefficients explique les données, selon le modèle de bruit retenu. Bayes combine les deux en une distribution **a posteriori (posterior)** :

$$p(\beta \mid X, y) \propto p(y \mid X, \beta)\, p(\beta).$$

Le symbole $\propto$ signifie « proportionnel à » : on multiplie l'a priori par la vraisemblance, puis on normalise le résultat pour obtenir une distribution. Pour ces coefficients et délais continus, p désigne une **densité** : une probabilité sur un intervalle s'obtient par l'aire sous la courbe, et non par sa hauteur en un point.

Régression linéaire bayésienne : de l'a priori à la prédiction



A posteriori ∝ Vraisemblance × A priori

Prédire : combiner l'incertitude sur les coefficients et le bruit des observations.

Régression bayésienne : combiner une distribution a priori sur les coefficients et la vraisemblance des observations pour obtenir une distribution a posteriori.

La notation $N(\mu, \sigma^2)$ désigne une loi normale de moyenne μ, d'écart-type σ et de variance σ^2. Le symbole ~ se lit ici « suit la loi ».

Dans cet exemple illustratif, $\beta_1 \sim N(5, 3^2)$ signifie une moyenne de 5 et un écart-type de 3 pour la pente, dans les unités choisies : ce 3 n'est pas le bruit des mesures. Les courbes du schéma montrent le principe, sans représenter un ajustement numérique. Pour prédire un nouveau délai, on combine l'incertitude restante sur les coefficients **et** le bruit d'observation. On obtient une distribution prédictive, dont la pertinence dépend des hypothèses et des données (scikit-learn developers 2026k).

Régression logistique : estimer un risque de classe

La **régression logistique** est un classifieur malgré son nom. En binaire :

$$p(y = 1 \mid x) = \frac{1}{1 + \exp[-(b + w^\top x)]}.$$

Ses paramètres sont ajustés en minimisant une perte logarithmique ; elle peut, par exemple, estimer un risque de saturation. Les log-cotes sont linéaires dans les variables retenues ; une probabilité produite n'est pas automatiquement calibrée. Le seuil de décision dépend du coût des fausses alertes et des incidents manqués. Une extension multinomiale traite plusieurs classes (scikit-learn developers 2026l).

SVM et SVR : marge et tolérance

SVM et SVR. Une machine à vecteurs de support (**SVM**) cherche une marge large entre classes. La marge souple autorise des violations, réglées par C : les données n'ont donc pas besoin d'être parfaitement séparables. Un noyau représente des frontières non linéaires. En régression, **SVR** emploie une zone de tolérance ε autour de la prédiction et pénalise les écarts au-delà. On peut comparer ces modèles pour classifier des signatures radio ou prévoir un délai. Échelle des variables, noyau, C et ε comptent ; l'entraînement à noyau peut devenir coûteux sur de nombreuses observations. Le score SVM n'est pas directement une probabilité (scikit-learn developers 2026s).

K plus proches voisins : comparer aux cas connus

Les **K plus proches voisins (KNN)** font voter les observations voisines pour une classe, ou moyennent leurs valeurs pour une régression. Comparer un profil de trafic aux profils étiquetés les plus proches est une référence simple. Il faut pourtant choisir distance, variables, échelle et K, conserver les données et rechercher les voisins à l'inférence. En grande dimension, les distances peuvent devenir peu discriminantes. L'absence d'ajustement de poids ne signifie ni absence de préparation, ni aptitude générale à apprendre avec quelques exemples (scikit-learn developers 2026p).

Arbre de décision : suivre une suite de questions

Un **arbre de décision** pose des questions successives : par exemple « taux de pertes élevé ? », puis « charge élevée ? ». À la fin d'un chemin, une **feuille** donne une classe, des fréquences de classes ou une valeur de régression. Un petit arbre est lisible et exige peu de mise à l'échelle. Pour éviter qu'il mémorise les données, on limite sa profondeur, on impose un nombre minimal d'observations par feuille ou on supprime certaines branches : c'est l'**élagage**.

L'arbre n'impose pas une relation linéaire, mais sa façon de chercher oriente ce qu'il peut apprendre. Un découpage **glouton** retient le meilleur choix immédiat, sans examiner toutes les suites possibles ; la représentation des variables compte aussi. Ces choix constituent son **biais inductif** (scikit-learn developers 2026c).

Compter des événements ou estimer un seuil

Une régression ne vise pas toujours une moyenne dans des unités continues. La **régression de Poisson** convient à des comptages, par exemple le nombre de pannes par relais et par journée. Elle relie les variables explicatives à un nombre moyen positif, généralement par une fonction exponentielle. Les durées d'observation doivent être comparables ou prises en compte : dix incidents en une heure et dix en un mois ne décrivent pas le même taux. Le modèle de Poisson suppose notamment une variance conditionnelle égale à la moyenne ; une dispersion plus forte invite à revoir cette hypothèse (scikit-learn contributors 2026c).

Une **forêt de régression quantile** cherche des quantiles conditionnels, au-delà de la moyenne prédite par une forêt ordinaire. Pour un délai, le quantile à 90 % est un seuil au-dessous duquel le modèle situe environ 90 % des délais, pour les variables d'entrée considérées. Ce n'est ni « 90 % de précision » ni une garantie de respect pour chaque requête. La méthode conserve davantage d'information sur les observations dans les feuilles des arbres, puis estime leur distribution pondérée (Meinshausen 2006). La couverture de ces seuils se vérifie sur des données indépendantes, notamment lorsque le trafic change.

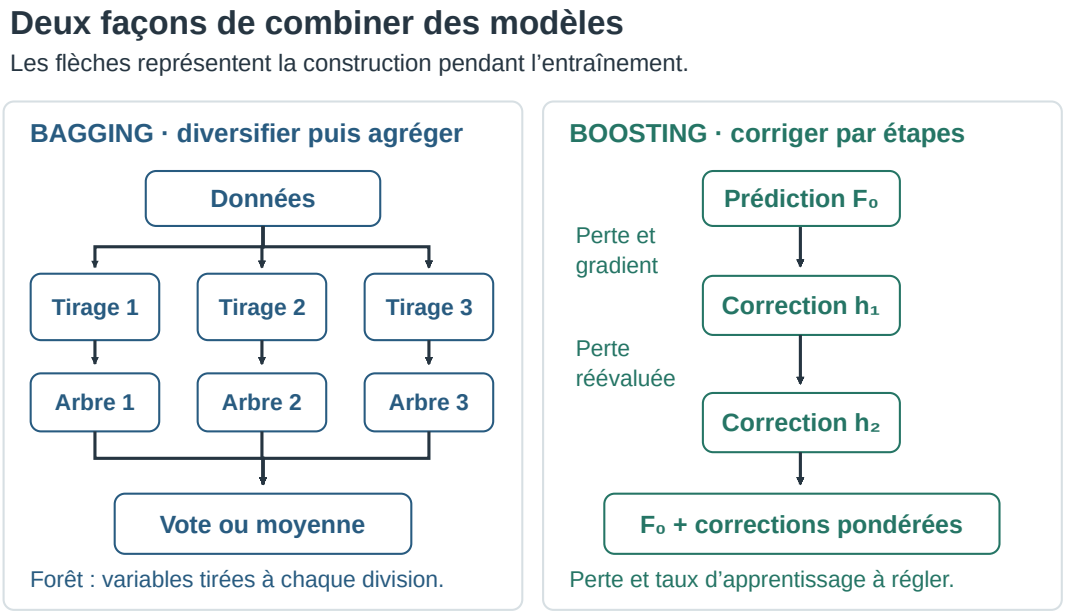
Combiner des modèles : agrégation et corrections successives

Le **bagging** entraîne plusieurs modèles sur des échantillons tirés au hasard dans les données d'entraînement, puis combine leurs prédictions. Ces tirages **bootstrap** permettent de choisir plusieurs fois la même observation. La **forêt aléatoire (Random Forest)** ajoute une sélection aléatoire de variables candidates aux divisions des arbres. Elle constitue un modèle de comparaison pour des indicateurs réseau présentés en tableau, en classification comme en régression. Ses arbres partagent néanmoins des données et peuvent rester corrélés : leur indépendance statistique n'est pas une hypothèse requise.

Le **boosting** combine des modèles entraînés successivement pour corriger les erreurs des précédents. **AdaBoost** augmente notamment le poids des exemples mal classés ; des étiquettes erronées peuvent alors peser lourd. Le **gradient boosting** entraîne chaque nouveau modèle à proposer une correction qui réduit la perte :

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x).$$

Ici F_m est la prévision après l'étape m, h_m la correction proposée par le nouveau modèle et η règle la taille de la correction ajoutée. h_m approxime le gradient négatif de la perte par rapport aux prédictions : celui-ci indique une direction locale de baisse de la perte. Pour la perte quadratique, cette direction est proportionnelle aux **résidus**, les écarts entre valeurs observées et prévisions actuelles. Le nombre d'étapes, la complexité des arbres et le pas η se règlent ensemble. Ces méthodes peuvent améliorer une prévision de congestion, sans garantir la meilleure performance sur tout jeu de données (scikit-learn developers 2026f).



Agrégation de modèles entraînés séparément et construction séquentielle de corrections ; les flèches décrivent l'entraînement.

Trois outils appartiennent à la famille du boosting d'arbres :

- **XGBoost** associe notamment objectif régularisé et calcul efficace des corrections ; c'est un candidat pour des KPI tabulaires et des interactions non linéaires (XGBoost developers 2026).
- **LightGBM** emploie des histogrammes et développe en priorité la feuille dont la division réduit le plus la perte ; limiter le nombre de feuilles et imposer un effectif minimal par feuille aide à maîtriser le surapprentissage, particulièrement sur un petit jeu de données (LightGBM developers 2026).
- **CatBoost** propose un traitement des variables catégorielles et un boosting ordonné pour réduire certaines fuites d'information sur la cible. Un modèle peut toutefois exploiter un identifiant d'équipement pour reconnaître un site sans apprendre une relation généralisable : il reste nécessaire de séparer les sites entre entraînement et évaluation (Prokhorenkova et al. 2018).

Une forêt aléatoire ou un modèle de boosting peuvent convenir à une application en temps réel : il faut mesurer latence, mémoire et débit de prédiction sur le matériel visé, avec le nombre et la profondeur d'arbres retenus. Le **stacking**, déjà introduit parmi les méthodes d'ensemble, ajoute un modèle qui combine les prédictions des modèles de base. Pour l'entraîner, chaque prédiction doit porter sur une observation qui n'a pas servi à ajuster le modèle de base qui la produit.

Regrouper sans étiquettes

Avant de choisir un regroupement, regarder la forme des observations, leur échelle et le sens d'une distance. Les méthodes suivantes ne font pas les mêmes hypothèses.

K-means : représenter chaque groupe par un centre

- **Principe** : alterner l'affectation au centre le plus proche et le recalcul des centres pour réduire la somme des distances euclidiennes au carré.
- **Usage réseau** : regrouper, par exemple, des profils journaliers de trafic.
- **Réglages** : choisir K, mettre les variables à une échelle pertinente et essayer plusieurs initialisations.
- **Limites** : la méthode favorise des groupes compacts. Des formes allongées, des tailles ou des densités très différentes peuvent rendre la partition peu utile. Des groupes exactement sphériques et égaux ne sont pas une condition formelle d'exécution.

Ce que minimise K-means. Avec k groupes $C_1, \ldots, C_k$, le centre μ_j représente le groupe j. Une écriture de l'objectif et de la mise à jour des centres est :

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \| x_i - \mu_j \|^2, \mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

x_i est un vecteur d'observations ; $\| \cdot \|$ sa distance euclidienne ; $|C_j|$ le nombre de points du groupe non vide. On affecte chaque point au centre le plus proche, puis on remplace chaque centre par la moyenne des points affectés. Pour les valeurs 2 et 4, le centre devient 3. Ces opérations sont répétées jusqu'à un critère d'arrêt. L'initialisation peut conduire à des solutions différentes ; un groupe vide exige un traitement explicite (scikit-learn developers 2026i).

Classification hiérarchique : choisir un niveau de détail

- **Principe** : le regroupement agglomératif fusionne des groupes selon une distance et une règle de liaison.
- **Intérêt** : son **dendrogramme** montre plusieurs niveaux de granularité, par exemple dans des profils d'équipements. Construire l'arbre n'exige pas de fixer K ; obtenir une partition exige de choisir son niveau de coupe.
- **Limites** : le coût en mémoire, la règle de liaison et l'effet de chaînage peuvent limiter l'usage.

DBSCAN : suivre des régions denses et repérer le bruit

- **Principe** : relier des régions suffisamment denses, définies par un rayon ε et un nombre minimal de voisins. Les points non rattachés à un groupe sont marqués comme « bruit ».
- **Intérêt** : accepter des groupes non convexes sans fixer leur nombre.
- **Limites** : des densités très variables ou des distances peu informatives peuvent mettre la méthode en défaut. Un point déclaré « bruit » n'est pas, de ce seul fait, une attaque (scikit-learn developers 2026a).

Trois autres façons de regrouper

Carte auto-organisatrice : rapprocher les profils sur une grille

- **Principe** : une **carte auto-organisatrice** (*Self-Organizing Map*, SOM ou carte de Kohonen) associe un vecteur représentatif à chaque case d'une grille. Pour une observation, la case dont le vecteur est le plus proche « gagne » ; son vecteur et ceux des cases voisines se rapprochent de cette observation. Le voisinage et l'amplitude des ajustements diminuent au cours de l'apprentissage.
- **Intérêt** : cette compétition construit une carte où des profils semblables tendent à occuper des cases proches.
- **Limite** : la grille est une représentation exploratoire ; elle ne garantit pas une partition vraie des équipements (Kohonen 1982).

Mean Shift : chercher les pics de densité

- **Principe** : déplacer un point vers la moyenne, éventuellement pondérée, des observations de son voisinage, puis recommencer. Les trajectoires convergeant vers un même pic de densité sont regroupées.

- **Intérêt** : le nombre de groupes apparaît sans choisir K à l'avance.
- **Réglages et limites** : une largeur de voisinage trop grande fusionne des pics ; trop petite, elle fragmente les groupes. Le calcul peut devenir coûteux sur un grand corpus (Comaniciu et Meer 2002).

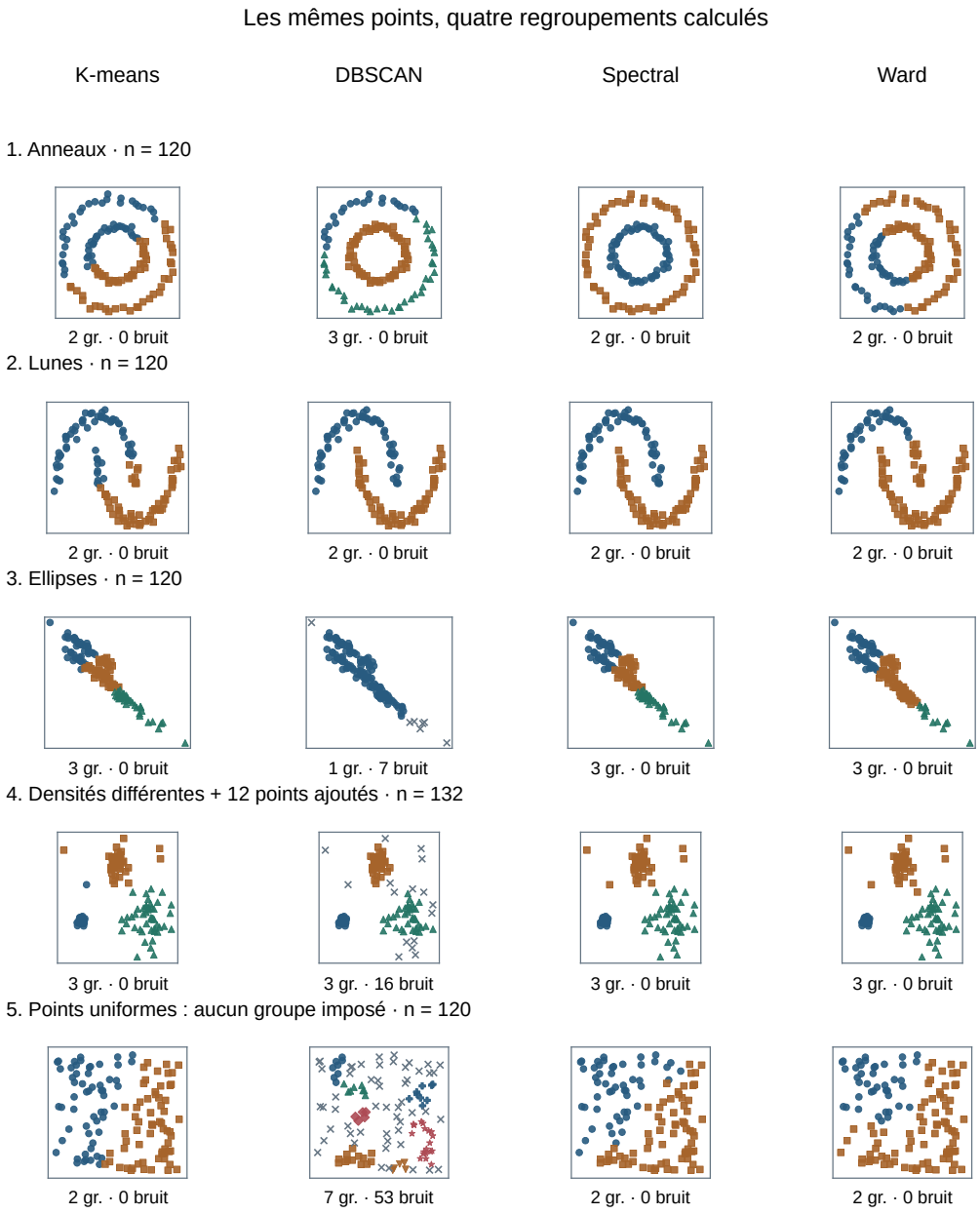
Clustering spectral : changer de représentation

- **Principe** : construire un graphe d'affinité dont les observations sont les sommets et les poids mesurent leur ressemblance. Les vecteurs propres d'une matrice issue du graphe fournissent une nouvelle représentation ; on y applique un regroupement, souvent K-means.
- **Intérêt** : des groupes difficiles à séparer dans les coordonnées initiales peuvent alors se distinguer.
- **Réglages et limites** : le résultat dépend du graphe, de ses poids et du nombre de groupes demandé. « Spectral » désigne ici les valeurs et vecteurs propres, sans nécessiter un signal radio (Ng et al. 2001).

Comparer des regroupements sur les mêmes points

Les mécanismes précédents deviennent plus visibles lorsque plusieurs méthodes reçoivent exactement les mêmes observations. La figure suivante a été calculée pour cet ouvrage, avec quatre méthodes et cinq jeux synthétiques à deux dimensions. Elle reprend le principe de comparaison sur géométries variées de la documentation scikit-learn, avec des données et des réglages propres à cette expérience (The scikit-learn developers 2026). Les étiquettes produites lors de la création des jeux ne sont jamais fournies aux méthodes.

Les deux coordonnées sont sans dimension. Dans chaque jeu, on centre chaque coordonnée sur sa moyenne puis on la divise par son écart-type ; cette préparation est identique pour les quatre méthodes. Chaque case d'une ligne contient les mêmes points, au même emplacement et à la même échelle. Couleurs et symboles représentent les groupes **calculés**. Ils n'imposent pas de correspondance entre deux méthodes ; une croix grise signale un point déclaré bruit par DBSCAN.



Vingt regroupements effectivement calculés : K-means, DBSCAN, clustering spectral et regroupement agglomératif de Ward, sur cinq jeux identiques entre les colonnes. « gr. » indique le nombre de groupes hors bruit. La dernière ligne ne contient aucune structure de groupes imposée lors de sa génération.

Conditions du calcul. La graine principale est 20261006 ; le jeu uniforme utilise la graine suivante. L'exécution repose sur Python 3.13.3, NumPy 2.3.3, SciPy 1.15.2, scikit-learn 1.7.2, Matplotlib 3.10.8 et threadpoolctl 3.6.0. n est le nombre de points, K le nombre de groupes demandé aux méthodes qui l'exigent et ε le rayon de DBSCAN dans les coordonnées standardisées.

Géométrie	n	K demandé	ε
Anneaux bruités	120	2	0,48
Deux lunes bruitées	120	2	0,40

Géométrie	n	K demandé	ε
Trois nuages étirés en ellipses	120	3	0,30
Densités différentes et points ajoutés	132	3	0,32
Points uniformes	120	2	0,30

K-means utilise 20 initialisations, au plus 300 itérations et une tolérance de 10^{-4}. DBSCAN demande au moins cinq points dans le voisinage, point lui-même compris. Le calcul spectral utilise un graphe de huit plus proches voisins, le solveur ARPACK et dix initialisations de sa partition finale. Ward utilise la distance euclidienne. Les rayons sont choisis pour cette illustration ; ils ne constituent pas des réglages conseillés pour un réseau réel.

Ce que montre cette exécution. Sur les anneaux, K-means et Ward coupent les couronnes, tandis que le calcul spectral les distingue. DBSCAN produit ici trois groupes : une partie de la couronne extérieure est séparée avec ce rayon et ce nombre de points. Le dessin rend donc visible la sensibilité au voisinage, même pour une forme adaptée au principe de densité.

Sur les nuages étirés, DBSCAN relie presque tout en un groupe et laisse sept points en bruit. Sur les densités différentes, il conserve trois groupes et marque 16 points en bruit ; les 12 points ajoutés à la génération ne sont pas une vérité terrain qu'il aurait exactement retrouvée. Le calcul spectral émet, sur ce même jeu, un avertissement de graphe non entièrement connecté, conservé avec la figure.

La dernière ligne est un contrôle utile : K-means, le calcul spectral et Ward fournissent les deux groupes demandés, et DBSCAN trouve sept groupes locaux avec 53 points en bruit. Une partition produite par un programme ne prouve donc pas l'existence de familles stables ou utiles. Cette comparaison illustre des effets de géométrie et de paramètres sur un petit exemple ; elle ne classe pas les méthodes pour des télémétries réelles à nombreuses dimensions.

Une variation à essayer. Sur un jeu synthétique d'anneaux construit pour l'essai, modifier seulement le rayon, puis la graine de génération, permet de voir si la séparation observée persiste. Les temps consignés correspondent à un seul ajustement par case, avec les bibliothèques numériques limitées à un fil de calcul ; imports, génération, préparation et dessin en sont exclus. Ils documentent cette exécution. Une comparaison de performances demande des répétitions et des volumes représentatifs sur une machine décrite.

Prévoir une série et repérer des observations atypiques

Séries temporelles et isolement d'anomalies

ARMA et ARIMA : prévoir à partir du passé

ARMA (*AutoRegressive Moving Average*) combine des valeurs passées de la série et des erreurs de prévision passées. « Moyenne mobile » désigne ici cette combinaison des innovations, pas une simple moyenne glissante des mesures. Le modèle vise une série stationnaire, dont les propriétés statistiques pertinentes ne changent pas avec le temps. **ARIMA** ajoute une différenciation : remplacer, par exemple, chaque mesure par son écart à la précédente pour traiter certaines tendances. Dans $ARIMA(p,d,q)$, p est l'ordre autorégressif, d le nombre de différenciations et q l'ordre de moyenne mobile. Un changement brusque de régime réseau peut invalider les hypothèses ; saisonnalité et variables explicatives demandent des extensions (NIST/SEMATECH 2026a).

Isolation Forest : mesurer la facilité d'isolement

Une **forêt d'isolement** (*Isolation Forest*) recherche les observations faciles à séparer : chaque arbre choisit une variable et une coupure aléatoires, puis recommence sur les sous-ensembles. Un point isolé après peu de coupures reçoit un score plus atypique. Plusieurs arbres réduisent la dépendance à un seul tirage. Cette méthode ne nécessite pas d'étiquettes d'incident ; elle ne transforme pas une observation rare en preuve d'attaque ou de panne (Liu et al. 2008).

Apprendre le fonctionnement habituel : le SVM à une classe

Le **SVM à une classe** (*One-Class SVM*) apprend une frontière autour d'observations représentant le fonctionnement habituel. Un nouveau point hors de cette région est signalé comme inhabituel ; cela ne suffit pas à identifier une panne ou une attaque. Pour la détection de nouveautés, l'ensemble d'entraînement est supposé surtout composé d'observations normales. La mise à l'échelle des variables et les paramètres de la frontière comptent. Un SVM supervisé sépare, lui, des classes déjà étiquetées ; Isolation Forest exploite la facilité d'isolement des points. Ces différences d'hypothèses guident la comparaison (scikit-learn contributors 2026b).

Recommander : rapprocher des contenus ou des usages

Un système de recommandation propose des éléments : vidéos, documents, produits ou actions à examiner. Le **filtrage par contenu** rapproche leurs caractéristiques de celles appréciées par une personne. Le **filtrage collaboratif** exploite des interactions entre utilisateurs et éléments. Un système hybride combine les deux.

Dans une matrice utilisateurs–éléments, les cases peuvent contenir des notes ou des interactions. Une case vide ne signifie pas « mauvaise note » : l'élément peut simplement ne pas avoir été rencontré. La **factorisation matricielle** apprend un petit vecteur pour chaque utilisateur et chaque élément ; leur produit scalaire estime une affinité. Elle se rapproche de l'idée de représentation compacte rencontrée avec la SVD, mais l'apprentissage doit tenir compte des cases réellement observées et de la régularisation. Remplir tous les manques par zéro puis appliquer une SVD ordinaire change le problème (Apache Software Foundation 2026b).

Les clics indiquent un comportement, sans prouver une préférence certaine. Un nouvel utilisateur ou un nouvel élément pose le problème du **démarrage à froid** ; les attributs de contenu peuvent alors aider. Une évaluation chronologique compare les recommandations à des interactions ultérieures, avec une référence simple comme les éléments populaires. La popularité, la diversité et les effets de retour — ce qui est montré influence ce qui sera cliqué — restent à examiner.

Systèmes experts : rendre une connaissance explicite et applicable

Problème. Certaines décisions reposent sur des connaissances métier déjà formulables, sans corpus d'exemples étiquetés suffisant pour les apprendre. Un **système expert à règles** sépare les faits observés, les règles et le moteur d'inférence qui applique les règles pertinentes (CLIPS 2026).

```
fait : interface administrativement activée
fait : porteuse absente
règle : si ces deux faits sont établis,
        alors ajouter « liaison physique à examiner » aux hypothèses
```

En **chaînage avant**, des faits déclenchent des règles, dont les conclusions peuvent alimenter d'autres règles. Un chaînage arrière part d'un but et recherche les conditions permettant de l'établir. Une trace des règles appliquées aide à expliquer une conclusion. Dans l'exemple, « liaison à examiner » n'affirme pas que le câble est la cause certaine : plusieurs défauts peuvent supprimer la porteuse.

Verrou levé et limite. L'expertise explicite devient réutilisable et inspectable. Mais extraire cette connaissance, gérer les exceptions et maintenir des règles cohérentes lorsque le réseau change reste difficile. L'absence d'un fait ne signifie pas sa fausseté : une mesure manquante doit pouvoir conduire à « inconnu ». La priorité entre règles concurrentes et le traitement des faits périmés doivent être définis. La traçabilité d'un raisonnement ne prouve pas la justesse de ses prémisses.

Autre mécanisme : explorer des possibilités. Une recherche dans un arbre énumère des choix et leurs suites possibles, puis utilise un coût, une heuristique ou une borne pour guider ou réduire l'exploration. Pour préparer une configuration, elle peut éliminer les suites incompatibles avec une contrainte. Elle n'exige pas nécessairement un apprentissage ; une heuristique peut être écrite par un expert ou apprise. Le verrou est l'exploration d'un espace de choix ; la limite reste notamment son explosion combinatoire. Les garanties d'optimalité dépendent de l'algorithme et de ses hypothèses.

Comprendre un neurone et un réseau multicouche

Du seuil linéaire aux interactions : comprendre XOR

Problème. Un perceptron simple calcule une somme pondérée des entrées et la compare à un seuil. Avec deux entrées, sa frontière de décision est une droite. Cette représentation suffit pour certaines séparations, mais pas pour toutes les interactions.

Considérons deux indicateurs binaires de disponibilité de chemins, x_1 et x_2. On veut détecter « exactement un chemin disponible », autrement dit une perte de redondance sans coupure totale. C'est la fonction **XOR**, ou « ou exclusif » :

Chemin 1	Chemin 2	Exactement un disponible
0	0	0
0	1	1
1	0	1
1	1	0

Dans le plan, les deux cas positifs occupent des coins opposés du carré. Aucune droite ne les sépare des deux cas négatifs. Ajouter des exemples d'entraînement ne lève pas cette impossibilité de représentation.

Mécanisme. Des unités intermédiaires peuvent calculer « au moins un chemin disponible » et « les deux disponibles ». Une dernière unité retient le premier résultat et exclut le second : XOR devient représentable par composition. Les activations non linéaires sont décisives : empiler seulement des transformations affines donne encore une transformation affine.

Verrou levé et limite. Des couches non linéaires peuvent représenter des interactions inaccessibles au seuil linéaire dans les variables initiales. Pour ces deux indicateurs connus, une règle XOR écrite directement reste la solution la plus simple ; le réseau neuronal sert ici à comprendre la capacité de représentation (scikit-learn developers 2026q).

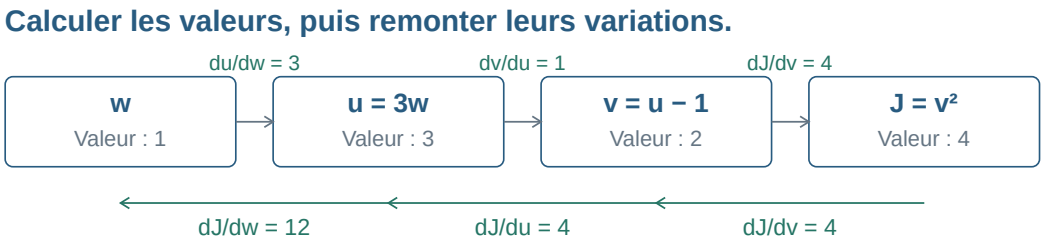
Un **perceptron multicouche (MLP)** compose des transformations affines et des activations non linéaires, par exemple $h = \sigma(W x + b)$. **Affine** signifie « somme pondérée plus décalage » : x rassemble les entrées, W les poids, b les biais et h les réponses des neurones. L'activation σ s'applique à chaque composante. La couche de sortie et la fonction de perte dépendent de la tâche : classification ou régression.

Un MLP peut exploiter des interactions entre KPI. Mise à l'échelle, capacité et régularisation se choisissent en comparant le modèle à des références simples. Le théorème d'approximation universelle précise ce qu'une seule couche cachée suffisamment large peut représenter ; cette existence ne garantit pas la réussite de l'apprentissage (Cybenko 1989). Ses hypothèses sont détaillées dans Un MLP à une couche cachée : approximation et apprentissage.

Entraîner : objectif, gradient et régularisation

Comment la machine calcule-t-elle une dérivée ?

Reprenons une petite fonction de coût : $J(w) = (3w - 1)^2$. En $w = 1$, on calcule successivement $u = 3w = 3$, $v = u - 1 = 2$, puis $J = v^2 = 4$. Ces opérations forment un **graphe de calcul** : une flèche indique quel résultat alimente l'opération suivante.



Au retour : 1 × 4 × 1 × 3 = 12. Les valeurs du passage aller sont réutilisées.

Le graphe de J(w) = (3w − 1)² : valeurs à l'aller et dérivées au retour.

Les dérivées locales sont 3 pour $u=3w$, 1 pour $v=u-1$ et $2v=4$ pour $J=v^2$. La règle de la chaîne les multiplie :

$$\frac{dJ}{dw}=\frac{dJ}{dv}\frac{dv}{du}\frac{du}{dw}=4\times1\times3=12.$$

La **différentiation automatique** applique ces règles aux opérations du programme et réutilise leurs valeurs. Dans le mode inverse illustré ici, elle remonte depuis la sortie ; si plusieurs chemins contribuent à un paramètre, leurs dérivées s'additionnent. La rétropropagation emploie ce mécanisme à travers les couches d'un réseau (Baydin et al. 2018).

On peut aussi transformer une formule à la main : c'est la dérivation symbolique. Ou estimer la pente en comparant deux valeurs proches : avec $h=0,01$, $(J(1+h)-J(1))/h=12,09$. Cette **différence finie** dépend du pas et des arrondis. La différentiation automatique compose les règles locales, avec les limites de précision du calcul numérique. Le même principe intervient dans les PINN : on dérive la sortie par rapport à l'entrée, par exemple le temps, pour calculer l'écart à l'équation physique.

Rétropropager : calculer la sensibilité de l'erreur aux poids

Problème. La sortie d'un réseau peut être fausse alors que des milliers de paramètres intermédiaires ont contribué au calcul. Modifier chaque poids au hasard ou tester séparément chaque petite perturbation devient coûteux.

Lire le calcul d'une couche. Numérotons les couches de 1 à L, la couche L étant la sortie ; les précédentes sont les **couches cachées**. Dans la notation locale suivante, $a_j^{(\ell)}$ est la réponse, ou **activation**, du neurone j de la couche ℓ. L'exposant entre parenthèses désigne la couche, pas une puissance. L'entrée est $a^{(0)}=x$. Si $n_{\ell-1}$ est le nombre de neurones de la couche précédente :

$$z_j^{(\ell)}=b_j^{(\ell)}+\sum_{i=1}^{n_{\ell-1}} w_{ji}^{(\ell)}a_i^{(\ell-1)},\quad a_j^{(\ell)}=\phi_\ell\left(z_j^{(\ell)}\right).$$

Le poids $w_{ji}^{(\ell)}$ relie le neurone précédent i au neurone courant j. Le **biais** $b_j^{(\ell)}$ décale la somme $z_j^{(\ell)}$. La fonction d'activation ϕ_ℓ transforme cette somme en réponse.

Pour une observation de régression, choisissons la **perte quadratique** :

$$J=\frac{1}{2}\sum_j\left(a_j^{(L)}-y_j\right)^2.$$

y_j est la cible attendue. On soustrait la cible à la prédiction, on met chaque écart au carré, puis on additionne. Le facteur $1/2$ simplifie la dérivée : celle de $(u-y)^2/2$ par rapport à u vaut $u-y$.

La dérivée de l'activation ϕ_ℓ' mesure la sensibilité de la réponse à une variation de la somme.

Mécanisme. Le calcul différentiel (merci à Newton, années 1660) fournit ici les dérivées. La **rétropropagation** applique la règle de dérivation en chaîne au graphe des opérations : une variation agit sur un résultat intermédiaire, qui agit sur le suivant, jusqu'à la perte. Pour un chemin $w \to h \to \hat{y} \to J$, les sensibilités locales se multiplient. La notation $\partial J / \partial w$ mesure la sensibilité de la perte J à une petite variation du poids w, les autres poids restant fixés. Une dérivée de 6 signifie qu'une augmentation de w de 0,001 augmente approximativement la perte de 0,006, près du point considéré :

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial w}.$$

Si un paramètre contribue par plusieurs chemins, leurs contributions s'additionnent. Le **gradient** regroupe les dérivées de la perte par rapport aux paramètres : il indique la sensibilité à chacun d'eux. Le passage avant calcule les valeurs nécessaires ; le passage arrière les réutilise pour calculer ces dérivées.

On définit le **signal de rétropropagation** $\delta_j^{(\ell)} = \partial J / \partial z_j^{(\ell)}$: il mesure la sensibilité de la perte J à cette somme reçue par le neurone.

Dernière couche : partir de la prédiction et de la cible

Pour une activation appliquée séparément à chaque neurone de sortie :

$$\delta_j^{(L)} = \frac{\partial J}{\partial a_j^{(L)}} \phi_L'\left(z_j^{(L)}\right).$$

On connaît l'effet de la prédiction sur la perte, puis on tient compte de la sensibilité de l'activation.

Pour la perte quadratique définie plus haut, on obtient donc :

$$\delta_j^{(L)} = \left(a_j^{(L)} - y_j\right) \phi_L'\left(z_j^{(L)}\right).$$

Avec une sortie **linéaire**, $\phi_L(z) = z$ et $\phi_L'(z) = 1$: le signal est alors simplement **prédiction moins cible**. Ce résultat dépend du couple activation–perte. En **classification**, la softmax transforme les sommes de sortie $z_j = z_j^{(L)}$ en probabilités qui totalisent 1 :

$$a_j = \frac{e^{z_j}}{\sum_k e^{z_k}}, J = -\sum_j y_j \ln a_j.$$

La cible y vaut 1 pour la classe attendue et 0 pour les autres. Cette perte, appelée **entropie croisée**, pénalise une faible probabilité attribuée à la bonne classe. La softmax couple les sorties : son dénominateur dépend de tous les z_k. En dérivant l'ensemble softmax–entropie croisée, on obtient néanmoins :

$$\delta_j^{(L)} = \frac{\partial J}{\partial z_j} = a_j - y_j.$$

Avec une prédiction $(0,7, 0,3)$ et la seconde classe attendue, $y = (0, 1)$, les signaux valent $(0,7, -0,7)$. La dérivation de la composition donne ce résultat ; une softmax associée à une perte quadratique demanderait un autre calcul (Zhang et al. 2023).

Couche cachée : remonter les contributions de la couche suivante

Un neurone caché ne possède pas sa propre cible mesurée. On remonte les sensibilités des neurones suivants, déjà calculées :

$$\delta_j^{(\ell)} = \phi_\ell'\left(z_j^{(\ell)}\right) \sum_{k=1}^{n_{\ell+1}} w_{kj}^{(\ell+1)} \delta_k^{(\ell+1)}.$$

Chaque neurone suivant k renvoie son signal $\delta_k^{(\ell+1)}$, multiplié par le poids qui le relie au neurone caché j. On **additionne** ces contributions parce que j influence plusieurs chemins. On multiplie ensuite par la dérivée de son activation : elle indique comment une variation de la somme reçue modifie la réponse.

Par exemple, pour une **sigmoïde**, si la réponse vaut a, la dérivée vaut $a(1-a)$. À une somme reçue de zéro, $a=0,5$ et la dérivée vaut 0,25. Un signal pondéré de 3 venu de la couche suivante devient donc $3\times 0,25=0,75$ pour ce neurone.

Passer du signal aux poids et aux biais. Les gradients sont :

$$\frac{\partial J}{\partial w_{ji}^{(\ell)}}=\delta_j^{(\ell)}a_i^{(\ell-1)},\quad \frac{\partial J}{\partial b_j^{(\ell)}}=\delta_j^{(\ell)}.$$

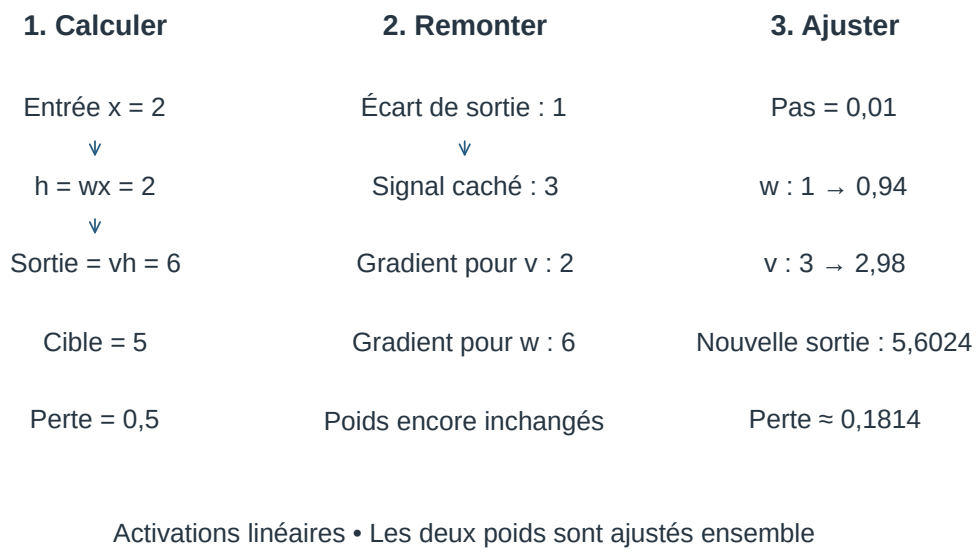
Une variation du poids agit proportionnellement à l'activation qui arrive du neurone précédent ; une variation du biais agit directement sur la somme. Tous ces gradients sont calculés avec les poids du passage avant. L'optimiseur les utilise ensuite pour modifier les paramètres (Rumelhart et al. 1986).

La **descente de gradient** modifie les paramètres θ pour réduire une perte J :

$$\theta_{t+1}=\theta_t-\eta_t\nabla_\theta J(\theta_t).$$

Ici, θ regroupe les paramètres, $\nabla_\theta J$ leurs dérivées et η_t est le **taux d'apprentissage**, qui règle la taille du pas à l'étape t. Le signe moins indique une direction de baisse locale de la perte ; un pas trop grand peut néanmoins l'augmenter.

Exemple calculable à la main. Prenons $h=wx$, $\hat{y}=vh$ et $J=(\hat{y}-y)^2/2$, avec $x=2$, $w=1$, $v=3$, $y=5$. Les activations de cet exemple sont linéaires, de dérivée 1. Le passage avant donne $h=2$, $\hat{y}=6$, $J=0,5$. À la sortie, $\delta^{(2)}=6-5=1$. Dans la couche cachée, $\delta^{(1)}=v\delta^{(2)}=3\times 1=3$. Le gradient du poids de sortie v vaut $\delta^{(2)}h=1\times 2=2$; celui du poids caché w vaut $\delta^{(1)}x=3\times 2=6$. Un pas de descente de 0,01, appliqué simultanément aux deux poids à partir de ces gradients, donne $w=0,94$, $v=2,98$, puis une perte d'environ 0,1814.



Trois étapes du même exemple : calculer la prédiction, remonter les sensibilités, puis ajuster simultanément les poids. Le passage arrière utilise encore les poids du passage avant.

Verrou levé et limite. On dispose d'un calcul efficace de la sensibilité locale de la perte à chaque poids. La rétropropagation **calcule les gradients** ; l'optimiseur choisit **comment modifier les poids**. Leur composition permet l'apprentissage, sans garantir un optimum global. Les gradients peuvent s'éteindre ou exploser. Une opération non

différentiable, comme un choix discret de route, exige un traitement adapté ; une chaîne de dérivées ne traverse pas automatiquement toute décision. L'exemple linéaire ci-dessus isole le calcul ; il ne résout pas XOR.

La version complète utilise toutes les observations pour chaque gradient. La **descente de gradient stochastique (SGD)** emploie une observation ou, dans l'usage courant, un mini-lot. Elle réduit le coût de chaque mise à jour mais introduit du bruit. **RMSprop** adapte l'échelle des pas par une moyenne mobile des gradients au carré. **Adam** utilise des moyennes des gradients et de leurs carrés, avec correction du biais d'initialisation. Aucun ne garantit le minimum global lorsque la fonction de perte à optimiser est non convexe. Une telle perte peut présenter plusieurs creux de hauteurs différentes : atteindre un creux local ne signifie pas avoir trouvé le plus bas de tous.

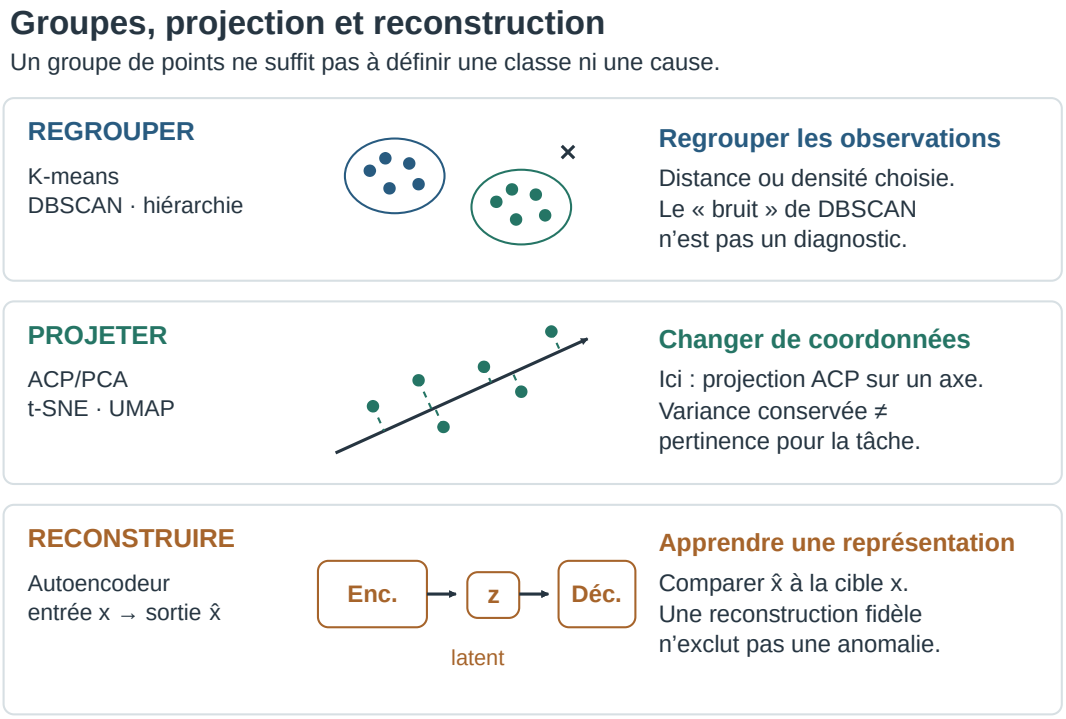
Une **planification du taux d'apprentissage (learning rate scheduling)** fait varier η_t, par exemple lorsque la performance sur le jeu de validation cesse de s'améliorer, ou selon une décroissance prévue à l'avance. Un taux trop élevé peut provoquer des oscillations ou une divergence ; un taux trop faible ralentit l'apprentissage. Comparer ces choix exige de préciser le budget de calcul de chaque essai (Goodfellow et al. 2016d).

La **régularisation L1/L2** pénalise certains paramètres, comme avec Lasso et Ridge. Le **dropout** désactive aléatoirement une partie des activations pendant l'entraînement ; l'inférence habituelle utilise le réseau sans ces masques, avec la convention de mise à l'échelle retenue. L'**arrêt anticipé** retient un état avant la dégradation du critère de validation. Ces moyens contrôlent le surapprentissage ; ils ne réparent ni une fuite de données ni un échantillon non représentatif (PyTorch contributors 2026b; scikit-learn developers 2026q).

Réduire la dimension et apprendre une représentation

L'**analyse en composantes principales (ACP/PCA)** projette des observations centrées sur des axes orthogonaux de variance décroissante. Elle se calcule par décomposition de la matrice de covariance ou par SVD. Elle peut résumer des indicateurs corrélés avec moins de variables. Une direction de faible variance peut cependant contenir l'information nécessaire pour détecter un incident rare. La variance expliquée ne prouve donc pas la conservation de l'information utile à la cible. Dans le livre complet, la section « Choisir une variante de l'ACP » développe le lien ACP–SVD–RPCA sur des données radio synthétiques (scikit-learn developers 2026d).

t-SNE cherche une carte de faible dimension préservant des relations de voisinage probabilistes. **UMAP** construit une représentation à partir d'un graphe de voisinage. Ces méthodes aident à explorer des signatures de trafic ; elles ne produisent pas à elles seules une classification validée. Les espaces vides, distances entre groupes et surfaces occupées en 2D ne se lisent pas comme des mesures fidèles de l'espace initial. Les paramètres, la distance utilisée et les graines aléatoires doivent être documentés. L'implémentation usuelle de t-SNE ne fournit pas de fonction permettant de projeter de nouvelles observations comme le fait l'ACP ; UMAP dispose d'une transformation, à évaluer sur les données futures (scikit-learn developers 2026m; UMAP developers 2026).



Trois objectifs distincts : attribuer des groupes, projeter les variables et reconstruire une observation à partir d'une représentation apprise.

Un **autoencodeur** apprend un encodeur $z = f_\theta(x)$ et un décodeur $\hat{x} = g_\phi(z)$ en minimisant une perte de reconstruction. Une contrainte de dimension, de parcimonie ou de bruit vise à éviter la simple copie de l'entrée ; l'architecture n'a pas à être symétrique. En débruitage, il reconstruit une observation non bruitée à partir d'une entrée bruitée. En détection d'anomalies, une erreur élevée peut déclencher une investigation, mais un modèle trop flexible peut aussi bien reconstruire une anomalie. Le seuil et les anomalies d'évaluation se définissent donc indépendamment de l'entraînement. Souvent classé parmi les méthodes non supervisées, ce modèle a une cible issue de l'entrée, qui permet aussi de décrire son objectif comme auto-supervisé (Goodfellow et al. 2016c).

Reconstruire pour transmettre moins. Dans *Real-Time Adaptive Image Compression*, Oren Rippel et Lubomir Bourdev combinent un autoencodeur, un codage adaptatif et une pénalité sur la longueur du message. L'enjeu est de conserver une image utile avec moins de bits. Leur entraînement adversarial travaille aussi l'aspect des images reconstruites à très faible débit. Cet article relie la représentation apprise à la compression avec perte : réduire l'erreur de reconstruction et réduire le nombre de bits sont deux exigences à concilier (Rippel et Bourdev 2017).

Préentraîner des autoencodeurs couche par couche

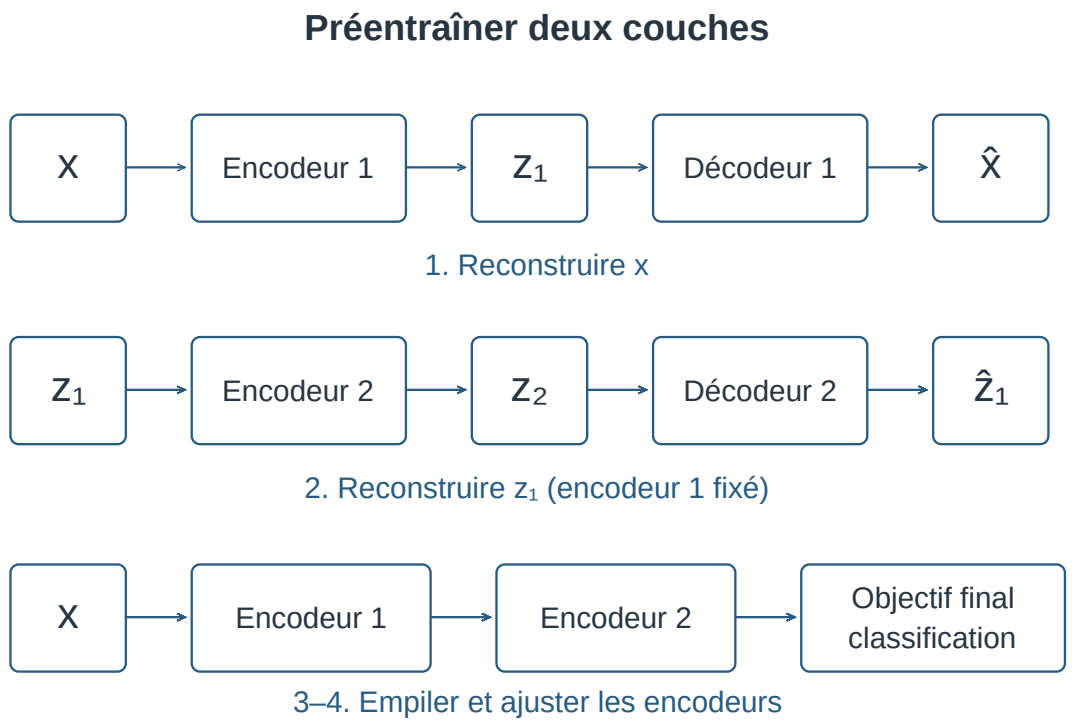
L'autoencodeur décrit plus haut peut aussi préparer les paramètres d'un réseau plus profond. Le **préentraînement couche par couche** apprend successivement des transformations, chacune avec un objectif local de reconstruction, avant de les réunir pour la tâche finale.

Notons x une observation, E_1 et E_2 deux encodeurs, D_1 et D_2 leurs décodeurs, et z_1, z_2 les représentations intermédiaires. Le déroulement est le suivant :

Étape	Ce qui est appris
1. Reconstruire l'entrée	Le premier autoencodeur apprend $z_1 = E_1(x)$, puis $D_1(z_1)$, avec une perte qui compare cette reconstruction à x.

Étape	Ce qui est appris
2. Reconstruire le premier code	On fixe provisoirement E_1 et calcule z_1 sur les données d'entraînement. Le deuxième autoencodeur apprend $z_2 = E_2(z_1)$, puis $D_2(z_2)$, pour reconstruire z_1.
3. Empiler les encodeurs	On compose $E_2(E_1(x))$. D'autres couches peuvent être préparées de la même manière.
4. Ajuster l'ensemble	On ajoute la sortie nécessaire et ajuste les encodeurs ensemble, par rétropropagation, selon l'objectif final : classification, régression ou reconstruction.

Pour une classification, les décodeurs locaux ont servi à préparer les encodeurs ; ils ne sont pas nécessaires à la sortie du classificateur. Pour un autoencodeur profond complet, on conserve ou construit une chaîne de décodage et ajuste aussi cette chaîne. La faible erreur d'une reconstruction locale ne garantit pas que le code conservera le détail utile à la tâche finale (Bengio et al. 2007).



Deux objectifs locaux préparent deux encodeurs : reconstruire x, puis reconstruire z_1. Les encodeurs sont ensuite empilés et ajustés ensemble selon la tâche finale ; les décodeurs locaux ne sont pas utilisés dans le classificateur représenté.

Cette démarche éclaire une étape historique de l'apprentissage profond. Hinton et Salakhutdinov (2006) préparent les couches d'un autoencodeur profond avec des machines de Boltzmann restreintes, puis ajustent l'ensemble. Ce ne sont pas les autoencodeurs locaux du tableau. Bengio et ses collègues, dans les actes de NIPS 2006 publiés en 2007, étudient aussi la variante qui prépare chaque couche par reconstruction (Hinton et Salakhutdinov 2006; Bengio et al. 2007).

Le mot **préentraînement** a un sens plus large que cette procédure. Un objectif auto-supervisé de masquage ou de contraste peut entraîner plusieurs couches ensemble, puis préparer leur adaptation à une autre tâche. La section sur l'apprentissage contrastif développe cette autre possibilité. Le découpage couche par couche aide ici à comprendre comment une représentation sert d'entrée à la suivante ; il n'est pas une étape obligatoire pour entraîner tout réseau profond.

Plongements et apprentissage contrastif

Un **plongement (embedding)** associe un vecteur appris à un objet. **Word2Vec** apprend des représentations de mots en prédisant des contextes ou des mots à partir du contexte. Pour des journaux réseau, une idée analogue peut rapprocher des événements employés dans des contextes similaires ; elle ne démontre pas leur équivalence technique (Mikolov et al. 2013).

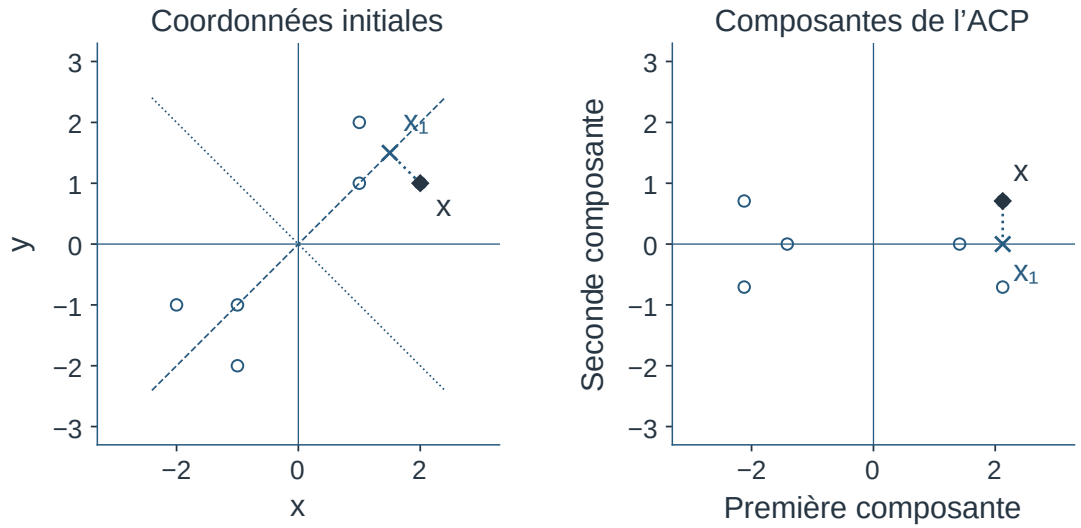
L'**apprentissage contrastif** rapproche des représentations de vues jugées équivalentes et, selon l'objectif, les sépare d'autres vues. SimCLR en fournit un exemple. Pour la radio, les transformations préservant une classe se choisissent en connaissant le signal : supprimer les indices d'une interférence à détecter invalide l'objectif. La représentation s'évalue ensuite sur la tâche cible (Chen et al. 2020).

Des polynômes de l'ENSI à Fourier et à l'ACP

Le corrigé ENSI de Mathématiques II, les séries de Fourier et l'ACP partagent un outil : **la projection sur des directions orthogonales**. Pour une direction non nulle e_i, le coefficient de x vaut $c_i = \langle x, e_i \rangle / \langle e_i, e_i \rangle$. Le produit scalaire mesure la composante ; le dénominateur tient compte de la longueur de la direction. x peut ici être un vecteur, un polynôme ou une fonction, avec le produit scalaire adapté.

Dans l'exercice ENSI, les directions sont des polynômes. Fourier utilise des sinus et des cosinus connus. L'ACP choisit ses directions à partir des données, pour y concentrer leur variance. Le choix des directions change ; le principe de calcul des composantes reste le même.

Un exemple à deux dimensions. Dans le jeu centré de la figure, l'ACP choisit $e_1 = (1,1)/\sqrt{2}$ et $e_2 = (1,-1)/\sqrt{2}$. Le point $x = (2,1)$ a donc pour coordonnées $c_1 = 3/\sqrt{2}$ et $c_2 = 1/\sqrt{2}$. Les deux composantes permettent de retrouver exactement x. Si l'on garde seulement la première, on reconstruit $(1{,}5, 1{,}5)$: l'écart $(0{,}5, -0{,}5)$ est perdu. **Changer de base conserve l'information ; tronquer la représentation peut en supprimer.**



x = (2 ; 1) x_1 = (1,5 ; 1,5)

Deux composantes : exact. Première seule : projection.

Projections sur les axes de l'ACP dans un jeu construit et centré : le point (2 ; 1) se reconstruit exactement avec deux composantes, mais devient (1,5 ; 1,5) si l'on garde seulement la première.

ACP, ICA, LDA et GED : quelles différences ?

Ces méthodes peuvent réduire ou réorganiser des variables ; elles ne cherchent pas la même propriété.

Méthode	Ce qui guide la transformation	Exemple et limite
ACP/PCA : analyse en composantes principales	conserver des directions de forte variance, sans étiquettes	résumer des KPI corrélés ; la variance maximale n'est pas forcément l'information utile au diagnostic
ICA : analyse en composantes indépendantes (*Independent Component Analysis*)	rechercher des composantes statistiquement indépendantes selon un modèle de mélange	séparer des sources mélangées ; cela demande des hypothèses d'identifiabilité, et l'indépendance est plus forte que la simple absence de corrélation
GED : décomposition généralisée en valeurs propres (*Generalized Eigendecomposition*)	maximiser un contraste entre une covariance de signal et une covariance de référence	extraire une combinaison de canaux ; le résultat dépend de la référence choisie
LDA : analyse discriminante linéaire (*Linear Discriminant Analysis*)	utiliser les classes connues pour rechercher des directions qui les séparent	préparer une classification ; la projection dépend des étiquettes d'entraînement

La **SVD** est une décomposition matricielle ; elle sert notamment au calcul de l'ACP, sans être synonyme de tous ces objectifs. Dans le modèle ICA linéaire classique, les mesures sont des mélanges de sources indépendantes, dont au plus une est gaussienne. Les solutions restent indéterminées en ordre et en échelle. LDA désigne ici l'analyse discriminante, et non le modèle de thèmes textuels *Latent Dirichlet Allocation*, qui partage le même sigle (scikit-learn developers 2026e, 2026j).

Dans la **GED**, w contient les coefficients qui combinent les canaux ; S est la matrice de covariance du signal ou du contraste recherché, R celle de la référence. Le critère et l'équation associée s'écrivent :

$$\frac{w^{\top} S w}{w^{\top} R w}, S w = \lambda R w .$$

La transposée $\top$ permet de former une variance scalaire : le numérateur mesure celle de la combinaison dans S, le dénominateur dans R. Une grande valeur propre λ signale un contraste élevé selon cette référence. Ce n'est un rapport signal/bruit que si les covariances représentent effectivement ces deux composantes. On demande une référence suffisamment régulière, généralement définie positive ou régularisée, et on estime les covariances sans utiliser le test final. L'exemple d'électroencéphalographie du support illustre la méthode ; il n'établit pas sa pertinence pour toute mesure réseau (Cohen 2022).

LDA thématique et hachage : deux autres représentations du texte

Le sigle **LDA** désigne aussi *Latent Dirichlet Allocation*, l'allocation de Dirichlet latente. Ce modèle thématique est distinct de l'analyse discriminante linéaire du tableau précédent. Il représente un document comme un mélange de thèmes, et chaque thème comme une distribution de probabilités sur les mots. Dans un exemple fictif, un rapport peut combiner un thème « radio » et un thème « consommation d'énergie ». Les thèmes sont appris sans étiquettes de sujet fournies ; leurs noms sont ensuite une interprétation humaine. Le modèle classique utilise les occurrences des mots sans leur ordre et demande notamment de choisir le nombre de thèmes. Une association de mots ne constitue pas une explication causale (Blei et al. 2003).

Le **hachage de caractéristiques** (*feature hashing*) répond à un autre besoin : produire un vecteur de taille fixe sans conserver un dictionnaire complet des mots. Une fonction déterministe envoie chaque terme dans une case ; les occurrences y sont accumulées, éventuellement avec un signe. Plusieurs termes peuvent partager la même case : c'est une **collision**. Dans un exemple schématique de quatre cases, si « relais » et « panne » arrivent tous deux dans la case 2, leurs contributions se combinent : on perd la possibilité de les distinguer dans cette case.

Augmenter la dimension réduit généralement les collisions, au prix d'un vecteur plus grand. Le codage reste identique entre entraînement et usage, avec les mêmes paramètres, mais il ne permet pas de retrouver systématiquement les termes depuis leurs indices. Il facilite le traitement par flux et le partage d'un codage entre machines. Ce hachage de variables

ne garantit ni chiffrement ni anonymisation, et ne correspond pas à la tokenisation d'un LLM (scikit-learn contributors 2026a).

Un cas réel pour relier ces outils. La thèse de Seif Eddine Hammami, *Dynamic Network Resources Optimization based on Machine Learning and Cellular Data Mining*, étudie la prévision de charge et l'optimisation des ressources de réseaux cellulaires à partir de données d'activité. Pour une première lecture, partir du résumé, repérer les mesures utilisées et chercher comment les prévisions interviennent dans une décision réseau. On retrouve ainsi le chemin « observer, prévoir, décider », au-delà du choix d'un modèle (Hammami 2018).

Représenter l'incertitude et les états cachés

Une densité n'est pas une probabilité

Un délai ou un nombre de pertes peut changer d'une observation à l'autre. Pour poser le vocabulaire : les variables aléatoires avec ParaMaths.

Pour une variable discrète, comme le nombre de paquets perdus, on peut affecter une probabilité à chaque valeur. Pour une variable continue **admettant une densité**, comme un délai modélisé sans arrondi, on décrit plutôt une fonction $f_T(t)$. Sa hauteur indique comment la probabilité se répartit autour de t ; c'est l'**aire sur un intervalle** qui donne une probabilité :

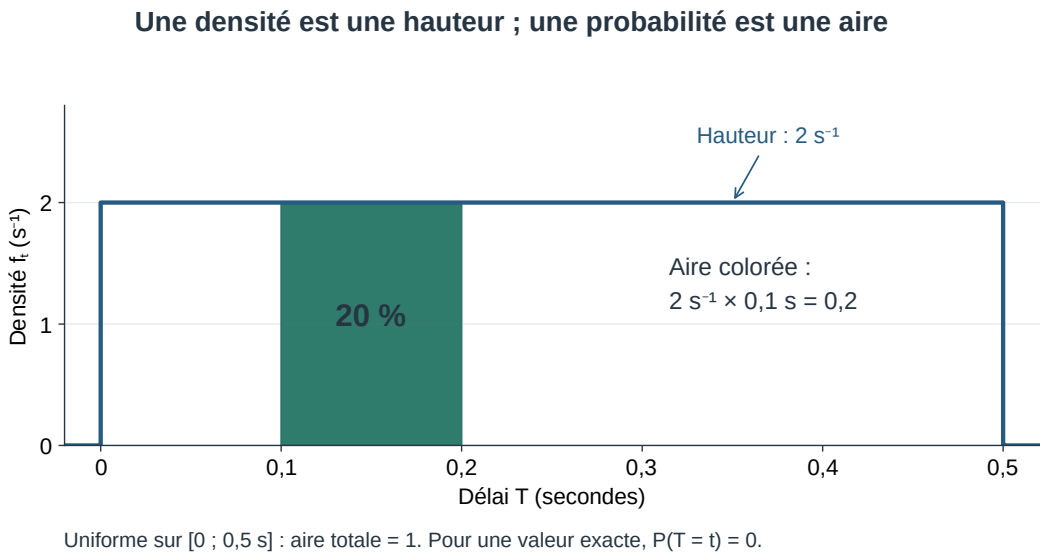
$$f_T(t) \geq 0, \int_{-\infty}^{+\infty} f_T(t)\,dt = 1, P(a \leq T \leq b) = \int_a^b f_T(t)\,dt.$$

Si t est exprimé en secondes, la densité est exprimée en secondes inverses. Pour une densité constante sur un intervalle, l'aire est le produit de la hauteur en s^{-1} par la largeur en secondes. Pour une densité variable, l'intégrale additionne les contributions de petits intervalles. Dans les deux cas, la probabilité obtenue est sans unité et comprise entre 0 et 1. **La hauteur d'une densité peut dépasser 1** ; sa valeur dépend aussi de l'unité choisie pour la variable.

Prenons un délai T uniforme entre 0 et 0,5 seconde (NIST/SEMATECH 2026c). La densité vaut $2\,s^{-1}$ dans cet intervalle et 0 en dehors. L'aire totale est $2 \times 0,5 = 1$. Sur l'intervalle de 0,1 à 0,2 seconde :

$$P(0,1 \leq T \leq 0,2) = 2 \times (0,2 - 0,1) = 0,2 = 20\,\%.$$

En millisecondes, le même délai est uniforme entre 0 et 500 ms ; la densité vaut $0,002\,ms^{-1}$. L'intervalle de 100 à 200 ms a toujours une probabilité $0,002 \times 100 = 0,2$. Changer d'unité modifie la hauteur de la densité, pas la probabilité de l'événement.



Densité uniforme d'un délai : la hauteur vaut 2 secondes inverses et l'aire colorée entre 0,1 et 0,2 seconde vaut 20 pour cent.

Dans ce modèle à densité, une valeur exacte occupe un intervalle de largeur nulle : $P(T=t)=0$, même si $f_T(t)>0$. Cela n'interdit pas qu'une mesure prenne une valeur ; chaque réalisation est un point parmi une infinité de possibilités. Une mesure arrondie à la milliseconde correspond en réalité à un intervalle de délais ; la probabilité d'obtenir cette valeur affichée peut donc être positive.

La **fonction de répartition** cumule les aires jusqu'à une valeur :

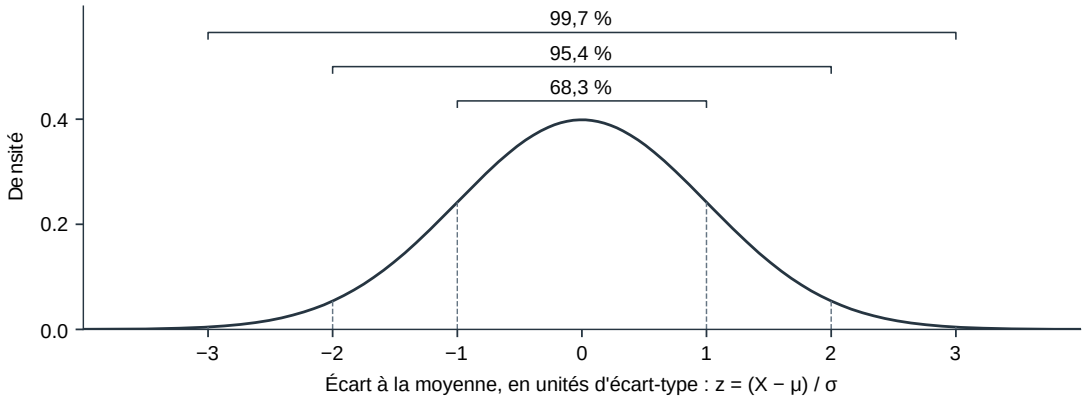
$$F_T(t)=P(T\le t)=\int_{-\infty}^{t} f_T(u)\,du.$$

Elle est sans unité, croissante au sens large et comprise entre 0 et 1. La probabilité de dépasser une échéance d vaut $1-F_T(d)$. C'est l'aire sous la courbe à droite de d : elle donne le risque de dépasser le délai maximal fixé pour le service. La hauteur $f_T(d)$ ne donne pas ce risque. Pour un petit intervalle où la densité varie peu, $P(t\le T\le t+\Delta t)\approx f_T(t)\,\Delta t$.

Enfin, dans un **histogramme de densité**, la hauteur de chaque barre est son effectif divisé par l'effectif total et par sa largeur. La somme des aires vaut 1, pas nécessairement la somme des hauteurs. Un mélange gaussien (GMM), décrit ci-dessous, estime une densité continue. Une forte densité signifie que le modèle concentre davantage de probabilité autour de cette valeur ; elle ne prouve pas que l'observation correspond à un fonctionnement normal.

La cloche gaussienne et les intervalles autour de la moyenne

Une loi **normale**, ou gaussienne, décrit une cloche symétrique autour d'une moyenne μ. Son écart-type $\sigma>0$ règle sa largeur. Si X suit cette loi, environ **68,3 %** des valeurs se trouvent entre $\mu-\sigma$ et $\mu+\sigma$, **95,4 %** entre $\mu-2\sigma$ et $\mu+2\sigma$, et **99,7 %** entre $\mu-3\sigma$ et $\mu+3\sigma$. Ces proportions sont des **aires**, pas des hauteurs (NIST/SEMATECH 2026b).



Loi normale centrée réduite : les intervalles à un, deux et trois écarts-types contiennent environ 68,3, 95,4 et 99,7 pour cent de la probabilité.

Exemple construit. Une mesure modélisée par une moyenne de 20 ms et un écart-type de 2 ms aurait environ 95,4 % de sa probabilité entre 16 et 24 ms. Cela suppose la loi normale ; calculer une moyenne et un écart-type ne démontre pas que les données suivent cette loi. Les délais réseau peuvent être asymétriques, multimodaux ou présenter des valeurs extrêmes : une cloche ajustée sans examen des données peut sous-estimer les dépassements.

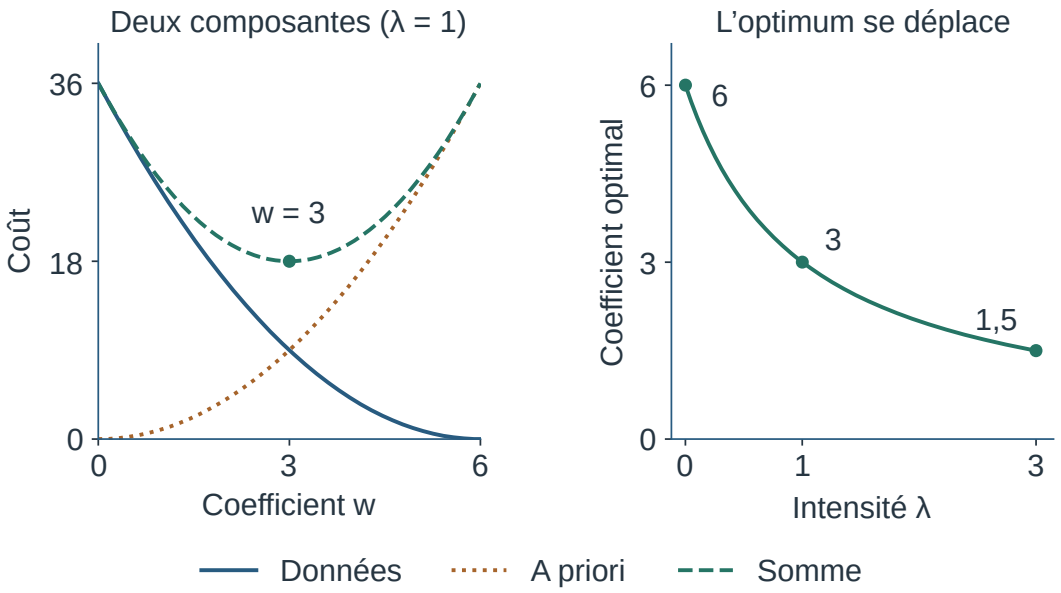
Un parallèle entre pénalité et a priori

Régulariser revient à demander au modèle de bien expliquer les données tout en évitant certains choix de paramètres. Avec des erreurs gaussiennes et un a priori gaussien centré sur zéro, chercher le coefficient le plus plausible après observation conduit à une pénalité ridge.

D'où vient la force de cette pénalité ? Prenons un seul coefficient w, une mesure égale à 6 avec une variance d'erreur $\sigma^2>0$, et un a priori de moyenne 0 et de variance $\tau^2>0$. Une petite variance décrit une distribution resserrée. Le calcul de Bayes combine les deux gaussiennes. Maximiser leur produit revient à minimiser son logarithme négatif. En retirant les constantes qui ne dépendent pas de w, puis en multipliant par $2\sigma^2>0$, on obtient :

$$J(w)=(w-6)^2+\lambda w^2,\ \lambda=\frac{\sigma^2}{\tau^2},\ w^*=\frac{6}{1+\lambda}.$$

Si les deux variances valent 1, $\lambda=1$ et $w^*=3$. Si l'a priori se resserre jusqu'à $\tau^2=1/3$, sans changer l'erreur de mesure, $\lambda=3$ et $w^*=1,5$. Une mesure plus précise favorise au contraire davantage la donnée. **La pénalité traduit ici une connaissance préalable et sa dispersion.** Ce lien dépend des hypothèses gaussiennes ; choisir le coefficient le plus plausible ne fournit pas, à lui seul, toute la distribution d'incertitude d'une régression bayésienne (scikit-learn developers 2026k).



Ridge et a priori : l'erreur et la pénalité se combinent ; leur somme est minimale en 3 pour une pénalité de poids 1. Le second graphique montre l'optimum selon ce poids, zéro signifiant sans pénalité.

Modèles probabilistes : quelles hypothèses et quelles sorties ?

Naive Bayes : combiner des informations conditionnelles

Le **classifieur bayésien naïf (Naive Bayes)** applique :

$$p(c/x) \propto p(c) \prod_j p(x_j/c).$$

Son hypothèse est l'indépendance des variables **conditionnellement à la classe** : une fois la classe fixée, le modèle traite les informations apportées par les variables séparément. Cela ne suppose pas leur indépendance sans connaître la classe, appelée **indépendance marginale**. Le modèle est simple et rapide pour classifier des tickets ou messages de journal. Des variables corrélées peuvent dégrader les probabilités sans rendre nécessairement la classification inutilisable ; la calibration doit être vérifiée (scikit-learn developers 2026o).

Réseau bayésien : expliciter les dépendances

Un **réseau bayésien** relie des variables par des flèches, sans cycle orienté : aucune suite de flèches ne ramène à son point de départ. Il combine des probabilités locales pour calculer celles de l'ensemble : c'est la **factorisation**. Les **parents** d'une variable sont ceux dont une flèche arrive sur elle. Le modèle décrit ses probabilités conditionnelles pour chaque combinaison des valeurs de ces parents. On peut ainsi relier pertes, charge et alarmes. Définir ou apprendre les liens et les tables de probabilités demande des données et des hypothèses explicites. Les flèches ne prouvent pas une causalité : estimer un état à partir d'observations et prévoir l'effet d'une intervention restent deux questions distinctes (pgmpy developers 2026).

À expérimenter. Les tutoriels de pyAgrum permettent de construire un petit réseau bayésien en Python et d'en calculer les probabilités. Reprendre « charge », « pertes » et « alarme », puis comparer la probabilité de pertes avant et après l'observation d'une alarme, donne un sens concret au « sachant » de Bayes (Équipe aGrUM 2026).

Réseau de neurones bayésien : représenter des poids incertains

Réseau bayésien et réseau de neurones bayésien : deux objets distincts. Dans le graphe précédent, les nœuds sont des variables, par exemple « charge », « pertes » et « alarme ». Une flèche décrit une dépendance dans la factorisation, pas

une connexion entre neurones. Observer une alarme peut aider à réviser une hypothèse sur les pertes ; une lecture causale exige des hypothèses supplémentaires.

Un **réseau de neurones bayésien (BNN, Bayesian Neural Network)** conserve une architecture neuronale, mais place des distributions a priori sur ses poids et cherche leur distribution a posteriori après l'entraînement. Au lieu de retenir seulement un poids $w = 0,82$, une approximation pourrait donner $w \sim N(0,82, 0,1^2)$. Ces nombres sont illustratifs : les distributions ne sont pas nécessairement gaussiennes ni indépendantes. Comparer les prédictions obtenues avec plusieurs jeux de poids plausibles aide à représenter l'incertitude du modèle (Blundell et al. 2015).

Point de comparaison	Réseau bayésien	Réseau de neurones bayésien
Structure	Graphe de variables aléatoires	Architecture neuronale
Flèches ou connexions	Dépendances dans une factorisation	Calculs entre unités
Paramètres	Lois conditionnelles, apprises ou fixées	Poids traités comme incertains
Exemple réseau	Diagnostic à partir de charge, pertes et alarmes	Prévision d'un délai avec incertitude sur les poids

À retenir. Le premier organise les dépendances entre variables ; le second représente notamment l'incertitude sur les paramètres d'un réseau de neurones. Aucun ne garantit des probabilités bien calibrées si les hypothèses ou les données sont inadéquates.

Mélange gaussien : attribuer des appartenances probabilistes

Un **modèle de mélange gaussien (GMM)** représente une densité comme une somme pondérée de gaussiennes. L'algorithme EM alterne estimation des appartenances probabilistes et ajustement des composantes. Il peut représenter plusieurs régimes de fonctionnement en attribuant à chaque observation une probabilité d'appartenance à chaque composante. K-means affecte au contraire chaque observation à un seul groupe. Nombre de composantes, structure de covariance, initialisation et régularisation influent sur le résultat ; une composante ne correspond pas automatiquement à une cause physique (scikit-learn developers 2026h).

Modèle de Markov caché : estimer un régime latent

Un **modèle de Markov caché (HMM)** distingue ce que l'on mesure et ce que l'on cherche à connaître. On observe, par exemple, les indicateurs d'une liaison pour estimer son régime « nominal, dégradé, panne ». Ce régime non directement observé est l'**état latent**. Il évolue selon une chaîne de Markov expliquée dans le livre complet. Dans la forme usuelle du modèle, l'état suivant dépend de l'état précédent ; chaque observation dépend de l'état caché au même instant. L'inférence estime les probabilités des états. L'algorithme **Viterbi** cherche la suite d'états la plus probable dans ce modèle, compte tenu des observations. Des dépendances longues ou des durées de régime mal représentées peuvent limiter sa pertinence (hmmlearn developers 2026).

Des distributions aux modèles génératifs : le VAE

Un **modèle génératif** apprend une distribution permettant de produire des exemples : une image, une séquence ou des mesures simulées. Ces modèles précèdent les VAE ; des réseaux profonds génératifs étaient déjà décrits en 2006 (Hinton et al. 2006). Les **autoencodeurs variationnels (VAE, Variational Autoencoders)**, proposés en 2013, constituent un jalon important de cette histoire (Kingma et Welling 2013).

Le lien avec la régression bayésienne est le raisonnement **a priori → observation → a posteriori**. L'objet incertain change : un VAE ordinaire cherche surtout une représentation cachée z d'une observation x, appelée **variable latente**. Un modèle simple choisit un a priori gaussien $p(z)$; le décodeur apprend une distribution $p_\theta(x \mid z)$. Bayes donnerait alors :

$$p_\theta(z \mid x) \propto p_\theta(x \mid z)\, p(z).$$

Ce posterior étant généralement difficile à calculer, l'encodeur apprend une approximation $q_\phi(z \mid x)$, par exemple une moyenne et une variance. θ et ϕ désignent les paramètres des deux réseaux. L'entraînement combine qualité de reconstruction et rapprochement de cette distribution avec l'a priori. Pour **générer**, on tire un z dans l'a priori, puis on utilise le décodeur pour produire une observation.

L'incertitude porte ici sur les **latents** ; les poids des réseaux ne reçoivent pas automatiquement un posterior bayésien comme dans un BNN. Un VAE pourrait proposer des profils de trafic synthétiques, mais ceux-ci devraient être confrontés aux contraintes et aux mesures du réseau. **Savoir générer un exemple plausible ne prouve pas qu'il représente fidèlement le monde.**

GAN : apprendre par confrontation entre deux réseaux

Un **GAN** (*Generative Adversarial Network*) associe un générateur G, qui transforme un vecteur aléatoire z en exemple, et un discriminateur D, qui estime si un exemple provient des données plutôt que du générateur. Le critère **minimax** oppose ici deux réglages : le discriminateur cherche à l'augmenter et le générateur à le diminuer. Les symboles min et max portent sur les paramètres de ces deux réseaux. Dans sa formulation originale :

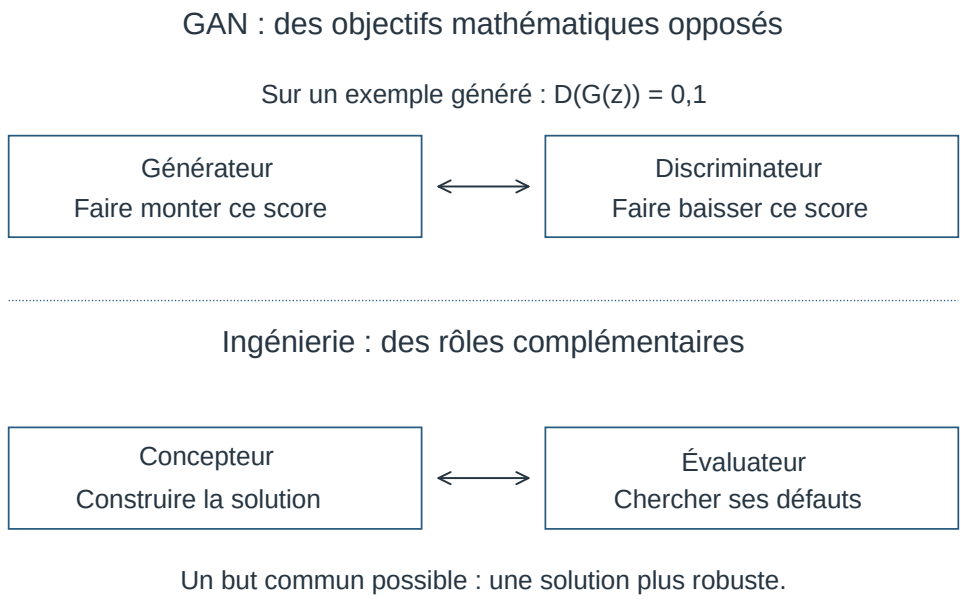
$$\min_G \max_D E_x[\log D(x)] + E_z[\log(1 - D(G(z)))].$$

x est un exemple réel ; E désigne une moyenne selon la distribution considérée. Le discriminateur cherche à augmenter ce critère, tandis que le générateur cherche à le diminuer. Les deux réseaux alternent leurs ajustements par gradient (Goodfellow et al. 2014).

On retrouve une forme de **minimax**, avec deux rôles opposés dans un entraînement alterné. Le VAE minimise au contraire un objectif combinant reconstruction et régularisation d'une distribution latente. Le parallèle avec les PINN auto-adaptatifs porte sur une alternance descente–montée, pas sur un mécanisme identique.

Deux objectifs sur un même exemple. Si le discriminateur attribue le score $D(G(z)) = 0,1$ à un exemple généré, le générateur cherche à faire monter ce score ; le discriminateur cherche à le faire baisser. Le terme $\log(1 - D(G(z)))$ diminue quand ce score augmente : pour cette composante, le générateur minimise ce terme et le discriminateur le maximise. Dans l'objectif original, cette opposition est inscrite dans une même expression mathématique (Goodfellow et al. 2014).

Elle rappelle les rôles complémentaires en cybersécurité et en innovation : l'un construit, l'autre cherche ce qui met la solution en défaut. Les personnes peuvent toutefois partager le but final d'améliorer le produit. Leurs intérêts et leurs responsabilités ne se réduisent pas à un jeu à somme nulle. Le rapprochement porte sur l'utilité d'éprouver le travail ; la formule d'un GAN ne garantit pas l'efficacité d'une organisation.



Deux formes de confrontation : dans le GAN original, générateur et discriminateur visent des directions opposées pour le score d'un exemple généré ; en ingénierie, concepteur et évaluateur confrontent une solution aux essais avec un but commun.

Un GAN peut proposer des signaux synthétiques. Une sortie visuellement plausible ne prouve ni sa fidélité physique ni la diversité du corpus. L'entraînement peut être instable et le générateur peut se concentrer sur quelques types d'exemples, phénomène appelé **effondrement des modes**.

EBM : donner un score à une solution entière

Imaginons une grille de deux lignes et deux colonnes : chaque ligne et chaque colonne doit contenir une fois le chiffre 1 et une fois le chiffre 2. Comparons deux propositions :

$$A=\begin{pmatrix}1 & 2\\ 2 & 1\end{pmatrix}, B=\begin{pmatrix}1 & 2\\ 1 & 2\end{pmatrix}.$$

Dans A, les quatre règles sont respectées. Dans B, les lignes conviennent, mais les deux colonnes répètent le même chiffre. Si l'on compte les lignes et colonnes incorrectes, le score vaut donc 0 pour A et 2 pour B. On sait ainsi comparer des propositions entières. Ce score est ici écrit à la main ; aucun apprentissage n'a eu lieu.

Un **EBM** (*Energy-Based Model*, modèle à énergie) reprend cette idée : il associe un nombre à une configuration. Dans un EBM neuronal, un réseau apprend à donner une énergie basse aux propositions compatibles avec le problème, et une énergie plus haute aux autres. « Énergie » désigne ici un score, pas une consommation en joules. On note ce score $E_\theta(x, y)$: x décrit le problème, y une proposition et θ les paramètres appris (LeCun et al. 2006).

Pour obtenir une réponse, on cherche une proposition de faible énergie :

$$\hat{y} \in \arg\min_y E_\theta(x, y).$$

Lire la formule. $\hat{y}$ est la réponse recherchée. « arg min » désigne les propositions donnant le plus petit score ; « min » seul donnerait la valeur de ce score. Dans notre grille, A fait partie des réponses de score nul. Elle n'est pas unique : la grille obtenue en échangeant 1 et 2 convient aussi.

Apprendre le score, puis chercher une réponse. À l'entraînement, on ajuste θ. Une méthode compare une bonne proposition y^+ à une mauvaise y^-, avec une marge $m>0$:

$$L=max\left(0,m+E_\theta(x,y^+)-E_\theta(x,y^-)\right).$$

L est la perte à réduire ; $+$¿ et $-$ repèrent les deux propositions, sans être des opérations. Avec $m=1$, une énergie de 2 pour la bonne réponse et de 1 pour la mauvaise donne $L=max(0,2)=2$. Si la mauvaise réponse reçoit ensuite une énergie de 3, en gardant l'autre à 2, la perte devient nulle : l'écart demandé est atteint. C'est un objectif d'apprentissage possible, pas une obligation pour tous les EBM (LeCun et al. 2006).

À l'inférence, les paramètres restent fixes : c'est la proposition y que l'on modifie. Pour des valeurs continues et une énergie différentiable, une descente de gradient peut guider cette recherche. Pour une grille de chiffres, il faut une recherche discrète, ou une représentation continue suivie d'un retour aux choix autorisés. Une recherche peut rester bloquée dans un minimum local, meilleur que ses voisins mais moins bon qu'une solution plus éloignée (Du s. d.). D'autres EBM servent à **générer** plusieurs exemples par échantillonnage : ils ne se limitent donc pas au classement de réponses (Du et Mordatch 2019).

Et les EBRM ? L'expression *Energy-Based Reasoning Models* désigne des modèles qui emploient cette approche pour le raisonnement. L'article ByteByteGo du 12 octobre 2026 présente notamment Kona, de Logical Intelligence (ByteByteGo 2026; Logical Intelligence 2026). Une voie de recherche consiste à ajuster des états internes numériques, appelés **latents**, avant de décoder une réponse. La prépublication de Johansson (2026) étudie cette démarche et rapporte aussi des cas où l'énergie diminue alors que la réponse se dégrade : un score appris n'est pas une preuve de justesse (Johansson 2026).

Le parallèle avec le routage et le MILP. x peut décrire la topologie, les capacités et les demandes ; y, une répartition du trafic entre plusieurs chemins. Un EBM pourrait apprendre à noter ces répartitions et guider leur recherche. Le MILP décrit explicitement un objectif et des contraintes ; son solveur peut établir une solution optimale lorsqu'il termine avec la preuve correspondante. On compare donc les résultats, le temps de calcul et les violations, sans supposer qu'un score appris remplace cette garantie.

Dans une solution composée, un LLM peut aider à exprimer la demande, un EBM proposer une configuration et un vérificateur contrôler les contraintes explicites. Pour notre grille, compter les violations suffit déjà ; le réseau neuronal n'apporterait rien. Pour un réseau de communication, une faible énergie ne dispense jamais de contrôler les capacités et les boucles. Le vérificateur établit le respect des règles qu'on lui a données, pas que ces règles décrivent tous les besoins réels.

Pour approfondir : le cadre de LeCun et ses collègues et le panorama de Yilun Du relient score, recherche, génération et composition de modèles.

Choisir une architecture neuronale

Apprendre aussi la fonction d'activation

Dans $h=\sigma(Wx+b)$, on apprend habituellement les poids W et les biais b, tandis que la forme de σ est fixée. Des recherches apprennent aussi des paramètres qui modifient cette forme : le calcul élémentaire devient lui-même adaptable.

Approche	Ce qui est appris
Activations adaptatives par morceaux (APL), 2014	Les pentes et les points de rupture d'une courbe propre à chaque neurone ; le nombre de morceaux reste choisi avant l'entraînement (Agostinelli et al. 2014).
PReLU, 2015	La pente pour les entrées négatives, ajustée avec les autres paramètres du réseau (He et al. 2015).
Réseaux de Kolmogorov–Arnold (KAN), 2024	Des courbes souples, appelées *splines*, sur les connexions ; les neurones additionnent les entrées ainsi transformées (Liu et al. 2024).

Le cas **PReLU (Parametric Rectified Linear Unit)** rend l'idée concrète :

$$\sigma_a(u) = max(0, u) + a\, min(0, u).$$

u est l'entrée de l'activation ; a est une pente apprise. Avec $a = 0$, on retrouve **ReLU**, qui conserve les valeurs positives et annule les négatives. Les KAN vont plus loin dans la forme des fonctions, mais changent aussi leur emplacement : leurs courbes sont sur les **connexions**, et non simplement une nouvelle activation de chaque neurone.

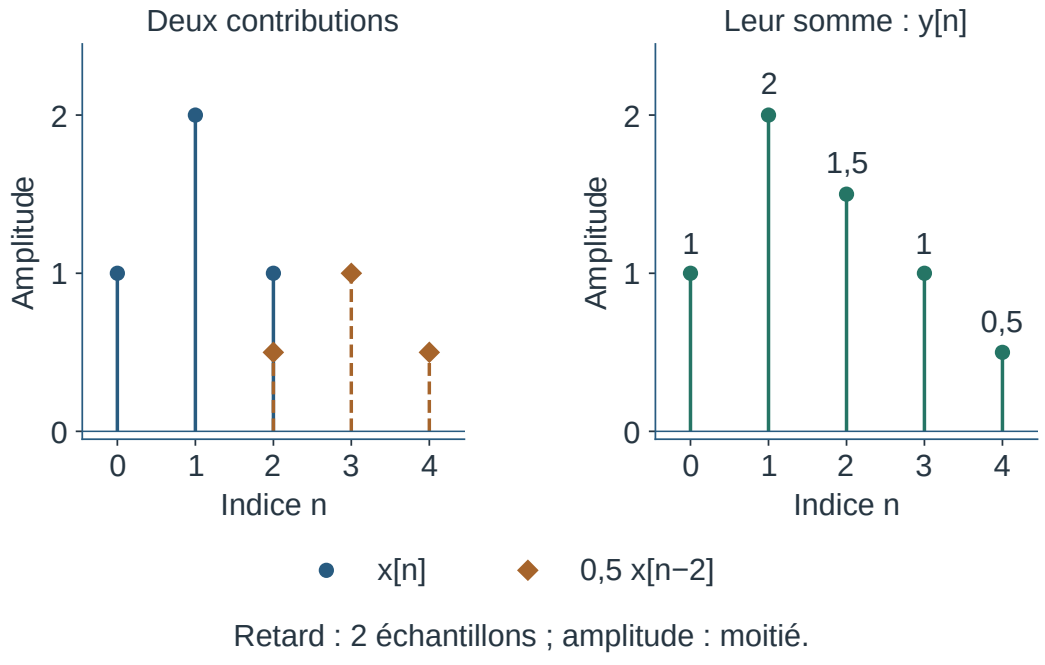
Cette liberté supplémentaire doit être évaluée. Les auteurs de KAN signalent le coût d'entraînement de leur implémentation. Une comparaison teste KAN et MLP à nombre de paramètres comparable, puis à budget d'opérations comparable : les avantages varient selon la tâche et le budget, sans établir une supériorité générale des KAN (Liu et al. 2024; Yu et al. 2024). Compter les opérations ne mesure directement ni le temps ni l'énergie.

La famille de courbes et ses contraintes restent des choix du concepteur. Apprendre leur forme est un autre choix que représenter l'incertitude des poids dans un BNN ; les deux peuvent se combiner. **L'apprentissage peut donc porter sur la forme du calcul, en plus de la force des connexions.**

Une grille et des filtres : le CNN

Un **réseau convolutif (CNN)** réutilise des filtres locaux sur une grille, par exemple un spectrogramme radio. Des convolutions 1D traitent également des séquences temporelles ou du texte ; les CNN ne sont pas réservés aux images. L'agrégation locale (*pooling*) est une possibilité architecturale, pas une obligation. Le voisinage couvert par les filtres et les transformations auxquelles le modèle est peu sensible doivent correspondre à la tâche (PyTorch contributors 2026a).

Un filtre, des décalages, une somme. La convolution décrit notamment des échos : avec un trajet direct et une copie atténuée retardée de deux échantillons, $y[n] = x[n] + 0{,}5\, x[n-2]$. Chaque sortie additionne deux contributions du même signal. Un filtre de CNN reprend l'idée de combiner un voisinage avec les mêmes coefficients à chaque position, mais ses coefficients sont appris. Le filtre $(1/3, 1/3, 1/3)$ appliqué à la fenêtre $(3, 6, 9)$ donne ainsi 6 : une moyenne locale. Le mécanisme élémentaire reste une somme pondérée. La comparaison porte sur ce calcul : les activations rendent le réseau complet non linéaire, et PyTorch emploie une corrélation croisée, qui ne renverse pas le filtre comme la convolution mathématique (PyTorch contributors 2026a).



Convolution et échos : additionner un signal discret et sa copie atténuée retardée de deux échantillons. Les valeurs sont nulles hors de leur support ; ce modèle linéaire ne représente pas un CNN entier.

Lire le calcul du filtre. Pour un noyau h de m coefficients :

$$y[n]=(x*h)[n]=\sum_{k=0}^{m-1} h[k]x[n-k].$$

n choisit la position de sortie ; k parcourt les coefficients du noyau. Le décalage $n-k$ choisit l'échantillon à multiplier. L'astérisque entre x et h désigne la convolution ; les crochets désignent un indice. Il faut préciser les valeurs utilisées hors du signal, par exemple des zéros.

Sur une image en niveaux de gris, le filtre combine les pixels d'une petite fenêtre. Avec les pixels $(2,0;1,3)$ et les coefficients $(1,0;0,-1)$, on calcule $1\times 2+0\times 0+0\times 1-1\times 3=-1$. Le **pas** (*stride*) règle le déplacement de la fenêtre ; le **padding** ajoute des valeurs autour de l'image. Un CNN apprend les coefficients des filtres par rétropropagation ; plusieurs couches associent ces calculs à des activations.

Une fenêtre, quatre produits, une addition.

Image X : 3 × 3

1	2	0
0	1	3
2	0	1

×

Filtre W : 2 × 2

1	0
0	-1

→

Fenêtre en haut à droite

1 × 2 + 0 × 0
+ 0 × 1 − 1 × 3 = −1

Le filtre reste dans cette orientation : c'est une corrélation croisée.

Une fenêtre d'image, quatre produits et une addition : le filtre reste dans la même orientation, comme dans la corrélation croisée utilisée par les CNN.

De l'audio à une entrée de CNN

Un enregistrement sonore fournit une suite d'**échantillons** : des valeurs d'amplitude prises à intervalles réguliers. La **fréquence d'échantillonnage** indique combien de valeurs sont acquises par seconde. Pour reconnaître un mot isolé, on peut transformer cette suite en une grille décrivant l'évolution de ses fréquences, puis apprendre un classificateur sur des extraits annotés. Ce choix rend visibles des structures locales dans le temps et dans les fréquences ; il ne rend pas un CNN obligatoire.

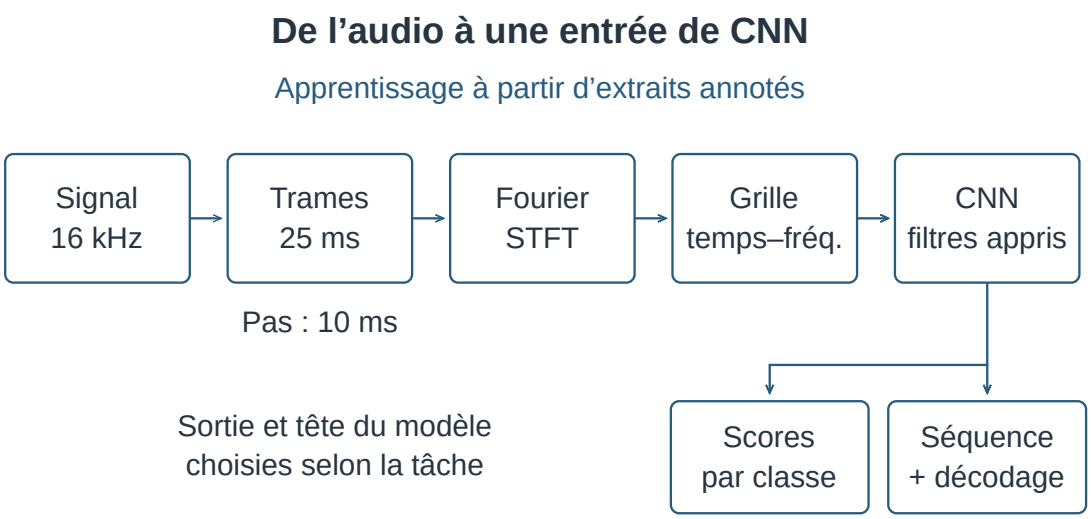
On découpe d'abord l'extrait en **trames**, des segments courts qui peuvent se chevaucher. Une fonction de fenêtre, par exemple de Hann, atténue les bords de chaque segment. On calcule ensuite une transformée de Fourier sur chaque trame : c'est la **transformée de Fourier à court terme** (**STFT**, *Short-Time Fourier Transform*). L'amplitude de ses coefficients forme une grille temps–fréquence ; leur carré décrit une puissance, selon la normalisation retenue. Une échelle logarithmique peut rendre des niveaux faibles plus visibles, en ajoutant une petite valeur positive avant le logarithme pour éviter celui de zéro (The SciPy community 2026). La STFT ne doit pas être confondue avec la transformée en cosinus discrète (DCT) présentée pour JPEG.

Exemple illustratif de dimensions. Prenons un signal réel d'une seconde à 16 000 échantillons par seconde. Une trame de 25 ms contient 400 échantillons ; avancer de 10 ms entre deux trames donne un pas de 160 échantillons. Notons N le nombre total d'échantillons, L la longueur d'une trame, H le pas et T le nombre de trames complètes. La notation $\lfloor\cdot\rfloor$ arrondit vers le bas. Sans prolongement du signal ni ajout de zéros aux extrémités :

$$T=1+\left\lfloor\frac{N-L}{H}\right\rfloor=1+\left\lfloor\frac{16\,000-400}{160}\right\rfloor=98.$$

Une transformée de 400 points conserve 201 coefficients fréquentiels distincts pour ce signal réel, de 0 à 8 000 Hz. L'entrée peut donc avoir **201 fréquences × 98 trames**, avec un canal d'amplitude. Le nombre change si l'on ajoute des

zéros, choisit une autre taille de transformée ou regroupe les fréquences. Ce calcul décrit une préparation possible, sans annoncer un résultat de reconnaissance.



Une chaîne audio possible : signal échantillonné, trames, STFT, grille temps–fréquence, puis CNN. La forme de la sortie dépend de la tâche et de la tête du modèle.

Pour l’entraînement, chaque grille est associée à une annotation, par exemple « oui », « non » ou « autre son ». Les filtres du CNN apprennent des motifs locaux ; une tête de classification transforme la représentation en scores de classes. Les travaux sur la détection de mots-clés donnent un exemple de cette articulation entre caractéristiques acoustiques et CNN (Sainath et Parada 2015). Choisir une classe pour un extrait ne transcrit pas une phrase : la reconnaissance automatique de la parole (**ASR**, *Automatic Speech Recognition*) doit produire une séquence de longueur variable et traiter son alignement ou son décodage. Un CNN peut participer à cette chaîne, dont il ne constitue pas à lui seul toutes les étapes.

La séparation des ensembles se fait **par locuteur ou session avant de dériver les trames et les extraits**, selon ce que l’on veut généraliser. Deux fenêtres issues du même enregistrement restent très proches : les répartir au hasard entre entraînement et test rendrait l’évaluation trop favorable. Normalisation et seuils se règlent ensuite avec les données réservées à leur apprentissage. Le même raisonnement relie traitement du signal et apprentissage dans les réseaux : préparer une grille radio ou acoustique, définir la cible, puis évaluer sur de nouvelles acquisitions. Des caractéristiques résumées et un modèle plus simple restent des points de comparaison utiles.

Réseaux récurrents et mémoire

Un **réseau récurrent (RNN)** met à jour un état caché à chaque pas, $h_t = f_\theta(x_t, h_{t-1})$, pour prévoir par exemple l’évolution du débit à partir de mesures successives. Les gradients peuvent s’éteindre ou exploser sur de longues séquences. **LSTM** et **GRU** introduisent des portes pour mieux gérer la mémoire, sans supprimer toute difficulté. Ordre temporel, horizon et coût du calcul séquentiel restent à évaluer (PyTorch contributors 2026d).

Mémoire des LSTM : conserver, atténuer, ajouter

Un **LSTM** (*Long Short-Term Memory*) conserve un état de cellule c_t. Dans sa forme courante, sa mise à jour s’écrit :

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t .$$

La **porte d’oubli** f_t règle la part de l’ancien état conservée. Ses valeurs sont comprises entre 0 et 1 : proches de 1, elles retiennent l’ancienne contribution ; proches de 0, elles l’atténuent fortement. La porte d’entrée i_t règle l’ajout du contenu candidat $\tilde{c}_t$. Le symbole $\odot$ désigne une multiplication composante par composante (PyTorch contributors 2026c).

Exemple scalaire construit. Avec $c_{t-1}=0,8$, $f_t=0,25$, $i_t=0,5$ et $\tilde{c}_t=0,6$, l'ancienne contribution vaut $0,25\times0,8=0,2$ et la nouvelle $0,5\times0,6=0,3$. L'état devient donc $c_t=0,5$.

Les portes varient avec l'entrée et l'état caché précédent, même lorsque les poids restent figés pendant l'inférence. Ce mécanisme module la mémoire de la séquence ; il n'efface pas directement les paramètres appris. La porte d'oubli introduite par Gers et ses collègues permet ainsi d'apprendre quand atténuer ou réinitialiser l'état au cours d'un flux continu (Gers et al. 2000). La mise à jour additive facilite la conservation d'information et la propagation du gradient, sans fournir une mémoire exacte et illimitée.

L'**oubli catastrophique** concerne un autre phénomène : en modifiant ses paramètres pour apprendre une nouvelle tâche, un modèle peut perdre des compétences antérieures (Kirkpatrick et al. 2017). Par exemple, un détecteur réentraîné seulement sur les alarmes d'un nouveau site peut devenir moins précis sur l'ancien. L'apprentissage continu cherche à limiter cette dégradation (voir l'évolution du modèle au fil du temps).

Le rapprochement avec Savoir oublier pour mieux apprendre reste une analogie fonctionnelle : sélectionner une information selon son utilité. La gestion d'un état numérique ne décrit pas à elle seule la mémoire biologique ni la remise en question d'une idée.

Encodeur–décodeur : produire une séquence de longueur différente

Problème. Classer une séquence produit une étiquette ; résumer dix alarmes en une phrase exige une sortie dont ni la longueur ni l'alignement avec les entrées ne sont fixés à l'avance.

Mécanisme. Dans une architecture **encodeur–décodeur**, l'encodeur construit une représentation de la séquence reçue. Le décodeur produit les éléments de sortie successivement, en tenant compte de cette représentation et des éléments déjà produits. Un symbole de fin permet de terminer la sortie. Dans une forme récurrente simple, l'entrée est résumée dans un vecteur de taille fixe (Sutskever et al. 2014).

```
10 alarmes -> encodeur -> représentation -> décodeur -> résumé de 18 jetons
```

Verrou levé et limite. Le modèle peut apprendre une transformation entre séquences de longueurs différentes. Mais condenser toute une longue entrée dans un seul vecteur crée un goulot d'information. L'**attention** permet au décodeur de consulter différentes représentations de l'entrée selon l'élément qu'il produit. Elle améliore l'accès aux informations disponibles ; elle ne garantit pas de conserver chaque alarme critique. L'exemple est pédagogique : la fidélité du résumé se vérifie sur les faits et leur ordre, pas sur sa fluidité.

Un **transformeur (Transformer)** utilise l'attention pour combiner les représentations des éléments d'une séquence selon leur contenu. Q, K et V signifient respectivement *queries*, *keys* et *values* : **requêtes, clés et valeurs**. Ce sont trois représentations numériques apprises pour chaque jeton, regroupées en matrices. Les requêtes sont comparées aux clés ; les scores obtenus servent à combiner les valeurs.

d_k est le nombre de composantes d'une clé ; la division par $\sqrt{d_k}$ ajuste l'échelle des scores. Pour chaque requête, la fonction **softmax** transforme ces scores en poids non négatifs dont la somme vaut 1.

Dans l'attention à produit scalaire, notons n_q le nombre de requêtes, n_k le nombre de clés et d_v le nombre de composantes d'une valeur. Les dimensions sont $Q : n_q \times d_k$, $K : n_k \times d_k$ et $V : n_k \times d_v$. La transposée $K^\top$ échange ses lignes et ses colonnes : $QK^\top$ contient donc $n_q \times n_k$ scores, un par paire requête–clé. Le produit final rassemble les valeurs en une sortie de dimension $n_q \times d_v$:

$$Attention(Q,K,V)=softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Un mécanisme de position informe le modèle de l'ordre. Avec l'attention dense usuelle, le nombre de comparaisons augmente comme le carré de la longueur de la séquence. La parallélisation de l'entraînement ne garantit donc ni une faible consommation de mémoire, ni une génération rapide. **BERT** illustre le préentraînement par masquage ; **GPT** la

prédiction autorégressive du prochain jeton. Ces architectures peuvent être adaptées par apprentissage supervisé après un préentraînement auto-supervisé. Elles peuvent servir à analyser des journaux ou des séquences de mesures, sous réserve d'évaluation sur le domaine (Vaswani et al. 2017; Devlin et al. 2019).

L'attention comme consultation pondérée. Une table clé-valeur permet de retrouver une information à partir d'une clé. L'attention réalise une consultation plus souple : la requête Q est comparée aux clés K, puis les informations V sont combinées selon les scores obtenus. Prenons une requête $(1,0)$, deux clés $(1,0)$ et $(0,1)$, et les valeurs 10 et 20. Ici, $d_k=2$: les scores valent $(1/\sqrt{2},0)$. La softmax donne les poids $(e^{1/\sqrt{2}}/(e^{1/\sqrt{2}}+1),1/(e^{1/\sqrt{2}}+1))$, soit environ $(0,670,0,330)$. Leur combinaison donne $0,670\times10+0,330\times20\approx13,30$. Dans un Transformer, ces valeurs sont des vecteurs appris. Le résultat est une représentation intermédiaire ; un poids élevé ne prouve ni la vérité d'une information ni une compréhension humaine (Vaswani et al. 2017).

De la notation d'Einstein au Transformer

La **convention de sommation d'Einstein** raccourcit l'écriture des produits. Dans la convention utilisée ici, pour des tableaux en coordonnées euclidiennes, un indice répété deux fois dans un produit est additionné. Les indices présents une seule fois restent dans le résultat.

Prenons U de dimension 2×3 et V de dimension 3×4. Leur produit peut s'écrire :

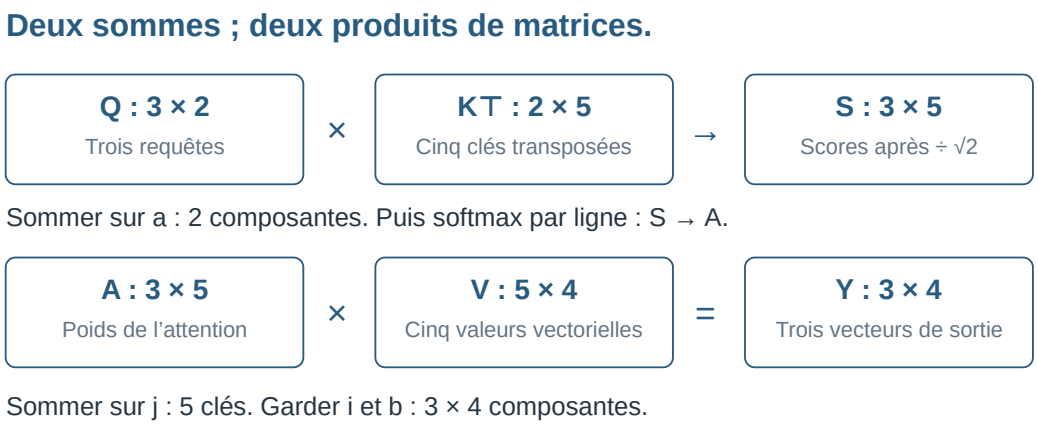
$$Z_{ij}=U_{ia}V_{aj}=\sum_{a=1}^{3}U_{ia}V_{aj}.$$

Pour calculer une case de Z, on fixe la ligne i et la colonne j, puis on additionne les trois produits repérés par a. a est un **indice sommé**, aussi appelé indice muet ; i et j sont les **indices libres**. Z a donc deux lignes et quatre colonnes. Un **tenseur**, dans ces calculs, est un tableau à plusieurs axes ; une **contraction** additionne sur des axes communs.

Appliquons cette lecture à l'attention. Notons S les scores, $A=softmax(S)$ les poids et Y la sortie :

$$S_{ij}=\frac{Q_{ia}K_{ja}}{\sqrt{d_k}},Y_{ib}=A_{ij}V_{jb}.$$

Dans le premier produit, a parcourt les composantes comparées. Dans le second, j parcourt les clés et leurs valeurs associées. i choisit une requête ; b une composante de sortie. La softmax s'applique par ligne, avec sa propre somme au dénominateur : la convention d'Einstein ne supprime pas toutes les sommes d'une formule.



Les deux contractions de l'attention : sommer sur les composantes pour comparer, puis sur les clés pour combiner les valeurs. Cet exemple relie trois requêtes et cinq clés.

En **auto-attention**, les requêtes, clés et valeurs viennent de la même séquence représentée par X. Trois matrices apprises donnent $Q=XW_Q$, $K=XW_K$ et $V=XW_V$: il y a alors autant de requêtes que de clés. Plusieurs **têtes** répètent le

calcul avec leurs propres paramètres, puis leurs sorties sont réunies et transformées. Le Transformer combine ces blocs avec des réseaux appliqués à chaque élément et des informations de position (Vaswani et al. 2017).

Dans l'auto-attention **causale**, une position peut consulter les jetons qui la précèdent et le jeton courant pour prédire le suivant. Le masque remplace les scores des positions futures par $-\infty$ avant la softmax : leurs exponentielles et leurs poids valent alors zéro. Au moins une clé doit rester accessible pour que la normalisation soit définie. L'entraînement peut ainsi traiter les positions en parallèle sans leur donner accès aux réponses futures.

Le filtre convolutif partage ses coefficients dans une fenêtre locale. L'attention calcule des poids selon le contenu comparé ; un masque détermine les positions accessibles. Les indices permettent de suivre ce que chaque calcul rassemble, et de vérifier les dimensions. Le compagnon Lire les mathématiques de l'IA reprend ces gestes avec des dessins et des exercices corrigés.

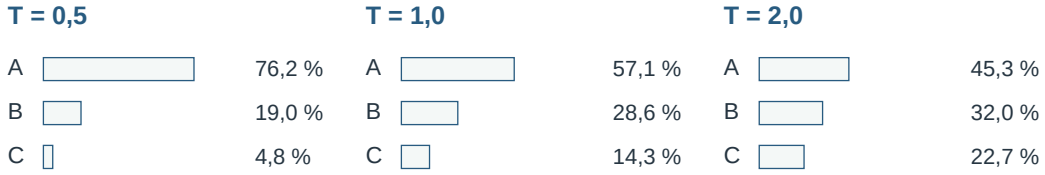
Des scores au prochain mot : que règle la température ?

Pour poursuivre une phrase, un modèle de langage produit un score pour chaque **jeton**, une unité de texte qui peut être un mot ou un fragment de mot. Ces scores de sortie, appelés **logits**, ne sont pas encore des probabilités. La softmax les transforme en probabilités dont la somme vaut un. Lorsque la génération utilise un tirage aléatoire, la **température** $T>0$ modifie cette répartition en divisant les scores avant la softmax :

$$p_i(T)=\frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

z_i est le score du candidat d'indice i ; $p_i(T)$ est sa probabilité de tirage. L'indice j parcourt tous les candidats. Prenons seulement trois candidats A, B et C, avec les scores $(\ln 4, \ln 2, 0)$. À $T=1$, les exponentielles valent $(4,2,1)$: on divise chaque valeur par leur somme, 7. Les probabilités sont donc $(4/7, 2/7, 1/7)$, soit environ $(57,1, 28,6, 14,3)\,\%$. À $T=0,5$, les scores sont doublés : les exponentielles deviennent $(16,4,1)$, et A reçoit $16/21$, soit environ 76,2 %.

La température change la répartition du tirage.



Mêmes scores : (ln 4 ; ln 2 ; 0). A, B et C désignent trois jetons possibles.

Mêmes scores, trois températures : les probabilités de tirage changent.

Une petite température favorise les jetons les mieux notés ; une grande température rapproche leurs probabilités. Ce réglage change le tirage, sans modifier les paramètres appris. Il ne mesure ni la créativité ni la fiabilité factuelle : favoriser une continuation fausse peut répéter l'erreur.

L'exemple isole la température, sans autre filtrage. Dans un outil réel, des réglages peuvent aussi limiter les candidats. Avec un choix **glouton**, on prend directement le jeton au score maximal ; aucun tirage n'a lieu (Hugging Face 2026c).

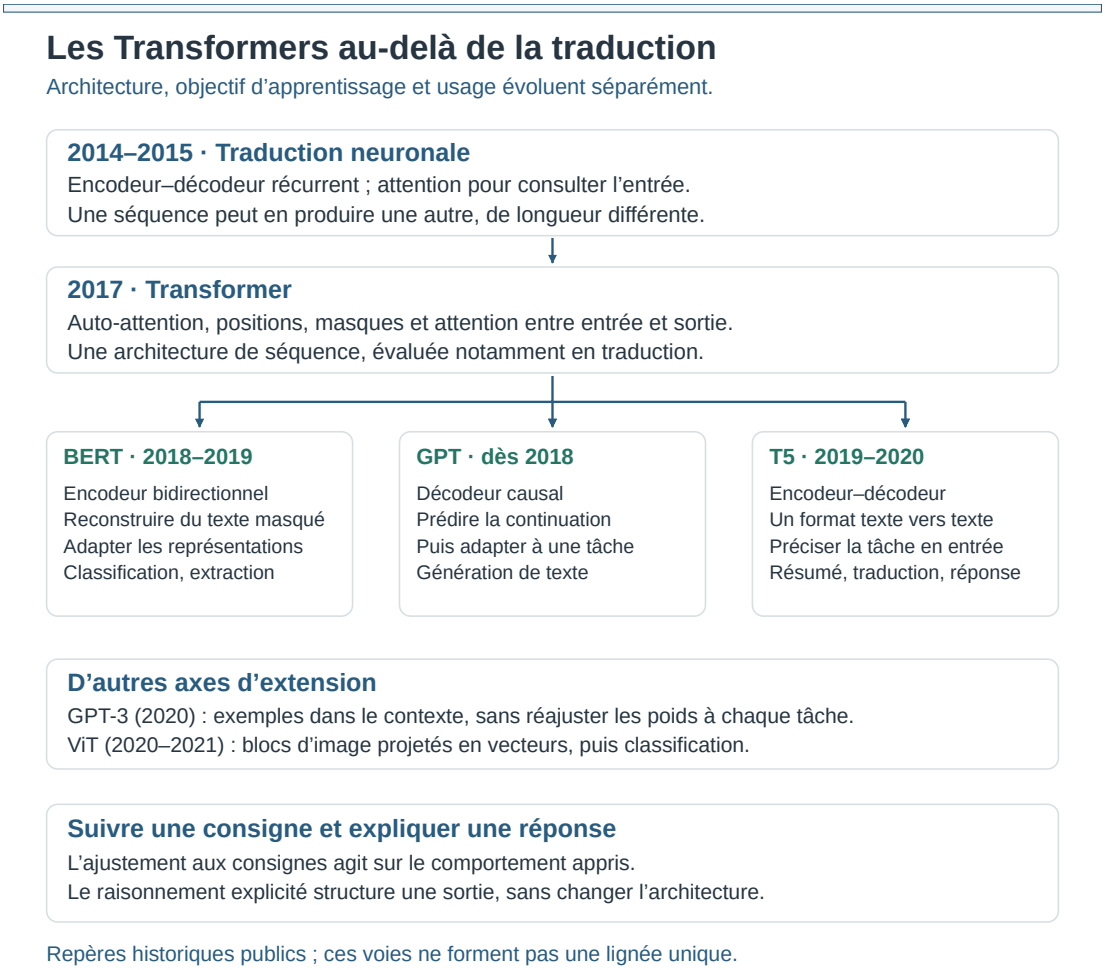
Comment les Transformers ont dépassé la traduction

L'évolution n'est pas celle d'un traducteur auquel on aurait simplement demandé de tout faire. Elle combine **une architecture réutilisable, de nouveaux objectifs d'apprentissage et de nouvelles façons de présenter les tâches**. La traduction a fourni un problème exigeant pour développer ces idées ; elle n'impose pas la limite des usages de l'architecture.

2014–2015 : représenter et aligner deux séquences. Les travaux encodeur–décodeur récurrents apprennent à passer d'une phrase à une autre, avec des longueurs et des ordres différents. L'attention de Bahdanau, Cho et Bengio permet au décodeur de consulter les représentations de l'entrée selon le mot à produire. Cette attention précède le Transformer ; les modèles concernés restent récurrents (Sutskever et al. 2014; Bahdanau et al. 2014).

2017 : changer le mécanisme de traitement. Le Transformer de Vaswani et ses coauteurs est évalué notamment en traduction. Il remplace la récurrence par des blocs combinant attention et transformations de chaque position. L'encodeur relie les éléments de l'entrée par auto-attention ; le décodeur utilise une auto-attention masquée sur la sortie et une attention vers l'encodeur. Positions et masques indiquent l'ordre et les informations accessibles. L'apprentissage peut traiter de nombreuses positions en parallèle, mais une génération autorégressive reste dépendante des jetons déjà produits (Vaswani et al. 2017).

Ce mécanisme calcule des relations entre représentations numériques. Il ne contient pas une obligation de traduire du français en anglais. Le choix des données, de l'objectif et de la sortie détermine ce qu'on lui apprend à faire. Cela n'implique pas que toutes les tâches deviennent identiques ou également faciles : chaque usage demande une représentation et une évaluation adaptées.



De la traduction à plusieurs familles d'usages : les dates sont des repères, les branches correspondent à des choix d'architecture et d'objectif.

2018–2019 : apprendre sur du texte, puis transférer. Deux voies deviennent emblématiques. Le premier GPT préentraîne un décodeur sur la continuation du texte, puis l'adapte à des tâches supervisées. BERT préentraîne un encodeur bidirectionnel, notamment en reconstruisant des éléments masqués, puis adapte ses représentations à la classification ou aux questions-réponses. BERT est présenté en 2018 et publié à NAACL en 2019 (Radford et al. 2018; Devlin et al. 2019).

Le changement décisif est aussi celui des données : on peut produire des cibles d'apprentissage à partir du texte lui-même, sans disposer d'une traduction humaine pour chaque passage. La réutilisation d'un préentraînement existait avant les Transformers ; ceux-ci en ont fourni de nouvelles réalisations puissantes. Un modèle préentraîné ne devient pas automatiquement un expert d'un réseau particulier : vocabulaire, données et critères doivent être vérifiés sur ce domaine.

2019–2020 : exprimer plusieurs tâches comme du texte vers du texte. T5 étudie un cadre commun dans lequel l'entrée précise la tâche et la sortie est textuelle : traduction, résumé, réponse à une question ou étiquette de classification. L'architecture encodeur–décodeur peut ainsi servir à plusieurs objectifs. Partager un format ne prouve pas que leurs difficultés ou leurs métriques sont interchangeables (Raffel et al. 2019).

2020 : fournir des exemples dans le contexte. GPT-3 étudie notamment l'adaptation à une tâche par quelques exemples donnés dans la requête, sans mise à jour des poids pour cette tâche à l'inférence. Ce qui change alors est le contexte fourni au modèle, pas son entraînement à chaque question. Cette capacité reste variable selon les tâches et les exemples ; elle ne constitue ni une mémoire permanente ni une garantie de généralisation (Brown et al. 2020).

2020–2021 : changer aussi la nature des entrées. Vision Transformer représente une image par une suite de blocs projetés en vecteurs, avec des informations de position, puis emploie un encodeur de type Transformer pour la classification. Un bloc d'image n'est pas un mot ; il devient une unité numérique traitable par une architecture de séquence. Les résultats dépendent du volume de préentraînement et du protocole de transfert. Ce travail illustre une extension au-delà du texte, pas une supériorité universelle sur les CNN (Dosovitskiy et al. 2020).

Famille	Information accessible dans le bloc principal	Usage illustratif
Encodeur, comme BERT	Les positions de l'entrée peuvent se consulter dans les deux sens.	Classer un ticket ou extraire une entité.
Décodeur causal, comme GPT	Une position ne consulte pas les positions futures de la séquence.	Produire une continuation ou une réponse.
Encodeur–décodeur, comme T5	L'entrée est représentée ; la sortie est générée en la consultant.	Transformer un document en résumé.

À retenir. Traduction, préentraînement, adaptation supervisée et utilisation d'exemples dans le contexte désignent des choses différentes. Le suivi de consigne constitue encore un objectif distinct. Le raisonnement explicité, présenté ensuite, est une façon de structurer une réponse ; ce n'est pas une quatrième architecture de Transformer.

Transformers tabulaires : des colonnes deviennent des jetons

Un tableau de mesures contient des colonnes hétérogènes : débit numérique, type d'équipement catégoriel, indicateur de valeur manquante. Chaque colonne porte une signification propre. Transformer ces valeurs en vecteurs permet à l'attention de représenter leurs interactions ; cela ne revient pas à donner un fichier CSV sous forme de texte à un modèle de langage.

TabTransformer contextualise les variables catégorielles par attention, puis combine leurs représentations avec les variables numériques dans un réseau de prédiction. **FT-Transformer** construit des jetons pour les caractéristiques numériques comme catégorielles et les traite ensemble. Dans ces architectures, l'attention s'applique aux caractéristiques d'une observation ; elle n'implique pas automatiquement une attention entre toutes les lignes du jeu de données. (Huang et al. 2020; Gorishniy et al. 2021)

Pour prévoir un incident, une ligne peut décrire un équipement avant l'incident : charge, température et catégorie de matériel. La cible reste extérieure aux entrées. Normalisation, imputation et vocabulaire des catégories se déterminent sur l'entraînement seulement. Un identifiant de machine peut favoriser une mémorisation trompeuse ; une mesure prise après la panne constitue une fuite de cible. La séparation temporelle et, selon l'usage, la séparation des équipements doivent correspondre au déploiement visé.

Le tableau suivant situe les choix sans décréter de vainqueur universel.

Approche	Représentation	Comparaison indispensable
Modèle linéaire	Colonnes préparées, relations simples	Point de départ lisible
Arbres renforcés	Découpages successifs des caractéristiques	Référence souvent forte sur tableaux
TabTransformer	Attention sur catégories, puis combinaison	Apport réel du contexte catégoriel
FT-Transformer	Jetons numériques et catégoriels	Gain face aux arbres et coût de calcul

La taille des données, les valeurs manquantes, les catégories nouvelles et la dérive comptent autant que le nom de l'architecture. Une meilleure moyenne sur plusieurs jeux publics ne démontre pas un avantage sur chaque réseau particulier.

Représenter, continuer et suivre une consigne

Ces trois objectifs demandent des comportements différents. Dans le préentraînement de **BERT**, reconstruire des éléments masqués à partir de leur contexte apprend des représentations utiles, par exemple pour classer un ticket ou reconnaître le nom d'un équipement. Cela ne constitue pas en soi une boucle de dialogue. Un modèle **GPT** autorégressif apprend à prédire la continuation d'une séquence ; produire une continuation plausible ne signifie pas nécessairement répondre à la demande de l'utilisateur.

Le verrou du suivi de consigne consiste à orienter une capacité de génération vers le comportement attendu. Un ajustement supervisé utilise des exemples de consignes et de réponses souhaitées. Des méthodes fondées sur les préférences comparent ensuite des réponses pour favoriser certains comportements ; l'apprentissage par renforcement à partir de retours humains peut contribuer à ce réglage, sans être systématique dans les assistants (Ouyang et al. 2022).

Pour « extraire les interfaces indisponibles dans ce journal », la réponse doit respecter le format, conserver les identifiants et signaler les cas ambigus. Inventer une interface fait échouer la tâche, même dans un texte convaincant. Un assistant conversationnel gère aussi les messages, le contexte et éventuellement des outils ; cette organisation ne garantit ni l'exactitude des faits ni le droit d'agir sur le réseau. L'évaluation sépare fidélité aux sources, respect de la consigne et réussite de la tâche.

Chaîne de raisonnement : utilité et limites

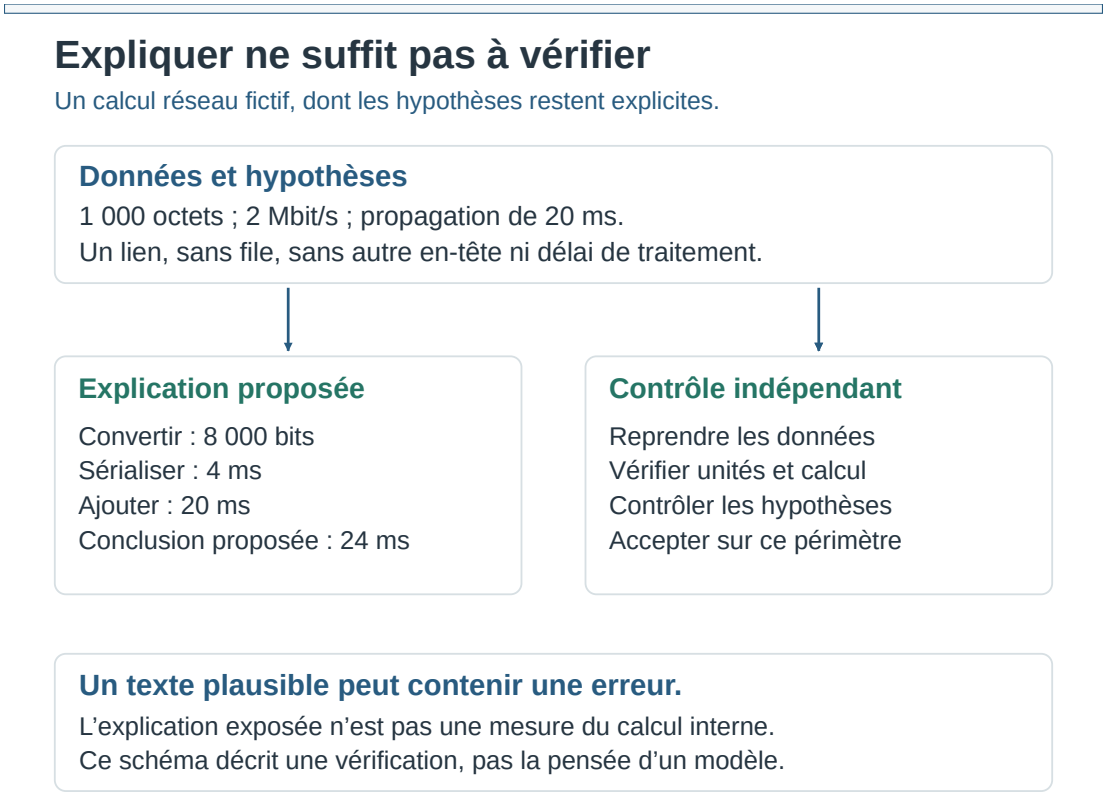
La **chaîne de raisonnement** (*chain of thought*, CoT) désigne une suite d'étapes intermédiaires produites avant la conclusion. Les travaux de Wei et ses coauteurs montrent en 2022 que des exemples de réponses ainsi structurées dans la requête peuvent améliorer certaines tâches arithmétiques, symboliques ou de sens commun. Le résultat est expérimental et dépend du modèle et de la tâche ; il ne suffit pas d'allonger une réponse pour la rendre correcte (Wei et al. 2022).

Une fois générées, ces étapes deviennent aussi une partie du contexte de la continuation. Elles peuvent expliciter une conversion d'unité, isoler une sous-question ou rendre une contradiction visible. Elles peuvent également contenir une erreur qui se propage. La CoT ne remplace ni les faits de départ, ni un outil de calcul, ni un protocole d'évaluation.

Exemple réseau fictif. Un paquet de 1 000 octets traverse un seul lien à 2 Mbit/s, avec 20 ms de propagation. On néglige les en-têtes supplémentaires, les files et le traitement. Le délai jusqu'à la réception du dernier bit est :

$$t = \frac{1\,000 \times 8}{2 \times 10^{6}} + 0{,}020 = 0{,}024\,s = 24\,ms.$$

L'explication utile identifie les hypothèses, convertit les octets en bits, calcule les 4 ms de sérialisation, puis ajoute la propagation. Une proposition de 20,5 ms qui aurait divisé les octets directement par le débit en bits serait fausse, même présentée dans un paragraphe convaincant. Un calcul indépendant contrôle l'arithmétique ; il faut encore vérifier que les hypothèses correspondent à la question posée.



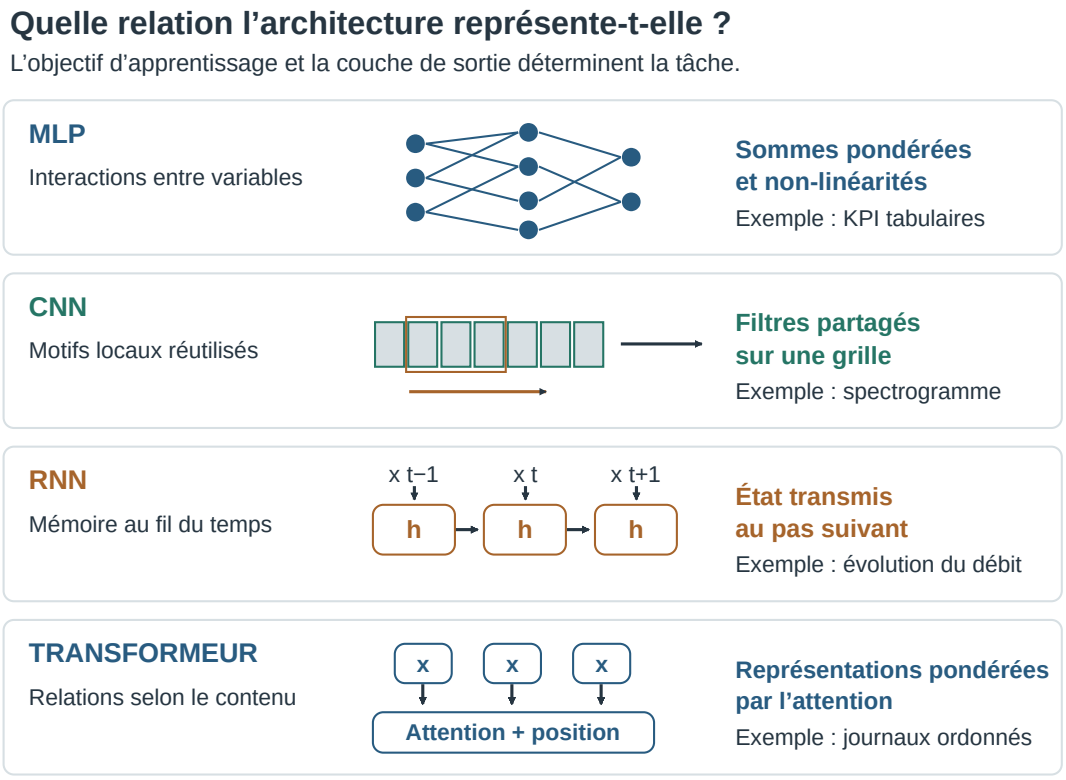
Explication générée et vérification : deux rôles distincts sur un exemple de délai réseau. Le schéma ne représente pas le calcul interne d'un modèle.

Trois objets à ne pas confondre. Le calcul interne du modèle est constitué d'opérations sur des représentations. Les étapes textuelles qu'il produit sont une sortie. L'explication remise à l'utilisateur peut enfin être une synthèse des éléments vérifiables. Ces trois objets ne sont pas équivalents. Les travaux de Turpin et ses coauteurs montrent que des explications peuvent rationaliser une réponse influencée par un biais sans signaler cette influence. Une CoT lisible n'est donc pas une transcription nécessairement fidèle des causes de la décision (Turpin et al. 2023).

La méthode de **sélection par concordance des réponses** (*self-consistency*) compare plusieurs raisonnements générés et agrège leurs réponses. Elle améliore certains résultats dans les expériences publiées, mais un accord majoritaire n'est pas une preuve indépendante : plusieurs réponses peuvent partager le même biais. Générer plusieurs propositions demande aussi davantage de calcul que produire une seule proposition comparable (Wang et al. 2022).

Pour évaluer l'intérêt du raisonnement explicité, comparer sur les mêmes cas la justesse de la réponse, la validité des étapes exposées, la fidélité aux données, le délai et les volumes traités. Un texte plus long peut être utile à la lecture sans améliorer la réponse finale. Le but pédagogique est une justification contrôlable ; celle-ci ne donne pas accès à toute la « pensée » d'un modèle.

Exercice. Refaire le calcul avec un débit de 4 Mbit/s, puis ajouter une attente en file de 3 ms. **Correction :** 2 ms de sérialisation, donc 22 ms sans file et 25 ms avec cette attente. Les trois calculs figurent dans `tools/verify_transformers_raisonnement_examples.py`. Ils illustrent des notions publiques, sans décrire le fonctionnement d'un système propriétaire.



Ces architectures peuvent servir plusieurs régimes d'apprentissage.

MLP, CNN, RNN et transformeur comparés selon leur mécanisme de calcul ; l'architecture se choisit en fonction des relations à représenter.

Normalisation par lots : stabiliser l'échelle des activations

La **normalisation par lots** (*batch normalization*, BatchNorm) transforme des activations intermédiaires à partir des statistiques d'un mini-lot. Pour une composante x_i, une écriture simplifiée est :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{v_B + \varepsilon}}, \; y_i = \gamma \hat{x}_i + \beta .$$

μ_B et v_B sont la moyenne et la variance calculées sur le lot ; $\varepsilon > 0$ évite une division par zéro. La pente γ et le décalage β sont appris : le réseau peut choisir l'échelle utile après normalisation. Avec un lot $(2, 4)$, on obtient une moyenne de 3, une variance de 1 et, en négligeant ε, les valeurs $(-1, 1)$.

L'entraînement utilise habituellement les statistiques des lots, tandis que l'inférence utilise des estimations accumulées pendant l'entraînement. Il faut donc passer le modèle en mode évaluation. Des lots très petits ou non représentatifs peuvent poser problème. BatchNorm ne remplace ni la préparation des entrées, ni la régularisation, ni le contrôle d'un changement de distribution. La **normalisation par couche** (*layer normalization*) calcule ses statistiques selon d'autres axes, généralement pour chaque exemple ; les deux mécanismes ne sont pas interchangeables (Ioffe et Szegedy 2015; Ba et al. 2016).

Augmenter les données est une autre opération. On construit de nouveaux exemples à partir de ceux de l'entraînement, par exemple en ajoutant un bruit contrôlé ou un décalage compatible avec l'acquisition. La transformation doit préserver la cible et les contraintes physiques. Tourner une image peut être pertinent pour une tâche et changer le sens dans une autre ; modifier un signal radio peut changer sa modulation ou son rapport signal/bruit. L'augmentation reste dans le jeu d'entraînement, après séparation des jeux : des variantes du même exemple placées dans le test créeraient une fuite. Elle n'invente pas des situations absentes du corpus (Keras developers 2026a).

PINN : apprendre avec une équation physique

Un **PINN** (*physics-informed neural network*, réseau de neurones informé par la physique) apprend à estimer une grandeur en réduisant aussi l'écart à une équation connue. La physique ajoute une contrainte ; le modèle physique lui-même reste à vérifier. Dans un **problème direct**, on connaît les paramètres physiques et on cherche les valeurs de la grandeur. Dans un **problème inverse**, on part des mesures pour retrouver certains paramètres (Raissi et al. 2019).

Prenons une décroissance normalisée, par exemple l'écart de température d'un équipement dans un modèle thermique très simplifié. On divise cet écart par sa valeur initiale non nulle et le temps écoulé par une durée caractéristique : u et t sont donc sans unité. Le temps $t=1$ correspond à cette durée, pas nécessairement à une seconde. L'équation devient :

$$u'(t)+u(t)=0, u(0)=1.$$

Le réseau $u_\theta(t)$ propose une valeur. La différentiation automatique calcule sa dérivée par rapport à l'entrée t, puis le résidu $r_\theta(t)=u'_\theta(t)+u_\theta(t)$. Ce résidu est évalué en des **points de collocation**, où une mesure réelle n'est pas nécessairement disponible. Une perte illustrative combine alors :

$$J(\theta)=\lambda_d MSE_{mesures}+\lambda_r \frac{1}{N_r}\sum_{i=1}^{N_r} r_\theta(t_i)^2+\lambda_0(u_\theta(0)-1)^2.$$

MSE signifie *mean squared error*, **erreur quadratique moyenne** : on calcule les écarts entre prédictions et mesures, on les élève au carré, puis on en prend la moyenne. $N_r>0$ est le nombre de points où l'équation est contrôlée. Les coefficients λ_d, λ_r et λ_0, non négatifs, règlent le poids des trois composantes. Sans mesure disponible, on omet le premier terme ; on ne calcule pas une moyenne sur un ensemble vide.

Le premier terme rapproche la prédiction des mesures disponibles ; le deuxième impose approximativement l'équation ; le troisième contrôle la condition initiale. Pour une équation aux dérivées partielles, des conditions aux limites peuvent s'ajouter. Les variables et les poids doivent être mis à l'échelle : additionner sans précaution des erreurs de dimensions ou d'amplitudes très différentes peut déséquilibrer l'apprentissage.

Contre-exemples calculables. La solution e^{-t} satisfait l'équation et la condition initiale. La fonction nulle a elle aussi un résidu nul, mais viole $u(0)=1$: la physique sans condition initiale ne suffit pas. La proposition e^{-2t} respecte la condition initiale mais son résidu vaut $-e^{-2t}$, soit environ $-0,1353$ à $t=1$. Ce calcul illustre les termes de la perte ; il ne simule pas l'entraînement d'un réseau.

Un PINN n'est pas automatiquement plus précis ni plus rapide qu'un solveur numérique. Une faible perte sur les points choisis ne garantit pas une faible erreur partout. Les équations raides, les échelles multiples et le déséquilibre des gradients peuvent rendre l'optimisation difficile (Wang et al. 2020). On évalue donc sur d'autres points, on contrôle les conditions aux limites et on compare à une solution analytique ou à un solveur de référence. Les erreurs du modèle physique, le bruit et l'identifiabilité des paramètres restent des questions distinctes.

Dans les réseaux, l'intérêt peut concerner un champ de propagation ou un modèle thermique d'équipement. Ajouter une pénalité à un classifieur de trafic ne le transforme pas automatiquement en PINN : l'équation, son domaine de validité et les grandeurs représentées doivent être explicités.

Minimax et PINN : deux composantes, une perte à équilibrer

Minimax choisit une décision en tenant compte de la réponse la plus défavorable d'un adversaire modélisé. Pour un coût à réduire, on minimise le plus grand coût possible. Exemple synthétique : un choix A coûte 2 ou 9 selon la réponse adverse ; B coûte 5 ou 6. Minimax retient B, dont le pire coût est 6, plutôt que A, dont le pire coût est 9.

Dans un arbre de jeu, l'algorithme évalue les feuilles, puis remonte leurs valeurs : un nœud MIN retient le minimum de ses enfants, un nœud MAX leur maximum, selon le joueur qui décide. L'**élagage alpha-bêta** évite d'explorer les branches qui ne peuvent plus améliorer la valeur retenue ; il conserve la valeur minimax du même arbre évalué. Il ne

supprime pas des connexions neuronales. La normalisation *min-max*, qui remet des valeurs sur une échelle donnée, est encore une autre opération. (Knuth et Moore 1975)

Deux composantes pour le PINN. On peut regrouper sa perte en :

- une composante L_d, qui mesure l'écart aux données disponibles ;
- une composante L_p, qui pénalise le résidu des équations et les conditions initiales ou aux limites. Dans l'exemple précédent, L_p regroupe donc deux termes.

Un PINN classique minimise une somme pondérée $\alpha L_d(\theta)+\beta L_p(\theta)$, avec des poids fixés. Deux composantes ne signifient pas deux objectifs opposés : on cherche à réduire les deux, après mise à l'échelle.

Un petit parallèle calculable. Supposons L_d et L_p mis sur une même échelle, sans unité. Un poids q peut privilégier l'une ou l'autre. Si l'on choisit le poids le plus défavorable, avec $0 \le q \le 1$:

$$\min_{\theta} \max_{0\le q\le 1} \left[q\,L_d(\theta)+(1-q)\,L_p(\theta)\right] = \min_{\theta} \max\{L_d(\theta), L_p(\theta)\}.$$

Le maximum retient la plus grande composante : $q=1$ privilégie les données, $q=0$ la physique. Pour deux propositions donnant les erreurs $(0,1,0,9)$ et $(0,4,0,4)$, ce min–max préfère la seconde : son pire écart vaut 0,4 plutôt que 0,9. Une somme pondérée donnant 90 % du poids aux données préférerait la première, avec 0,18 contre 0,4. C'est une illustration de deux règles de choix, pas la définition d'un PINN.

Dans certaines variantes **auto-adaptatives (SA-PINN)**, les paramètres du réseau descendent la perte tandis que des poids par point montent pour accentuer les erreurs persistantes. Ces poids locaux ne sont pas le seul q du petit exemple : ils donnent une priorité à chaque point difficile (McClenny et Braga-Neto 2020). Leur domaine et leur règle d'évolution comptent : un poids positif sans borne pourrait croître indéfiniment si l'erreur reste non nulle. Cette dynamique ne garantit donc pas un maximum fini ni une convergence. Le parallèle porte sur la descente/montée continue ; il n'introduit ni arbre de jeu ni élagage alpha-bêta.

Évaluer et adapter en préservant le jeu de test

Données, évaluation comparative et GPU : trois verrous distincts

Apprendre une relation. Dans un apprentissage supervisé, les annotations associent des observations à la réponse attendue. Pour des spectrogrammes, elles peuvent nommer un type de signal ; pour des tickets, une cause confirmée. Des annotations incohérentes ou un corpus qui ignore certains équipements limitent ce qui peut être appris. Davantage de données semblables ne corrige pas nécessairement ce défaut de couverture. Un grand corpus d'images tel qu'ImageNet illustre le rôle d'une ressource annotée ; ses performances ne qualifient pas un diagnostic réseau (ImageNet project 2026).

Mesurer un progrès. Un **banc d'évaluation comparative** (*benchmark*) combine une tâche, des données, un protocole et des mesures. Il rend les comparaisons interprétables lorsque ces conditions sont partagées. Un jeu de données seul ne fixe ni le budget de calcul ni les règles d'évaluation. Séparer entraînement et test évite de mesurer seulement la mémorisation ; consulter le test de façon répétée pour choisir un modèle finit toutefois par l'utiliser comme validation. Pour un réseau, une séparation par site ou par période teste mieux certains changements de domaine qu'un tirage aléatoire de lignes presque identiques.

Rendre le calcul réalisable. Un **GPU** exécute en parallèle de nombreuses opérations similaires, notamment sur les matrices et les lots d'exemples. **CUDA** fournit un environnement de programmation pour les GPU NVIDIA : ce n'est ni un modèle d'apprentissage ni un jeu de données (NVIDIA 2026). Des bibliothèques exploitent ce matériel pour entraîner et exécuter les modèles. Le gain dépend du travail disponible, de la mémoire, des transferts et des opérations prises en charge. Un petit calcul peut rester plus rapide sur CPU.

Ce qui reste à vérifier. Le GPU réduit potentiellement le temps de calcul ; il ne répare pas des annotations fausses. Un meilleur score sur un banc d'évaluation comparative ne prouve pas la robustesse sur un autre réseau. Une architecture

adaptée, des données pertinentes et un budget de calcul suffisant répondent à des contraintes complémentaires : aucun de ces trois éléments ne remplace les autres.

La séparation **entraînement / validation / test** réserve respectivement l'ajustement des paramètres, les choix de modèle et l'estimation finale de performance. La **validation croisée** répète l'ajustement sur plusieurs **plis**, des groupes gardés hors de l'entraînement à tour de rôle. Avec cinq plis, on entraîne cinq fois sur quatre groupes et on valide sur le cinquième, puis on compare les scores. Si elle sert à choisir les hyperparamètres, une évaluation externe reste nécessaire.

Pour le réseau, séparer par temps, site ou équipement évite de placer des observations presque identiques des deux côtés. Normalisation, PCA et sélection de variables s'ajustent uniquement sur le pli d'entraînement. La section « Paramètres, hyperparamètres et choix du modèle » détaille la validation croisée ; L23 illustre la séparation entraînement / validation / test (scikit-learn developers 2026b).

Mesure	Ce qu'elle mesure et ses limites
Exactitude (*accuracy*)	proportion d'observations correctement classées ; peut masquer l'échec sur une classe rare
Matrice de confusion	vrais/faux positifs et négatifs au seuil retenu ; révèle la répartition des erreurs
Précision, rappel, F1	précision des alertes, fraction d'incidents détectés et moyenne harmonique ; F1 n'intègre pas les vrais négatifs ni tous les coûts métier
RMSE	racine de l'erreur quadratique moyenne, dans l'unité de la cible ; sensible aux grands écarts, mais ne mesure pas directement le risque de délais extrêmes
ROC-AUC et aire sous la courbe précision–rappel	classement des scores sur plusieurs seuils ; préciser la courbe et, pour PR, la convention d'intégration ; elles ne prouvent pas la calibration

La performance doit être accompagnée d'effectifs, de dispersion et du seuil choisi. Les biais, variance et bruit, détaillés dans le livre complet, expliquent pourquoi améliorer l'ajustement d'entraînement peut dégrader la généralisation (scikit-learn developers 2026n).

Lire une erreur de régression

On dispose de n observations : y_i est la valeur mesurée, $\hat{y}_i$ la prévision, et $e_i = \hat{y}_i - y_i$ leur écart. La somme $\sum_{i=1}^{n}$ ❑ additionne les écarts sur les n exemples.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |e_i|, MSE = \frac{1}{n}\sum_{i=1}^{n} e_i^2, RMSE = \sqrt{MSE}.$$

La **MAE**, erreur absolue moyenne, compte chaque écart en valeur absolue. La **MSE**, erreur quadratique moyenne, élève les écarts au carré : les grandes erreurs pèsent davantage. La **RMSE**, sa racine carrée, retrouve l'unité de la mesure. Pour deux erreurs de 1 et 3 ms, la MAE vaut 2 ms, la MSE 5 ms² et la RMSE environ 2,24 ms. Aucun de ces nombres ne donne, à lui seul, la probabilité de dépasser une échéance.

La **MAPE**, erreur absolue moyenne en pourcentage, rapporte chaque erreur à la valeur mesurée :

$$MAPE = \frac{100}{n}\sum_{i=1}^{n} \left|\frac{e_i}{y_i}\right|, R^2 = 1 - \frac{\sum_{i=1}^{n} e_i^2}{\sum_{i=1}^{n} (y_i - \overline{y})^2}.$$

La MAPE est indéfinie si une valeur y_i est nulle et devient très sensible aux valeurs proches de zéro. Une erreur de 2 sur une valeur de 20 compte pour 10 %, mais la même erreur sur 2 compte pour 100 %. Modifier le dénominateur change la métrique : il faut annoncer cette convention.

Dans R^2, $\overline{y}$ est la moyenne des valeurs mesurées. Le dénominateur représente l'erreur d'une prévision constante égale à cette moyenne. $R^2=1$ indique des prévisions exactes ; $R^2=0$ une erreur quadratique égale à celle de cette référence ; une valeur négative indique une erreur supérieure. Ce n'est pas un pourcentage de bonnes réponses. Si toutes les cibles sont identiques, le dénominateur est nul et cette formule ne définit pas le score. Les conventions logicielles doivent être précisées (scikit-learn developers 2026n).

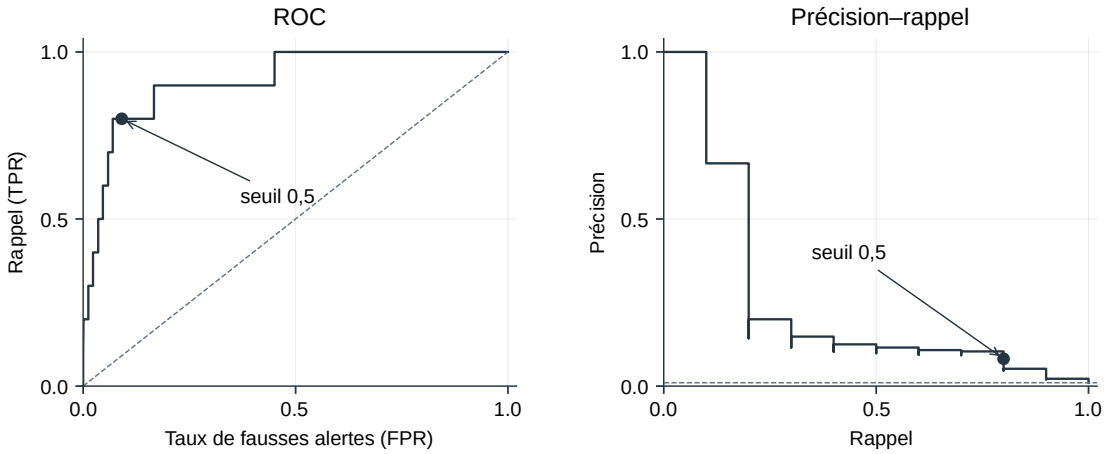
Du seuil aux courbes ROC et précision–rappel

Un classifieur peut donner un score, puis déclencher une alerte quand ce score dépasse un **seuil**. On compte les **vrais positifs** (VP : incidents signalés), **faux positifs** (FP : alertes sans incident), **faux négatifs** (FN : incidents manqués) et **vrais négatifs** (VN : situations normales sans alerte).

$$rappel = TPR = \frac{VP}{VP+FN}, FPR = \frac{FP}{FP+VN}, precision = \frac{VP}{VP+FP}.$$

TPR (*True Positive Rate*) est le taux de détection parmi les incidents ; **FPR** (*False Positive Rate*) le taux de fausses alertes parmi les situations normales. La précision compte la part d'incidents parmi les alertes. Un dénominateur nul demande une convention explicite.

Sur 1 000 observations dont 10 incidents, supposons 8 VP, 2 FN, 90 FP et 900 VN. Le rappel vaut 80 %, le FPR environ 9,1 % et la précision seulement 8,2 % : les alertes sont nombreuses pour peu d'incidents. L'exactitude de 90,8 % masque cette difficulté.



Les mêmes scores synthétiques donnent une courbe ROC et une courbe précision–rappel. Le point au seuil 0,5 correspond à 8 incidents détectés et 90 fausses alertes sur 1 000 observations.

La courbe **ROC** (*Receiver Operating Characteristic*) place le FPR en abscisse et le TPR en ordonnée quand le seuil varie. Son **AUC** (*Area Under the Curve*) résume l'aire sous cette courbe : plus elle est proche de 1, meilleur est le classement des deux classes dans ce protocole. Une aire élevée ne choisit pas le seuil opérationnel et ne garantit pas des probabilités calibrées.

La courbe **précision–rappel** montre plus directement la qualité des alertes quand les incidents sont rares. La proportion d'incidents constitue un repère de précision pour un classement sans information. Son aire dépend de la convention d'interpolation ; la **précision moyenne** (*average precision*, AP) additionne les gains de rappel pondérés par la précision obtenue, au lieu d'appliquer une simple aire trapézoïdale. Le score, le seuil et le coût des erreurs répondent à trois questions distinctes (Saito et Rehmsmeier 2015; scikit-learn developers 2026n).

Deux compléments : Hamming et Matthews

Quand plusieurs étiquettes peuvent être vraies simultanément, la **perte de Hamming** compte la fraction de décisions d'étiquette erronées. Pour trois étiquettes attendues $(1,0,1)$ et prédites $(1,1,1)$, elle vaut $1/3$. L'exactitude du vecteur complet vaut pourtant zéro : les deux métriques ne posent pas la même question.

Le **coefficient de corrélation de Matthews** (**MCC**) utilise les quatre cases de la matrice de confusion binaire :

$$MCC = \frac{VP \cdot VN - FP \cdot FN}{\sqrt{(VP+FP)(VP+FN)(VN+FP)(VN+FN)}}.$$

Avec un dénominateur non nul, il varie de -1 (désaccord parfait) à 1 (accord parfait) ; zéro correspond à l'absence de corrélation dans ces observations. Il fournit un complément à F1 et à l'exactitude, notamment lorsque les classes sont déséquilibrées. Il ne remplace pas l'examen du coût des incidents manqués et des fausses alertes ; les cas où le dénominateur est nul demandent aussi une convention (Chicco et Jurman 2020).

Les paradigmes spécialisés modifient surtout les données disponibles et la façon de les exploiter :

- **Semi-supervisé** : combiner peu d'étiquettes et beaucoup d'observations non étiquetées. Les pseudo-étiquettes peuvent amplifier une erreur initiale.
- **Auto-supervisé** : fabriquer une cible à partir de l'entrée, par masquage, prédiction de contexte ou contraste. La tâche auxiliaire doit préparer la tâche réseau visée.
- **Transfert (transfer learning)** : réutiliser des représentations ou paramètres appris ailleurs, puis éventuellement les ajuster. Un décalage entre domaines peut provoquer un transfert négatif.
- **Apprentissage actif (active learning)** : choisir quels exemples faire annoter, par exemple les alarmes les plus incertaines. Il faut compter le coût humain et conserver une évaluation indépendante de cette sélection.
- **Méta-apprentissage (meta-learning)** : apprendre sur plusieurs tâches une initialisation ou une procédure d'adaptation. Quelques exemples sur une nouvelle tâche ne suffisent que si les tâches d'apprentissage ont préparé cette adaptation ; ce n'est pas une propriété générale de KNN.

Ces distinctions prolongent les axes « signal d'apprentissage » et « réutiliser et adapter un modèle ». Elles se combinent avec les familles précédentes ; leur comparaison doit annoncer données, budget d'annotation, tâche et protocole. Les références sur l'apprentissage semi-supervisé, le contraste, l'apprentissage actif et MAML donnent des formulations précises (scikit-learn developers 2026r; Chen et al. 2020; Settles 2009; Finn et al. 2017).

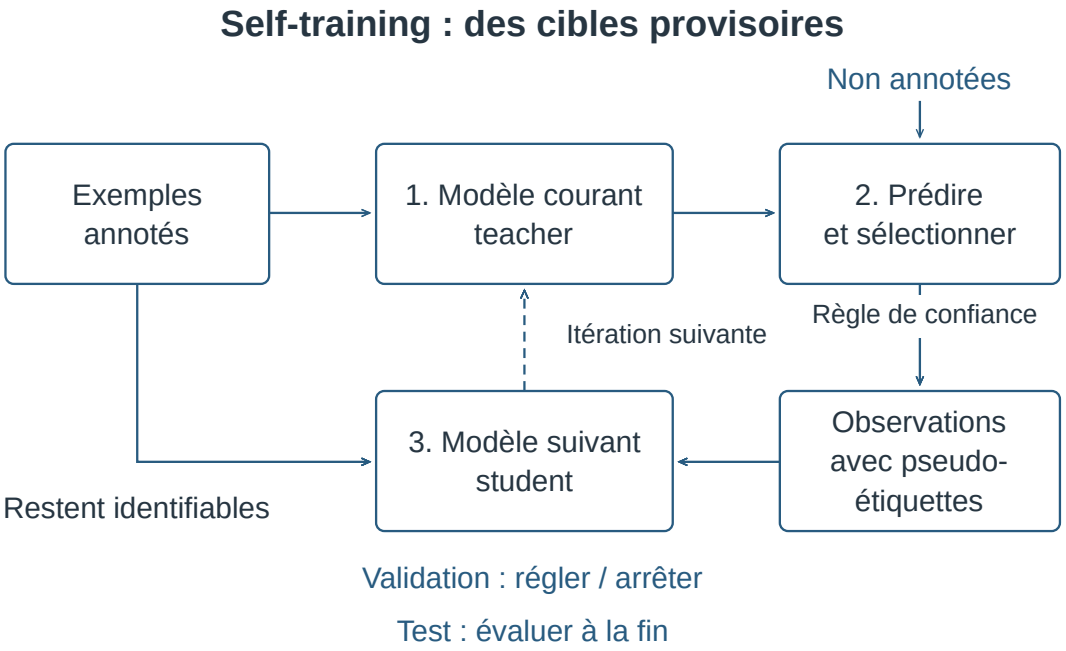
Une boucle de self-training avec des pseudo-étiquettes

Le **self-training**, ou apprentissage par réutilisation de ses propres prédictions, est une manière de réaliser un apprentissage semi-supervisé. Un modèle entraîné sur des exemples annotés propose des étiquettes pour d'autres observations. Ces **pseudo-étiquettes** servent de cibles provisoires lors d'un entraînement suivant. Elles gardent leur origine automatique : les accepter ne les transforme pas en annotations humaines.

La boucle élémentaire comporte trois étapes :

1. **Apprendre un modèle initial.** On utilise les exemples annotés du seul ensemble d'entraînement. Le modèle courant joue le rôle de **teacher** (« enseignant ») lorsqu'il produit les prochaines cibles.
2. **Prédire et sélectionner.** Le teacher calcule des scores sur les observations non annotées. On retient les propositions qui satisfont une règle annoncée, par exemple un seuil de confiance, puis on conserve la classe proposée et l'origine de la pseudo-étiquette.
3. **Entraîner le modèle suivant.** Le **student** (« élève ») apprend sur les observations associées à leurs annotations humaines ou aux pseudo-étiquettes sélectionnées, avec une pondération éventuellement différente. On examine la validation avant de répéter. Le student peut devenir le teacher de l'itération suivante ; les deux rôles n'exigent pas deux architectures différentes.

Cette famille de boucles est ancienne ; l'utilisation de prédictions sélectionnées pour étendre progressivement des données d'apprentissage apparaît notamment dans les travaux de Yarowsky (Yarowsky 1995).



Dans le self-training, les exemples annotés alimentent l'apprentissage initial et les suivants. Le modèle courant propose des pseudo-étiquettes sélectionnées, puis le modèle suivant peut reprendre ce rôle. Validation et test restent hors de la boucle d'entraînement.

Confiance et hypothèses. Un score élevé n'est pas automatiquement une probabilité fiable. La calibration doit être vérifiée avant d'interpréter ce score comme une probabilité de succès. On règle la sélection avec une validation distincte de l'entraînement ; le test reste réservé à l'évaluation finale (Guo et al. 2017). La boucle suppose aussi que les observations non annotées relèvent des classes et des conditions que le modèle doit apprendre. Un changement de domaine peut rendre une prédiction erronée très confiante.

Exemple illustratif. Supposons 20 extraits audio annotés et 200 sans annotation. Pour trois extraits, le teacher attribue à sa classe préférée des scores de 0,97, 0,91 et 0,62. Une règle de sélection au seuil de 0,95 ne retient que la première proposition. Ces nombres servent à montrer la règle ; ils ne donnent ni la qualité du modèle ni un seuil conseillé. Même la proposition à 0,97 peut être fausse.

Le **biais de confirmation** apparaît lorsque le modèle réapprend ses propres erreurs et les renforce. Arazo et ses collègues étudient ce problème dans le pseudo-étiquetage de réseaux profonds (Arazo et al. 2019). Une classe rare, initialement mal reconnue, peut aussi fournir peu de pseudo-étiquettes et devenir moins présente dans les entraînements suivants. Examiner les effectifs sélectionnés, les erreurs et le rappel par classe aide à repérer ce déplacement. La validation sert à décider de poursuivre, modifier ou arrêter la boucle ; le nombre de cibles acceptées, à lui seul, ne prouve pas un gain. On conserve toujours les annotations humaines identifiables et un test indépendant des choix successifs.

Expliquer une sortie et éprouver sa robustesse

L'**IA explicable** (**XAI**, *Explainable Artificial Intelligence*) cherche à rendre une sortie ou un fonctionnement examinable. Une explication **locale** porte sur une décision particulière ; une analyse **globale** sur des tendances du modèle. Un petit arbre peut exposer directement sa règle de décision. Une méthode d'attribution appliquée après calcul estime, selon sa propre convention, la contribution de variables.

Pour une alarme réseau, indiquer que « pertes » a fortement contribué au score peut orienter une enquête. Cela ne prouve pas que les pertes ont causé l'incident. On examine la **fidélité** de l'explication au calcul effectué et sa **stabilité** lorsque l'entrée change légèrement. Les tests d'Adebayo et ses collègues montrent qu'une carte visuellement convaincante peut

même rester presque identique après modification du modèle. Une explication doit donc être éprouvée ; elle n'est pas un certificat de confiance (Adebayo et al. 2018).

Une attaque par **évasion** modifie l'entrée utilisée à l'inférence pour provoquer une mauvaise sortie. Un **empoisonnement** altère les données ou le processus d'apprentissage ; une **porte dérobée** cherche à faire apparaître un comportement lorsqu'un déclencheur précis est présent. La petite perturbation d'une image n'épuise pas ces risques : signaux, télémétrie, textes et documents consultés peuvent aussi servir d'entrées hostiles.

Le test de robustesse précise ce que l'attaquant contrôle, son budget de modification et le domaine d'entrées admissible. On mesure ensuite les erreurs, les abstentions et le fonctionnement du repli sur des cas indépendants. Une défense efficace contre une famille d'attaques ne prouve pas la résistance à toutes les autres (NIST 2025).

Des compétences communes, des rôles différents

La **compétence** est ce que l'on sait faire ; le **rôle**, la mission confiée et son objectif. En cybersécurité, le défenseur protège le système ; le testeur cherche à mettre cette protection en défaut, dans un périmètre autorisé. Ils peuvent maîtriser les mêmes outils, mais leurs critères de réussite diffèrent. Leurs objectifs locaux s'opposent au service d'un but commun : améliorer la sécurité. Un test d'intrusion sans faille découverte ne démontre pas l'invulnérabilité (Scarfone et al. 2008).

Cette organisation est aussi utile en IA. Le concepteur construit ou adapte une solution ; l'évaluateur recherche ses échecs, ses biais et les moyens de contourner ses garde-fous. Il éprouve les choix de conception sur des tests réservés à l'évaluation, y compris lorsque les défauts signalés retardent la livraison (National Institute of Standards and Technology 2023).

En innovation, le risque d'être **juge et partie** apparaît lorsqu'on porte une idée et décide seul si elle a tenu ses promesses. L'attachement au projet peut conduire à privilégier les résultats qui le confortent. Fixer les critères avant les essais, comparer avec une solution de référence et faire intervenir un évaluateur libre de conclure à l'échec aident à limiter ce biais (National Institute of Standards and Technology 2023).

Avec des agents de codage, l'un peut proposer un correctif tandis qu'un autre cherche un contre-exemple et écrit un test, par exemple sur une entrée vide. Le désaccord devient utile lorsqu'il produit une preuve que l'on peut rejouer. Deux instances du même LLM peuvent toutefois partager les mêmes angles morts : leur dialogue ne suffit pas à constituer un audit indépendant.

Cette confrontation peut débloquer des situations et accélérer les itérations : une forme d'**hyperagilité façon Thunderbolt**, pour parler Mac.

Passer du panorama à une expérience réseau

Pour estimer un délai en congestion, comparer régression linéaire, forêt aléatoire et boosting sur les mêmes horizons, sites et budgets. Pour classer des signaux radio, comparer un SVM sur variables choisies à la main à un CNN qui apprend sa représentation. K-means, GMM et DBSCAN explorent des régimes inconnus selon des hypothèses différentes ; une projection 2D aide à les inspecter sans prouver leur séparation. Décider sur les mesures et le coût des erreurs.

Reprendre l'exemple de l'interruption de liaison. Une forêt aléatoire est un choix de modèle ensembliste pour la classification supervisée.

Sans étiquettes, DBSCAN peut regrouper les observations et marquer certains points comme « bruit », mais il ne détermine pas lesquels correspondent à une interruption de liaison. Une ACP peut préparer les variables des deux approches. Adam serait un choix d'optimiseur pour un modèle différentiable, pas une alternative à « supervisé ».

Enfin, le score F1 résume la précision et le rappel pour un seuil de décision donné ; il sert à évaluer les prédictions, sans mesurer le coût opérationnel des erreurs.

Vérifier sa compréhension. Pourquoi « supervisé ou forêt aléatoire ? » est-il un faux choix ? Pourquoi une carte UMAP ne suffit-elle pas à identifier une interruption de liaison ? Pourquoi faut-il séparer choix du seuil et test final ? Nommer l'axe concerné aide à situer ces questions avant de choisir un algorithme.

Les laboratoires L23 à L27 et le notebook ACP/SVD/RPCA apportent des expériences ciblées de classification rare, de RL contraint et de réduction de dimension, sans couvrir tous les algorithmes du panorama. Pour les étendre, garder le protocole fixe et préciser les nouvelles hypothèses.

Quatre cas industriels en 2023

Ma présentation *Tous Compétents IA* de 2023 illustrait ces méthodes par quatre cas. Voici leurs résultats et les suites alors envisagées, sans bilan de déploiement en 2026 (Morard 2023).

Antenne satellite : relier mesures et mise au point. Pour l'IVVQ (intégration, vérification, validation et qualification), des mesures du **RSSI**, indicateur de puissance du signal reçu, étaient fusionnées pour construire par régression une surface de prévision. Détection d'anomalies et classification aidaient ensuite la mise au point du produit. L'**ACU** (*Antenna Control Unit*) est l'unité de commande de l'antenne. L'exemple illustrait la chaîne mesure–modèle–diagnostic, sans effectif ni gain chiffré.

COMTICS : un démonstrateur de connexion par visage. Une preuve de concept (*POC*) intégrait un algorithme Gemalto de reconnaissance faciale dans un terminal pour expérimenter la connexion et l'expérience utilisateur. La vidéo et la démonstration montraient le fonctionnement, sans taux d'erreur ni comparaison de robustesse selon la population ou l'éclairage. Le cadre des données et des usages biométriques complète aujourd'hui la lecture de cet exemple.

IFF : calibrer avec une trajectoire de référence estimée. L'**IFF** (*Identification Friend or Foe*) désigne un système d'identification coopérative. Le cas partait de plots, des détections successives d'un aéronef. Un lissage construisait une trajectoire théorique ; la trajectoire réelle demeurait inconnue. Régression, classification et localisation aidaient à distinguer les plots directs de ceux dus à des réflexions et à estimer les performances de l'équipement pour sa calibration. Le gain de temps d'installation visé n'était pas chiffré. La trajectoire lissée reste une estimation ; l'utiliser comme vérité indépendante introduirait un raisonnement circulaire.

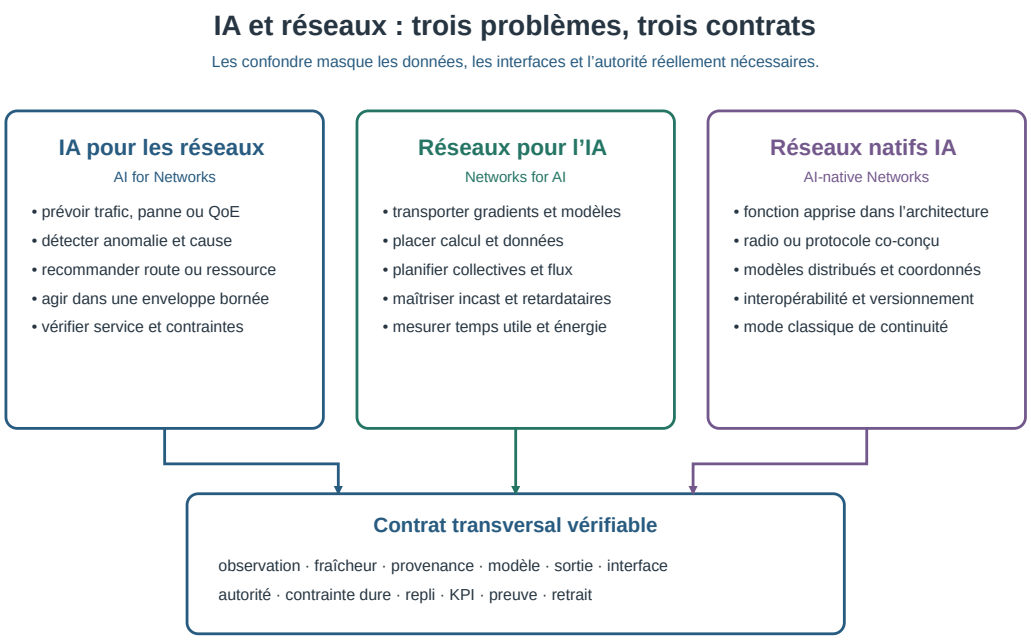
ACAS-X : remplacer une grande table par une approximation. Un modèle apprenait la fonction donnant un coût à partir de plusieurs variables, au lieu de conserver toute la table du système anticollision. La présentation annonçait « 5 Go → 10 Mo », sans préciser les formats ni le protocole de mesure. Sur 60 scénarios parmi 4 000 modèles de rencontre, elle rapportait un critère de sécurité égal : l'IA était jugée meilleure dans 33 cas, la table dans 27, selon un critère de dangerosité dont le protocole de comparaison n'était pas détaillé.

Ces 60 cas couvrent **1,5 %** du corpus — le support indiquait 1,3 % — et ne prouvent pas l'équivalence sur les 3 940 cas restants. Les essais en vol et le suivi des travaux EUROCAE WG-114 / SAE G-34 étaient envisagés, sans réalisation ni certification établies.

Ces cas relient analyse de données, apprentissage, intégration et preuve. Ils montrent surtout pourquoi une démonstration, une amélioration sur quelques scénarios et une qualification complète ne sont pas des résultats interchangeables.

Trois domaines à distinguer

« IA et réseaux » recouvre trois directions qui partagent des outils, mais pas le même objet à optimiser ni la même preuve. Le périmètre d'un projet se précise avec une étiquette principale et, si nécessaire, une étiquette secondaire ; la seule formule « IA native » ne distingue pas ces directions.



Taxonomie des trois domaines : IA pour les réseaux, réseaux pour l'IA et réseaux natifs IA

Domaine	Objet principal	Question d'ingénierie	Exemples dans ce guide	Preuve décisive
AI for Networks — IA pour les réseaux	le réseau et ses services sont le système à observer ou commander	une estimation ou une décision apprise améliore-t-elle un effet de service sans violer les invariants ?	prévision de trafic, NWDAF/MDAF, détection d'anomalie, routage appris, RIC, RCA	comparaison appariée à des méthodes de référence, contraintes, mesure du service réel, repli
Networks for AI — réseaux pour l'IA	le réseau est le substrat d'entraînement ou d'inférence	le transport, le placement et la reprise servent-ils correctement une charge IA distribuée ?	collectives, incast, placement, ECN, hiérarchie de workers, transfert de modèles	temps de tâche, débit utile, équité, coût, énergie et reprise sous panne
AI-native Networks — réseaux natifs IA	l'apprentissage et l'inférence font partie du contrat architectural d'une fonction réseau	le cycle données–modèle–décision–assurance est-il conçu comme une fonction de réseau versionnée et interopérable ?	fonction radio apprise, communications sémantiques, modèles servis par un RIC, fonction de contrôle conçue autour d'une inférence	contrat d'interface et de modèle, sûreté de la boucle complète, interopérabilité, comportement sans modèle

Cette taxonomie appartient au vocabulaire d'ingénierie du guide ; **elle n'est pas une classe de conformité commune aux SDO**. Au 30 juillet 2026, « AI-native network » reste une direction d'architecture. Le rapport de recherche O-RAN NGRG *Native AI Architecture Description* v1.2 n'est pas une spécification technique. Les services publiés A1-ML/R1 et la gestion 3GPP AI/ML fournissent des briques normatives, dont aucune ne suffit seule à certifier un « réseau natif IA ».

Une optimisation de routage exécutée hors ligne relève de l'**IA pour les réseaux**. Le réseau qui transporte son entraînement relève aussi des **réseaux pour l'IA**, mais cela ne rend pas le routeur « natif IA ». Cette dernière qualification exige que le modèle, son contexte, son cycle de vie, ses interfaces, son budget temporel, son autorité et son repli soient des éléments explicites de l'architecture de la fonction.

Le test de frontière tient en trois questions :

1. **Qu'est-ce qui échoue si le modèle disparaît ?** Une aide d'exploitation peut s'arrêter ; une fonction native doit avoir un mode dégradé spécifié.

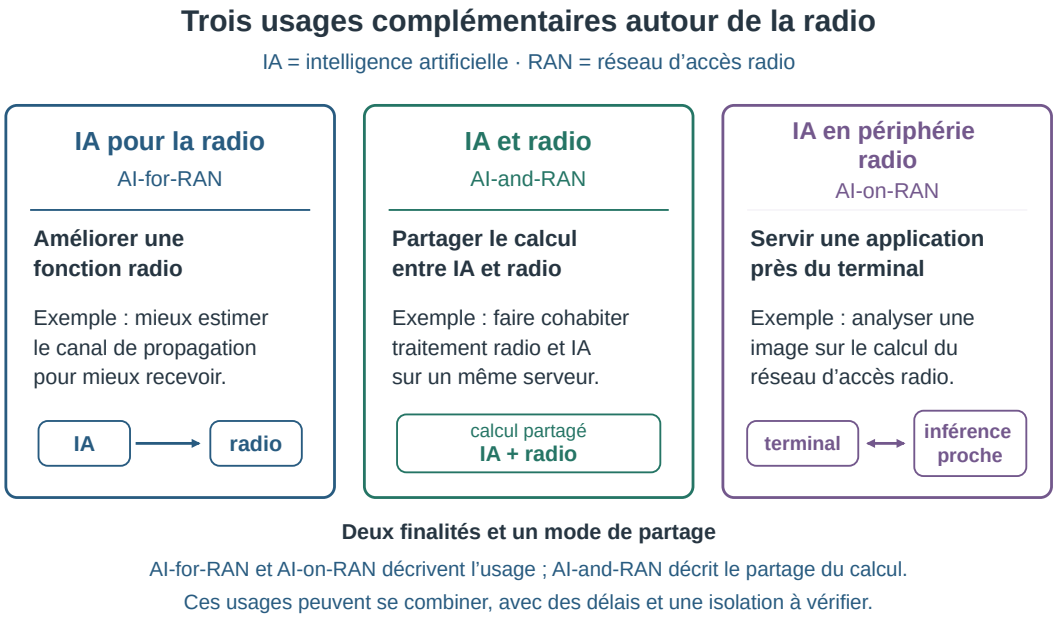
2. **Où se trouve l'autorité ?** Un service d'analyse ou une rApp peut recommander ; seul un consommateur autorisé ou un contrôleur exposant l'action peut agir.
3. **Quel contrat est interopérable ?** Un fichier de poids ne suffit pas : il faut sémantique des entrées et sorties, version, compatibilité, provenance, ressources, contraintes temporelles, validation et retrait.

Cette taxonomie organise le reste du guide : les sections **Rang éditorial P1** à **Apprentissage distribué et confidentialité** traitent surtout de l'IA pour les réseaux ; **Direction inverse — le réseau pour l'IA** traite du substrat ; les cas radio, sémantiques et de contrôle ne deviennent natifs IA que lorsqu'ils satisfont le contrat architectural décrit plus loin dans **Architecture opérationnelle inter-SDO**.

AI-RAN : trois usages complémentaires de l'IA et de la radio

Le **réseau d'accès radio**, ou **RAN** (*Radio Access Network*), relie les terminaux mobiles au réseau de l'opérateur par les stations de base. L'**IA** désigne ici l'intelligence artificielle. Le vocabulaire de l'AI-RAN Alliance décrit trois usages complémentaires (AI-RAN Alliance 2026) :

- **AI-for-RAN : l'IA pour améliorer la radio.** Un modèle estime, par exemple, le canal de propagation pour aider la station à transmettre. Le service à améliorer reste le service radio.
- **AI-and-RAN : l'IA et la radio sur une infrastructure partagée.** Des traitements radio et des travaux d'IA utilisent un même ensemble de ressources de calcul. Il faut décider qui reçoit quelle capacité et comment protéger les échéances de chacun.
- **AI-on-RAN : des applications d'IA en périphérie radio.** Une application est hébergée sur l'infrastructure de calcul du RAN, près du terminal, par exemple pour analyser une image. L'inférence, c'est-à-dire l'exécution d'un modèle déjà entraîné, sert alors l'application de l'utilisateur.



Deux finalités, améliorer la radio et héberger une application proche du terminal, peuvent partager une infrastructure de calcul.

Ces expressions répondent à deux questions : **à quoi sert le calcul ?** et **où ses ressources sont-elles partagées ?** Elles ne sont ni trois générations successives ni trois niveaux de maturité. Elles complètent la taxonomie précédente : héberger une application d'IA près d'une antenne ne suffit pas à rendre la fonction radio elle-même « native IA ».

Le rapport d'architecture de l'Alliance, version 1.2, décrit cette organisation fonctionnelle (§§ 1–3, p. 3–6). C'est une proposition d'architecture industrielle, pas une certification de performance ou une norme 3GPP. Les exemples du livre

développent séparément le partage des ressources dans l'idée X, l'inférence distribuée et la compression du canal dans l'idée XII.

Vue d'ensemble — catalogue par rang éditorial

P1 = aéronaval et défense (rang éditorial orienté mission), **P2 = double usage directement mobilisable en environnement contesté**, **P3 = infrastructure civile / opérateur**.

Ces symboles sont des **rangs éditoriaux propres à ce document**. Ils ne doivent pas être injectés dans une politique réseau : le scénario de contention du Routing Experiment Lab emploie au contraire une priorité de service numérique où `3 = critique`, `2 = mission` et `1 = trafic volumineux` (`bulk`).

Le catalogue inclut des **fonctions opérationnelles**, des **risques ou modes de défaillance**, et des **mécanismes de validation ou d'assurance**. La colonne « Domaine » rend cette nature visible : les derniers ne sont pas des services réseau supplémentaires, mais les moyens qui empêchent les premiers de rester de simples démonstrations.

Rang	#	Domaine	Cas d'usage	Décision produite	Section source
P1	1	**Aéronaval**	**Détection d'incohérence de navigation** (trajectoire, radar, inertiel, GNSS, AIS)	épisode inhabituel / source commune probable ; sortie sûre = « position non fiable »	§ Ormuz — Rôle borné de l'IA
P1	2	**Aéronaval**	**Prévision de qualité de lien** à partir de trajectoire et mesures	anticiper la perte du lien	§ IA et boucles
P1	3	**Aéronaval**	**Choix de relais ou de support**	recommandation de relais	§ IA et boucles
P1	4	**Aéronaval**	**Détection d'incohérence de temps, position ou topologie**	alerte de cohérence	§ IA et boucles
P1	5	**Aéronaval**	**Sélection des données à transmettre / réduction vidéo**	quoi émettre sous fenêtre rare	§ IA et boucles
P1	6	**Aéronaval**	**Prévision de saturation du nœud de fusion**	délestage anticipé	§ IA et boucles
P1	7	**Défense**	**Autonomie bornée et contrôle local déconnecté** (modèles préchargés)	rerouter / réduire / isoler / allouer dans une enveloppe	§ 27
P1	8	**Défense**	**QoS comme commande de mission** sous contrainte et incertitude	conserver commande et alerte, dégrader le reste	§ VII.19
P1	9	**Défense**	**Chemin, exposition et autorité**	route autorisée, pas seulement la plus courte	§ VI.13
P1	10	**Défense**	**Orchestration sur liaisons rares et contestées** (prépositionnement de modèles)	déplacer / prépositionner / différer une charge	§ 36

Rang	#	Domaine	Cas d’usage	Décision produite	Section source
P1	11	**Défense**	**PACE et SWaP-C** : politique de service mini-drone / relais mobile	bascule de moyen, rôle du nœud	§ 16–17, L19
P1	12	**Défense**	**IA native et radio : surface de vulnérabilité** (adversarial, isolement)	employer ou non le modèle	§ 26
P1	13	**Défense**	**Découvrir sans se dévoiler** ; identité, pseudonymie, signature	quoi émettre pour être joignable	§ 11, § 10
P1	14	**Défense**	**Confiance locale, autonomie et minimisation**	portée de confiance accordée localement	§ 20
P2	15	Routage	**Décision de routage apprise** (Routing Experiment Lab ; modules de calcul de `sb3_trainer_project`)	coefficients d’arêtes, allocations ou chemin selon la méthode	§ Passerelle 5
P2	16	Routage	**Routage conjoint chemin–faisceau–puissance** (radio maillée)	route + faisceau + puissance	§ Repère Lahsen-Cherif
P2	17	Routage	Routage sensible aux applications (charge, QoE)	choix du chemin par flux	§ VI.10
P2	18	Routage	Stabilité du plan de contrôle : bagottement induit par l’IA	changer / ne pas changer de route	§ VI.7
P2	19	Radio / MAC	Planification et allocation radio assistée par IA	blocs, MCS, puissance, priorités	§ I.4 quater
P2	20	Radio / PHY	Chaîne de réception assistée : estimation de canal, égalisation, débruitage, suppression d’interférences, détection MIMO, compensation matérielle, récepteur neuronal	LLR, symboles, coefficients de canal	§ I.4 ter
P2	21	Plan de transfert	Admission, marquage ECN, abandon de paquets	admettre / marquer / déprioriser / jeter	§ 19 bis
P2	22	Média	Communications sémantiques	quoi transmettre pour accomplir la tâche	§ 19 bis

Rang	#	Domaine	Cas d’usage	Décision produite	Section source
P2	23	Sécurité	Cybersécurité et détection d’intrusions	alerte priorisée, regroupement de campagnes	§ 19 bis
P2	24	Vie privée	Apprentissage fédéré	mise à jour de modèle agrégée	§ 19 bis
P2	25	Énergie	Orchestration énergétique	veille, déplacement de calcul, puissance radio	§ 19 bis
P2	26	Validation	**Calibration noyau séparée de la récompense**	mesurer l’écart simulation–noyau sur débit utile / RTT / perte / gigue et la conformité de chemin	`routing_training_project` + `routing_emulation_cli`
P2	27	Validation	Jumeau numérique du réseau	tester un changement et ses hypothèses avant déploiement	§ 24
P2	28	Contrôle	Contrôle proactif sous horizon et délai d’action	prépositionner, réserver ou préparer une bascule	§ 24 bis
P2	29	Sécurité	Sécurisation du contrôleur et de la chaîne IA	autoriser, signer, borner, journaliser ou refuser une action	§ 28
P2	30	Assurance	Cycle de vie, preuve et retrait d’un modèle réseau	promouvoir, maintenir, replier ou retirer une version	§ 9, § 19 bis
P2	31	Diagnostic	Causalité et expérience discriminante	choisir la prochaine mesure ou expérience sûre	§ 21
P2	32	Observabilité	Estimation de topologie et qualité de télémétrie	état connu, inconnu, périmé ou incohérent	§ 24, extension locale `observed_partial`
P2	33	Spectre	Détection d’occupation et classification d’interférences	recommander un canal ou s’abstenir dans une enveloppe réglementaire	§ 29 — Spectre partagé
P2	34	Assurance	Prédiction du risque d’un changement de configuration	accepter, tester davantage, déployer sur un périmètre témoin ou refuser	§ 19 bis, § 24
P2	35	Diagnostic	Sélection active de sondes et mesures	prochaine observation discriminante à coût borné	§ 21

Rang	#	Domaine	Cas d’usage	Décision produite	Section source
P3	36	Antennes	Calibration IA 5G/6G, sélection de faisceaux	phases, amplitudes, tilts, associations	§ A.8 bis
P3	37	Trafic	Prévision de trafic et ingénierie de congestion	déplacement de flux, métriques, capacité	§ 19 bis
P3	38	QoE	Optimisation de QoE	débit, résolution, FEC, tampon, chemin	§ 19 bis
P3	39	QoE	Optimisation conjointe congestion–routage–QoE	action prévalidée parmi un ensemble	§ 19 bis
P3	40	RAN	RAN, mobilité, transfert intercellulaire, découpage en tranches de réseau	faisceau, transfert intercellulaire, ressources de tranche	§ 19 bis
P3	41	Média	ABR assisté par IA (diffusion en continu à débit adaptatif)	qualité du prochain segment	§ 7 bis
P3	42	Média	Encodage, restitution et génération IA des médias	représentation compacte, lieu d’exécution	§ 7 ter
P3	43	Calcul	Allocation de ressources et placement à l’échelle	placement, réallocation, préemption	§ X.2 bis
P3	44	Exploitation	Maintenance prédictive	inspecter / basculer / commander une pièce	§ 19 bis
P3	45	Exploitation	Corrélation d’alarmes et analyse de cause racine	hypothèses de cause classées	§ 19 bis
P3	46	Exploitation	Gestion de la connaissance, recherche sémantique, RAG	réponse ancrée dans des sources, ou abstention	§ 19 bis
P3	47	Exploitation	Agents génératifs pour la configuration	proposition de configuration / différences / test	§ 19 bis
P3	48	Infrastructure IA	Réseau pour entraînement et inférence distribués	placement, collective, pacing, ECN, reprise	§ 22–23
P3	49	Transport	Adaptation apprise de congestion ou de paramètres TCP/QUIC	rythme d’envoi ou choix borné de paramètres	prolongement de § VII.11 — BBR

Repère — Pourquoi communiquer en mer reste difficile

Imaginons un navire scientifique qui collecte des mesures loin des côtes. À bord, l'équipe doit surveiller les instruments, comprendre les résultats et échanger avec un laboratoire à terre. Sous l'eau, un capteur peut avoir encore un autre moyen de communication. **Le réseau maritime est une chaîne de liaisons différentes**, avec des contraintes de portée, d'énergie, de capacité et de disponibilité.

Trois situations à distinguer

Situation	Moyens possibles	Première question à poser
Navire près des côtes ou d'autres navires	Radio maritime, notamment VHF ; accès terrestre lorsqu'il est disponible	La station est-elle à portée et le service adapté au message ?
Navire au large	Liaison satellitaire ; radio HF pour certains échanges	Quelle couverture, quelle capacité et quelle continuité de service sont disponibles ?
Instrument sous l'eau	Liaison acoustique ou câble vers un navire, une bouée ou la terre	Quelle quantité de données peut traverser cette première liaison, avec quelle énergie ?

Les moyens radio maritimes répondent à des besoins différents : échanges entre navires, avec la côte, informations de navigation ou communications à longue distance (International Telecommunication Union 2026). L'horizon radio, le mouvement des antennes et les conditions de propagation expliqués dans le livre restent déterminants. Une liaison satellitaire ajoute ses propres contraintes de visibilité, de terminal et de capacité partagée.

Sous l'eau, les ondes radio utilisées habituellement pour le Wi-Fi ou la téléphonie mobile sont fortement atténuées. Une bouée peut servir de relais : elle reçoit des mesures par signaux acoustiques, puis les transmet par satellite. La bouée Nootka, décrite par la Woods Hole Oceanographic Institution, fournit un exemple réel de cette chaîne et du stockage des mesures avant leur envoi (Woods Hole Oceanographic Institution 2026).

Une alerte de détresse, un message d'exploitation et une archive scientifique n'ont pas les mêmes exigences. Le **Système mondial de détresse et de sécurité en mer** (SMDSM, ou GMDSS en anglais) associe des moyens terrestres et satellitaires dédiés à la sécurité. Un accès Internet ne décrit donc pas à lui seul les moyens de détresse d'un navire (International Maritime Organization 2026).

Que transmettre quand tout ne peut pas passer ?

Dans un exemple pédagogique, transférer **10 Mo décimaux**, soit 80 Mbit, sur une liaison offrant un débit utile constant de **100 kbit/s** prend au minimum **800 secondes**, soit **13 minutes et 20 secondes**. Une coupure allonge encore ce délai. Ces chiffres sont des hypothèses de calcul, pas une performance annoncée pour un équipement maritime.

On peut conserver les mesures complètes à bord, envoyer d'abord les informations nécessaires à la tâche et reprendre les transferts interrompus. Cela exige d'identifier les messages qui peuvent attendre, ceux qui se périment et les données encore manquantes. Un résumé reste relié aux mesures sources.

Les petits modèles locaux peuvent aider à lire une documentation embarquée ou à préparer un compte rendu pendant une coupure vers la terre. Leur utilité se vérifie sur ces tâches : **répondre hors connexion ne signifie pas disposer d'informations à jour sur le monde extérieur.** Des règles simples peuvent suffire aux transferts ; mesurer ce que l'IA apporte en plus.

Ce que l'article de Rouxel et ses collègues apporte

Rouxel, Sarzeaud et Lequentrec-Lalancette étudient la précision de modèles d'anomalies de gravité marine obtenus à partir de l'altimétrie satellitaire. Il s'agit de cartes numériques des variations du champ de gravité au-dessus des océans, et non de modèles de langage. Ils les confrontent à des mesures effectuées en mer et tiennent compte de l'incertitude de ces mesures. Ils proposent ensuite une estimation de l'erreur du modèle, variable selon les zones, en combinant plusieurs sources d'information (Rouxel et al. 2023).

Cette étude de géophysique illustre la différence entre **un modèle, une mesure et leur incertitude**. Elle n'évalue pas une liaison de communication. Le rapprochement pédagogique proposé ici est le suivant : transmettre une valeur correctement ne prouve pas que cette valeur représente fidèlement la réalité.

Pour une collecte marine, poser trois questions : **la mesure est-elle fiable ? Le message est-il arrivé ? Est-il encore utile ?** Préserver origine, date, position, unité, version du traitement et incertitude connue permet d'y répondre. Évaluer toute compression ou sélection selon ce qu'elle laisse encore vérifier.

Ce cas relie les idées du livre : mesurer, transmettre, conserver, traiter localement et confronter les modèles aux observations.

---

1. Rang éditorial P1 — Aéronaval et défense

> **Périmètre (livre).** Les passages de défense reposent **uniquement sur des principes, normes et sources ouvertes**. Ils portent sur architecture, résilience, QoS, sûreté de fonctionnement et compréhension défensive de l'environnement électromagnétique. Ils ne donnent ni fréquences opérationnelles, ni paramètres de formes d'onde, ni procédures de brouillage, ni configurations réelles. Objectif : **concevoir un service qui continue à fonctionner dans un environnement mobile, congestionné, intermittent ou contesté.**

1.1 Détection d'incohérence de navigation — GNSS, AIS, radar, inertiel (§ Ormuz)

Un modèle peut détecter une incohérence entre trajectoire prévue, radar, inertiel, GNSS et AIS, classer un épisode comme inhabituel, ou proposer la source commune probable.

Dans ce scénario maritime, l'IA peut signaler une incohérence et hiérarchiser les vérifications. Elle ne remplace ni les contrôles indépendants ni l'autorité humaine, et doit pouvoir conclure que la position reste indéterminée.

> **La sortie sûre est parfois :** « position non fiable ; revenir aux procédures et sources indépendantes prévues ».
>
> **Leçon Ormuz — La redondance compte seulement si les sources sont indépendantes.** Copier une position GNSS dans cinq systèmes crée cinq affichages, pas cinq preuves.

Rappel de contexte : en environnement maritime, la réflexion sur la mer, les gradients de température et d'humidité et le relief côtier font varier le niveau reçu, rendent un émetteur lointain momentanément audible et créent des zones d'ombre proches. **La réponse n'est pas de conclure automatiquement à une attaque** : il faut corréler mesures radio, position, météo, heure, plusieurs capteurs et événements réseau.

1.2 Aide aux boucles aéronavales (§ Scénario aéronaval)

Les cinq fonctions retenues par le livre :

1. prévoir la qualité d'un lien à partir de trajectoire et mesures ;
2. recommander un relais ou un support ;
3. détecter une incohérence de temps, de position ou de topologie ;
4. réduire une vidéo ou sélectionner les données à transmettre ;
5. prévoir la saturation d'un nœud de fusion.

Bornes obligatoires :

- une liste d'actions autorisées ;
- un délai maximal d'inférence ;
- une méthode classique de repli ;
- une mesure de confiance et la possibilité de s'abstenir ;
- une validation sur des trajectoires, météos, charges et pannes **absentes de l'entraînement** ;

- un journal permettant d’expliquer la décision.

> **Leçon aéronavale — Le réseau est un graphe spatio-temporel.** L’altitude, l’horizon, le temps, les droits et la fraîcheur déterminent le chemin utile autant que le débit nominal.

Contexte système à ne pas perdre de vue (§ 3) : plusieurs bâtiments de surface, aéronefs habités ou non, un nœud de guet/commandement servant aussi de fusion — mais **le nœud aéroporté ne « centralise » pas tout** ; la proximité des équipements radio (**co-site**), le pont d’envol et le multitrajet contraignent la radio ; en multicast, c’est **le récepteur le plus défavorisé** qui dicte le service ; et toutes les données n’ont pas la même échéance.

1.3 Autonomie bornée et contrôle local déconnecté (§ 27)

Le réseau doit continuer lorsque le contrôleur central est inaccessible. Sont préchargés : politiques, chemins de repli, classes QoS, plafonds, clés ou certificats à durée bornée, **modèles**, règles de sécurité, scénarios PACE.

```
autorisé :
  - rerouter parmi des chemins approuvés
  - réduire des services
  - isoler une charge suspecte
  - allouer une capacité bornée

interdit sans autorité supérieure :
  - étendre un domaine de confiance
  - introduire une nouvelle racine
  - exporter un préfixe non prévu
  - lever un cloisonnement critique
```

Au retour de connectivité : transmettre le journal, comparer les états, résoudre les conflits, rétablir l’autorité, **éviter une reconfiguration massive instantanée**.

1.4 QoS comme commande de mission (§ VII.19)

En environnement contesté, le transport doit distinguer autant que possible : congestion interne, qualité radio fluctuante, mobilité, indisponibilité temporaire, **limitation volontaire pour discrétion**, perte durable du chemin.

Une réponse uniforme — « retransmettre davantage » — peut empirer l’occupation du canal, la latence, l’énergie, **la détectabilité** et la congestion.

Sous capacité dégradée, la politique préserve d’abord commandes et alertes, réduit ensuite la voix et la vidéo, diffère les volumes non critiques, puis les reprend lorsque la connectivité revient.

> **La QoS devient une commande de mission : préserver l’effet utile plutôt que la forme nominale.** C’est le cadre dans lequel toute IA de priorisation doit s’inscrire — elle propose, la politique de mission décide.

1.5 Chemin, exposition et autorité (§ VI.13)

Le routage de défense n’optimise pas seulement une métrique : un chemin techniquement valide peut être interdit d’emprunt. Toute route proposée par un modèle est donc soumise à une politique d’autorité et d’exposition, jamais l’inverse. Voir aussi § 15 — géopolitique d’une table de routage.

1.6 Orchestration sur liaisons rares et contestées (§ 36)

La capacité n’est pas seulement variable : **elle peut disparaître.** L’orchestrateur raisonne sur coût de déplacement d’une charge, taille des images et **modèles**, cache local, énergie, fenêtre de liaison, sensibilité des données, autonomie, **signature des synchronisations**, priorité de mission.

> Déplacer un conteneur léger peut être trivial ; déplacer un modèle de plusieurs dizaines de gigaoctets sur une liaison contrainte ne l’est pas. Il faut **prépositionner, compresser, versionner et reprendre par blocs**.

Pour une mise à jour commune : diffusion signée, cache local, validation avant activation, reprise de blocs manquants, retour arrière, délai d’activation coordonné ou tolérance aux versions mixtes.

1.7 PACE, SWaP-C et plateformes contraintes (§ 16–17 ; L19)

Sur mini-drone, tout se dispute : énergie, masse, calcul, thermique, spectre et co-site. PACE est **une chaîne de moyens, pas une liste décorative**. Le laboratoire L19 propose de comparer meilleur lien seul, relais fixe, relais tournant et politique SWaP-aware avec hystérésis, en mesurant effet utile, énergie par type de trafic, gigue, échéances manquées, durée de vie du relais et **oscillations d'orchestration**.

1.8 IA native et radio : nouvelle surface de vulnérabilité (§ 26)

Le traitement technique des fonctions apprises, de leur dérive et de leur repli figure au § 2.4. La conséquence propre au contexte de défense est plus simple : **le modèle, son service de distribution et sa télémétrie ne doivent pas devenir une dépendance centrale cachée**. Le système documente ce qui reste possible avec une version ancienne, sans mise à jour et sans contrôleur distant.

> **Une radio qui ne sait fonctionner qu'avec son modèle central devient fragile en isolement.** Cette dépendance doit être traitée comme un critère de sélection explicite en défense.

1.9 Discrétion, identité et confiance locale

- **Découvrir sans se dévoiler** (§ 11) : toute fonction apprise d'assistance à la découverte doit être évaluée sur ce qu'elle fait émettre.
- **Identité, pseudonymie et signature** (§ 10) : traverser un réseau ne prouve pas qui parle.
- **Confiance locale, autonomie et minimisation** (§ 20) : la portée de confiance accordée localement borne ce qu'un agent peut décider seul.

1.10 Trajectoire d'adoption recommandée

```
observation -> recommandation -> simulation
            -> action bornée -> vérification -> extension prudente
```

Jumeaux numériques, émulateurs RF et campagnes reproductibles servent à tester charge, mobilité, météo, panne, interférence, dérive et données adversariales **avant** d'accorder une autorité.

> **Lecture des programmes DARPA / MILCOM.** Un programme public indique une hypothèse de recherche, pas une capacité déployée. Demander maturité, métrique, environnement d'essai, interopérabilité, SWaP-C, supply chain, autorité, comportement dégradé et preuve de transition.
>
> **Invariant — La tendance MILCOM est la composition contrôlée.** Aucun support, cloud, modèle IA ou programme de recherche ne suffit seul.

---

2. Couche physique et radio

2.1 Chaîne de réception assistée (§ I.4 ter)

Un modèle entraîné sur échantillons, pilotes, mesures de canal et symboles connus peut approximer une fonction difficile à écrire explicitement.

Bloc assisté par IA	Entrée possible	Sortie utile
estimation de canal	pilotes, historique, position, vitesse	coefficients ou distribution du canal
égalisation apprise	échantillons et canal estimé	symboles corrigés ou LLR
débruitage	signal bruité et contexte	signal ou représentation restaurée
suppression d'interférences	mélange reçu, pilotes, signatures	composante utile ou interférence estimée
détection conjointe MIMO	signaux de plusieurs antennes	symboles ou LLR par couche

Bloc assisté par IA	Entrée possible	Sortie utile
compensation matérielle	échantillons avec distorsions	correction non-linéarité / phase / I/Q
récepteur neuronal	grille temps-fréquence + contrôle	LLR pour le décodeur

Trois niveaux à ne pas confondre :

```
IA d'assistance           : prédit un paramètre pour une chaîne classique
bloc appris               : remplace un égaliseur ou un détecteur délimité
récepteur de bout en bout : apprend plusieurs opérations conjointement
```

Limites explicites — l'IA ne répare pas ce qui n'a jamais été mesuré : saturation du front-end RF, composante hors bande d'échantillonnage, signal sous la plage dynamique. « Une reconstruction plausible n'est donc pas une récupération démontrée. » Elle ne supprime ni le budget de liaison ni Shannon, et un débruiteur optimisé sur des formes d'onde fréquentes peut effacer précisément le signal faible ou nouveau recherché.

2.2 Planification et allocation radio (§ I.4 quater)

L'allocation multi-utilisateur, multi-cellule et multiobjectif devient combinatoire. L'IA prédit trafic et mobilité, estime la valeur d'une allocation ou approxime une solution en temps borné.

- **Observations** : CQI/SINR et fraîcheur, files et délais de tête, échéances, ACK/NACK et HARQ, mobilité et faisceaux, tampon et QoE applicative, énergie et température, contrats de tranche de réseau.
- **Actions bornées** : blocs autorisés, puissance, MCS, priorités, quotas, rythme de changement.
- **Évaluation** : utilisateurs non servis, percentiles de délai et de débit utile, échéances manquées, équité, BLER, stabilité, énergie, respect des minimums.
- **Repli** : un ordonnanceur classique reprend la main sur modèle indisponible, trop lent ou hors distribution, en utilisant le même état de files.

> **Invariant.** L'allocation est une politique de rareté : un gain de débit moyen ne compense pas une famine silencieuse ou une échéance critique manquée.

2.3 Calibration IA des antennes 5G/6G (§ A.8 bis)

Boucle : signaux de référence → mesures internes/externes → rapports terminaux → carte radio ou jumeau numérique → optimiseur (phases, amplitudes, inclinaisons, puissances, associations cellule-utilisateur) → déploiement progressif avec mesure avant/après et retour arrière.

L'IA apprend les non-linéarités résiduelles, prévoit la couverture, accélère la sélection de faisceaux, détecte une dérive matérielle. Ses entrées restent partielles, retardées et biaisées vers les terminaux qui rapportent. En TDD, la réciprocité du canal ne dispense pas de calibrer les chaînes ; en FDD, l'inférence montant→descendant dépend encore plus du modèle.

> **Garde-fou.** Une meilleure moyenne n'est pas une preuve de meilleure couverture : tester percentiles de bord de cellule, zones mortes, mobilité, charge, météo, défaillances de voies.

2.4 IA native et radio (§ 26)

Cas : estimer le canal, allouer des ressources, prédire une mobilité, choisir un mode, détecter une anomalie. À gérer : distribution différente de l'entraînement, données adversariales, explicabilité, temps de décision, validation, repli classique, cohérence entre nœuds, mise à jour du modèle, certification. **Une radio qui ne sait fonctionner qu'avec son modèle central devient fragile en isolement.**

2.5 Détection d'occupation et classification d'interférences (§ 29)

À partir de spectres, statistiques temporelles, rapports de plusieurs nœuds et contexte autorisé, un modèle peut estimer qu'une ressource est occupée, classifier une famille d'interférences ou recommander un canal parmi ceux qui sont

permis. Il ne transforme pas une absence de détection en preuve d'absence et ne déduit pas automatiquement qu'une perturbation est hostile.

Évaluer probabilité de détection et de fausse alarme selon niveau reçu, matériel, mobilité et distributions inconnues ; tester rapports coopératifs retardés ou empoisonnés. Réglementation, utilisateurs primaires, puissance, durée et zones interdites restent des contraintes externes au modèle. En cas d'incertitude, la sortie sûre est l'abstention ou une politique conservatrice, pas l'exploration libre du spectre.

---

3. Trafic, files et qualité de service

3.1 Prévision de trafic et ingénierie de congestion

Entrées : séries temporelles, matrices de trafic, files, topologies. Actions proposées : déplacement de flux, modification de métriques ou de tunnels, réservation de capacité, changement de placement, adaptation du codec, activation anticipée d'une ressource.

> La prévision n'est utile que si **son horizon dépasse le délai d'action**. Déplacer des flux trop tard ne fait que déplacer la congestion.

Comparaison obligatoire à une politique classique simple : sans gain robuste, la complexité n'est pas justifiée.

3.2 Admission, marquage et abandon de paquets

```
observation -> classe et échéance estimée -> score d'utilité
            -> politique bornée
            -> admettre | marquer ECN | mettre en file
                        | déprioriser | abandonner
```

Risques propres : affamer un utilisateur mal couvert, favoriser les flux faciles, masquer une panne par la disparition de sa télémétrie, créer un déni de service. Les contraintes du plan de transfert (séparation, débit minimal, classes protégées, règles de sécurité) **restent indépendantes du score appris**. Métriques : débit utile, latences de queue, échéances manquées, abandons et marquages par classe, retransmissions induites, équité, faux abandons.

3.3 Optimisation de QoE

Relier mesures réseau, état du terminal, codec et métriques applicatives pour choisir débit, résolution, FEC, tampon ou chemin. La fonction objectif doit empêcher l'optimisation du seul utilisateur moyen : percentiles, accessibilité, équité, consommation, coût, niveau minimal des services critiques. Données personnelles : minimisation, rétention bornée, contrôle d'accès.

3.4 Optimisation conjointe congestion–routage–QoE

Ce que l'IA apporte réellement :

- prévoir la matrice de trafic avant une pointe récurrente ;
- approximer rapidement une optimisation combinatoire de chemins ;
- estimer la QoE à partir de mesures réseau et applicatives ;
- apprendre l'effet d'une action dans un jumeau numérique ;
- choisir parmi un ensemble d'actions prévalidées.

Méthode de mise en œuvre en 10 étapes : définir classes et échéances → inventorier chemins, capacités, files, domaines de panne → mesurer réseau et qualité applicative → construire une politique classique de référence → entraîner et tester sur pointes, pannes et OOD → simuler chaque famille d'action → déployer en observation seule, puis en recommandation → autoriser un petit ensemble d'actions réversibles dans un déploiement canari → comparer QoE moyenne, quantiles de queue, équité et stabilité → étendre l'autorité seulement si le gain reste robuste.

Les **quantiles de queue** décrivent les extrémités d'une distribution. Pour les cas défavorables, on examine par exemple les quantiles élevés des délais et les quantiles bas du débit ou de la qualité. Il faut préciser le quantile choisi et la grandeur mesurée : une « mauvaise moyenne » et un délai dépassé par 1 % des observations ne décrivent pas le même phénomène.

3.5 ABR assisté par IA (§ 7 bis)

Observation : tampon, derniers débits, tailles de segments, latence, terminal. Comparaison à des heuristiques simples sur les interruptions de lecture pour remplissage du tampon (*rebuffering*), la qualité moyenne et les quantiles de queue, la fréquence et l'amplitude des changements, la latence de diffusion en direct, l'équité entre lecteurs et les cas de chute brutale, de transfert intercellulaire ou de sortie du domaine évalué (OOD). Règle sûre conservée : tampon proche du vide ou estimation incertaine → choisir une représentation décodable plus petite. **L'IA ne doit pas apprendre par interruptions réelles sur des utilisateurs non consentants.**

3.6 RAN, mobilité et découpage en tranches (*network slicing*)

Sélection de faisceau, prévision de mobilité, déclenchement de transfert intercellulaire (*handover*), répartition blocs/puissance, ajustement des ressources d'un profil logique. Ces décisions interagissent : améliorer une cellule augmente l'interférence de sa voisine ; retarder un transfert intercellulaire économise de la signalisation mais dégrade le bord de cellule. **Une tranche de réseau (*slice*) n'est pas une capacité physique indépendante.**

3.7 Allocation de ressources de calcul (§ X.2 bis — Allocation assistée par IA)

Prévoir la demande, estimer la durée d'une tâche, résoudre un placement combinatoire, proposer une réallocation — sans inventer la capacité ni violer quota, localisation, isolation, énergie ou échéance. Évaluation : travail utile et SLO, attente et quantiles de queue, équité et famine, fragmentation, préemptions et migrations, coût réseau et énergétique, comportement en panne et OOD, comparaison à un ordonnanceur classique.

3.8 Contrôle de congestion et paramètres de transport appris (§ VII.11)

Un émetteur peut estimer bande passante de livraison, RTT, files, pertes et marquages ECN pour choisir un rythme d'envoi, une fenêtre ou un profil de transport borné. Cette boucle rapide interagit avec des flux employant parfois d'autres algorithmes : un gain isolé peut devenir une prise de capacité injuste à l'échelle du réseau.

Comparer au contrôle standard sur liens variables, pertes non congestionnelles, files courtes et longues, ACK compressés, changement de chemin et concurrence hétérogène. Mesurer débit utile, délai, équité, retransmissions, stabilité et temps de reprise. Le repli doit respecter le protocole et reprendre avec un état cohérent ; apprendre en ligne en provoquant des pertes sur des utilisateurs réels n'est pas un plan d'évaluation acceptable.

---

4. Routage et choix de chemin

Le **Routing Experiment Lab** (`routing_training_project`) est la surface opérateur actuelle du cas de routage, le plus développé du livre. Il construit et vérifie l'expérience en réutilisant les modules d'entraînement de `sb3_trainer_project`, surface d'implémentation et de compatibilité dont les workers entraînent ou évaluent une politique.

Le volume en présente l'architecture et le protocole expérimental. Ces applications ne font pas partie des supports distribués avec le livre. L'étude de cas permet d'en suivre les entrées, les décisions, les résultats et les limites.

Le point de départ est un rappel de vocabulaire (§ Passerelle 1) : un **lien** relie deux voisins, un **chemin** est une suite de liens, une **route** est *la décision* de choisir le prochain passage. Un modèle qui choisit une route **ne contrôle pas** pour autant :

- le débit réellement disponible sur chaque lien ;
- les files déjà présentes ;

- les décisions des autres organisations réseau ;
- la qualité radio future ;
- la durée pendant laquelle le chemin restera valide.

Invariant (§ VI.4). L'algorithme de chemin est souvent la partie la plus simple du routage.

4.1 Décision de routage apprise — Routing Experiment Lab (§ Passerelle 5)

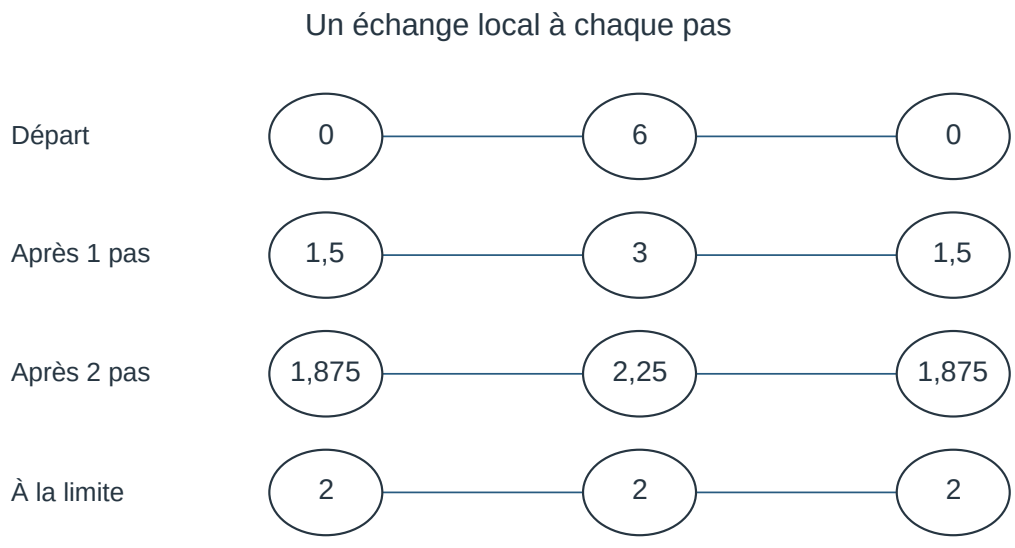
Avant ce cas. Un **graphe** représente des nœuds et leurs liaisons. Un **GNN** construit des représentations en tenant compte des voisins. L'**apprentissage par renforcement** apprend des choix d'action à partir des récompenses observées après ces actions. Une politique PPO-GNN combine ces deux mécanismes. Les bases sont expliquées dans la passerelle du livre : représentation des nœuds et de leurs voisins, familles d'apprentissage sur les graphes et décisions successives en renforcement.

De la chaleur aux échanges sur un graphe. La chaleur se répartit entre zones voisines ; une règle de diffusion sur un graphe peut, de même, atténuer les écarts entre valeurs voisines. Prenons une chaîne de trois nœuds portant les valeurs $(0, 6, 0)$. À chaque pas, chaque nœud échange un quart de son écart avec chacun de ses voisins :

$$h_i^{(t+1)} = h_i^{(t)} + \frac{1}{4} \sum_{j \sim i} \left(h_j^{(t)} - h_i^{(t)} \right).$$

$h_i^{(t)}$ est la valeur du nœud i au pas t ; $j \sim i$ désigne ses voisins. Après un pas, on obtient $(1, 5, 3, 1, 5)$, puis $(1,875, 2,25, 1,875)$. Avec cette règle, les trois valeurs tendent vers leur moyenne, 2.

Certains **GCN**, réseaux convolutifs sur graphes, associent une agrégation de ce type à des transformations apprises. Multiplier les couches peut alors rendre les représentations des nœuds trop semblables : c'est le **lissage excessif** (*over-smoothing*) (Li et al. 2018). La diffusion éclaire cette partie du calcul. Un GCN peut aussi apprendre des poids et appliquer des fonctions non linéaires ; ses valeurs ne deviennent pas pour autant des températures, ni son calcul une loi physique.



Même moyenne : 2. Des écarts de plus en plus petits.

Règle illustrative de diffusion ; un GCN ajoute d'autres calculs.

Diffusion illustrative sur une chaîne de trois nœuds : (0 ; 6 ; 0), puis (1,5 ; 3 ; 1,5), puis (1,875 ; 2,25 ; 1,875) ; les valeurs tendent vers (2 ; 2 ; 2). La règle conserve la moyenne et atténue les écarts ; elle illustre une agrégation, pas un GCN complet.

L’étude de cas compare plusieurs façons de prendre des décisions de routage dans un monde simulé, lues comme trois blocs :

```
MONDE
positions -> liaisons possibles -> graphe et demandes

DÉCISION
observation -> politique -> action -> allocation réalisable

PREUVE
effet observé -> indicateurs -> comparaison -> artefacts vérifiables
                                      -> rejeu si une commande existe
```

Le projet **ne compare pas des objets de même niveau** : il combine modes d’apprentissage, représentations, politiques et solveurs.

Surface expérimentale	Rôle	Sortie ou preuve exacte
`fcas_routing_ppo`	apprentissage interactif on-policy	politique plate ou PPO-GNN selon la configuration
`fcas_routing_ppo_inference`	exécution d’un modèle sauvegardé	coefficients d’arêtes puis allocations réalisées par l’environnement
`fcas_routing_supervised_gnn`	supervision par allocations cibles	fractions prédites par un MPNN personnalisé ; pas un GIN/GINE (*Graph Isomorphism Network*, GINE en étant la variante exploitant les attributs d’arêtes)
`fcas_routing_ilp`	optimisation centralisée sous contraintes	référence d’allocation si le solveur et ses entrées sont effectivement disponibles
`fcas_routing_path_ac`	gradient de politique sur instantanés	score de chemins candidats avec masque de validité
comparaison de métriques	expérience synthétique contrôlée	capacité, WCETT et airtime sur les mêmes entrées ; pas trois politiques entraînées
`fcas_routing_priority_contention`	régression sémantique stricte	preuve sur une fixture à deux niveaux que la priorité `3` l’emporte sur la priorité `1` ; le niveau `2` n’est pas exercé et ce n’est pas une politique générale
`fcas_routing_protocol_comparison`	comparaison de contrôleurs	alignement de snapshots (v1) ou exécution appariée dans `TimeRoutingEnv` (v2)
`fcas_routing_emulation_evidence`	vérification hors ligne	liaison hachée vers une exécution noyau Linux déjà produite ; aucune opération privilégiée

PPO décrit comment une politique est mise à jour ; un **GNN** comment le graphe est représenté ; un **algorithme génétique** cherche des paramètres sans gradient ; une **ILP/MILP** cherche une solution explicite sous contraintes. « GNN contre PPO » n’est donc pas une comparaison de même nature. Le Path Actor-Critic, traitant chaque instantané séparément et sans bootstrap temporel multi-pas, relève plutôt du **bandit contextuel combinatoire** entraîné par gradient de politique que d’un RL séquentiel général.

Les modules d’entraînement génétique de file relais et PPO de maillage actif restent des expériences de la couche d’implémentation réutilisée ; ils ne doivent pas être présentés comme des surfaces opérateur stables de `routing_training_project`. De même, `gine_metric_spf` est encore une identité réservée aux snapshots : le modèle GINE de métriques, l’oracle strict et la réparation de faisabilité restent une proposition, pas une capacité exécutée de bout en bout.

La question commune aux deux métiers. Qu'a vu le décideur, qu'avait-il le droit de faire, quel effet le simulateur a-t-il calculé, et quelle preuve permet de vérifier — puis éventuellement de rejouer lorsqu'une commande existe — cette conclusion ?

Deux distinctions. Une récompense élevée ne prouve ni qu'une **route est déployable**, ni qu'un faisceau est disponible, ni que la décision restera bonne après un changement de topologie. La conformité d'une route à une métrique n'établit pas non plus que le modèle est correctement entraîné et généralisable.

Dans le mode `execute_controllers_end_to_end`, le candidat PPO-GNN émet un **vecteur borné de coefficients multiplicatifs associé aux arêtes**. Une politique neutre émet des coefficients égaux à un ; les deux décisions sont exécutées dans des environnements `TimeRoutingEnv` frais construits à partir du même instantané reçu. L'environnement applique ensuite sa sélection de chemins candidats et ses métriques de capacité, WCETT ou airtime. Ce contrat est plus précis que « l'IA choisit une route », mais il ne faut pas le renommer abusivement en OSPF ou OLSRv2 : un coefficient multiplicatif de capacité n'est pas, en général, un coût additif de protocole de routage.

Le mode de comparaison est **optionnel, désactivé par défaut**, et conserve `paired_snapshot` comme choix prudent. Son schéma v1 aligne des sorties fournies par l'opérateur : identité des contrôleurs et lignée du scénario ne sont pas prouvées. Le schéma v2 exécute les deux décideurs dans des environnements frais, mais son vérificateur ne les réexécute pas ; validité des routes, correspondance des demandes, généralisation, significativité et équité scientifique restent à démontrer. Dans les deux cas, `observed_partial` consomme une vue reçue déterministe dont la provenance est `modeled_not_packet_capture` : ce n'est ni du trafic de contrôle mesuré, ni une exécution RFC 7181/7188.

4.2 Échelle de preuve — du modèle au paquet mesuré

Le risque principal est que le modèle analytique note lui-même la politique qu'il a entraînée. La progression correcte sépare les niveaux de preuve :

Niveau	Substrat	Preuve visée	Portée de la réalisation étudiée
1	formules et fixture synthétique	contraintes, invariants, résultat attendu sur le cas construit	implémenté ; ne prouve pas un plan de transfert
2	deux contrôleurs dans des `TimeRoutingEnv` appariés	même scénario, même vue reçue, action bornée et sorties comparables	implémenté ; ne prouve ni protocole distribué, ni paquets, ni généralisation
3	`routing_emulation_cli`, netns/veth/tc	débit utile UDP, perte, gigue, RTT et conformité des interfaces sur deux chemins déjà choisis	producteur et vérificateur implémentés ; la preuve n'existe que pour chaque exécution Linux privilégiée terminée et vérifiée
4	daemon de routage et captures de contrôle	traduction de métriques, convergence, stabilité et trafic de contrôle	non implémenté dans ce contrat
5	banc RF ou terrain instrumenté	effet du médium, du matériel, de la mobilité et de l'environnement	hors périmètre ; ne généralise jamais au-delà des conditions mesurées

Chaque niveau associe un dispositif expérimental, une preuve visée et une limite d'implémentation. Une conclusion porte au plus sur le niveau réellement exécuté et vérifié : renommer une simulation « test réseau », ou un essai noyau « terrain », ne constitue pas une promotion de preuve.

Le niveau 3 dispose désormais d'un contrat producteur séparé et privilégié : NetworkSim produit un scénario hashé ; le CLI compile un plan déterministe, installe les chemins capacité et WCETT déjà sélectionnés, mesure les paquets dans le noyau Linux, journalise les commandes et nettoie chaque ressource possédée. Le Routing Experiment Lab ne reçoit aucun privilège : il vérifie hors ligne la chaîne `résumé NetworkSim → scénario → plan → exécution` et refuse un artefact incomplet ou non passant.

Lorsqu'une exécution Linux réussie est explicitement liée, cette calibration apporte une mesure noyau produite par des outils paquet séparés de la fonction de récompense. Les capacités et délais de netns/tc restent paramétrés depuis le même scénario modélisé : il s'agit d'une calibration croisée, pas d'une mesure terrain indépendante de toutes les hypothèses.

Elle reste volontairement étroite. Les paires veth sont indépendantes : elles ne reproduisent ni médium partagé ni occupation airtime radio. Le CLI ne mesure pas le délai aller simple faute d'horodatage unidirectionnel instrumenté — les espaces de noms réseau partagent l'horloge du noyau, mais l'outil ne rapporte qu'un aller-retour — ne choisit pas le meilleur contrôleur et ne démontre pas qu'une action PPO se traduit sans perte en coût FRR/OSPF/OLSRv2. Ces affirmations exigeraient les niveaux 4 et 5, avec contrat de traduction, capture des paquets de contrôle et scénarios multi-graines.

4.3 Routage sensible aux applications (§ VI.10)

Choisir la route la moins chargée pour chaque paquet est tentant, mais **une mesure de charge est déjà ancienne lorsqu'elle arrive**. Si tous les routeurs réagissent au même signal, ils déplacent simultanément le trafic, surchargent l'autre chemin, puis reviennent : oscillation.

Un bon mécanisme définit : fréquence de mesure, lissage, hystérésis, limites de changement, domaine de décision, priorité des objectifs, comportement en cas de données absentes. Ce sont les mêmes questions que pour la **mise à l'échelle automatique** (*autoscaling*).

4.4 Stabilité du plan de contrôle — le bagottement (§ VI.7)

```
mesure bruitée autour du seuil
 -> changement d'état
 -> annonce et recalcul
 -> trafic déplacé, paquets perdus ou réordonnés
 -> nouvelle mesure
 -> changement inverse
```

> **Une IA peut aggraver ce phénomène** si son score franchit alternativement un seuil, si elle poursuit chaque prédiction instantanée, ou si plusieurs agents réagissent ensemble. Elle n'en est toutefois ni la définition ni la cause nécessaire : le bagottement est antérieur à l'IA.

Stabilisation : hystérésis (seuils distincts entrée/sortie), durée de confirmation et durée minimale de résidence, lissage et fraîcheur maximale des mesures, temporisation avant réannonce, **budget de changements et limitation du rythme d'action**, annonce seulement après confirmation sans cacher les oscillations aux diagnostics, abstention sur forte incertitude et repli indépendant. Métriques : nombre de transitions, temps de convergence, paquets abandonnés ou réordonnés, effet de service.

> **Point de vigilance — Réactivité et stabilité se règlent ensemble.** Réduire la temporisation accélère la réaction, mais augmente l'exposition au bruit de mesure et aux oscillations ; l'allonger ne doit jamais repousser la reprise au-delà de l'échéance du service.

4.5 Routage conjoint chemin–faisceau–puissance (§ Repère Lahsen-Cherif)

Dans un réseau maillé radio à antennes directionnelles, « choisir une route » ne suffit pas : il faut choisir **ensemble chemin, faisceau et puissance**, avec un compromis débit/énergie. La démarche citée formule d'abord une **référence d'optimisation linéaire en nombres entiers**, puis emploie **Ant-Q** (Q-learning + colonies de fourmis) pour approximer la décision dans des simulations ns-3 ; des travaux ultérieurs portent sur la prédiction adaptative de latence de bout en bout avec des estimateurs peu complexes.

> **Portée de cette approche.** L'IA ne décide pas « du réseau » en général. Elle produit une estimation ou choisit une action dans un espace défini par l'ingénieur. Variables, contraintes, politique classique de référence, coût d'inférence et durée de validité doivent être explicites.

4.6 Contextes où le graphe bouge

- **MANET** (§ VI.11) : la topologie varie avec mouvements, environnement radio et énergie — le graphe bouge plus vite que sa carte. Routage proactif (état maintenu, coût de contrôle) contre réactif (route à la demande, délai initial et risque d'inondation).
- **DTN** (§ VI.12) : router vers une **occasion future** plutôt que vers un chemin permanent.
- **Défense** (§ VI.13) : chemin, exposition et autorité — le chemin le plus court n'est pas forcément celui qu'on a le droit d'emprunter.
- **Autonomie bornée** (§ 27) : rerouter **parmi des chemins approuvés** est autorisé hors contrôleur ; exporter un préfixe non prévu ne l'est pas.

4.7 Ce qui rend une route apprise déployable

Reprendre la méthode en 10 étapes du § 19 bis et la trajectoire `observation → recommandation → simulation → action bornée → vérification → extension prudente`. Le livre insiste sur trois points propres au routage : politique classique de référence conservée, actions limitées à un ensemble prévalidé et réversible, et validation sur pointes, pannes et données hors distribution — pas sur la seule récompense simulée.

---

5. Exploitation, sécurité et connaissance

> **Repère réglementaire — EUCS et SecNumCloud ne décrivent pas un type d'IA. Ne pas confondre EUCS et EUCC.** L'EUCC, adopté par le règlement d'exécution (UE) 2024/482 et applicable depuis le 27 février 2025, certifie des produits TIC et des profils de protection ; il ne certifie pas les services cloud. L'EUCS reste le schéma distinct consacré à ces services (European Commission 2024).
>
> L'**EUCS** (*European Cybersecurity Certification Scheme for Cloud Services*) est un projet de schéma européen de certification de cybersécurité des services cloud — IaaS, PaaS, SaaS et autres services — élaboré dans le cadre du *Cybersecurity Act*. Il porte sur le service cloud et sa chaîne de fourniture ; il ne certifie pas, à lui seul, la sûreté fonctionnelle d'un modèle ou d'une décision IA (European Union Agency for Cybersecurity (ENISA) 2020).
>
> **État au 3 septembre 2026.** L'EUCS n'est pas encore adopté. L'ANSSI le présente comme un schéma en cours de rédaction et indique que SecNumCloud sert de base au niveau d'assurance élevé (Agence nationale de la sécurité des systèmes d'information (ANSSI) 2026). Il est donc prématuré d'écrire que l'EUCS « remplace » déjà SecNumCloud. Le mécanisme juridique est conditionnel : lorsqu'un schéma européen entre en vigueur, les schémas nationaux couverts cessent de produire leurs effets à la date fixée par l'acte d'exécution, tandis que les certificats nationaux existants restent valides jusqu'à leur échéance (European Parliament et Council of the European Union 2019).
>
> La formulation rigoureuse est donc : **une fois adopté, l'EUCS a vocation à organiser la transition depuis les schémas nationaux dont il couvre le périmètre ; pour SecNumCloud, la portée et la date exactes de cette substitution dépendront de l'acte d'exécution — il ne s'agit pas d'effacer indistinctement toute exigence française**.

Pour une fonction réseau fondée sur l'IA et hébergée dans le cloud, les preuves sont complémentaires : l'EUCS ou SecNumCloud renseigne la confiance dans le service cloud ; l'assurance IA renseigne les données, le modèle, son intégration, son autorité et son comportement. Les deux dossiers doivent être reliés par des interfaces et des hypothèses explicites, jamais fusionnés en une certification générique.

5.1 Le socle (§ 19)

L'apprentissage automatique peut aider à : détecter des anomalies ; prévoir la charge ; regrouper des alarmes ; identifier des dépendances ; estimer un risque de panne ; proposer une cause ; optimiser un placement ; classifier des flux ; générer ou vérifier des changements.

Mais un modèle peut : apprendre un historique biaisé ; dériver ; confondre corrélation et cause ; être contourné ; halluciner une configuration ; réagir à une télémétrie empoisonnée ; optimiser la mauvaise métrique.

5.2 Cybersécurité et détection d'intrusions

Données : flux, DNS, identités, journaux, graphes de communication, séquences d'événements. Le modèle apprend des profils, détecte une rupture de comportement, regroupe des campagnes, priorise les alertes.

> **Une anomalie n'est pas une attaque.** Une mise à jour, un nouveau service ou une panne peut ressembler à une intrusion ; une attaque lente peut ressembler au fonctionnement normal.

Les décisions irréversibles restent à une politique explicitement autorisée. Tests requis : faux positifs, faux négatifs, trafic chiffré, dérive, évasion, empoisonnement des données.

5.3 Maintenance prédictive

Signaux : température, puissance optique, taux d'erreur, dérive d'oscillateur, reprises de lien, ventilateurs, alimentation, évolution de l'EVM (*Error Vector Magnitude*, erreur vectorielle de modulation). La cible pertinente n'est pas « panne dans sept jours » mais **une décision** : inspecter, basculer, réduire la puissance, commander une pièce, ne rien faire. À traiter : coût des faux positifs, rareté des pannes étiquetées, changements de génération matérielle, **censure statistique à droite** (équipement remplacé avant sa panne).

5.4 Corrélation d'alarmes et RCA

Une coupure produit des centaines d'alarmes en aval. Graphe de dépendances, ordre temporel et changements récents permettent de regrouper les symptômes et de classer les causes possibles.

> Le système ne doit pas confondre **classement** et **preuve**. Une cause est confirmée par une observation discriminante, une requête ciblée, une expérience sûre ou la disparition reproductible du symptôme après correction.

Une extension utile est la **sélection active de mesures** : choisir la sonde, la trace ou le test qui sépare le mieux les hypothèses restantes sous un budget de trafic, de temps et de risque. Le modèle recommande l'observation ; la politique interdit les sondes intrusives ou susceptibles d'aggraver l'incident.

Repère sur les outils cités. Tokki encadre les actions d'assistants d'IA et conserve des traces récupérables pour vérifier leurs résultats et reprendre le travail. AGILAB organise l'exécution d'applications sur des ressources de calcul. Ces deux logiciels illustrent les liens entre une intention, les outils employés et la preuve du résultat.

Six dossiers de lecture. Les sections suivantes ouvrent des questions différentes. Choisir celle qui vous intéresse, puis rejoindre les fonctions d'exploitation à partir de § 5.11 ; leur lecture n'exige pas de suivre les six dossiers dans l'ordre.

Question	Dossier
Retrouver une explication fiable dans les documents	Connaissance et RAG, § 5.5
Relier intention, code, expérience et résultat	Cas Tokki × AGILAB, § 5.6
Choisir où envoyer une requête d'IA	Routage des requêtes, § 5.7
Faire travailler plusieurs agents ensemble	Équipes d'agents, § 5.8
Adapter les modèles aux moyens disponibles	Modèles locaux, § 5.9
Qui peut bénéficier de ces outils ?	Accessibilité et égalité des chances, § 5.10

5.5 Gestion de la connaissance, recherche sémantique et RAG

Quatre niveaux à ne pas confondre :

Niveau	Exemple réseau	Question de contrôle
donnée	`loss_rate = 4,2 %` à 14 h 03	qui l'a mesurée, où et comment ?
information	la perte a augmenté après une bascule	quelles mesures, quelle chronologie ?
connaissance candidate	ce firmware a un défaut connu dans cette condition	quelle source, version, domaine ?
décision	revenir à la version précédente	qui autorise, quelle preuve, quel retour ?

Chaîne complète :

```
documents + tickets + configurations + topologie + traces + mesures
   -> extraction, découpage, normalisation et dédoublonnage
   -> entités, relations, métadonnées, droits et provenance
   -> index lexical + index vectoriel + éventuellement graphe de connaissances
   -> recherche sous contrôle d'accès
   -> passages et faits candidats
   -> réponse ancrée dans les sources, ou abstention
   -> validation humaine / outil / expérience
   -> correction, versionnement, expiration et audit
```

Interventions de l'IA : extraction d'entités (équipements, interfaces, versions, symptômes, actions), classement et étiquetage, **résolution d'entités**, **recherche sémantique** par embeddings, rapprochement alarme ↔ runbook ↔ changement ↔ incident antérieur, résumé et détection de contradictions ou de contenus périmés, proposition de relation manquante dans un graphe (à confirmer).

Un graphe extrait automatiquement n'est pas un graphe de vérité ; une proximité d'embedding ne prouve ni identité de cause ni compatibilité de solution.

Relier la recherche documentaire à une réponse vérifiable

La section Du texte aux vecteurs : corpus, comptage et TF-IDF introduit les fondements de cette chaîne. L'index lexical retrouve les termes, notamment les codes d'erreur exacts ; l'index vectoriel cherche des proximités apprises. Le **RAG** (*retrieval-augmented generation*, génération augmentée par recherche documentaire) associe la recherche de passages à leur utilisation par un modèle génératif. Ajouter ces passages au contexte d'une réponse ne modifie pas les poids du modèle. Entraîner le modèle ou le moteur de recherche reste une opération distincte, qui peut compléter cette organisation (Lewis et al. 2020).

Exemple fictif : quelle procédure utiliser pour un firmware 2.0 ? Parmi les documents auxquels le lecteur a accès, la recherche donne :

Document	Score de similarité fictif	Utilisation possible
Guide pour le firmware 1.2	0,96	procédure incompatible avec la version 2.0
Ticket sans version indiquée	0,93	piste à examiner, sans procédure validée
Guide pour le firmware 2.0	0,88	procédure à examiner pour cette version

Le score classe une proximité ; **0,96 ne signifie pas 96 % de chances d'avoir la bonne solution**. Le guide moins bien classé est ici le candidat applicable. Si sa procédure est ensuite mal interprétée, la réponse reste fausse : retrouver le bon passage et l'utiliser correctement sont deux contrôles distincts. Une citation ne suffit pas. Sans passage applicable, il vaut mieux demander la version manquante ou s'abstenir de proposer une procédure.

Scénario pédagogique : une interface retombe après un rétablissement. L'objectif est de préparer un diagnostic, avec la procédure applicable à la bonne version du logiciel :

1. Un outil extrait les messages, leurs horodatages et l’identité de l’interface ; il conserve aussi les lignes de contexte nécessaires.
2. La recherche retrouve une procédure candidate par code d’erreur et par description du symptôme. Sa version, son domaine et sa provenance sont vérifiés avant utilisation.
3. Un modèle propose une explication en distinguant faits observés, hypothèses et information manquante. Une instruction trouvée dans un journal reste une donnée à analyser, pas un ordre autorisé.
4. L’ingénieur confronte cette proposition aux compteurs et à une observation discriminante. Toute modification suit la chaîne de changement sûre.

Le cas ci-dessous illustre une organisation possible de ces opérations ; il ne rapporte pas une expérience sur un réseau réel.

> **Encadré daté — Qwen3.8, GPT-6 et Tokki, 24 septembre 2026.** Le modèle local retenu pour ce cas est `ddalcu/Qwen3.8-27B-MLX-Serve-4bit`, une conversion pour MLX, une bibliothèque de calcul pour l’IA sur les puces Apple (Apple Silicon). « 27B » désigne une classe de nombre de paramètres, pas la taille du vocabulaire. La **quantification** représente certaines matrices de poids avec une précision réduite. Selon la fiche de conversion, les principales matrices concernées sont en 4 bits, tandis que la table `embed_tokens` reste en BF16 : tous les vecteurs ne sont donc pas en 4 bits (ddalcu 2026).
>
> GPT-6 Astra est ici le modèle distant identifié par `gpt-6-astra` dans les métadonnées d’une session de Codex, un assistant d’IA pour la programmation, ayant produit des réponses. Cette observation établit l’identité déclarée par le logiciel, pas une attestation indépendante du calcul fournisseur. La documentation officielle décrit son interface ; on ne lui attribue ni la dimension des embeddings ni l’architecture interne de Qwen (OpenAI 2026).
>
> **Tokki** est un dispositif local qui encadre l’usage des outils et conserve des éléments de preuve autour des agents. Il peut conserver des sorties volumineuses localement et en transmettre des extraits accompagnés de références récupérables. Une extraction déterministe n’exige pas d’appel à Qwen. Une analyse locale n’a lieu que si une requête est effectivement routée vers ce modèle. La présence d’un serveur ou d’un nom dans une configuration ne prouve pas cet appel.

Dans ce scénario, une analyse locale peut être transmise au modèle distant avec les extraits nécessaires et leurs références. Ce sont du texte ou des données structurées qui circulent, pas les vecteurs internes du modèle local. Un document privé traité localement ne devient pas automatiquement transmissible à un service distant. La sélection doit respecter les droits et la destination autorisée des données.

Trois leviers agissent sur des objets différents :

Levier	Objet modifié	Contrôle utile
Représentation vectorielle	nombres décrivant le texte pour une tâche	qualité de recherche ou de prédiction
Sélection du contexte	informations transmises à une exécution	faits conservés, provenance et omissions
Quantification des poids	précision de certaines matrices du modèle	mémoire utilisée et qualité mesurée

Une conversion en 4 bits ne réduit pas le nombre de jetons du texte. Un extrait plus court ne garantit ni une meilleure réponse ni une économie facturée. Pour comparer les chemins local et distant, conserver l’identité déclarée du modèle, les appels terminés, les délais, les compteurs de jetons disponibles et une vérification indépendante du diagnostic. Les compteurs de deux tokenizers ne sont pas nécessairement comparables.

Ce cas adapte l’article « Du texte aux vecteurs : comprendre Qwen3.8 et GPT-6 dans Tokki » et sa note de vérification du 24 septembre 2026 (dossier privé `articles/`). Qwen a été choisi par l’opérateur. L’exemple n’ajoute ni essai de génération ni comparaison de performances ; il décrit une association de modèles, sans gain établi de qualité, de latence ou de coût.

5.6 De l’intention à la preuve : Tokki × AGILAB

Les deux affiches de cette étude se complètent : la première décrit les fonctions nécessaires pour construire une solution ; la seconde, une réalisation et ses résultats observés. La question directrice est : **quel verrou chaque fonction lève-t-elle, et quelle preuve permet de le vérifier ?** Les exemples réseau ci-dessous sont des transferts pédagogiques, pas des essais déjà réalisés sur un réseau opérationnel.

Sept briques et une boucle de travail

Le poster « IA, des solutions 2027 » propose une organisation de travail que l’on peut lire par **verrous à lever**, plutôt que comme une chronologie. Son millésime désigne un horizon éditorial, pas une date de mesure.

Brique	Verrou traité	Preuve attendue
Cap : problème, contraintes, objectif	une intention trop vague pour être testée	critère de réussite et cas d’échec explicites
Réalité observée : données et mesures	raisonner sur une représentation sans lien vérifié avec le terrain	provenance, unités, couverture et limites des données
Moyens de calcul	une expérience impossible à exécuter dans le budget disponible	mémoire, durée et ressources effectivement mesurées
Jumeau numérique, si pertinent	tester des conséquences sans perturber le système réel	domaine de validité et confrontation aux observations
Agent de codage	transformer une intention en code exécutable	modifications inspectables et tests indépendants
Tokki : contrôle local et preuves	encadrer les opérations selon les règles et conserver un contexte récupérable	autorisations applicables, résultats contrôlés et accès aux traces
AGILAB : exécution applicative	relier application, ressources et résultats	exécution reproductible et résultats vérifiés

Ces briques ont des rôles distincts. Le jumeau numérique est utile lorsque la simulation répond à une question vérifiable ; il n’est pas un préalable universel à une solution d’IA. L’agent produit une proposition de code ; le cadre d’exécution rend possible son essai ; les tests déterminent si elle répond au problème. Aucun de ces rôles ne garantit à lui seul la qualité.

Pour un diagnostic réseau, la chaîne devient : définir l’erreur à détecter, réunir des journaux représentatifs, choisir les moyens de calcul, simuler les cas dangereux si nécessaire, préparer le contexte, construire l’application, puis vérifier ses résultats sur des cas conservés à part. Les échecs de test alimentent une nouvelle itération documentée. Le verrou final est donc la **preuve d’utilité**, pas la seule disponibilité d’un modèle.

Lire les flèches. Les numéros de l’affiche repèrent des rôles, pas un ordre d’exécution. Les entrées du projet (1 à 4) et les demandes alimentent Tokki (6), qui prépare le contexte de l’agent (5) et encadre son passage aux outils selon les règles configurées. L’agent modifie des fichiers et utilise des outils ; AGILAB (7) organise l’exécution applicative. Tests et résultats reviennent ensuite enrichir le contexte de l’itération suivante. Conserver un échec avec ses entrées et sa trace rend possible une correction vérifiable ; conserver seulement la dernière réponse ferait perdre cette preuve.

L’affiche vise l’horizon 2027. Les logos repèrent les acteurs, sans valoir certification. Son PDF original, joint au livre, permet de la consulter en pleine résolution.



Affiche 1 — Construire une solution d'IA : les sept briques et leur boucle.

Distinguer modèle, agent, contexte et exécution

Le second support décrit une **supervision hybride** : Codex fournit en ligne consignes, diagnostics et contrôles de livraison. Sur chaque nœud local, Tokki encadre les opérations et conserve contexte et preuves récupérables. OpenCode organise la boucle dans laquelle l'assistant lit les fichiers et utilise les outils ; Qwen produit les réponses pour construire le code. Le modèle, l'agent et le cadre d'exécution ont donc des rôles distincts. AGILAB assure l'intégration applicative ; une galerie Hugging Face expose les démonstrations sous Linux.

« Cinq installations autonomes » signifie ici cinq environnements locaux distincts. Cela ne prouve ni une réalisation sans supervision, ni cinq entraînements de modèle, ni l'exécution simultanée de toutes les applications. Le schéma montre trois machines RTX 4090 utilisées pour les résultats : `rtx1` pour le cas free-threading, `rtx4` pour MILP et `rtx5` pour les trois autres applications. Les deux RTX 3090 (`rtx2`, `rtx3`) sont indiquées disponibles ; le tableau ne fournit aucune mesure sur ces cartes.

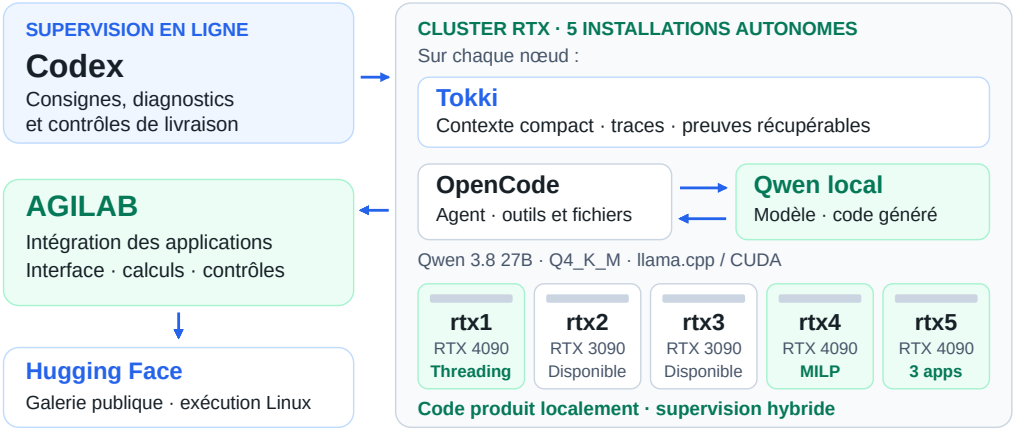
Quatre distinctions restent essentielles : réponse du modèle et action d'outil ; action réussie et application correcte ; réussite sur un cas et validation en production ; génération locale et supervision distante. Pour chaque étape d'un diagnostic réseau, préciser les données transmises, le lieu d'exécution et le résultat observé.

Lecture de l'affiche intégrée. Les observations datent du 22 septembre 2026. Les colonnes de temps ont des périmètres différents : leur juxtaposition ne démontre pas une accélération RTX/MLX. La colonne Mac ne permet pas d'identifier son moteur comme MLX. Ces limites s'appliquent à toute lecture de l'affiche, dont le PDF original est également joint au livre.

TOKKI × AGILAB

Un environnement. Des résultats.

5 applications · 3 versions — Résultats du 22 septembre 2026



Résultats des builds livrés

Durées en h:mm:ss — Reprises et validations incluses

APPLICATION Nœud du build RTX	GPT-6 Astra	Qwen Mac	Qwen RTX	KLOC RTX	Lignes/min RTX
Iris rtx5 · RTX 4090	0:04:45	0:34:09	**0:08:02**	**0,261**	**32,5**
Text Atlas rtx5 · RTX 4090	0:07:44	0:20:08	**0:20:54**	**0,304**	**14,5**
Chronos Forecast rtx5 · RTX 4090	0:06:01	1:29:22	**0:26:55**	**0,407**	**15,1**
Python free-threading rtx1 · RTX 4090	0:12:31	4:27:47	**4:26:27**	**1,312**	**4,9**
MILP Energy rtx4 · RTX 4090	0:24:00	4:25:00	**4:23:58**	**0,584**	**2,2**

15 variantes testées · 5 bundles RTX vérifiés

KLOC : code Python livré, app + notebook.
1 KLOC = 1 000 lignes, hors commentaires, docstrings, tests et code fourni.
Lignes/min = volume livré ÷ durée.
Aucune mesure de qualité du code.

RTX : temps actif des builds et reprises.
Hors installation, téléchargements et tests navigateur. Astra/Mac : attentes incluses.
Matériels et périmètres différents.
Comparaison non contrôlée.

Génération locale. Supervision en ligne.

Démo et reçus : huggingface.co/spaces/jpmorard/tokki

Affiche 2 — Environnement et résultats : observations, sans classement matériel.

Cinq applications, cinq questions à comprendre

Les applications ne sont pas cinq façons de mesurer un GPU. Elles permettent d'interroger des mécanismes différents. Les reçus archivés décrivent des contrôles fonctionnels ; ils ne remplacent pas une évaluation propre au domaine dans lequel on veut transférer l'application.

Application	Compréhension et verrou levé	Transfert pédagogique aux réseaux
Iris	classifier des observations et régler la capacité du modèle	catégoriser des états réseau, puis mesurer les erreurs sur des cas distincts
Text Atlas	transformer un corpus en représentations exploitables	explorer des journaux ou tickets ; vérifier la pertinence des regroupements et de la recherche
Chronos Forecast	prévoir une série sans confondre ajustement et anticipation	prévoir une charge ; évaluer sur une période ultérieure sans fuite de données
Python free-threading	distinguer parallélisme demandé et exécution réellement parallèle	traiter de la télémétrie ; mesurer le temps avec l'état du GIL et le coût de coordination
MILP Energy	optimiser sous contraintes et reconnaître l'absence de solution admissible	affecter des ressources sous capacité ; expliciter le coût d'une demande non servie

Ce que les contrôles archivés établissent. Pour Iris, un réglage de profondeur du classifieur a été exercé et des résultats ont été calculés. Pour Text Atlas et Chronos, les contrôles mentionnent le déclenchement réel de l'analyse et l'obtention de résultats. Cela ne mesure ni la pertinence d'un RAG, ni la précision d'une prévision sur une nouvelle période. Pour free-threading, les traces rapportent un essai réel, le téléchargement des observations de GIL et de temps, et la validité du paquet de sources après exécution ; le résultat reste propre à cette charge et à cet interpréteur.

Pour MILP, les contrôles portent sur des conséquences discriminantes : un coût de démarrage modifie l'optimum, une capacité insuffisante produit une infaisabilité, l'autorisation de délestage restaure la faisabilité en laissant une demande non servie, et les exécutions série puis à deux travailleurs sont effectivement déclenchées. **Le chemin RTX mesuré utilise un lanceur Python généré ; le moteur AGILAB fourni sert de référence de source et n'est pas le moteur mesuré.** Une proximité de code ou une mention dans l'interface ne suffit donc pas à attribuer un temps au moteur de référence.

Distinguer trois niveaux de preuve. Les quinze variantes correspondent à cinq applications déclinées suivant trois voies de construction. Les cinq paquets de sources RTX ont été téléchargés et leurs empreintes contrôlées. Une empreinte établit l'identité d'un fichier ; un contrôle fonctionnel établit un comportement sur les entrées testées ; seule une évaluation représentative peut étayer l'utilité dans le domaine cible. Les contrôles archivés sont des observations datées, pas une vérification permanente de la galerie en ligne ni une preuve de généralisation aux réseaux.

Interpréter des résultats de construction sans en faire une évaluation comparative

Le second poster, « Tokki × AGILAB — environnement et résultats », décrit cinq applications réalisées suivant trois variantes. Les observations archivées portent sur le **22 septembre 2026**. Elles illustrent une chaîne de travail avec génération locale et supervision en ligne ; elles ne démontrent pas une autonomie complète ni une supériorité matérielle.

Les durées, volumes RTX et débits en lignes par minute concordent avec les deux fichiers de métriques archivés. Vérifier le détail des phases demanderait aussi les événements bruts ; les empreintes seules ne suffisent pas.

Application	Astra : durée historique	Mac : durée historique	RTX : périodes actives
Iris	0:04:45	0:34:09	0:08:02
Text Atlas	0:07:44	0:20:08	0:20:54
Chronos Forecast	0:06:01	1:29:22	0:26:55
Python free-threading	0:12:31	4:27:47	4:26:27
MILP Energy	0:24:00	4:25:00	4:23:58

Durées en heures:minutes:secondes, arrondies à la seconde. **Les colonnes n’ont pas le même périmètre.** Selon la note de mesure, le RTX additionne les périodes actives de construction et de réparation, en excluant les intervalles entre reprises, l’installation, les téléchargements et les tests navigateur. Les mesures historiques Astra/Mac incluent des attentes. Une division de ces colonnes ne mesure donc pas une accélération du matériel.

Les productions diffèrent également : pour MILP, les métriques comptent 584 lignes Python livrées côté RTX contre 1 768 côté Mac ; pour Text Atlas, 304 contre 236. Le volume inclut application et cellules de code du notebook, hors commentaires, docstrings, tests et fichiers fournis. Les lignes par minute représentent ce volume divisé par la durée retenue : elles ne mesurent ni le débit de génération en jetons, ni la qualité, ni la performance d’exécution de l’application.

RTX et MLX ne désignent pas le même type d’objet. RTX identifie ici le matériel NVIDIA ; MLX désigne un environnement logiciel. Le poster déclare un chemin Qwen quantifié Q4_K_M avec llama.cpp, un logiciel d’exécution de modèles de langage, et CUDA sur RTX. Sa colonne « Qwen Mac » et les extraits de métriques archivés n’établissent pas l’identité du moteur Mac, du modèle chargé et de sa quantification. On ne peut donc pas renommer cette colonne « MLX », ni l’assimiler au modèle MLX retenu dans l’encadré précédent. **Aucun classement RTX/MLX n’est établi.**

Il manque un **protocole commun** : tâche et critères d’acceptation identiques, modèles et quantifications documentés, contexte et paramètres conservés, mêmes budgets de reprise, matériel et versions consignés, plusieurs répétitions. Mesurer préparation, attente, chargement, génération, outils, réparations et validation ; publier durée totale et sous-durées. Pour isoler l’inférence, garder les mêmes entrées et distinguer traitement de la requête, premier jeton et génération. Sans ce protocole, on compare des parcours de construction, pas les performances d’un banc d’évaluation.

Sources privées et contrôle reproductible. Les deux posters sources sont conservés dans `posters/ia-des-solutions-2027/` et `posters/tokki-agilab/`. Le contrôle associé au livre, `tools/verify_tokki_agilab_poster_metrics.py`, vérifie les empreintes et recalcule les valeurs publiées ; son résultat est enregistré dans `tokki-agilab-verification-chiffres-fr.json`. Il ne remplace pas les reçus bruts et ne prouve pas l’exécution d’un moteur MLX. Les contrôles fonctionnels proviennent notamment de `evidence/live-rtx-final/browser-validation.json` et de `evidence/live-milp-runtime-serial/browser-validation.json`, dans l’archive du second poster. Les deux affiches sont reproduites sur des pages dédiées, accompagnées de leurs PDF originaux en pièces jointes. Les explications et tableaux en précisent le sens et les limites ; les observations ne sont pas transformées en garanties.

Tokki : suivre l’arbre de contrôle

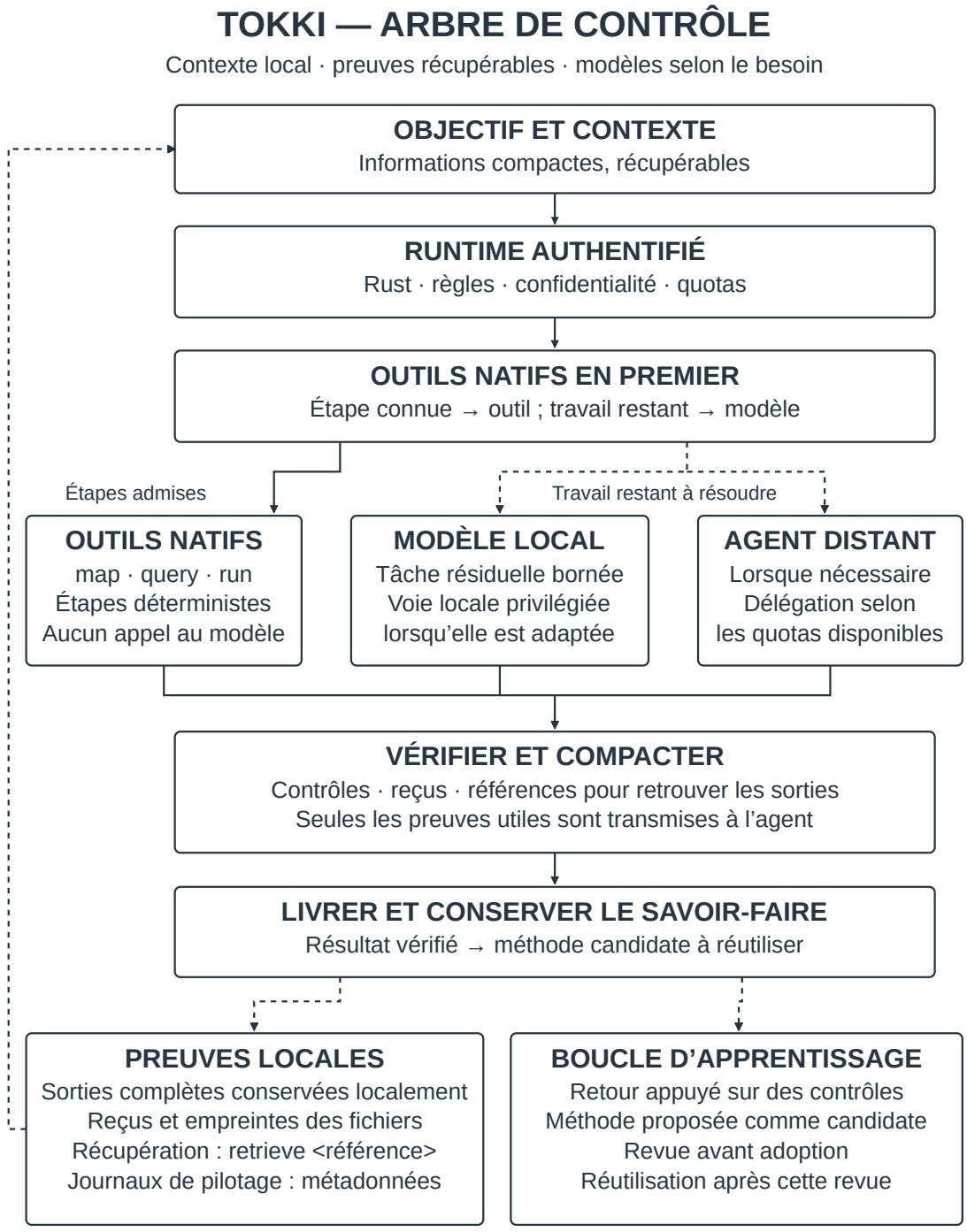
Le poster **« Tokki Control Tree » du 5 octobre 2026** rassemble la boucle de travail de ce chapitre. Son trajet central se lit ainsi : préciser l’objectif et le contexte, passer par l’environnement d’exécution authentifié, puis choisir comment effectuer l’opération.

Une **opération connue**, admise par les règles applicables, peut être effectuée par un outil natif : rechercher un symbole, lire un fichier ou exécuter une commande ne demande pas nécessairement un appel à un modèle. Le **travail restant à résoudre** peut mobiliser un modèle local, lorsque cette voie convient, ou un agent distant selon le besoin et les quotas. « Local en premier » décrit une préférence de routage ; cela ne promet pas que toute tâche sera résolue localement.

Les branches se rejoignent ensuite : **vérifier le résultat**, conserver les sorties complètes localement et transmettre à l’agent les éléments utiles, avec une référence permettant de retrouver le détail. Un *reçu* est ici une trace de l’opération et de son résultat ; une *empreinte* permet de vérifier l’identité d’un fichier. Ces traces aident à reprendre le travail sans confondre une réponse plausible avec une action effectivement contrôlée.

La boucle latérale concerne la réutilisation : un résultat vérifié peut faire naître une **méthode candidate**, qui demande encore une revue avant son adoption. Il s’agit de capitaliser une façon de travailler ; cette boucle n’entraîne pas les poids du modèle de langage.

Lecture pour l’impression. La figure suivante reprend le sens du poster avec des libellés français, des traits fins et un fond blanc. L’original anglais, conservé sans modification, est joint au PDF sous le nom `tokki-control-tree-poster-en-20261005.png`.



Affiche 3 — Tokki : contexte, exécution, preuves et réutilisation.

Migrer AGILAB sans perdre le notebook : un cas de codage autonome

Une migration d'interface commence souvent par une envie de moderniser un écran. Dans AGILAB, la question de départ était plus exigeante : quel intérêt y aurait-il à passer à React, quel serait le coût rapporté au gain, et pourrait-on toujours exporter une application dans un notebook ? Cette dernière contrainte a guidé les choix. Un notebook devait rester un document lisible, modifiable et exécutable, avec ses calculs Python et ses interactions.

Ce cas, arrêté au **4 octobre 2026**, décrit une réalisation menée par un agent de codage et des agents spécialisés, que j'ai pilotés. Il distingue l'intégration initiale du framework de la vérification ultérieure des paquets installés, des applications intégrées et de l'environnement privé. Chaque jalon conserve ses preuves et ses limites ; la publication publique reste une étape séparée.

Du besoin à un périmètre vérifiable

La première friction était concrète. Une vue exportée qui ne disposait pas d'un rendu intégré pouvait ouvrir Streamlit, le logiciel qui hébergeait alors l'interface. L'export devenait un point de départ vers une autre application, au lieu de rester dans Jupyter. Corriger cette rupture sur les vues courantes apportait un bénéfice observable.

Le périmètre a ensuite progressé par décisions explicites : privilégier la carte et les courbes d'analyse ; construire l'interface principale en React ; conserver les vues Python spécialisées ; enfin retirer entièrement la dépendance à Streamlit. Ce chemin est instructif. Une proposition d'architecture n'autorise pas à elle seule une réécriture générale. Chaque extension doit conserver le but et les contraintes déjà acceptés.

Le travail autonome a porté sur l'exécution : chercher les points d'entrée, modifier les composants, adapter les paquets, tester, corriger les défauts observés et préparer des changements inspectables. La personne a fixé la destination, choisi les priorités et demandé les étapes suivantes.

Une interface partagée, des calculs conservés

Réécrire tous les traitements en JavaScript aurait ajouté un second chantier. Le choix retenu a été de conserver les contrats et calculs Python, puis de changer la manière de les présenter.

Élément	Solution retenue	Vérification utile
Interface principale, carte et courbes	Composants React servis localement	Affichage et interactions dans un vrai navigateur
Éditeur de pipeline, vues géographiques spécialisées et administration	Vues Python reliées à l'hôte natif agi_web	Commandes, formulaires et état de session
Export notebook	Ressources React embarquées dans un widget AnyWidget	Exécution et interaction dans Jupyter
Distribution	Paquets et lanceurs sans dépendance Streamlit	Installation neuve et métadonnées des paquets

L'**hôte natif** est ici le programme Python qui sert les ressources locales et reçoit les interactions de l'interface. Il remplace le rôle d'hébergement nécessaire aux vues conservées. **AnyWidget** fournit le lien entre la partie Python d'un widget et son rendu dans le notebook.

Partager une carte ou une courbe ne signifie pas rendre toutes les pages identiques. Le navigateur et Jupyter ont des cycles de vie différents : navigation et sessions dans le premier, cellules, noyau Python et sorties dans le second. Le composant commun doit s'adapter à chacun. L'utilisateur du notebook n'a pas à installer Node.js pour simplement utiliser les ressources déjà construites.

L'autonomie se voit dans les réparations

Le travail a été réparti entre les chemins de lancement, les dépendances et les exports notebook. Des revues distinctes ont ensuite recherché des défauts de correction et de maintenance. Cette répartition permet d'avancer en parallèle, mais

l’intégration reste une responsabilité commune : une interface peut fonctionner depuis les sources et échouer après installation.

Plusieurs problèmes concrets ont été corrigés avec des vérifications ciblées : soumission de formulaires, interactions en attente, durée de vie des caches, rafraîchissement d’une vue notebook, interférence entre ses styles et ceux de Jupyter, et choix dont la valeur peut être absente. Ce sont des différences de comportement, pas seulement de présentation.

Une autre réparation a concerné deux vues cartographiques conservées. Leurs dépendances PyDeck et Parquet devaient être déclarées par les bons paquets. Une installation neuve a permis de vérifier les points d’entrée, les imports et un aller-retour de données Parquet. Un environnement de développement déjà équipé aurait pu masquer cet oubli.

Tokki a encadré le passage aux outils et fourni des résultats récupérables. Les changements Git, les tests et les contrôles d’installation ont constitué des points de reprise. L’enjeu était de reprendre à partir du dernier résultat concret, sans effacer les modifications de l’utilisateur ni confondre un code prêt avec un environnement effectivement mis à jour.

Ce qui a été prouvé au jalon de migration

La migration principale correspond à la proposition de changement AGILAB n° 1064, intégrée le 3 octobre 2026. Le commit d’intégration est dce2392. Les chiffres suivants proviennent des rapports de ce jalon. Ils décrivent des profils différents et ne doivent pas être additionnés.

Contrôle	Résultat enregistré	Portée
Suite locale du projet	9 074 tests réussis, 33 ignorés	Régressions couvertes par ce profil
Suite Windows	3 998 tests réussis, 26 ignorés	Profil de validation Windows
Installation neuve de l’interface et du notebook	120 distributions compatibles, absence de Streamlit	Paquets réellement installés
Interface principale installée	Accueil, projet et analyse contrôlés sur ordinateur et mobile	Parcours de navigation observés
Exports et vues notebook	Interactions Jupyter et exports portables contrôlés	Usage hors de l’ancien hébergement

Les contrôles navigateur de l’interface principale n’ont relevé ni erreur de navigateur, ni requête échouée, ni requête externe. Les formulaires et le rendu Python ont aussi été exercés dans un navigateur réel. Les environnements locaux principaux et l’outil global ont été mis à jour sans Streamlit.

Détecter onze projets intégrés et afficher un espace de travail ne prouve pas l’exécution de tous leurs traitements métier. Les tests ignorés conservent leur raison : dépendance scientifique optionnelle, plateforme différente ou scénario soumis à activation explicite. Ce jalon ne certifie donc pas l’ensemble des applications.

Publier et valider le catalogue : une seconde chaîne de preuve

Le lecteur qui installe une ancienne version publiée ne reçoit pas automatiquement les nouveaux fichiers. La suite demandée consiste donc à préparer puis vérifier les distributions du framework et des applications intégrées, ainsi que les applications privées dans leur circuit propre.

Le contrôle doit partir d’un environnement neuf : installer les versions prévues, vérifier les dépendances, charger chaque application et exercer ses parcours représentatifs avec des données adaptées. Une bibliothèque scientifique optionnelle reste nécessaire lorsque le calcul de l’application l’utilise. Retirer Streamlit ne retire ni cette exigence ni les besoins de configuration.

Les applications privées ne doivent pas être copiées dans une publication publique pour simplifier la validation. On peut partager une méthode de test et publier un état agrégé, tout en conservant leurs sources, noms, données et preuves détaillées dans l’espace autorisé. Au jalon décrit ici, cette publication et la validation exhaustive du catalogue ne sont pas annoncées comme achevées.

Suivi du 4 octobre 2026 : des sources aux paquets installés

Un second jalon prépare la version coordonnée **2026.10.4**. Il répond à une question que le premier changement d'interface laissait ouverte : que reçoit réellement une personne qui installe AGILAB et ses applications dans un environnement neuf ? Le code proposé à la publication est désormais suivi dans la proposition n° 1065.

Les preuves couvrent plusieurs parcours distincts :

- **Catalogue intégré.** Les quatorze applications passent leurs contrôles fonctionnels depuis les paquets installés. Les scénarios comprennent un entraînement PyTorch réel, l'exécution de calculs R et la production de données persistées. Pour le graphe de tâches DAG, la preuve porte sur la planification ; l'exécution sur un cluster distant reste hors de ce contrôle.
- **Distribution.** Trente-huit distributions ont été construites et inspectées, dont trente-cinq destinées à PyPI et trois à des archives de release. Leurs dépendances ne comprennent plus Streamlit ni les extensions retirées avec lui. Les quatorze applications conservent leur politique de distribution : onze paquets promus et trois archives.
- **Usage installé.** L'interface native a été exercée dans de vrais navigateurs, sur ordinateur et en disposition mobile. Les exports ont également été utilisés dans JupyterLab avec les vues Python, les callbacks et les formulaires. Un clone neuf du code committé a parcouru l'installation, l'import notebook, l'exécution, l'analyse et l'export.
- **Régression.** Le profil complet de l'interface rapporte 5 283 tests réussis et six tests ignorés. Ce nombre décrit ce profil précis. Les contrôles métier des quatorze applications et les parcours dans le navigateur apportent des preuves complémentaires.

Ces parcours ont encore révélé des défauts. Les opérations d'une vue Python échouaient lorsque le noyau Jupyter avait déjà une boucle asynchrone active. Le clonage devait adapter les chemins du framework et des bibliothèques d'interface. Le catalogue réellement produit par son générateur devait correspondre aux quatorze applications prévues. L'isolation des sessions et l'exécution des gardes Git dans un environnement au chemin de recherche limité ont également été corrigées. Les régressions ont été reproduites avant correction, puis vérifiées sur le code corrigé.

La préparation des applications privées suit la même démarche dans leur circuit autorisé. Onze distributions ont été reconstruites. L'environnement principal a ensuite été activé et contrôlé : 457 distributions compatibles, aucune dépendance Streamlit, neuf formulaires totalisant 167 contrôles, et six imports réutilisables sans session d'interface. Un notebook neuf a aussi exercé un formulaire et vérifié la persistance de ses réglages. Le SDK a préparé les projets de construction des neuf workers, puis leurs dépendances ont été résolues.

Une revue indépendante a aussi détecté un import qui exigeait indûment une session, des références vers des dossiers de développement absents après installation, des caches inclus dans les archives source et une normalisation de notebook qui changeait à chaque exécution. Les corrections ont été recontrôlées.

Les dix suites métier existantes rapportent 971 tests réussis, un scénario Linux privilégié ignoré et quatorze scénarios de compilation ou de cluster écartés de cette passe. Ces derniers restent à exercer avec le montage système requis. Leurs noms, données et preuves détaillées restent dans l'espace privé.

La prépublication privée a été publiée dans son circuit autorisé. L'intégrité des artefacts a été contrôlée par leurs empreintes locales et les digests fournis par le serveur. Cette preuve n'est pas présentée comme l'exécution de tous les pipelines ou d'un entraînement complet.

L'instantané public relu est be61965. Les trente-huit roues testées sont identiques à celles produites depuis 2e823f3. Les trente-cinq versions prévues pour PyPI sont préparées et **ne sont pas encore publiées** à la date de ce suivi. Leur publication dépend des contrôles requis et de l'approbation d'un autre mainteneur : l'environnement GitHub interdit l'auto-approbation. L'agent prépare les artefacts et poursuit les opérations autorisées jusqu'à cette frontière explicite.

Les versions antérieures restent disponibles pour la reproductibilité des installations existantes. La migration sera reçue par l'installation de la nouvelle version coordonnée, une fois publiée. L'état « construit et vérifié », l'état « publié » et l'état « activé chez l'utilisateur » gardent ainsi chacun leur preuve.

Le coût rapporté au gain

Le gain immédiat est facile à observer : l'export reste utilisable dans le notebook et l'interface principale fonctionne sans l'ancien hébergement. La carte et les courbes partagées réduisent aussi le nombre de rendus à maintenir.

Le coût dépasse pourtant l'écriture de composants. Il comprend le transport des interactions Python, les sessions, les caches, les formulaires, les installations, les exports, les dépendances optionnelles et la documentation. Les contrôles sur des paquets installés sont une partie du travail, pas une formalité finale.

Ce cas ne fournit pas une mesure fiable d'économie financière ou de productivité. Il manque un parcours comparable sans agent, un relevé homogène du temps humain, des temps d'attente et des coûts d'inférence. Les estimations initiales ont servi à choisir une direction ; elles ne constituent pas un coût final mesuré. Le bénéfice démontré est la continuité d'usage obtenue et la suppression vérifiée d'une dépendance dans le périmètre contrôlé.

Le TP sur le gain réel d'une assistance IA propose un relevé pour une prochaine expérience : référence sans assistance, temps humain complet, dépenses et résultats acceptés.

Ce que ce cas apprend aux projets de réseau

Un agent de codage peut conduire un changement qui traverse interface, calcul, installation et documentation. L'autonomie utile consiste à poursuivre ce travail autorisé, corriger ce qui échoue et apporter les preuves qui permettent de juger le résultat.

Le même principe s'applique à un simulateur de réseau : préserver modèles et expériences, changer leur présentation, puis vérifier les résultats sur des scénarios connus. Une carte affichée ne prouve pas un routage correct ; un test réussi ne prouve pas un déploiement opérationnel. Chaque affirmation doit être reliée à l'expérience qui la soutient.

Pour reproduire cette méthode, il faut définir quelques parcours d'acceptation, séparer les chantiers qui peuvent avancer en parallèle, faire relire leurs frontières, installer les artefacts dans un environnement neuf et garder un point de retour. L'agent assure la continuité des opérations ; la personne conserve le choix du but et l'évaluation de l'intérêt du changement.

Sources et statut du cas

- Proposition AGILAB n° 1064 : description publique du changement, contrôles et limites.
- Préparation de release n° 1065 et source relue be61965 : paquets installés, parcours notebook et statut de publication du second jalon.
- Source intégrée dce2392 : point de référence du framework au jalon.
- Guide du dépôt public, « Native React UI and notebook export » : contrats de l'interface et de l'export.
- Dossier local de validation associé à cet article : empreintes des rapports de migration, contrôle de l'insertion et des exports PDF. Les chemins locaux, journaux de session et détails d'applications privées restent hors de l'article.

Du code plausible au résultat vérifié

Un simulateur annonce que tous les paquets sont arrivés. Sa carte paraît cohérente et son premier test passe. Pourtant, il compte deux fois une livraison et continue d'emprunter un lien coupé. Comment s'en apercevoir avant de confier ses résultats à une décision ?

Cette séquence part d'un petit modèle que l'on peut vérifier à la main. Elle prolonge le cas de migration : les coûts manquants dans ce cas restent inconnus. Le protocole proposé ici permet de les relever lors d'un nouvel essai ; il ne reconstitue pas les mesures absentes.

Une démonstration qui réussit à tort

Quatre nœuds, A, B, C et D, sont reliés par des liens orientés. Les coûts de A vers B et de B vers D valent chacun 1 ; ceux de A vers C et de C vers D valent chacun 2. Ces nombres sont des coûts abstraits, sans unité. Le chemin A–B–D

coûte donc 2, contre 4 pour A–C–D. Le routage choisit le chemin disponible de coût minimal ; il ne répartit pas les paquets entre plusieurs chemins. Aucun algorithme d'apprentissage n'intervient.

Trois paquets distincts sont proposés au modèle : p1 contient 1 000 octets, p2 en contient 1 500 et p3 en contient 500. Un octet représente huit bits. Après p2, une notification de sa livraison est répétée, puis le lien B–D est coupé avant le passage de p3. Le programme conserve l'ancien chemin. Chaque événement porte l'état des liens au moment de son passage.

```
avant = {("A", "B"), ("B", "D"), ("A", "C"), ("C", "D")}
apres = avant - {("B", "D")}
journal = [
    ("p1", 1000, ("A", "B", "D"), avant),
    ("p2", 1500, ("A", "B", "D"), avant),
    ("p2", 1500, ("A", "B", "D"), avant),
    ("p3", 500, ("A", "B", "D"), apres),
]

def bilan_plausible(evenements):
    return sum(taille for _, taille, chemin, _ in evenements
               if chemin[0] == "A" and chemin[-1] == "D")

assert bilan_plausible(journal[:1]) == 1000
assert bilan_plausible(journal) == 4500
```

Le premier test vérifie un cas heureux. Le second confirme ce que calcule le programme, mais pas ce que demande le problème. Le programme voit A au départ et D à l'arrivée ; il ignore les liens traversés et l'identité des paquets déjà comptés. Avant de lire la correction, calculez le nombre d'octets distincts dont le passage est valide.

Calcul de référence. p1 et p2 contribuent une seule fois : 1 000 + 1 500 = **2 500 octets**. La seconde notification de p2 n'ajoute rien. Le passage de p3 par B–D est invalide après la coupure. Les 4 500 octets annoncés dépassent même les 3 000 octets distincts proposés. Ce contrôle de conservation aurait dû déclencher une recherche d'erreur.

Construire une vérification indépendante

Le cahier des charges précède la correction : compter chaque paquet valide une seule fois, vérifier chaque lien dans l'état fourni avec l'événement, et signaler la réutilisation d'un identifiant pour une autre taille. Pour cet exercice, le journal est fourni par l'enseignant ; les identifiants et tailles sont supposés correctement renseignés. Sa validation ne constitue ni une capture réseau ni une mesure de réception sur un équipement réel.

Une correction possible complète le programme précédent :

```
def bilan_verifie(evenements):
    recus = {}
    for identifiant, taille, chemin, liens in evenements:
        valide = (
            len(chemin) >= 2
            and chemin[0] == "A" and chemin[-1] == "D"
            and all((a, b) in liens for a, b in zip(chemin, chemin[1:]))
        )
        if not valide:
            continue
        if identifiant in recus and recus[identifiant] != taille:
            raise ValueError("Identifiant réutilisé pour une autre taille")
        recus[identifiant] = taille
    return sum(recus.values())

assert bilan_verifie(journal) == 2500
assert bilan_verifie(journal + [journal[1]]) == 2500

journal_corrige = journal[:-1] + [
    ("p3", 500, ("A", "C", "D"), apres),
]
assert bilan_verifie(journal_corrige) == 3000
```

La correction du compteur laisse p3 non livré dans le premier journal. La correction du routage remplace ensuite son chemin par A–C–D, dont le coût vaut 4. Les deux corrections portent sur des responsabilités différentes. Le compteur ne prouve pas à lui seul que le chemin choisi est optimal. Ce petit exemple ne modélise ni files d'attente, ni capacité, ni pertes aléatoires, ni délai. Il ne permet donc d'annoncer aucun débit reçu.

L'indépendance vient du calcul et des règles écrites avant la correction. Demander au même agent de produire le programme et des tests peut reproduire la même erreur dans les deux. Un autre agent peut aussi partager cette erreur, surtout s'il lit le premier résultat avant de chercher sa propre réponse. On lui donne d'abord le problème et le journal, puis on confronte les raisonnements. L'accord entre agents complète les contrôles ; il ne remplace pas le calcul de référence.

Changer une hypothèse. Sans assistance, ajoutez p4, de 700 octets, sur A–C–D après la coupure. Le journal initial doit alors donner **3 200 octets** valides. Expliquez ensuite pourquoi rejouer sa notification ne doit pas changer ce total. Enfin, proposez un test pour un identifiant réutilisé avec une autre taille. Le lecteur doit pouvoir défendre ces réponses même si un agent a écrit la correction.

TP — Mesurer le gain réel d'une assistance IA

But et prérequis. Savoir lire une boucle Python, additionner les coûts d'un chemin et distinguer une notification d'un paquet suffit pour commencer. Prévoyez une séance de préparation du protocole et une séance d'essais. Le programme s'exécute localement avec Python, sans dépendance supplémentaire et sans agir sur le réseau. Les blocs ci-dessus peuvent être réunis dans un même fichier. Lors des essais comparatifs, l'enseignant distribue des variantes et conserve leurs corrections à part.

La question à tester est précise : **à critères de qualité identiques, l'assistance réduit-elle le temps humain et le coût d'un résultat accepté ?** On peut constater un gain sur une dimension et une perte sur une autre. Une impression de facilité se relève séparément des mesures.

1. **Fixer les cas avant les essais.** Préparer plusieurs petits journaux comparables : doublon, coupure, chemin de secours, destination inaccessible ou identifiant incohérent. Écrire pour chacun le résultat attendu à la main. Conserver quelques variantes pour le contrôle final, sans communiquer leur solution aux participants ni aux agents. Les liens restent orientés, le routage suit le coût minimal disponible et les tailles sont fixées. Les mêmes règles de service et d'acceptation s'appliquent à toutes les conditions.
2. **Comparer trois conditions.** Prévoir un travail sans assistance générative, un travail avec un agent et un travail avec plusieurs agents. Tous disposent de la même documentation, du même matériel et des mêmes outils Python. Répartir les cas entre des participants de niveau comparable et tirer l'affectation au sort. Une personne ne résout pas trois fois le même cas : elle connaîtrait déjà sa correction. Si les variantes diffèrent, conserver cette différence dans l'analyse au lieu de les déclarer identiques.
3. **Borner les moyens.** Fixer avant le départ la même durée maximale par cas et le même plafond total d'inférence pour les deux conditions assistées. Ce plafond couvre l'ensemble des agents, leurs reprises et leurs échanges. Noter modèle, version, date, consignes et outils autorisés. Enregistrer séparément le temps humain actif, l'attente et la durée écoulée. Si deux personnes relisent simultanément pendant dix minutes, relever vingt minutes humaines.
4. **Accepter selon le même contrat.** Le résultat doit passer tous les cas réservés, respecter le périmètre local et être expliqué par son auteur. Un relecteur applique la même grille, sans connaître la condition si cela est possible. Compter aussi les essais refusés, les abandons et les délais dépassés. Une exécution interrompue sans résultat final reste non vérifiée.
5. **Livrer les éléments de jugement.** Conserver cahier des charges, journaux fournis, code, résultats attendus, contrôles, temps et dépenses. Le tableau ci-dessous accompagne les conclusions. Quelques essais de classe décrivent ces cas ; ils ne suffisent pas à établir un effet général de l'IA.

Relevé par condition	Sans assistance générative	Un agent	Plusieurs agents
Cas tentés, acceptés, refusés ou inachevés	à relever	à relever	à relever

Relevé par condition	Sans assistance générative	Un agent	Plusieurs agents
Préparation, écriture et coordination humaines, en minutes	à relever	à relever	à relever
Revue et corrections humaines, en minutes	à relever	à relever	à relever
Durée écoulée et attente, en minutes	à relever	à relever	à relever
Inférence et exécution, en euros, tarif daté	à relever	à relever	à relever
Explication et transfert sans assistance	à évaluer	à évaluer	à évaluer

Pour plusieurs personnes, les minutes humaines sont additionnées par personne et par activité, sans compter deux fois le même intervalle. Pour des agents en parallèle, la durée écoulée suit l'horloge ; la dépense additionne leurs consommations. Le relevé évite ainsi de confondre délai, travail et facture.

Pourquoi garder la condition sans assistance ? Elle fournit la référence qui manque à une simple démonstration. Si elle n'a pas été réalisée, écrire « référence absente » et laisser le gain non calculé. Comparer un agent à plusieurs agents répond à une autre question : l'intérêt de la répartition du travail, une fois l'assistance déjà présente.

Calculer le coût d'un résultat accepté

Pour une condition, on additionne les dépenses de **tous** les essais, y compris ceux qui échouent. Le temps humain couvre préparation, écriture, coordination, revue et reprises. Avec un tarif horaire explicite :

$$C_{total} = C_{inférence} + C_{exécution} + \frac{t_{humain}}{60} r_{horaire}, C_{accepté} = \frac{C_{total}}{N_{acceptés}}.$$

Le temps est exprimé en minutes, le tarif en euros par heure et les coûts en euros. Si aucun résultat n'est accepté, le coût par résultat accepté est non calculable ; on rapporte le coût engagé et l'absence de réussite. Lorsque l'inférence est incluse dans un abonnement sans ventilation par essai, conserver cette limite ou déclarer la règle d'allocation choisie. Une consommation inconnue ne devient pas une dépense nulle.

Exemple de calcul, entièrement fictif. Un cas accepté sans agent mobilise 120 minutes humaines et 2 euros d'exécution. Au tarif supposé de 60 euros par heure, son coût vaut 122 euros. Avec un agent, le même cas accepté mobilise 90 minutes humaines, 12 euros d'inférence et 2 euros d'exécution : son coût vaut 104 euros. La réduction de coût vaut alors $(122 - 104)/122 \simeq 14,8\,\%$, tandis que la réduction du temps humain vaut 25 %. Ces valeurs illustrent la méthode ; elles ne sont ni une mesure d'AGILAB ou de Tokki, ni une prévision de gain. Des coûts de maintenance ultérieurs restent hors de ce calcul tant qu'ils n'ont pas été suivis.

Avant de conclure, examiner chaque refus et le travail nécessaire à sa reprise. Une condition qui produit davantage de propositions peut aussi demander plus de revue. Si la qualité finale diffère, présenter cette différence avec les temps et les coûts plutôt que d'annoncer une économie à qualité constante.

Ce qui ferait changer la conclusion

Un agent réduit le temps d'écriture, mais la revue et les reprises absorbent ce gain : le protocole doit pouvoir le montrer. Plusieurs agents trouvent des erreurs supplémentaires, mais leur coordination dépasse le budget : leur utilité dépend alors du résultat attendu et de son coût. À l'inverse, un gain qui se répète sur des cas réservés, à qualité comparable, fournit un motif d'adopter l'assistance dans ce périmètre. Conserver les essais défavorables et publier les limites permet de discuter cette décision.

Pour la condition à plusieurs agents, introduire volontairement une note erronée : « vérifier le premier et le dernier nœud suffit ». Faire d'abord établir le calcul de référence sans cette note, puis observer si elle entraîne une erreur commune. Ce

second essai étudie la propagation d'une consigne fausse ; il reste séparé de la comparaison de productivité. Le contre-exemple montre pourquoi multiplier les avis ne multiplie pas automatiquement les preuves.

Repères pour lire les études. L'expérience de Peng et ses coauteurs sur GitHub Copilot, publiée en 2023, porte sur la construction d'un serveur web en JavaScript. L'essai de METR publié en juillet 2025 porte sur des développeurs expérimentés travaillant dans leurs propres dépôts.

Leurs résultats concernent des tâches, des outils et des populations différents. Dans sa mise à jour de février 2026, METR décrit des biais de sélection et des difficultés de mesure du temps qui limitent l'interprétation du nouvel essai. La date, la référence, la sélection des tâches et le critère de réussite accompagnent donc toute conclusion. La synthèse de Vahe Aslanyan publiée sur freeCodeCamp le 12 octobre 2026 offre une lecture complémentaire sur la valeur de l'expertise ; elle reste une source secondaire, distincte des expériences citées.

5.7 Routage réseau et routage de requêtes d'IA

Le mot **routage** désigne dans les deux cas un choix sous contraintes, mais les objets orientés et les critères de réussite diffèrent. Cette comparaison porte sur des concepts publics. Elle ne décrit ni l'architecture interne de Tokki, ni ses règles de décision, ni une recette pour le reproduire.

Une abstraction commune, des réussites différentes

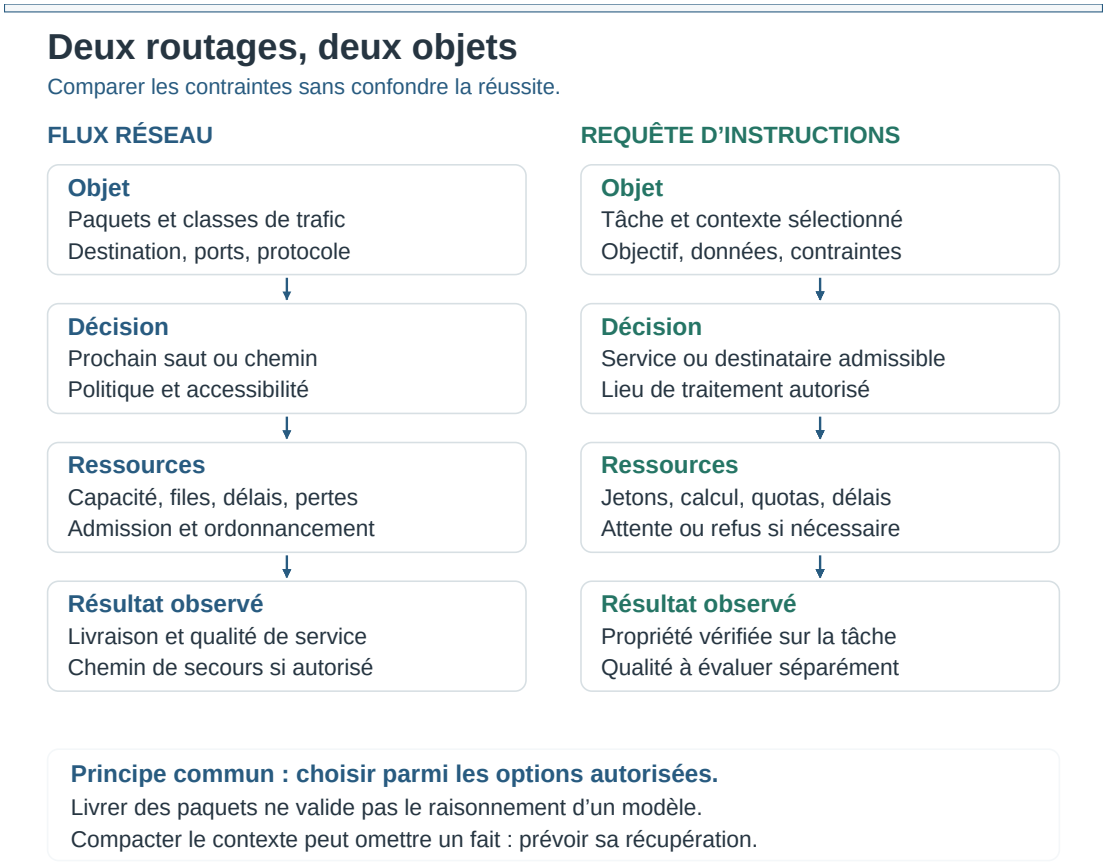
Pour comparer deux routages, commencer par quatre éléments : l'objet orienté, le but, les transitions possibles et les contraintes. Un **plan** décrit des étapes ; une **politique** dit comment choisir lorsque les observations changent. Une tâche d'IA peut avoir pour but une propriété vérifiée, alors qu'un paquet a une destination réseau : atteindre l'adresse ne démontre pas que la tâche est réussie.

Cette abstraction aide à transférer des questions, par exemple « que faire si aucune option n'est admissible ? », sans supposer qu'un algorithme se transpose tel quel. Le livre développe les graphes d'états, la composition des métriques et les compromis de Pareto dans l'approfondissement **« Le routage comme problème de décision »**, dans l'Idée VI du livre complet.

Des paquets à une tâche applicative

Dans un réseau IP, le plan de contrôle construit des routes ; le transfert utilise ensuite les informations disponibles pour choisir un prochain saut. Le traitement usuel se fait paquet par paquet, même si des mécanismes de répartition cherchent à maintenir les paquets d'un flux sur un même chemin. Un flux peut être décrit par un quintuplet d'adresses, de ports et de protocole. Le routeur IP ne comprend pas pour autant la demande applicative contenue dans ces paquets (Baker 1995).

Une **requête d'instructions** (*prompt*) exprime une tâche et peut contenir des données. Son routage applicatif consiste à orienter cette demande vers un service apte à la traiter. L'adresse du service ne suffit pas à établir son aptitude. La confidentialité, la capacité disponible, la nature du travail et les moyens d'évaluer le résultat sont des critères possibles. Le terme ne prescrit pas une méthode : une règle explicite, un classifieur ou une décision humaine peuvent intervenir selon le système considéré.



Routage réseau et routage de requêtes d'IA : comparaison fonctionnelle, sans correspondance avec l'architecture d'un produit.

Question	Flux réseau	Requête d'instructions
Qu'oriente-t-on ?	Des paquets appartenant à un trafic.	Une tâche et les informations nécessaires.
Entre quoi choisit-on ?	Prochains sauts, chemins ou tunnels.	Services ou destinataires admissibles.
Qu'observe-t-on ?	Accessibilité, capacité, délai, pertes.	Nature de tâche, disponibilité et qualité évaluée.
Que contraint-on ?	Politique, capacité, priorité, isolation.	Confidentialité, autorité, ressources et aptitude.
Que vérifie-t-on ?	Livraison et qualité de service observées.	Respect des critères de la tâche.

La classification du trafic et la qualité de service ajoutent une autre dimension : une classe peut recevoir un traitement particulier sans changer de route. Le routage, l'ordonnancement et le contrôle d'admission doivent donc rester distincts (Blake et al. 1998). Pour une requête d'IA, choisir un service, attendre une capacité disponible et autoriser une transmission sont également des questions différentes.

Un cas fictif : choisir un service autorisé

Un exercice propose trois services d'analyse de documents. Les temps sont des estimations synthétiques. Le document impose un traitement dans un périmètre autorisé et un délai maximal de 20 secondes. Les trois services sont supposés aptes à la tâche pour les besoins de l'exercice ; cette aptitude devrait être établie séparément dans une application réelle.

Service fictif	Périmètre autorisé	Délai estimé	Admissible dans l'exercice ?
A	Oui	12 s	Oui

Service fictif	Périmètre autorisé	Délai estimé	Admissible dans l’exercice ?
B	Non	4 s	Non : destination interdite.
C	Oui	30 s	Non : délai supérieur à la limite.

Le service A est le seul admissible. La rapidité de B ne compense pas l’interdiction. Si A devient indisponible, l’ensemble admissible est vide : on ne déduit pas de ce tableau une autorisation d’utiliser B. Il faut revoir la demande avec l’autorité compétente ou accepter qu’elle ne puisse pas être servie sous les contraintes actuelles. Ce raisonnement ne dépend d’aucune architecture particulière et ne décrit pas une politique de Tokki.

Ce que l’analogie ne permet pas de conclure

Un modèle transforme l’information et peut produire une erreur sémantique ; un routeur IP transfère une charge utile sans avoir à en valider le raisonnement. Une réponse rapide et syntaxiquement correcte peut être fausse. Les délais d’un réseau et la difficulté d’une tâche ne s’additionnent pas selon une même loi. Une requête n’est donc pas simplement un paquet plus volumineux, et « routage de prompts » ne signifie pas nécessairement calcul de plus court chemin.

L’analogie avec la QoS éclaire les ressources et les priorités ; elle ne transforme pas la qualité d’une réponse en débit ou en latence. Il faut définir un critère adapté à la tâche : résultat numérique exact, propriété testée, source vérifiable ou jugement humain explicite. Une seconde réponse ne constitue pas automatiquement une vérification indépendante de la première.

Enfin, un contexte plus court peut omettre un fait décisif. Pour comparer des volumes, notons T la taille initiale et C la taille sélectionnée : la réduction relative est $(T - C)/T$, pour $T > 0$. Passer de 12 000 à 3 000 jetons représente une réduction de 75 %. Cela ne fournit ni une méthode de sélection, ni une preuve de qualité, ni une économie facturée. Les mesures doivent employer une convention de comptage explicite et porter sur les traitements réellement effectués.

Exercice. Ramener le délai estimé de C à 18 secondes. Quels services deviennent admissibles ? Les données suffisent-elles pour désigner le meilleur ? Puis expliquer pourquoi 75 % de texte en moins ne signifie pas 75 % de qualité en plus.

Repères de correction. A et C deviennent admissibles ; B reste exclu. Il manque un objectif de choix : délai minimal, coût, qualité mesurée ou autre critère justifié. Réduire le volume ne démontre pas que les faits utiles sont conservés. La note `routage_requetes_ia_sources_fr.md` rassemble uniquement les références publiques de cette comparaison et des compléments généraux.

5.8 Faire travailler plusieurs agents comme une équipe

Ici, un **agent d’IA** utilise un modèle de langage et des outils pour accomplir une tâche. En apprentissage par renforcement, le terme est plus général : l’agent choisit des actions selon une politique, sans nécessairement employer un modèle de langage. Plusieurs agents peuvent former une petite équipe : une personne fixe le but, un agent principal répartit le travail et des agents spécialisés prennent en charge certaines étapes.

Imaginons que l’on veuille comparer deux façons de faire circuler les données dans un réseau. Un agent lit les documents et relève les contraintes. Un autre prépare le programme d’essai. Un troisième examine les résultats et rejoue les contrôles. L’agent principal réunit leurs contributions et signale ce qui reste incertain. La personne responsable décide si le résultat répond au besoin.

Du modèle abstrait à une réalisation concrète

Le mot **modèle** peut désigner plusieurs choses. Un **modèle abstrait** est une représentation simplifiée qui aide à raisonner. Ici, le plan « une personne fixe le but, un coordinateur répartit le travail, des spécialistes répondent » décrit une organisation. Il ne dit pas encore quels logiciels seront installés ni si le résultat sera bon.

Une **réalisation concrète** précise les choix : les modèles entraînés et leur version, les documents accessibles, les outils, les règles de délégation et l'ordinateur utilisé. Un modèle d'IA entraîné est lui-même un objet concret, avec une architecture et des poids enregistrés. Mais il ne constitue qu'une partie du système qui rend le service.

Étape	Dans notre exemple	Ce qu'elle permet d'affirmer
Décrire	Dessiner l'équipe et le travail de chaque agent	L'organisation est définie
Réaliser	Relier les modèles, les documents et les outils sur une machine	Un prototype peut être exécuté
Vérifier	Lui soumettre des cas et examiner ses résultats	Certaines capacités sont établies dans les conditions testées

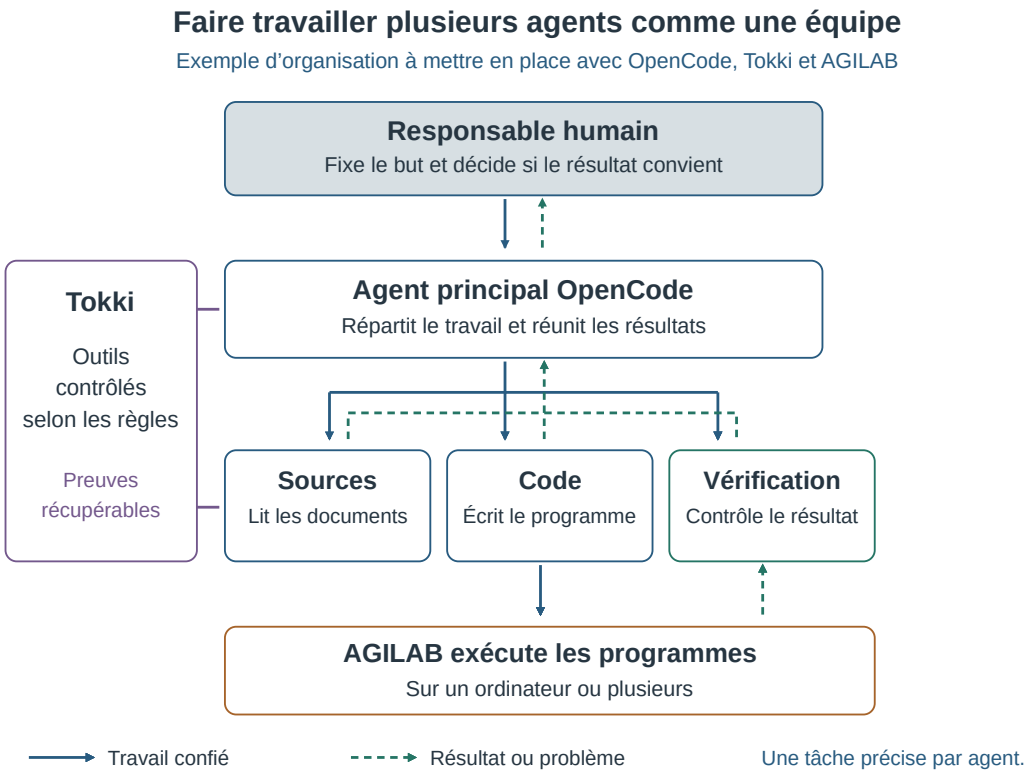
Distinguer **le plan, le programme qui tourne et les résultats qui montrent ce qu'il sait faire**. Un modèle généraliste est aussi concret qu'un petit spécialiste : la distinction abstrait/concret ne dit rien de sa taille ni de ses connaissances. Une simulation reste une représentation ; la faire fonctionner ne prouve pas le fonctionnement du réseau réel.

L'abstraction intervient aussi dans le raisonnement : « vérifier qu'un chemin existe avant d'accuser le serveur » est un principe général ; examiner la table de routage d'un équipement précis, son application concrète. Notre petit spécialiste doit relier les deux dans des situations nouvelles. Son évaluation inclut donc ces situations, au-delà des exemples mémorisés.

Trois outils, trois rôles

Outil	Son rôle	Dans cet exemple
OpenCode	Faire travailler un agent principal et des sous-agents spécialisés	Confier la recherche documentaire, le code et la vérification à des agents différents
Tokki	Encadrer l'usage des outils selon les règles et conserver les preuves accessibles	Contrôler une inspection autorisée et fournir les éléments utiles avec accès aux détails
AGILAB	Exécuter les programmes d'une application, sur un ou plusieurs ordinateurs	Lancer les essais et recueillir leurs résultats

Les documentations publiques présentent les trois projets (OpenCode 2026; Tokki 2026; ThalesGroup 2026). Le rôle de contrôle de Tokki renvoie au projet de l'auteur expliqué dans l'avant-propos ; sa portée dépend des règles installées. L'organisation décrite ici est un exemple à mettre en place : installer les trois outils ne crée pas automatiquement cette équipe.



Une personne fixe le but, les agents se partagent le travail et AGILAB exécute les programmes ; Tokki encadre l'usage des outils et conserve des preuves récupérables.

Il faut donner à chaque agent une tâche précise, les informations nécessaires, les outils qu'il peut utiliser et une limite de temps ou de coût. Il doit rendre un résultat que l'on peut vérifier, ou expliquer ce qui l'a empêché d'aboutir. Un agent chargé de lire des documents n'a pas besoin des mêmes droits qu'un agent autorisé à modifier des fichiers.

Partager un travail complexe expose aussi aux limites d'une équipe humaine : consignes floues, doublons, désaccords et coordination. Plusieurs agents peuvent répéter la même erreur : conserver tests, sources et décision humaine pour les choix qui lui sont réservés. Un seul agent peut suffire aux petites tâches.

Le TP sur le gain réel d'une assistance IA compare ces organisations avec le même contrat de qualité. Il propose aussi un essai distinct où une consigne fausse peut contaminer plusieurs agents.

5.9 Petits modèles locaux : faire aussi bien sur une tâche précise

L'objectif est qu'un petit modèle local atteigne la qualité d'un grand modèle sur une tâche bien délimitée. Par exemple, un assistant consacré aux configurations d'une famille d'équipements réseau : expliquer une commande, repérer une incohérence et proposer une correction que l'on peut tester. On le juge sur ces tâches, sans lui demander de maîtriser tous les sujets.

On peut préparer ce modèle avec des exemples du domaine et lui fournir les documents utiles. La **distillation** offre une autre piste : entraîner un modèle plus simple à reproduire une partie du comportement d'un modèle ou d'un ensemble de modèles plus coûteux (Hinton et al. 2015). Ces méthodes aident à construire une spécialisation ; elles ne démontrent pas d'avance que le petit modèle égale le grand.

Pour l'établir, on leur soumet des cas représentatifs qu'ils n'ont pas vus pendant leur préparation, avec les mêmes informations et les mêmes outils. On compare la justesse des réponses, les erreurs graves, le délai et les ressources nécessaires. Hors de son domaine, l'assistant doit signaler sa limite et demander de l'aide, au lieu de produire une réponse assurée.

Autonome sur son poste, ou *standalone*, signifie ici que le modèle, les documents et les outils nécessaires fonctionnent sans service distant. Une connexion n'est alors pas indispensable pour chaque utilisation. Le matériel et la maintenance

ont toujours un coût, mais cette organisation peut limiter la dépendance à un abonnement et faciliter l'usage dans un atelier, une école ou un site mal connecté.

Dans une équipe d'agents, chaque rôle pourrait ainsi utiliser un petit modèle adapté à son travail. Le modèle chargé des configurations réseau ne serait pas nécessairement celui chargé de traduire ou de rédiger. L'enjeu est de choisir un spécialiste qui réussit la tâche, puis de vérifier son résultat.

Apple Silicon : la mémoire unifiée et les LLM locaux

Un processeur peut passer plus de temps à attendre des données qu'à calculer. Il faut distinguer la **capacité mémoire**, la quantité de données que l'on peut garder, de la **bande passante mémoire**, la quantité que l'on peut lire ou écrire par seconde.

Avec M1, présenté en 2020, Apple a réuni processeur central (CPU) et processeur graphique (GPU) autour d'une **mémoire unifiée** : accéder aux mêmes données évite des copies entre mémoires séparées lorsque le logiciel exploite cette organisation (Apple 2020). MLX, son framework de calcul numérique, l'utilise pour les opérations CPU et GPU (Apple 2025). Le partage exige toujours de coordonner les accès et gérer les données.

Les poids d'un LLM local peuvent utiliser cette mémoire commune au lieu de tenir dans la seule mémoire dédiée d'une carte graphique. Elle doit aussi accueillir système, résultats intermédiaires et **cache d'attention** (*KV cache*). K signifie *keys*, les clés, et V *values*, les valeurs. Pendant la génération, ce cache garde celles des positions précédentes pour éviter de les recalculer à chaque nouveau jeton. Le calcul d'attention du nouveau jeton reste à faire (Hugging Face 2025a). MLX-LM fournit des outils d'exécution et de gestion de ces caches (Apple Machine Learning Research 2026).

Combien de mémoire prend ce cache ? Pour une seule séquence et des couches d'attention de même taille, qui conservent toutes les positions, on peut compter les octets ainsi :

$$M_{KV} = 2\,L\,T\,n_{KV}\,d\,b\,.$$

Le 2 compte K et V ; L est le nombre de couches ; T, le nombre de jetons conservés, ceux de la demande puis ceux de la réponse ; n_{KV}, le nombre de têtes K/V par couche ; d, le nombre de valeurs dans chaque tête et b, le nombre d'octets par valeur. Certaines architectures partagent les clés et valeurs entre plusieurs têtes de requête : il faut compter les têtes K/V, pas les têtes de requête (Ainslie et al. 2023).

Prenons un modèle fictif : 32 couches, 8 têtes K/V, 128 valeurs par tête, 8 192 jetons conservés et 2 octets par valeur. Son cache occupe $2 \times 32 \times 8\,192 \times 8 \times 128 \times 2 = 1\,073\,741\,824$ octets, soit **1 Gio**. Doubler le contexte double ici le cache.

Si ce modèle possède huit milliards de poids codés sur 4 bits, leurs seuls codes occupent $8 \times 10^9 \times 4/8 = 4 \times 10^9$ octets, soit **4 Go**. Les poids et le cache totalisent déjà environ **5,07 Go**, avant les paramètres de quantification, les résultats intermédiaires et le système. Un Go vaut un milliard d'octets ; un Gio vaut 2^{30} octets.

Des poids en 4 bits n'impliquent pas un cache en 4 bits. La quantification du cache est un choix distinct. Plusieurs séquences simultanées augmentent aussi la mémoire nécessaire ; à l'inverse, une attention à fenêtre limitée ne conserve pas tout le passé. La formule décrit donc cet exemple, pas toutes les architectures (Hugging Face 2025b).

Faire tenir le modèle et produire rapidement ses réponses sont deux problèmes différents. La génération successive des jetons peut être limitée par la lecture des poids et du cache. La quantification réduit la place occupée en représentant des valeurs avec moins de bits ; son effet sur la qualité et la vitesse doit être mesuré. Ajouter de la mémoire ne suffit donc pas si le calcul attend toujours les données.

Rapprocher mémoire et calcul : l'exemple de Huawei

La distance intervient, mais **le débit n'est pas universellement inversement proportionnel à la distance**. Le délai de propagation augmente avec la longueur du trajet. Le débit dépend aussi du support, de la largeur des interfaces, du signal, des protocoles et des accès concurrents. Sur un réseau, un aller-retour plus long peut réduire le débit TCP si trop

peu de données peuvent rester en vol ; ce plafond dépend aussi des fenêtres du protocole et du goulot d'étranglement (Constantine et al. 2011). Rapprocher la mémoire du calcul peut faciliter des connexions larges et réduire le coût des échanges ; la distance seule ne fixe pas le débit.

Dans sa présentation du 18 septembre 2025, Huawei annonce pour Ascend 950 de la **HBM** (*High Bandwidth Memory*, mémoire à haute bande passante) assemblée avec le circuit de calcul dans son boîtier. Les mémoires HiBL et HiZQ répondent à des besoins différents selon les phases d'inférence : traiter le texte d'entrée (*prefill*), puis générer les jetons (*decode*). Cette annonce rapproche mémoire et calcul ; elle ne signifie pas que toute la mémoire est gravée dans le même morceau de silicium que le processeur (Xu 2025).

La leçon dépasse le choix d'un fabricant : **accélérer l'inférence suppose d'organiser ensemble calcul, mémoire et transferts**. Les annonces de produits illustrent ce choix d'architecture ; les gains restent à vérifier sur le modèle, le contexte et la charge réels.

AirLLM : utiliser un grand modèle avec peu de mémoire GPU

Un grand modèle contient beaucoup de **poids**, c'est-à-dire les valeurs numériques apprises pendant son entraînement. Les garder tous dans la mémoire d'une carte graphique peut demander un matériel coûteux.

L'idée d'AirLLM est de **charger les poids couche par couche**, au moment où ils sont nécessaires (Li 2023). L'image d'un gros livre sur un petit bureau aide à comprendre : on garde le livre dans une bibliothèque et l'on apporte seulement le chapitre que l'on est en train de lire. On peut ainsi travailler sans que tout le livre tienne sur le bureau.

Pour le modèle, on charge une couche, on effectue son calcul, on conserve le résultat utile, puis on libère ses poids avant de poursuivre. D'autres outils, comme Accelerate, proposent aussi de déplacer des poids entre le disque, la mémoire de l'ordinateur et le GPU (Hugging Face 2026a). Ce principe concerne l'organisation de la mémoire ; il ne rend pas le modèle plus petit.

Le prix à payer est le temps de transfert. Aller chercher chaque chapitre prend du temps. De même, déplacer les poids peut ralentir fortement la production d'une réponse, surtout si ces déplacements doivent être répétés pour chaque nouveau jeton. Il faut aussi de la place pour les résultats intermédiaires et les informations conservées pendant le calcul. La quantité de mémoire nécessaire ne se résume donc pas à la taille d'une couche.

AirLLM propose également des options de compression et de chargement anticipé pour agir sur ces coûts (Li 2023). Leur intérêt doit être mesuré sur le modèle et la machine choisis. **Faire fonctionner un modèle avec moins de mémoire ne signifie pas obtenir une réponse plus vite.**

Dans notre équipe, cette approche pourrait servir à une analyse approfondie que l'on accepte d'attendre. Un modèle plus réactif pourrait assurer les échanges courants. Il faudrait vérifier que les logiciels communiquent correctement, puis mesurer le délai et les ressources consommées. Aucune connexion automatique entre AirLLM, OpenCode et AGILAB n'est supposée ici.

MoE et AirLLM : choisir les calculs ou déplacer les poids

Le **MoE**, pour *Mixture of Experts* ou **mélange d'experts**, organise le calcul à l'intérieur d'un modèle. Certaines couches contiennent plusieurs blocs appelés experts. Un mécanisme de sélection, le routeur, choisit lesquels utiliser pour chaque jeton, c'est-à-dire chaque morceau de texte traité. Dans Mixtral 8x7B, par exemple, deux experts parmi huit sont sélectionnés à chaque couche concernée. Leurs sorties sont combinées avec des poids donnés par le routeur (Jiang et al. 2024). Pour une composante du vecteur de sortie, si les experts produisent 10 et 20 avec des poids de 0,7 et 0,3, on obtient $0,7 \times 10 + 0,3 \times 20 = 13$. C'est une moyenne pondérée, comme dans l'attention, mais elle porte ici sur les sorties d'experts plutôt que sur les valeurs associées aux positions du texte.

Ces experts sont des parties du réseau neuronal. Ce ne sont ni huit modèles de langage complets ni nécessairement des spécialistes nommés « réseau », « médecine » ou « droit ». Choisir entre plusieurs modèles complets ou entre des agents reste une organisation différente, même si l'idée de sélection des spécialistes est comparable.

MoE choisit quoi calculer ; AirLLM choisit quoi charger en mémoire. Activer deux experts sur huit ne permet pas de conclure à un temps total divisé par quatre : l'attention et les autres opérations restent à exécuter. Les poids des experts doivent toujours être stockés et rendus disponibles. Un modèle MoE peut donc rester volumineux. AirLLM, lui, cherche à limiter les poids présents sur le GPU en les chargeant au fur et à mesure. Cela peut coûter du temps de transfert.

Choix	Idée principale	Point à vérifier
Petit modèle spécialisé	Viser une très bonne qualité sur une tâche précise	Réussit-il aussi bien que le grand sur cette tâche ?
MoE	N'activer qu'une partie des experts internes	Quel calcul et quelle mémoire faut-il réellement ?
AirLLM	Charger les poids lorsqu'ils sont nécessaires	Le délai de réponse reste-t-il acceptable ?

Ces choix peuvent se combiner si l'implémentation le permet. Un petit modèle local peut être dense, c'est-à-dire sans sélection d'experts, ou utiliser un MoE. Un grand modèle MoE peut employer un chargement progressif. Pour une IA accessible au quotidien, le point de départ reste le besoin : **la qualité attendue, sur une tâche précise, avec les moyens disponibles**.

On sépare ainsi quatre choix : **le domaine** du modèle, **son architecture** (dense ou MoE), **sa façon d'utiliser la mémoire** et **l'organisation du travail entre agents**. Le plan abstrait décrit ces choix ; la réalisation concrète permet de les tester ensemble.

Aller plus loin : construire un assistant réseau local

Pour transformer cette idée en expérience, commençons par un domaine limité : une famille d'équipements, une version connue et quelques opérations courantes. L'ordinateur conserve les modèles, la documentation et les outils nécessaires. L'utilisateur peut choisir le type de demande dans un menu ; une règle simple peut suffire à désigner le spécialiste. Il n'est pas obligatoire de faire appel à un grand modèle pour organiser chaque échange.

Travail demandé	Spécialiste envisagé	Moyen de vérifier
Expliquer une alarme	Petit modèle aidé par la documentation locale	Passage cité, version du document et correspondance avec l'alarme
Proposer une correction	Petit modèle préparé aux configurations visées	Analyse du fichier, règles connues et essai isolé
Comparer deux solutions	Agent qui prépare un programme d'essai	Exécutions AGILAB sur les mêmes données et examen des résultats

Confier les calculs exacts et les contrôles connus aux programmes ordinaires ; le modèle aide à comprendre et préparer. Tokki encadre les opérations et conserve leurs preuves dans un contexte compact, OpenCode porte les agents et AGILAB exécute les programmes. Leur intégration reste à tester : ce tableau propose une expérience, pas un système livré.

Protocole pédagogique. Comparer d'abord trois solutions sur les mêmes cas : des règles seules, le petit spécialiste avec ses documents et outils, puis un grand modèle disposant des mêmes documents et outils. Les cas d'évaluation doivent rester séparés des exemples utilisés pour préparer le spécialiste. Inclure des demandes normales, des erreurs et des situations hors de son domaine. Fixer avant l'essai la qualité attendue, les erreurs inacceptables et le temps de réponse maximal.

Noter les réponses correctes, les erreurs, les demandes d'aide justifiées, le temps de réponse, la mémoire et le coût complet. Vérifier séparément que le fonctionnement local ne dépend d'aucun service distant. Si le petit modèle atteint le niveau du grand, la conclusion porte sur ce domaine et ces conditions. S'il échoue, rechercher si le problème vient du modèle, des documents, du choix du spécialiste ou de l'outil de vérification.

Cette démarche permettrait de constituer peu à peu une bibliothèque de spécialistes locaux, chacun accompagné de son domaine d'emploi, de ses tests et de ses limites. Leur assemblage doit aussi être évalué : de bons résultats individuels ne garantissent pas que la bonne demande arrive au bon spécialiste.

Exercice. L'ordinateur manque de mémoire GPU et l'utilisateur veut une réponse en deux secondes. AirLLM suffit-il à résoudre le problème ? Un MoE le garantit-il ?

Repères de correction. AirLLM peut aider à faire tenir les poids, mais les transferts peuvent dépasser le délai. MoE sélectionne des calculs, sans garantir à lui seul la mémoire ou la vitesse. Il faut mesurer le système complet ; un petit modèle spécialisé ou des règles peuvent être plus adaptés.

5.10 Une IA qui donne sa chance à chacun

Mon passage du **bac F2** à la **prépa TA**, une voie ouverte aux bacheliers techniciens (Ministère de l'Éducation nationale 1985), nourrit mon attachement à l'égalité des chances. J'en raconte les rencontres et les obstacles dans mon parcours de formation.

Je souhaite que l'IA aide davantage de personnes à apprendre, créer et travailler, même avec peu d'argent ou de matériel : un étudiant face à un sujet difficile, une petite entreprise qui souhaite essayer une idée, une association qui construit un outil utile. L'objectif est d'ouvrir ces possibilités au-delà des grandes entreprises et des pays les plus riches.

Des écarts sociaux qui se creusent dans certaines évaluations. Une baisse du niveau moyen ne démontre pas, à elle seule, un recul de l'égalité des chances : il faut comparer les résultats selon le milieu social et leur évolution. Voici les données disponibles au 30 septembre 2026, avec leurs années d'observation :

Indicateur	Dates et valeurs	Ce que l'on peut en conclure
Sciences en troisième, Cedre ; écart entre les contextes scolaires les plus et les moins favorisés	30 points en 2018 → 34 en 2024 ; publication en avril 2026	Léger creusement de l'écart social observé (Blanche et al. 2026)
Histoire-géographie et enseignement moral et civique au CM2, Cedre ; même comparaison	38 points en 2017 → 43 en 2023 ; publication en février 2025	Creusement de 5 points, avec une baisse plus forte dans les écoles les moins favorisées (Conceiçao et al. 2025)
Sciences à 15 ans, PISA ; écart entre les quarts d'élèves les plus et les moins favorisés	100 points en France en 2025, contre 85 dans l'OCDE ; publication en septembre 2026	Inégalité persistante, mais écart réduit depuis 2015 : baisse des favorisés et stabilité des défavorisés (Organisation de coopération et de développement économiques (OCDE) 2026)

Cedre regroupe les établissements selon leur indice de position sociale moyen ; PISA classe les élèves selon leur situation économique, sociale et culturelle. Les deux premières lignes concernent la France hors Mayotte, dans le public et le privé sous contrat. Ces échelles sont différentes : leurs points ne s'additionnent pas. La DEPP précise aussi qu'une évolution de la mixité sociale entre collèges peut modifier l'écart observé en sciences. Ces résultats documentent donc des aggravations précises, sans établir un recul uniforme dans toutes les disciplines ni en isoler une cause unique.

L'accès aux filières sélectives reste une autre préoccupation. À la rentrée 2025-2026, 50,0 % des inscrits en CPGE viennent de familles de cadres et professions intellectuelles supérieures, contre 6,4 % de familles ouvrières (Sous-direction des systèmes d'information et des études statistiques (SIES) 2026). Cette composition ne donne ni la probabilité d'admission de chaque milieu, ni son évolution : il faut aussi connaître les candidats et leurs conditions de préparation.

Le cas de Polytechnique. Selon la Cour des comptes, les enfants de cadres et professions intellectuelles supérieures représentent **73 % des admis français au concours 2019 du cycle ingénieur** (Cour des comptes 2020, 459). Un autre indicateur, plus récent, permet de suivre l'ouverture sociale de la voie CPGE française (École polytechnique 2025, 33) :

Part de boursiers parmi…	2016	2025
les inscrits au concours	30,8 %	32,8 %
les entrants à l’École	13,3 %	10,5 %

La représentation des boursiers augmente parmi les candidats, mais diminue de 2,8 points parmi les entrants, avec des fluctuations entre ces dates. Ce sont des compositions de groupes, pas des taux individuels de réussite. Le statut de boursier dépend des ressources et des règles d’attribution ; il ne se confond pas avec la profession des parents.

Les acquis français dans la comparaison internationale. PISA évalue les élèves de 15 ans scolarisés dans chaque pays, quelle que soit leur nationalité. Ses scores ne sont pas des notes sur 20. Les évolutions publiées par la DEPP donnent un recul français plus marqué en valeur brute entre 2022 et 2025 :

Domaine PISA	France, 2022 → 2025	Variation France	Variation moyenne OCDE
Mathématiques	474 → 458 points	−16 points	−9 points
Compréhension de l’écrit	474 → 456 points	−18 points	−14 points

En 2025, les moyennes de l’OCDE sont respectivement de 463 et 461 points. Malgré les écarts numériques, la DEPP situe la France dans la moyenne de l’OCDE compte tenu de l’incertitude statistique. Les variations OCDE portent sur les pays présents aux deux enquêtes : elles ne se déduisent pas en soustrayant des moyennes de périmètres différents (Fernandez et al. 2026).

Le retard et la baisse restent deux constats distincts. En mathématiques au CM1, TIMSS 2023 donne 484 points à la France contre 524 pour les 22 pays de l’Union européenne participants. Mais le score français ne diffère pas significativement des 485 points de 2019 (Ministère de l’Éducation nationale 2024). Ces mesures étayent une inquiétude sur les acquis et l’accès aux études ; elles ne constituent pas une mesure globale du « déclin de la France ».

Et les notes françaises et étrangères au concours de l’X ? Les deux derniers livrets consultés donnent, pour l’écrit de mathématiques A en MP :

Session et population publiée	Moyenne des candidats français	Moyenne des candidats internationaux
2024, banque X–ENS–ESPCI	8,59/20 ; 1 768 copies	6,77/20 ; 626 copies
2025, candidats X	9,15/20 ; 1 462 copies	7,94/20 ; 463 copies

Sources : livrets officiels, p. 17 de chaque édition (École polytechnique 2024, 2025). Dans cette épreuve, la moyenne française est supérieure dans les deux éditions. Le périmètre publié change entre les éditions ; les sujets changent aussi. La différence entre les deux lignes ne mesure donc pas une progression à difficulté constante. Enfin, la nationalité ne désigne pas le pays de scolarisation. Ce concours sélectionne des candidats déjà très particuliers : ses notes ne peuvent remplacer les enquêtes représentatives pour comparer les systèmes éducatifs.

L’enjeu pour une IA éducative devient concret : aider en priorité ceux qui disposent de moins de ressources, puis vérifier leur progression sur de nouveaux exercices. Une moyenne en hausse ne suffirait pas si les écarts d’apprentissage continuaient à se creuser.

Éviter les échanges inutiles, partager le travail et utiliser une machine modeste peuvent réduire les coûts. L’accès au logiciel exige aussi apprentissage, vérification, explications dans sa langue et usage possible malgré handicap ou connexion limitée. Compter matériel, abonnement, énergie, installation et aide nécessaire donne le coût réel.

Garder une **liberté de choix** suppose de protéger ses données, changer de fournisseur et travailler avec d’autres. Partager connaissances et outils compatibles peut construire l’indépendance technologique. L’ambition est que cette indépendance s’appuie sur un savoir accessible et que l’IA élargisse les chances.

Les recommandations de l'UNESCO et les principes de l'OCDE donnent des repères : inclusion, éducation et partage des bénéfices (UNESCO 2021; OECD 2026). Ils n'établissent pas que les outils cités ont atteint ce but. Il faut vérifier qui peut les utiliser, à quel coût et avec quel progrès dans sa capacité à apprendre ou à agir.

Les sources et les limites des exemples figurent dans `agents_hierarchie_airllm_6g_sources_fr.md`.

Reprendre le fil de l'exploitation. Les dossiers précédents ont étudié comment préparer un raisonnement et élargir l'accès aux outils. Nous revenons maintenant aux opérations sur le réseau : proposer une configuration, organiser les ressources et observer les changements.

5.11 Agents génératifs pour la configuration

Traduire une intention en configuration, expliquer les différences, préparer une procédure, rechercher une incohérence, générer un test — **sans appliquer directement du texte plausible à l'infrastructure** :

```
intention -> proposition structurée -> validation de schéma
          -> analyse statique -> simulation/jumeau -> déploiement témoin
          -> observation -> promotion ou retour arrière
```

Sources, hypothèses et diffs restent visibles. **Les secrets ne sont pas placés dans le contexte du modèle.** Outils autorisés, portée, durée et nombre de changements sont bornés.

5.12 Orchestration énergétique

Mettre en veille des capacités, déplacer du calcul vers un site mieux alimenté, adapter puissance radio et couverture, planifier selon énergie disponible et intensité carbone. L'économie se mesure sur le système complet : réveils, migrations, réchauffement de caches, perte de redondance, usure des batteries peuvent annuler le gain local. Couverture, disponibilité et sûreté sont des **bornes**, pas des pénalités négociables.

5.13 Estimation de topologie et qualité de télémétrie

Les décisions apprises supposent souvent un graphe déjà connu. Dans un réseau mobile ou partiellement observé, une fonction distincte doit qualifier ce que le décideur sait réellement : voisin reçu, lien connu actif, lien connu inactif, état périmé ou lien inconnu. Inférer une arête probable peut guider une sonde ou une recommandation ; cela ne crée pas une liaison utilisable.

Le contrat `observed_partial` du Routing Experiment Lab illustre cette limite : la décision est liée à l'identité d'un nœud, à une preuve de topologie reçue, à sa provenance et à un âge maximal. Une précision moyenne de reconstruction ne suffit pas ; mesurer faux liens, liens manqués, âge, asymétrie, partition et effet sur les routes et services critiques.

Dans l'implémentation actuelle, cette vue reçue est **modélisée puis validée** ; le laboratoire ne reconstruit pas la topologie avec un modèle appris. Une telle reconstruction constituerait un producteur distinct, avec ses propres données, erreurs, provenance et critères d'abstention.

Le banc `telemetry_evidence_lab/run_public_network_lab.py` complète désormais ce contrat par trois extraits publics dont les niveaux conceptuels restent volontairement séparés :

Profil	Niveau observé	Preuve bornée
fixe, CESNET-TimeSeries24	trafic d'infrastructure	48 intervalles horaires et 2 395 flux ; continuité des compteurs, pas nature du médium
mobile avec infrastructure, ENE5AI	qualité d'un lien d'accès 5G	180 observations de six liens `cell` ; RSSI observé et code BER `99` traité comme sentinelle inconnue
ad hoc/MANET, ENE5AI	état d'une topologie distribuée	114 observations de quatre liens dirigés, 31 transitions, six absences attendues et un état invalide

CESNET-TimeSeries24 est épinglé par le DOI `10.5281/zenodo.13382427` sous CC BY 4.0 ; ENE5AI par le DOI `10.57745/DFMCYS` sous licence Etalab 2.0. Le manifeste conserve versions, tailles et empreintes des archives ainsi que la règle déterministe d'extraction. Les extraits portent `synthetic: false` mais aussi `production_evidence: false` : ils permettent de tester la détection d'un changement ou d'une absence sur les liens retenus, pas d'affirmer une partition de tout le réseau ni de qualifier un déploiement opérationnel.

5.14 Risque de changement et assurance de configuration

Historique des incidents, différences structurées, dépendances, topologie, politiques et état courant peuvent servir à estimer le risque d'un changement. La sortie utile est une décision bornée : observer sans appliquer, demander un test ciblé, réduire le périmètre témoin ou refuser. Un score élevé ne remplace pas la validation de schéma, l'analyse statique, le jumeau ni le retour arrière.

Séparer la **génération** du changement et son **assurance** : un même modèle ne doit pas être l'unique auteur, juge et preuve de ses propres modifications. Tester changements inédits, dépendances absentes de l'historique, succès trompeurs, retour arrière partiel et dérive entre configuration désirée et réellement appliquée.

---

6. Média et représentation

6.1 Génération et restitution IA des médias (§ 7 ter)

Où exécuter l'IA : en périphérie (réduit parfois le trafic et protège les données brutes, mais consomme énergie et mémoire), dans le cloud (mutualise accélérateurs et modèles, mais ajoute transport, latence, dépendance et exposition), ou en hybride (représentation compacte transmise, restitution locale) à condition de versionner le modèle et de définir le comportement lorsqu'il manque.

Pour un contenu opérationnel ou probatoire : conserver l'original, signaler toute transformation générative, tracer modèle/version/paramètres, attacher une provenance vérifiable. **Une commande critique ne doit jamais être remplacée silencieusement par une phrase « probable » générée pour masquer une perte.**

6.2 Communications sémantiques

Transmettre ce qui sert une tâche : objets et trajectoires plutôt qu'image complète, ou représentation latente comprise par un modèle distant. **Le gain de débit déplace le contrat vers les modèles** : des versions divergentes peuvent altérer le sens, et la compression omettre un détail jugé inutile à l'entraînement mais critique dans une situation nouvelle. Définir fidélité, incertitude, accès à la source et repli interopérable.

---

7. Apprentissage distribué et confidentialité

7.1 Apprentissage fédéré

Deux sites peuvent apprendre ensemble en gardant leurs exemples sur place. Dans **FedAvg** (*Federated Averaging*, moyenne fédérée), l'agrégateur envoie un même modèle aux participants d'un cycle. Chacun ajuste ses paramètres sur ses données, puis renvoie une mise à jour. L'agrégateur combine ces résultats et redistribue le modèle obtenu (McMahan et al. 2017).

Pour deux participants A et B, une moyenne pondérée par leurs nombres d'exemples n_A et n_B, tous deux positifs, s'écrit :

$$w_{nouveau} = \frac{n_A w_A + n_B w_B}{n_A + n_B}.$$

w_A et w_B désignent les paramètres après les ajustements locaux. Les mêmes positions doivent représenter les mêmes paramètres ; les modèles partent ici d'une version commune. La formule s'applique à chaque coordonnée d'un vecteur de poids.

Un calcul à la main. Pour un seul paramètre, A obtient $w_A = 2$ avec 100 exemples et B obtient $w_B = 6$ avec 300 exemples. Le résultat vaut $(100 \times 2 + 300 \times 6)/400 = 5$, contre 4 pour une moyenne donnant le même poids à chaque site. Le choix des pondérations exprime l'objectif poursuivi. Le résultat 5 est un paramètre à évaluer, pas un score de qualité ni la garantie d'un meilleur modèle.

Les difficultés restent à traiter. Des données **non-IID** ne sont pas indépendantes et identiquement distribuées : les sites peuvent observer des populations différentes ou des exemples corrélés. Plusieurs ajustements locaux peuvent éloigner leurs modèles ; la moyenne ne corrige pas à elle seule ce désaccord. Il faut aussi prévoir les clients absents, les mises à jour empoisonnées, la reprise et le coût des échanges.

Garder les données sur place ne garantit pas la confidentialité. Gradients et paramètres peuvent révéler une partie de leur information. L'agrégation sécurisée et la confidentialité différentielle apportent des protections différentes, développées dans l'apprentissage fédéré et la confidentialité.

7.2 Calcul sur données chiffrées

Le chiffrement homomorphe permet à un calculateur de manipuler des chiffrés sans recevoir la clé privée de déchiffrement. Pour les réseaux, on peut étudier une somme de compteurs ou une inférence sur des caractéristiques chiffrées. Paillier illustre l'addition ; BFV/BGV traitent de l'arithmétique entière modulaire et CKKS des calculs approchés. Un modèle comportant des comparaisons ou des fonctions non linéaires demande une adaptation explicite.

Cette technique peut participer à une agrégation fédérée, mais elle ne valide ni les mises à jour ni leur innocuité et ne remplace pas la politique de publication des résultats. Les clés, la précision et le coût de bout en bout font partie du protocole. Le chapitre sur la confiance développe le mécanisme, les limites et le TP Paillier L28 ; les sources sont réunies dans la bibliographie sur le calcul chiffré.

---

8. Direction inverse — le réseau *pour* l'IA

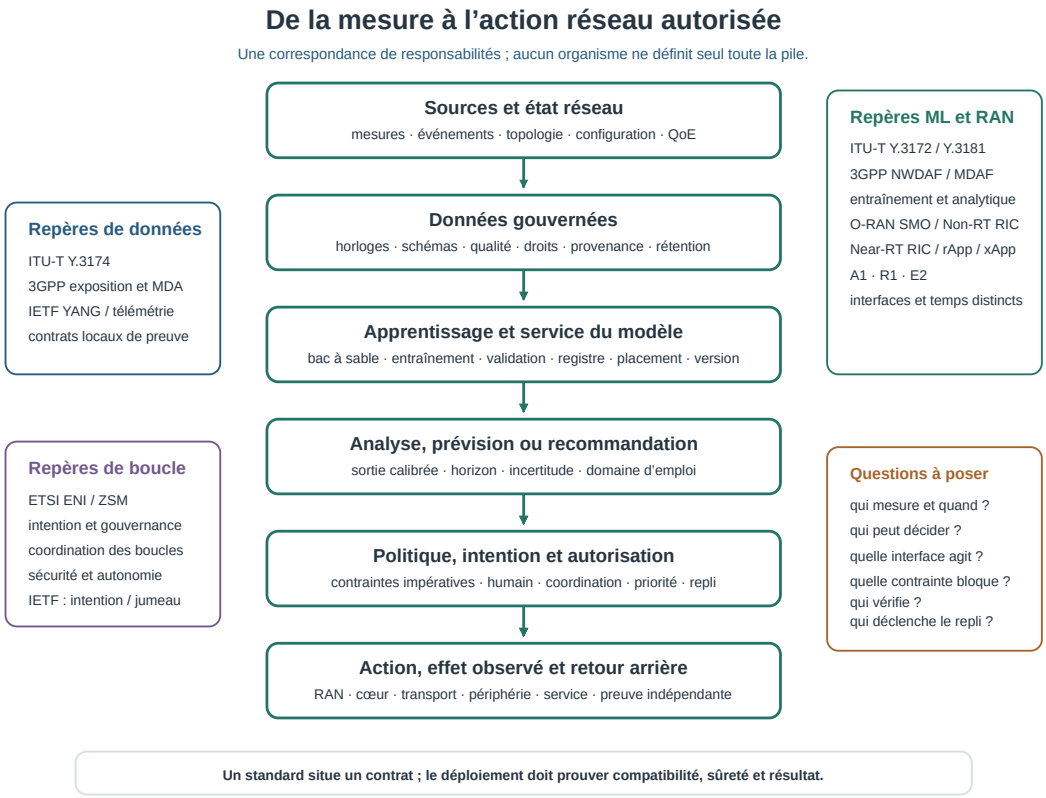
À ne pas confondre avec les cas ci-dessus : ici l'IA est la charge, pas le décideur.

Sujet	Section
Réseaux pour charges IA	§ 35
Réseaux pour l'IA distribuée (synchronisation collective, incast, flux périodiques)	§ 22
Cas de synthèse — entraînement distribué d'un modèle	§ Synthèse 5
Laboratoire **L18 — Projet IA/réseau** (workers synchrones, retardataire, incast ; placement, limitation, hiérarchie, taille de groupe, reprise)	§ Laboratoires — L18

8 bis. Architecture opérationnelle inter-SDO

Les organismes ne découpent pas le problème de la même façon. L'UIT-T fournit une architecture ML générique ; 3GPP définit des fonctions et services du cœur et du système de gestion ; O-RAN spécialise les boucles RAN ; ETSI structure l'intelligence et l'automatisation de bout en bout ; l'IETF fournit des concepts et des briques de gestion IP. La carte suivante est une **mise en correspondance d'ingénierie**, pas une déclaration d'équivalence normative.

Pour les cas de défense, cette cartographie reste limitée aux documents publics des SDO, aux rôles logiques et aux contrats de preuve. Elle ne déduit ni topologie, ni capacité, ni fréquence, ni configuration, ni identité, ni calendrier de déploiement, ni procédure d'une plateforme réelle.



Pile opérationnelle de l'IA pour les réseaux et correspondance entre UIT-T, 3GPP, O-RAN, ETSI et IETF/IRTF

Une chaîne opérationnelle commune

```
objectif de service et autorité
  -> demande d'analytics ou intention ML
  -> contrat de données et de télémétrie
  -> entraînement, validation et bac à sable
  -> modèle identifié, compatible et approuvé
  -> inférence ou analytics avec incertitude
  -> recommandation, politique ou commande bornée
  -> contrôleur de domaine et actionneur
  -> mesure indépendante, assurance et repli
```

La chaîne contient deux frontières qu'aucun sigle ne doit masquer :

- un **résultat d'analyse n'est pas une commande** ; son consommateur décide si et comment il est traduit en action ;
- un **canal de commande n'accorde pas l'autorité** ; l'identité, la politique, le périmètre, la durée, les contraintes et le retour arrière viennent du système d'exploitation et de gouvernance.

Correspondance fonctionnelle, sans fausse équivalence

Étape d'ingénierie	UIT-T	3GPP	O-RAN	ETSI	IETF / IRTF	Contrat restant à fournir
objectif, intention, politique	intention ML et orchestration dans Y.3172	demande d'analyse et politiques des fonctions consommatrices	rApp et services R1 ; politique A1 vers Near-RT RIC	politiques et contexte ENI ; intentions et services ZSM	vocabulaire d'intention dans RFC 9315	propriétaire, priorité, ambiguïtés, critères d'acceptation
collecte et préparation	cadre de traitement de données Y.3174	données des NF vers NWDAF ; données de gestion vers MDAF	O1 et services de données R1 ; mesures E2 selon E2SM	ingestion, connaissance et contexte ENI/ZSM	cadre RFC 9232 ; abonnements YANG-Push RFC 8639/8641	schéma, unité, horloge, fraîcheur, droits, pertes et coût
entraînement et cycle de modèle	chaîne de traitement et MLFO Y.3172 ; service de modèle Y.3179	MTLF/AnLF dans NWDAF ; gestion AI/ML TS 28.105	services de flux de travail AI/ML par R1 ; gestion de modèle A1-ML	blocs cognitifs ENI ; boucles et services de gestion ZSM	aucun cycle de modèle réseau commun normalisé	lignée, format, compatibilité, signature, ressources, retrait
bac à sable et preuve avant action	bac à sable (*sandbox*) Y.3181	émulation et gestion de cycle dans TS 28.105 selon le cas	validation hors ligne, tests R1/A1/E2 et environnement opérateur	gouvernance ZSM ; NDT étudié dans GR ZSM 015	architecture NDT encore Internet-Draft	fidélité, scénarios, oracle, seuils et écart simulation –réel
analyse et inférence	nœuds de la chaîne ML Y.3172	NWDAF pour le 5GC ; MDAF pour la gestion	rApp/Non-RT RIC ou xApp/Near-RT RIC selon le délai	cognition ENI ; analyse des boucles ZSM	consommation laissée aux applications	domaine de validité, calibration, latence et abstention
politique et contrôle	points de référence architecturaux, sans actionneur universel	une NF ou fonction de gestion autorisée consomme le résultat d'analyse et agit	A1 oriente ; E2 contrôle seulement via capacités d'un E2SM ; O1 configure/gère	services de boucle, délégation et coordination ZSM	NETCONF/ RESTCONF/ YANG sont des mécanismes, pas une autorité IA	action exacte, contraintes impératives, conflit, résidence et retour arrière
assurance et coordination	évaluation du niveau d'intelligence Y.3173, hors périmètre détaillé ici	suivi de précision et exposition de résultats selon release	retours A1/E2, tests d'interface et assurance par SMO	gouvernance, coordination, délégation et escalade ZSM	assurance d'intention conceptuelle dans RFC 9315	mesure indépendante, corrélation action–effet, audit et arrêt

Cette table ne permet donc pas de remplacer `NWDAF` par `Non-RT RIC`, `MDAF` par `ENI`, ou `A1` par `E2`. Ils peuvent contribuer à une même chaîne, mais leurs périmètres, consommateurs et sémantiques d'action diffèrent.

UIT-T : architecture ML, données et bac à sable

UIT-T Y.3172 définit l'architecture de haut niveau qui relie la chaîne ML, le bac à sable, la gestion et l'orchestration — dont l'orchestrateur de fonctions ML — au réseau sous-jacent. Elle donne un vocabulaire et des points de référence pour composer une application ML de réseau. Elle ne rend pas, à elle seule, deux produits interopérables au niveau protocolaire.

UIT-T Y.3174 approfondit le traitement des données : collecte, préparation, stockage, modèles de données et métadonnées, ainsi que l'application des sorties ML au réseau sous-jacent. Son apport opérationnel est de forcer la

séparation entre données, chaîne de traitement et action ; il ne dispense pas de définir les schémas, unités, horloges, droits et politiques de conservation propres au déploiement.

UIT-T Y.3181 spécifie les exigences, composants, points de référence et API de haut niveau d'un bac à sable ML. Le bac à sable sert à entraîner, tester et évaluer l'effet d'un modèle avant promotion. Il ne devient une preuve du réseau réel que si sa fidélité, sa fraîcheur et son erreur simulation–mesure sont publiées.

Ces trois recommandations sont complémentaires : Y.3172 place la chaîne de traitement, Y.3174 contracte les données et Y.3181 isole l'évaluation. Pour une réalisation, il faut encore sélectionner les protocoles, profils, modèles de données, formats de modèle et contrôleurs de domaine.

3GPP : trois plans qui ne se substituent pas

NWDAF — analyse des données du cœur 5G. Le terme anglais *analytics* désigne ici les résultats d'analyse fournis aux fonctions qui les consomment. 3GPP TS 23.288 définit l'architecture de services d'analyse des données du 5GS. Un NWDAF peut contenir une **Analytics Logical Function (AnLF)** et une **Model Training Logical Function (MTLF)**. Il expose des résultats d'analyse ou des modèles à des consommateurs autorisés au moyen des services `Nnwdaf` ; les API de stade 3 sont spécifiées notamment dans TS 29.520. La sortie peut être une statistique, une prévision ou une information de confiance. Elle ne possède pas automatiquement le droit de modifier une politique de cœur.

MDAF — analyse des données du système de gestion. 3GPP TS 28.104 place la **Management Data Analytics Function** dans le plan de gestion. Une MDAF produit un service de gestion fournissant des analyses, peut consommer des données de gestion et peut participer à une composition domaine/cross-domaine. Ses consommateurs incluent des fonctions de gestion ou, si l'architecture et les droits le prévoient, d'autres producteurs ou consommateurs de résultats d'analyse. Une MDAF n'est donc ni un autre nom du NWDAF, ni un RIC.

Gestion AI/ML. 3GPP TS 28.105 définit des cas, exigences, fonctions et ressources de gestion du cycle AI/ML, dont entraînement, évaluation ou émulation, déploiement et inférence selon le cas réalisé. Cette couche gère le modèle ; elle ne définit pas à elle seule la sémantique d'un résultat d'analyse NWDAF, d'une action MDAF ou d'une fonction radio.

Un dossier 3GPP doit donc figer **la release et la version de chaque TS**, le service producteur, le consommateur, l'`Analytics ID` ou le type de résultat, les filtres, la fenêtre temporelle, le délai admissible, puis la fonction qui transforme ce résultat en décision. Dire seulement « conforme NWDAF » ne décrit ni le modèle employé ni l'autorité accordée.

O-RAN : SMO, RIC, applications et interfaces

La pile logique se lit dans le sens suivant :

```
rApp -- R1 --> services du SMO / Non-RT RIC
                  |
                  +-- A1-P / A1-EI / A1-ML --> Near-RT RIC
                                                  |
                                           xApps + plateforme
                                                  |
                                      E2AP + E2SM choisi --> nœud E2

SMO -- O1 --> fonctions et éléments gérés ; SMO -- O2 --> O-Cloud
```

- **SMO** : porte des services de gestion et d'orchestration. Le **Non-RT RIC**, logiquement situé côté non temps réel, expose des services que les rApps consomment via R1 et produit politiques, informations d'enrichissement et services de flux de travail.
- **R1** expose des services aux rApps : enregistrement/découverte, accès aux données et fonctions de flux de travail AI/ML selon les profils publiés. R1 n'est pas une voie directe permettant à une rApp d'écrire arbitrairement dans un nœud radio.
- **A1** relie le Non-RT RIC au Near-RT RIC. Les familles de service couvrent politique (`A1-P`), information d'enrichissement (`A1-EI`) et gestion de modèles (`A1-ML`). L'Application Protocol v6.00, répertorié par le bulletin

officiel du 27 février 2026, définit `A1-ML/v1` ; cela spécifie un service de modèle, pas la qualité du modèle ni le droit universel de commander toute fonction RAN.

- **Near-RT RIC** : héberge les xApps et médie leur accès. **E2AP** porte les procédures entre Near-RT RIC et nœud E2 ; un **E2SM** détermine la sémantique effectivement exposée — par exemple mesures KPM ou paramètres/actions RAN Control. Sans E2SM pris en charge des deux côtés, `E2 CONTROL` n'est pas une API générique de contrôle du RAN.
- **O1** sert la gestion des fonctions et éléments O-RAN ; **O2** sert la gestion de l'O-Cloud. Les mélanger avec E2 confondrait configuration, infrastructure et boucle proche temps réel.

Le bulletin officiel O-RAN du 27 février 2026 publie aussi des tests R1 et A1 couvrant plusieurs flux de travail AI/ML. L'existence d'un test de conformité d'interface ne constitue toutefois ni une certification du modèle, ni une validation du KPI de service, ni une autorisation de déployer.

ETSI : intelligence expérientielle et boucles de bout en bout

ENI définit une architecture de système cognitif, orientée modèles, contexte, politiques, connaissance et boucles de contrôle autour d'un système assisté. ETSI GS ENI 005 V4.1.1 est une spécification de groupe publiée : elle structure les blocs fonctionnels et points de référence externes. Elle n'impose pas un modèle ML particulier et ne transforme pas ses API en interfaces radio ou cœur 3GPP.

ZSM (*Zero-touch network and Service Management*) structure les domaines de gestion, services de management et automatisation horizontale et verticale de bout en bout. ETSI GS ZSM 002 est l'architecture de référence. ETSI GS ZSM 009-1 ajoute les mécanismes de gouvernance, cycle de vie, coordination, délégation et escalade des boucles. C'est la bonne couche pour arbitrer deux contrôleurs qui agissent sur une même ressource ; ce n'est pas un algorithme d'arbitrage prêt à l'emploi.

ETSI GR ZSM 015 V1.1.1 étudie le jumeau numérique de réseau et distingue notamment un jumeau d'analyse d'un jumeau pouvant participer au contrôle. Le préfixe **GR** signifie *Group Report* : ce rapport éclaire l'architecture, mais ne doit pas être présenté comme une interface normative de jumeau numérique.

IETF et IRTF : concepts et briques, pas un RIC générique

- RFC 9315 définit les concepts d'*Intent-Based Networking*, dont réalisation et assurance. C'est un RFC **IRTF informatif**, explicitement hors Standards Track.
- RFC 9232 est un cadre IETF **informatif** de télémétrie réseau ; il classe les sources et techniques, mais ne définit pas une interface unique.
- RFC 8639 et RFC 8641 sont des **Proposed Standards** pour les abonnements YANG et YANG-Push. Ils peuvent transporter une partie de l'observation ; ils ne prescrivent ni features ML, ni labels, ni politique de conservation.
- `draft-irtf-nmrg-network-digital-twin-arch-13`, daté du 1er juillet 2026, reste un Internet-Draft actif expirant le 2 janvier 2027. Il doit être cité comme **travail en cours**, pas comme RFC ni norme publiée.

Le statut compte : un RFC peut être informatif, un Internet-Draft peut expirer, et un modèle YANG Standards Track ne normalise pas pour autant l'architecture AI/ML qui le consomme.

Échelles de temps : budget mesuré, pas placement par sigle

Boucle	Ordre de grandeur architectural	Placement typique	Ce qui doit rester vrai
temps réel radio / plan de transfert	**moins de 10 ms**	fonction locale dans le nœud, matériel ou logiciel temps réel	aucune dépendance synchrone à un service de management distant ; WCET et repli local

Boucle	Ordre de grandeur architectural	Placement typique	Ce qui doit rester vrai
proche temps réel O-RAN	**environ 10 ms à 1 s**	Near-RT RIC, xApp, E2 et E2SM pris en charge	budget bout en bout incluant collecte, file, inférence, conflit, signalisation et réalisation
non temps réel	**1 s ou plus**	Non-RT RIC/rApp, SMO, optimisation de politique	fraîcheur, horizon, cadence de politique et absence d'oscillation
analyse des données du cœur ou de gestion	**dépend du service et de la version**	NWDAF, MDAF, ENI, boucle ZSM	ne pas déduire « temps réel » du nom ; spécifier période, délai, horizon et expiration
entraînement, bac à sable, planification	minutes à mois selon le cas	MTLF, plateforme AI/ML, bac à sable Y.3181, jumeau	version des données, capacité, reproductibilité, durée de promotion et dérive

Les bornes O-RAN `10 ms–1 s` et `≥ 1 s`, rappelées dans le rapport officiel O-RAN NGRG sur l'évolution du Near-RT RIC, sont des classes architecturales, pas un SLO garanti par le simple choix d'un RIC. Une action dont l'échéance est inférieure au pire cas mesuré de la boucle doit être déplacée vers une fonction locale ou refusée. NWDAF, MDAF, ENI et ZSM ne portent pas une classe temporelle universelle comparable : leur contrat doit être dérivé du service exact.

Autorité : de l'observation à l'action native

Niveau	Sortie permise	Surfaces possibles	Contrôle obligatoire
observation	mesure, événement, résultat d'analyse, incertitude	Y.3174, Nnwdaf, MDA MnS, R1 data, E2SM-KPM, YANG-Push	qualité, fraîcheur, confidentialité, aucune action implicite
recommandation	option classée ou proposition	NWDAF/MDAF vers consommateur, rApp, ENI	propriétaire de décision, abstention, justification et journal
orientation	intention ou politique déclarative	ZSM/ENI, A1-P, services R1/SMO	traduction validée, priorités, conflits, durée et périmètre
modèle servi	artefact pour une fonction d'inférence identifiée	Y.3179, TS 28.105, R1 AI/ML, A1-ML	provenance, compatibilité, signature, déploiement témoin et retrait
commande bornée	action d'un catalogue explicite	E2AP + E2SM, contrôleur 3GPP autorisé, API de domaine	contraintes hors modèle, quota, résidence minimale, retour d'état
configuration	différences transactionnelles sur objets gérés	O1, NETCONF/RESTCONF/YANG, MnS	prévalidation, verrouillage, validation confirmée ou retour arrière
fonction native	décision dans la fonction réseau elle-même	PHY/MAC/plan de transfert ou fonction locale conçue à cet effet	enveloppe certifiée, watchdog, état sûr et comportement sans modèle

La progression n'est jamais automatique. Passer d'un résultat d'analyse à une politique, ou de politique à commande, exige une décision de promotion distincte, testée et révocable.

Photographie versionnée et maturité documentaire au 30 juillet 2026

Les numéros ci-dessous sont ceux visibles dans les catalogues officiels à la date de revue. Une intégration doit les recopier dans son manifeste plutôt que demander « la dernière version » au démarrage.

Corpus	Version ou édition contrôlée	Statut officiel au 2026-07-30	Maturité que l’on peut revendiquer	Limite de la revendication
UIT-T Y.3172	06/2019	recommandation **en vigueur**	architecture ML de référence publiée	pas un protocole ni un profil d’interopérabilité complet
UIT-T Y.3174	02/2020	recommandation **en vigueur**	cadre de traitement des données publié	schémas, transports et politiques restent à profiler
UIT-T Y.3181	09/2022	recommandation **en vigueur**	architecture de bac à sable publiée	pas une preuve automatique de fidélité au réseau réel
3GPP TS 23.288	Rel-20 **20.1.0**, téléversée le 2026-06-22	TS **Under Change Control**	architecture normative NWDAF, propre à une version	conformité à vérifier avec TS de stade 3 et profils produits
3GPP TS 28.104	Rel-20 **20.0.0**, téléversée le 2026-07-09	TS **Under Change Control**	architecture normative MDA/MDAF, propre à une version	ne normalise pas l’algorithme d’analyse
3GPP TS 28.105	Rel-19 **19.6.0**, téléversée le 2026-07-10	TS **Under Change Control**	gestion AI/ML normative, propre à une version	ne confère pas une autorité d’action au modèle
O-RAN architecture	Architecture Description **v17.0** ; SMO Architecture **v3.00**	documents annoncés le 2026-07-17	architecture O-RAN publiée et versionnée	spécification O-RAN, pas preuve de déploiement multi-vendeur
O-RAN R1	GA&P **v14.00** ; AP pour services R1 **v11.00**	documents annoncés le 2026-07-17	services rApp/SMO et flux de travail AI/ML spécifiés	une API publiée ne qualifie pas le modèle ni son KPI métier
O-RAN A1	GA&P **v5.02** ; AP **v6.00** ; Test **v5.00**	suite répertoriée dans le bulletin du 2026-02-27 ; AP définit `A1-ML/v1`	politique, enrichissement et gestion de modèle spécifiés	pas de contrôle radio arbitraire ni de garantie de latence
O-RAN Near-RT/E2	Near-RT RIC Architecture **v8.00** et E2AP **v8.00** ; E2SM-KPM **v8.00**, E2SM-RC **v10.00**, E2SM-CCC **v7.00**	KPM, RC et CCC annoncés le 2026-07-17 ; autres versions issues des trains précédents	boucle et sémantiques E2 versionnées	l’E2SM et les capacités réellement pris en charge font foi
ETSI GS ENI 005	**V4.1.1**, 2026-01-15	Group Specification **publiée**	architecture ENI publiée	pas un profil 3GPP/O-RAN ni un modèle ML imposé
ETSI GS ZSM 002	**V1.1.1**, 2019-08-13	Group Specification **publiée**	architecture ZSM publiée	intégration concrète et profils de service à réaliser
ETSI GS ZSM 009-1	**V1.1.1**, 2021-06-15	Group Specification **publiée**	enablers de boucles et gouvernance publiés	« stage 2 » : pas une implémentation universelle de boucle
ETSI GR ZSM 015	**V1.1.1**, 2024-02-22	Group Report **publié**	étude structurée du jumeau réseau	rapport informatif, pas interface normative
IRTF RFC 9315	octobre 2022	RFC **informatif**, hors Standards Track	terminologie stable de l’intention	ne définit pas un protocole IBN
IETF RFC 9232	mai 2022	RFC **informatif**	cadre et taxonomie de télémétrie	ne définit pas une technologie unique

Corpus	Version ou édition contrôlée	Statut officiel au 2026-07-30	Maturité que l’on peut revendiquer	Limite de la revendication
IETF RFC 8639 / 8641	septembre 2019	**Proposed Standards**	abonnement YANG et YANG-Push normalisés	seulement une partie du contrat de données
IRTF NMRG NDT	`draft-irtf-nmrg-network-digital-twin-arch-13`, 2026-07-01	Internet-Draft, **travail en cours**, expiration 2027-01-02	architecture de recherche révisable	ni RFC, ni norme, ni base suffisante de conformité

Lecture de maturité. « En vigueur », « publié » et « Under Change Control » qualifient un document, pas une implantation. La revendication de production exige en plus une matrice de conformité, des tests d’interopérabilité sur les versions choisies, la preuve de service, la sécurité, le repli et l’exploitation dans la durée.

Dossier minimal d’une intégration multi-SDO

Avant de relier deux briques de la carte, conserver dans un manifeste :

1. SDO, document, release, version, date et clauses réellement implémentées ;
2. producteur, consommateur, identité, domaine administratif et modèle de confiance de chaque interface ;
3. service/API, opération, schéma, encodage, transport, version et capacités négociées ;
4. sémantique des données : unité, horloge, fraîcheur, granularité, valeurs manquantes, confidentialité et rétention ;
5. identité du modèle, lignée, signature, format, environnement d’exécution, accélérateur, compatibilité et consommation ;
6. classe temporelle, échéance, WCET mesuré, file d’attente, expiration et comportement quand la donnée arrive trop tard ;
7. niveau d’autorité, catalogue d’actions, contraintes impératives, quotas, résolution des conflits et durée de résidence ;
8. test de conformité d’interface **et** test indépendant de l’effet de service, avec déploiement témoin, arrêt et retour arrière ;
9. propriétaire de la boucle, personne ou fonction qui accepte le risque, journal corrélé et procédure d’incident ;
10. comportement lors de perte de R1, A1, E2, Nnwdaf, MnS, télémétrie, modèle, horloge, bac à sable ou contrôleur.

Cette fiche rend la cartographie falsifiable : si un champ reste « selon le standard » sans référence, version ou test, l’interopérabilité et l’autorité ne sont pas démontrées.

---

9. Transverse — ce qui rend un cas d’usage déployable

9.1 Cycle de vie et assurance des modèles réseau

Un modèle opérateur est un composant de production. Le **registre de modèles** associe données, code, paramètres, métriques, domaine d’emploi, signature et propriétaire. Le déploiement surveille dérive des entrées, dérive de performance, incertitude, latence d’inférence et consommation. La détection **hors distribution (OOD)** provoque abstention ou repli classique. Chaque version possède une phase témoin, des critères d’arrêt et un **retour arrière indépendant de la disponibilité du modèle**. Pour une fonction critique, la certification porte sur la boucle complète — données, modèle, intégration, autorité, repli — et non sur un score de test isolé.

Les rôles derrière le cycle de vie : de l’analyse au MLOps

Une même personne peut tenir plusieurs rôles ; leurs responsabilités restent distinctes.

Rôle	Contribution à rendre vérifiable
Responsable produit	préciser l'usage attendu, la valeur recherchée et les critères qui justifient de poursuivre ou d'arrêter
Expert du domaine	préciser le problème, les unités, les situations importantes et les conséquences d'une erreur
Ingénieur des données (*data engineer*)	construire la collecte et les transformations, en documenter la qualité et la provenance
Analyste et scientifique des données (*data analyst*, *data scientist*)	explorer, formuler les hypothèses, choisir les comparaisons et évaluer les modèles
Ingénieur ML (*machine learning engineer*)	intégrer un modèle versionné, maîtriser ses dépendances, ses performances et ses interfaces
Exploitation, sûreté et cybersécurité	surveiller le service, traiter les incidents et éprouver les conditions d'arrêt et de repli

Le **MLOps** rapproche développement et exploitation des systèmes d'apprentissage : tests des données et du code, suivi des expériences, registre de modèles, déploiement et surveillance. La chaîne de traitement, ou *pipeline*, relie ces opérations ; son automatisation ne rend pas ses hypothèses vraies. Une dérive détectée peut déclencher une analyse, puis un réentraînement soumis aux critères de validation. Elle ne justifie pas à elle seule une mise à jour automatique du modèle en production (Google Cloud 2024).

Une **preuve de concept** (**POC**) vérifie une faisabilité précise. Un **produit minimum viable** (**MVP**) met une fonction utile entre les mains d'utilisateurs pour éprouver le besoin. La **production** ajoute les exigences de service, de maintenance et d'assurance. Passer du POC au MVP puis à la production demande donc d'autres preuves que l'amélioration du score du modèle.

Réduire le coût d'un modèle : élagage et quantification

L'**élagage neuronal** (*pruning*) retire ou neutralise des connexions, voire des groupes de paramètres, jugés peu utiles selon un critère explicite. Il réduit ce que le modèle conserve. La **quantification** réduit la précision avec laquelle certains poids, et parfois les activations, sont représentés : par exemple de 32 à 8 bits. Les deux méthodes peuvent se combiner ; elles répondent à des questions différentes : **combien de paramètres garder, et avec combien de bits les représenter ?** (Han et al. 2015)

Un élagage **non structuré** peut annuler des poids isolés, dispersés dans les matrices. Un élagage **structuré** retire des groupes entiers, par exemple des canaux ou des filtres : les dimensions de certaines opérations peuvent alors diminuer. La seconde forme se prête souvent mieux aux bibliothèques de calcul usuelles ; la première demande un format et des opérations capables d'exploiter les zéros. Le taux de poids retirés ne détermine donc pas à lui seul le gain de vitesse. Le résultat dépend aussi de la structure conservée et de l'implémentation cible (Wen et al. 2016).

Pour un million de poids, 32 bits représentent 4 Mo de valeurs brutes, contre 1 Mo à 8 bits, en unités décimales. Ce calcul exclut les échelles de quantification, les métadonnées, les activations et le cache. Des poids mis à zéro ne réduisent pas forcément la taille d'une matrice encore stockée intégralement ni son temps de calcul. Le format et le matériel doivent exploiter la réduction.

Que devient un poids ? Prenons un entier signé sur huit bits, compris entre -128 et 127, et un pas $s = 0,1$. On divise le poids w par ce pas, on arrondit à l'entier le plus proche, puis on limite le résultat à l'intervalle représentable :

$$q = clip_{[-128,127]}\big(arrondi(w/s)\big),\ \hat{w} = s\,q.$$

q est le nombre stocké ; $\hat{w}$, la valeur qu'il représente. L'opération **clip** ramène une valeur hors de l'intervalle à la borne la plus proche : c'est la **saturation**.

Poids w	Entier q	Valeur représentée $\hat{w}$
$0,26$	3	$0,3$

Poids w	Entier q	Valeur représentée $\hat{w}$
$-0,26$	-3	$-0,3$
14	127, après saturation	$12,7$

Sans saturation, l'erreur d'arrondi ne dépasse pas la moitié du pas : ici $0,05$. Avec saturation, elle peut être bien plus grande : $1,3$ pour le dernier poids. Ce choix de pas sert à comprendre le mécanisme ; en pratique, on le règle pour les valeurs à représenter. D'autres quantifications ajoutent un décalage du zéro (Jacob et al. 2018).

Après transformation, on réévalue la qualité, la latence et la mémoire sur la cible. Un réajustement ou un entraînement tenant compte de la quantification peut limiter la dégradation. La quantification des poids du modèle est distincte de celle des échantillons d'un signal ; l'élagage neuronal est distinct de l'élagage alpha-bêta décrit avec minimax.

Concevoir pour une latence cible et explorer les architectures

Réduire le nombre de paramètres ou d'opérations donne des indices sur le coût d'un modèle. Pour prévoir son délai sur un équipement, il faut aussi considérer **calcul, transferts, accès mémoire et synchronisations**. Une opération attend parfois ses données ou une autre branche ; une copie vers un accélérateur peut prendre plus de temps que le calcul qu'elle prépare. Dans une exécution séquentielle, on peut additionner les durées des étapes sans les compter deux fois. Lorsque ces étapes se chevauchent, leur somme ne donne plus le délai : ce sont les dépendances et les ressources disponibles qui déterminent la fin du calcul.

Le nombre d'opérations, souvent résumé en **FLOPs** pour les opérations en virgule flottante, ne mesure donc directement ni la latence ni l'énergie. Une conception **guidée par la latence** (*latency-aware design*) choisit aussi les opérateurs, la largeur des couches, les formes de tenseurs et les échanges en fonction de la cible. Le modèle exporté se mesure avec le logiciel et le matériel prévus, des entrées représentatives et la taille de lot retenue. Pour un service, le budget comprend en outre l'attente, la préparation et l'utilisation du résultat ; le placement des boucles réseau ne se décide pas sur le seul temps du réseau neuronal (voir budgets de temps mesurés).

La **recherche d'architecture neuronale**, ou **NAS** (*Neural Architecture Search*), automatise une partie de ces choix. On définit un espace de candidats — profondeurs, largeurs, opérateurs, connexions — puis une procédure propose et évalue des architectures. Pour chaque candidate, les poids doivent encore être entraînés ou estimés selon un protocole précis. Les critères peuvent associer qualité sur validation, délai, mémoire et énergie, avec des contraintes explicites. Les premiers travaux de NAS par renforcement ont ainsi fait générer des descriptions de réseaux par un contrôleur appris (Zoph et Le 2016).

MnasNet illustre ensuite une recherche tenant compte de la plateforme : son objectif combine qualité de classification et latence effectivement mesurée sur un téléphone. Cette démarche évite de supposer qu'un modèle ayant moins de FLOPs sera forcément plus rapide sur cette cible (Tan et al. 2019). Pour un autre équipement ou une fonction réseau, il faut définir ses propres mesures et contraintes.

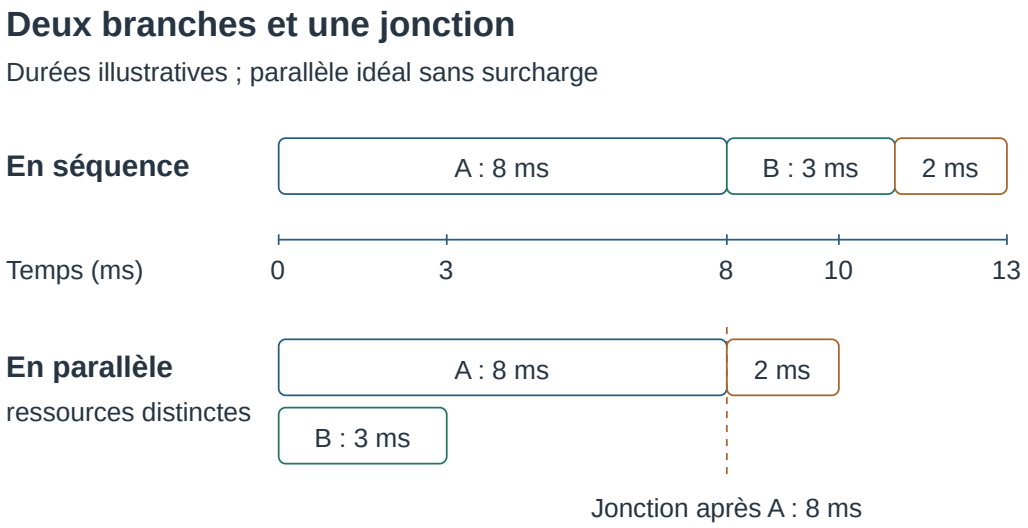
La recherche elle-même consomme calcul et énergie. Son budget — nombre de candidats, durée d'entraînement et durée totale — fait partie de la comparaison avec une architecture conçue directement. L'espace exploré, les évaluations approximatives et ce budget limitent le résultat : NAS ne promet pas de trouver automatiquement la meilleure architecture possible. Le jeu de test final reste indépendant des choix guidés par la validation. Après sélection, l'élagage ou la quantification peuvent encore modifier le compromis ; leur gain effectif se mesure à nouveau.

Paralléliser des branches : suivre le chemin critique

Le **graphe de calcul** décrit les opérations et leurs dépendances. Il est distinct du graphe de relais reçu en entrée d'un GNN. Deux branches peuvent s'exécuter ensemble si elles disposent de leurs entrées et si aucune n'attend le résultat de l'autre. L'opération qui les rejoint attend toutefois la plus lente : cette dépendance contribue au **chemin critique**, la suite d'opérations qui fixe la fin du calcul.

Prenons un exemple illustratif, sans mesure d'un équipement réel. Deux branches indépendantes durent respectivement **8 ms** et **3 ms**, puis leur résultat commun demande **2 ms**. En séquence, le délai vaut $8+3+2=13$ ms. Avec des ressources

distinctes et sans surcharge, l'exécution parallèle idéale donne $max(8,3)+2=10$ ms. La branche de 3 ms terminée plus tôt n'avance pas le point de jonction avant la fin de celle de 8 ms.



Chronologie illustrative : deux branches de 8 et 3 ms, suivies d'une jonction de 2 ms, donnent 13 ms en séquence et 10 ms en parallèle idéal

Dans la réalisation, lancement des opérations, transferts et synchronisations ajoutent un coût. Des branches partageant les mêmes unités de calcul ou la même bande passante mémoire peuvent aussi se ralentir. Il faut donc mesurer l'ordonnancement sur la cible. Multiplier les branches ne supprime ni les dépendances ni les ressources partagées (voir limites du parallélisme).

Dense–Sparse–Dense : une contrainte temporaire d'entraînement

Le cycle **Dense–Sparse–Dense**, ou **DSD**, proposé en 2016, utilise l'élagage comme étape d'entraînement et de régularisation. **Dense** signifie ici que les connexions sont disponibles ; **parcimonieux** (*sparse*) signifie qu'une partie des poids est contrainte à zéro. Le cycle comporte trois phases :

1. entraîner le réseau dense pour apprendre ses poids ;
2. élaguer des connexions selon la magnitude des poids, puis poursuivre l'entraînement en maintenant cette contrainte de parcimonie ;
3. retirer la contrainte, réinitialiser à zéro les poids précédemment élagués et entraîner de nouveau l'ensemble du réseau dense.

```
entraînement dense → entraînement sous contrainte parcimonieuse
                   → retour dense, connexions réouvertes et réapprises
```

Le retour dense est décisif : les poids réintroduits peuvent redevenir non nuls. DSD vise à améliorer l'apprentissage ; il ne garantit donc ni une parcimonie finale exploitable, ni une réduction de mémoire ou de latence à l'inférence (Han et al. 2016). L'élagage pour le déploiement cherche au contraire une réduction conservée et exploitable sur la cible. La quantification modifie la précision de représentation. La **distillation** entraîne un autre modèle à partir d'un modèle professeur (voir réutilisation et adaptation des modèles). Ces méthodes peuvent se combiner, mais leur objectif et leur résultat se vérifient séparément.

9.2 Chaîne de preuve IA et données (§ 9)

Encadré — cadre d'assurance inspiré d'EUROCAE WG-114

La méthode proposée ici s'inspire de la séparation pratiquée par EUROCAE WG-114 entre préoccupations de système, cycle de développement du constituant ML, implémentation/vérification et sujets transverses. Elle est transposée aux réseaux ; elle ne constitue ni une reprise du standard ni une revendication de conformité aéronautique (EUROCAE 2026b).

La chaîne d'assurance se lit du contexte vers la preuve :

1. **fonction prévue et domaine opérationnel** — mission, population de trafic, topologie, enveloppe de charge et conditions hors domaine ;
2. **rôle système et autorité** — sortie informative, recommandation, action bornée ou commande, avec repli et responsabilité humaine ;
3. **données** — origine, représentativité, séparation des jeux, qualité, dérive, droits et traçabilité ;
4. **constituant ML** — exigences mesurables, apprentissage, robustesse, vérification et limites connues ;
5. **implémentation et intégration** — équivalence du modèle déployé, interfaces, ressources, délais, modes de panne et interaction avec le reste du réseau ;
6. **exploitation et changement** — surveillance, incidents, retour arrière, configuration, réentraînement et réouverture proportionnée du dossier de preuve.

Les facteurs humains, la cybersécurité, la chaîne d'approvisionnement et la gestion de configuration traversent ces six niveaux : ce sont des dimensions d'assurance, non des familles d'algorithmes. Pour chaque niveau, le patron éditorial retenu ici, inspiré des préoccupations du *statement of concerns*, est : **hypothèse héritée → rupture possible introduite par le ML → préoccupation → preuve attendue → limite résiduelle** (EUROCAE 2021).

À la date de référence du présent texte, ED-324 est encore indiqué comme projet par EUROCAE, avec une cible au 31 décembre 2026. Son programme public commence par l'apprentissage supervisé hors ligne et prévoit des extensions ultérieures ; il ne faut donc pas extrapoler ce premier périmètre à toute forme d'IA (EUROCAE 2026a).

```
capteurs et captures
  -> contrat de données et vérité terrain
  -> préparation et séparation train/validation/test
  -> méthode de référence et entraînement
  -> calibration, robustesse et latence d'inférence
  -> quantification ou compilation pour la cible
  -> observation sans action, autorité bornée, abstention et repli
  -> surveillance du service et de la dérive
```

Chaque jeu de données conserve unité, horloge, incertitude, valeurs manquantes, topologie, configuration, firmware, action réellement appliquée et résultat observé. Séparations par temps, site et domaine de panne pour éviter les fuites.

> **Une meilleure récompense simulée ne prouve ni l'effet sur le service, ni la tenue sur la cible, ni la sécurité de l'action.**

Données personnelles et règles européennes : repère au 5 octobre 2026

Le **RGPD** s'applique aux traitements de données personnelles : finalité, base légale, minimisation, sécurité et exercice des droits se précisent dès la conception. Calculer sur un terminal limite certains transferts, mais n'établit pas, à lui seul, la conformité. Un modèle peut aussi mémoriser des données personnelles ; sa mise à disposition appelle une analyse propre (CNIL 2025).

Le règlement européen sur l'IA, ou **AI Act**, ajoute des règles selon la finalité, le risque et le rôle de l'acteur. Interdictions et dispositions d'acculturation s'appliquent depuis février 2025, gouvernance et modèles d'IA à usage général depuis août 2025, puis obligations de transparence depuis août 2026, sous les exceptions et transitions prévues. L'amendement de juillet 2026 reporte certaines obligations des systèmes à haut risque à décembre 2027 et des systèmes associés aux

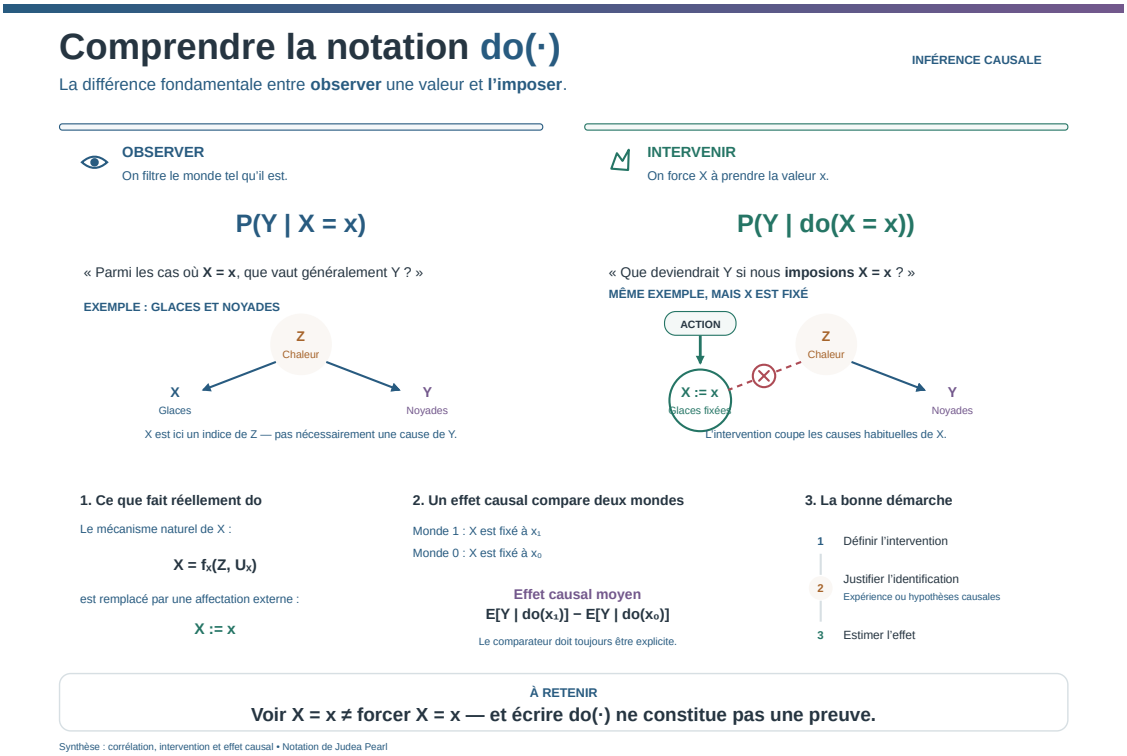
produits à août 2028. Ces échéances restent futures à la date de ce livre (Commission européenne 2026b; Union européenne 2026).

Vérifier un visage contre l'identité revendiquée (**1:1**) et rechercher une personne dans une population (**1:N**) sont deux usages différents. Leur régime ne se déduit pas du seul mot « reconnaissance faciale ». Les systèmes destinés exclusivement à la défense, à l'usage militaire ou à la sécurité nationale sont hors du champ de l'AI Act ; cela ne supprime pas les autres règles applicables (Commission européenne 2026b, 2026a).

Une preuve formelle reste possible sur un périmètre précis. Reluplex a montré dès 2017 la vérification de propriétés de réseaux neuronaux du prototype anticollision **ACAS Xu**, sur des domaines d'entrée définis. Cette preuve porte sur un réseau, une propriété et des hypothèses : elle ne certifie ni toute la perception, ni toutes les situations de vol, ni l'ensemble d'un équipement. Le prototype ACAS Xu étudié dans cet article est distinct du cas historique ACAS-X présenté plus loin (Katz et al. 2017).

9.3 Causalité (§ 21)

L'IA peut **prioriser des hypothèses**. La preuve vient de données et d'expériences : topologie de dépendances, ordre temporel, changements récents, expériences contrôlées, traces, invariants, connaissance des boucles.



Notation do(·) : distinguer l'observation conditionnelle de l'intervention qui impose une valeur

La sortie utile n'est donc pas seulement « cause probable = CPU ». Elle peut être : « mesurer la file de telle interface », « comparer avant/après le dernier changement », « rediriger temporairement un flux témoin », ou « aucune expérience sûre n'est disponible ». Une expérience discriminante doit séparer au moins deux hypothèses, être réversible et ne pas masquer l'incident qu'elle cherche à comprendre.

9.4 Jumeau numérique et contrôle proactif (§ 24–24 bis)

Un jumeau numérique représente topologie, configuration, politiques, routes, capacité, pannes et parfois charge. Il permet d'évaluer un changement, de générer des données rares ou de chercher une action avant de toucher le réseau. Il n'est pas automatiquement fidèle parce qu'il porte le nom de « jumeau ».

Son contrat expose :

- la date et la source de chaque état ;

- les protocoles et comportements matériels modélisés ou absents ;
- l'erreur entre prédictions et mesures indépendantes ;
- les scénarios où le modèle est invalide ;
- la commande de reconstruction et les versions de dépendances ;
- les conditions qui interdisent de promouvoir une action vers le réel.

Le contrôle proactif ajoute une contrainte temporelle : **l'horizon utile de la prévision doit dépasser le délai nécessaire pour agir**. Prévoir une rupture de lien dix secondes à l'avance ne sert pas à établir un relais qui exige trente secondes. La fraîcheur de l'observation, le coût du prépositionnement et le risque d'une fausse alerte font partie du résultat.

```
observer avec une fraîcheur connue
 -> prévoir avec une incertitude calibrée
 -> préparer sans basculer prématurément
 -> autoriser dans une enveloppe
 -> mesurer le service réel
 -> maintenir, réconcilier ou revenir en arrière
```

9.5 Sécuriser le contrôleur et la chaîne de traitement (§ 28)

Une automatisation réseau compromise possède un périmètre d'impact supérieur à celui d'un outil d'analyse isolé. Le contrôle porte donc sur toute la chaîne :

- entrées autorisées, schémas fermés, unités et fraîcheur vérifiées ;
- séparation des rôles entre entraînement, approbation et exploitation ;
- code, dépendances, modèles et configurations identifiés et vérifiés ;
- secrets exclus des prompts, jeux d'entraînement et journaux non protégés ;
- modèle chargé seulement depuis une origine de confiance ;
- déploiement témoin, budget de changements, durée et périmètre d'autorité ;
- journal append-only ou attesté, synchronisation temporelle et corrélation des identités ;
- arrêt, retour arrière et récupération testés sans dépendre du modèle fautif.

Une empreinte prouve que les octets n'ont pas changé ; elle ne prouve ni la confiance dans le producteur, ni la qualité de l'entraînement, ni l'identité de l'opérateur. Dans le Routing Experiment Lab, le chargement d'un modèle Stable-Baselines3 — qui peut désérialiser des objets Python — exige donc un acquittement de confiance explicite en plus de l'empreinte.

9.6 Coordonner plusieurs boucles fermées

Routage, codec, admission, radio, mise à l'échelle automatique et énergie peuvent observer le même symptôme et agir sur les mêmes ressources. Les optimiser séparément crée des décisions contradictoires ou des oscillations. Une architecture exploitable définit pour chaque boucle :

Propriété	Question de coordination
propriétaire	qui décide et qui peut préempter ?
échelle de temps	la boucle agit-elle en millisecondes, secondes ou minutes ?
ressource	quel budget commun est consommé ?
priorité	quelle boucle cède en cas de conflit ?
état partagé	quelle observation et quelle fraîcheur font foi ?
stabilité	hystérésis, résidence minimale et budget de changements ?
reprise	dans quel ordre les contrôleurs reviennent-ils ?

Cette coordination est un contrat de contrôle, pas une tâche que l'on délègue implicitement à un second modèle. Les travaux ETSI sur l'automatisation en boucle fermée distinguent notamment gouvernance et coordination des boucles ; ces deux fonctions restent nécessaires même lorsque la décision interne est apprise.

9.7 Contrat minimal d'un artefact de preuve

Une expérience interprétable conserve au minimum :

```
identité de l'exécution et statut final
révision du code, environnement et dépendances
empreintes des entrées, du modèle et des sorties
scénario, graine, horloge, unités et domaine de connaissance
contrat d'observation, d'action et de réalisation
méthode de référence, métriques, seuils d'acceptation et résultats manquants
commande de vérification et, si elle existe, commande de rejeu
limitations, opérateur ou attestation lorsque l'origine compte
```

Le manifeste est publié **en dernier**, après validation des sorties. Un statut `failed`, `skipped` ou `unsupported` reste visible ; il n'est jamais transformé en succès par l'absence silencieuse d'une métrique.

---

10. Choisir l'outil — y compris ne pas employer l'IA

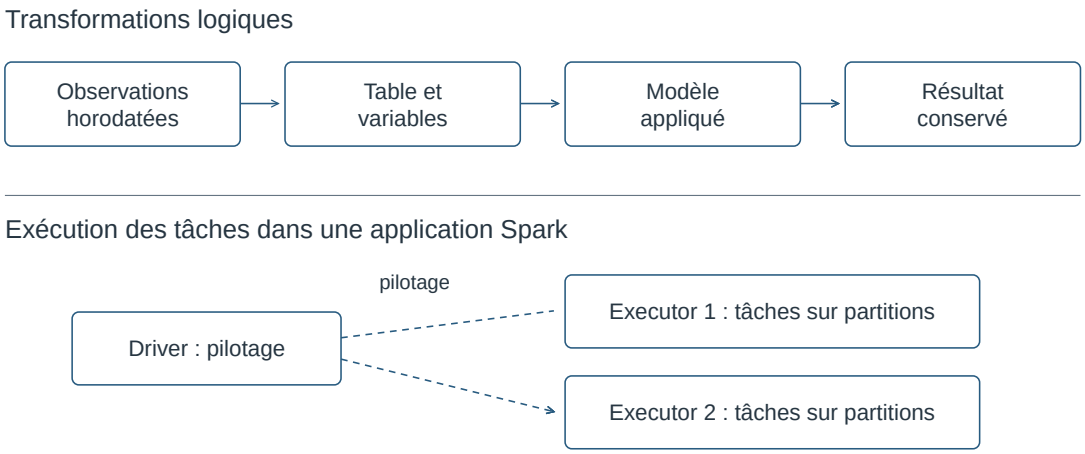
Besoin dominant	Outil de départ	Sortie attendue	Échec à surveiller
règle connue et stable	règle, protocole ou automate	décision déterministe	complexité ajoutée sans gain
contraintes combinatoires explicites	programmation linéaire, MILP, CP-SAT (solveur de programmation par contraintes fondé sur SAT) ou heuristique	solution et faisabilité	temps de calcul, approximation cachée
prévision temporelle	modèle statistique puis ML supervisé	distribution ou intervalle futur	fuite temporelle, rupture de régime
classification avec vérité terrain	supervisé calibré	classe et probabilité	biais d'étiquettes, surconfiance
anomalie sans labels complets	statistique robuste / non supervisé	score et observations contributrices	prendre la rareté pour une attaque
graphe relationnel variable	algorithme de graphe puis GNN/MPNN	représentation, score ou allocation	extrapolation à une taille/topologie inconnue
décision séquentielle à effet retardé	contrôle classique puis RL borné	action dans une enveloppe	exploration dangereuse, récompense mal spécifiée
texte et connaissance documentaire	recherche hybride, RAG, LLM outillé	réponse sourcée, différences ou test	hallucination, droits contournés, invite hostile
question causale	modèle de dépendances et expérience	hypothèse discriminée	confondre prédiction et intervention

Deux hybrides sont souvent préférables à un modèle seul : **solveur + modèle** (le modèle propose, le solveur répare ou certifie la faisabilité) et **protocole + modèle** (le modèle estime ou repondère, le protocole conserve ses invariants). L'hybridation doit toutefois dire quelle composante possède le dernier mot.

Des données au résultat : une chaîne de traitement distribuée

Un modèle n'est qu'une étape d'une chaîne : recevoir des observations, les préparer, calculer des variables, entraîner ou appliquer un modèle, puis conserver et utiliser le résultat. Lorsque les volumes ou les délais le justifient, ces étapes peuvent être réparties entre machines. **Apache Spark** fournit un moteur pour organiser une partie de ce travail.

Une **partition** est une portion de données sur laquelle une tâche peut travailler. Les méthodes de découpage et d'équilibrage expliquent comment construire ces portions ; les sémaphores limitent leur accès simultané à une ressource commune. Le processus de pilotage, appelé **driver**, planifie les tâches ; des processus **executors** les exécutent et conservent des données de l'application. Ils peuvent fonctionner sur un équipement ou sur plusieurs. La répartition ne supprime pas les transferts : regrouper toutes les mesures d'un même relais peut déplacer des données entre partitions. Cet échange, appelé **shuffle**, consomme réseau, mémoire et parfois disque (Apache Software Foundation 2026a).



Deux vues distinctes d'une chaîne distribuée : transformations logiques des observations et pilotage de tâches sur des partitions

Les modules désignent des fonctions complémentaires ; ils ne sont pas des modèles concurrents.

Fonction	Rôle dans une chaîne Spark
SQL et DataFrames	manipuler des tables à colonnes nommées : filtrer, joindre, agréger
Structured Streaming	appliquer des calculs incrémentaux à des observations qui continuent d'arriver
MLlib	préparer des variables, entraîner et évaluer des méthodes d'apprentissage ; l'interface DataFrame `spark.ml` est la principale
GraphX	traiter des graphes avec l'interface historique fondée sur les RDD, collections distribuées
Stockage et gestion des ressources	fournir les données et attribuer les ressources ; ces fonctions restent distinctes du modèle

Cette lecture actualise l'empilement présenté dans le support de 2023. Les guides Spark **4.2.0** donnent des repères de version conservables : SQL et DataFrames, Structured Streaming et MLlib (Apache Software Foundation 2026e, 2026f, 2026d, 2026c).

Voici une agrégation sur des données fictives. La table `mesures` contient le relais, la latence en millisecondes et un indicateur de panne valant 0 ou 1. La même intention peut s'exprimer en SQL dans Spark ou dans une base locale ; cet exemple n'exige pas de cluster.

```
SELECT relais,
       AVG(latence_ms) AS latence_moyenne_ms,
       SUM(panne) AS nombre_pannes
FROM mesures
GROUP BY relais
ORDER BY relais;
```

Avec deux lignes pour R1, de latences 10 et 30 ms et d'indicateurs 0 et 1, le résultat est **20 ms et 1 panne**. Une ligne pour R2 de 8 ms et d'indicateur 0 donne **8 ms et 0 panne**. La moyenne n'est pas un quantile de délai ; le comptage dépend de la durée et de la manière d'enregistrer les événements.

Pour un flux, on définit aussi la fenêtre temporelle et le traitement des arrivées tardives. Le **temps de l'événement** est celui de la mesure ; le **temps de traitement** est celui où elle est reçue et calculée. Un seuil de retard accepté (*watermark*) aide à décider jusqu'à quand conserver l'état des fenêtres. Un point de reprise (*checkpoint*) conserve des informations nécessaires à la reprise du traitement. Les garanties effectives dépendent aussi des sources, des destinations et du mode d'exécution (Apache Software Foundation 2026f).

Un calcul distribué peut coûter plus cher qu'un calcul local lorsque les données sont petites ou les échanges dominants. Son intérêt se mesure sur la chaîne entière, avec ses transferts et ses reprises (voir limites du parallélisme).

Bibliothèques, modèles et matériel : des choix distincts en 2026

Un modèle préentraîné fournit des paramètres appris ; une bibliothèque permet d'en définir ou d'en exécuter les calculs. Un catalogue de modèles n'est donc pas un classement des bibliothèques.

Besoin	Exemples et rôle
Méthodes statistiques et ML classique	scikit-learn ; R pour l'analyse statistique
Modèles différentiables	PyTorch, TensorFlow et JAX : calculs, gradients et exécution selon leur environnement
API de conception de réseaux	Keras 3 : plusieurs moteurs, dont JAX, TensorFlow et PyTorch pour l'entraînement ; OpenVINO pour l'inférence
Traitement d'images et préparation	OpenCV : opérations de vision et outils associés
Échange et exécution d'un modèle	ONNX : représentation d'un modèle ; ONNX Runtime : exécution avec des opérateurs et fournisseurs compatibles
Expériences et cycle de vie	MLflow : suivi des expériences et gestion d'artefacts de modèles

Les versions, licences, formats, opérateurs et contraintes d'installation comptent davantage qu'un superlatif tel que « meilleur pour la recherche ». Keras n'est plus limité à TensorFlow ; exporter en ONNX ne rend pas tous les opérateurs compatibles avec tous les matériels (Keras developers 2026b; ONNX Runtime developers 2026).

Un **CPU** est un processeur généraliste ; un **GPU** exploite un grand parallélisme ; un **NPU** (*Neural Processing Unit*) est un accélérateur spécialisé dans certaines opérations neuronales. Un petit modèle peut s'exécuter sur CPU. Le placement entre terminal (*edge*) et centre de calcul (*cloud*) dépend aussi des transferts, de la latence, de la mémoire et de la confidentialité ; le calcul local n'assure pas automatiquement une moindre consommation.

Les **TOPS** comptent des milliers de milliards d'opérations par seconde ; les **TFLOPS**, des milliers de milliards d'opérations en virgule flottante. Comparer ces annonces demande la précision numérique, la convention de comptage, la sparsité éventuelle et la charge réellement exécutée. Après élagage et quantification, on mesure qualité, latence, mémoire et énergie sur la cible (MLCommons 2026).

Un tableau de vitesse a toujours un protocole. Le catalogue Keras Applications contient encore les valeurs du support de 2023, mais décrit un protocole particulier : validation ImageNet pour la précision, lots de 32, moyenne sur 30 lots et 10 répétitions pour les temps, avec CPU AMD EPYC 92 cœurs, 1,7 To de RAM et GPU Tesla A100. Ces temps ne prédisent pas ceux d'un terminal radio (Keras developers 2026c).

Un repère économique, avec son périmètre. L'AI Index publié en 2026 rapporte environ **344,7 milliards de dollars d'investissements privés mondiaux en 2025**, dont 285,9 aux États-Unis et 12,4 en Chine. Ces observations ne comptent pas l'ensemble des investissements publics et privés cités dans la présentation de 2023. Elles situent une échelle

économique, sans mesurer la qualité d'un modèle ni le gain d'un projet (Stanford Institute for Human-Centered Artificial Intelligence 2026).

Ce que les pratiques de 2026 ajoutent à cette carte

Les familles classiques restent utiles : leur simplicité donne un point de comparaison. Ce qui a changé est surtout la possibilité de réutiliser des représentations et des modèles déjà appris, puis d'adapter leur exécution à une tâche et à un budget.

Des tableaux aussi. Les forêts et le *gradient boosting* servent toujours de références sur des données tabulaires. Les modèles préentraînés pour les tableaux ajoutent une option. Le travail TabPFN publié en 2025 en explique le principe ; la famille TabPFN-3.5, publiée en septembre 2026, étend cette voie. On compare sur les mêmes données, avec une séparation temporelle ou par groupe lorsque le problème l'exige, et avec le coût complet de l'inférence. Les résultats d'un catalogue ne désignent pas le meilleur modèle pour toute entreprise (Hollmann et al. 2025; Prior Labs 2026).

Des images au-delà de la convolution. Un CNN apprend des filtres locaux ; un Transformer visuel met en relation des portions de l'image. Le préentraînement auto-supervisé permet d'apprendre une représentation sans étiquette humaine pour chaque image. DINOv3 en est un exemple documenté en 2025. On peut réutiliser cette représentation pour une tâche, avec son propre contrôle de qualité. Reconnaître des images dans un banc d'essai et comprendre toute scène réelle restent des objectifs différents (Simeoni et al. 2025).

Adapter quelques paramètres. Un nouveau besoin n'impose pas toujours de réentraîner tout le réseau. LoRA ajoute de petites matrices apprises ; **QLoRA** combine cette adaptation avec un modèle de base quantifié. La bibliothèque PEFT met en œuvre ces adaptations. Elles se distinguent de la recherche documentaire RAG, qui fournit des informations au moment de la réponse, et du post-entraînement par renforcement, qui ajuste le comportement à partir de récompenses. Un défaut peut aussi venir du code ou des données : on corrige d'abord sa cause (Hugging Face 2026b).

Réduire le coût réel. Les outils actuels distinguent quantification des poids seuls, des activations et calcul en faible précision. TorchAO documente notamment INT8, INT4 et FP8, avec des contraintes de matériel et de moteur. Quatre fois moins de bits pour les poids ne signifie pas quatre fois moins de mémoire pour toute l'application, ni quatre fois plus de vitesse. Pour un LLM, on mesure séparément la lecture de l'entrée (*prefill*) et la génération (*decode*), en comptant aussi le cache des clés et valeurs de l'attention (PyTorch developers 2026).

Donner davantage de calcul à une réponse. On peut développer une solution plus longtemps, produire plusieurs candidats ou chercher dans un arbre de possibilités. C'est le *test-time scaling* : augmenter le calcul pendant l'inférence. Le travail de synthèse publié en août 2026 distingue ces stratégies et leurs protocoles d'évaluation. Des candidats qui répètent la même erreur et un vérificateur peu fiable limitent le bénéfice. La comparaison porte donc sur la solution complète, avec son budget, plutôt que sur le seul nom du modèle (Hariri et al. 2026).

Un même principe, plusieurs produits. Le nom d'un algorithme ne décrit pas encore l'usage ; la sortie et le coût des erreurs le précisent.

Exemple	Sortie et contrôle utile
Détection d'incendie dans une vidéo	Localiser des indices visuels, puis mesurer faux positifs, omissions et délai ; la détection ne remplace pas la procédure d'alerte
Segmentation d'un corps, pose d'une main ou profondeur	Produire un masque, des points articulaires ou des distances ; vérifier les unités, les occultations et les conditions de prise de vue
Amélioration d'une vidéo ou légende d'une photographie	Produire des pixels ou du texte ; une restitution plausible peut ajouter des détails absents de l'observation
Classement de courriels ou commande vocale	Associer texte ou signal à une intention ; séparer erreur de transcription, erreur de classement et action autorisée
Jeu à information incomplète, tel le bridge	Combiner règles, recherche, estimation et apprentissage ; une bonne prédiction ne remplace pas les règles du jeu

Pour approfondir les méthodes sans figer un catalogue de 2023 : guide des modèles de scikit-learn, documentation PEFT, quantification TorchAO et rapport TabPFN-3.5. Les articles sont conservés avec leur année et leur identifiant ; les documentations de logiciel permettent de suivre les évolutions.

Un nom voisin peut désigner un autre mécanisme. Un *Spatial Transformer Network* apprend une transformation géométrique de l'image, par exemple un recadrage ou une rotation. Ce n'est pas le Transformer de séquences qui calcule l'attention entre des jetons Jaderberg et al., 2015.

Les modèles génératifs d'images ne se réduisent pas aux GAN et aux autoencodeurs. Une **diffusion** apprend à retirer progressivement le bruit ajouté pendant l'entraînement ; un modèle de **flow matching** apprend un champ de déplacements reliant une distribution de départ à celle des données. Ces familles peuvent utiliser des représentations latentes et des architectures à attention. Les articles DDPM et Flow Matching permettent de retrouver les objectifs d'apprentissage.

Adapter une moyenne n'est pas effacer des poids. Pour suivre une mesure qui évolue, une moyenne exponentielle peut employer $m_t = \gamma\, m_{t-1} + (1-\gamma)\, x_t$, avec $0 \le \gamma < 1$. L'influence d'une ancienne mesure diminue à chaque pas. La décroissance des poids d'un réseau, l'oubli catastrophique et ce filtre temporel décrivent des mécanismes distincts.

Une découverte dans les données. En 2017, l'analyse neuronale des observations du télescope Kepler a contribué à identifier Kepler-90i, une huitième planète dans ce système. Le modèle analyse un signal mesuré ; il ne voit pas sans instrument. Cette distinction garde tout son intérêt pour l'IA scientifique annonce de la NASA.

S'orienter dans l'écosystème et poursuivre sa formation

Une carte d'outils devient plus lisible lorsque l'on sépare leurs rôles : collecter et stocker les données ; les préparer et les orchestrer ; entraîner ou adapter un modèle ; l'exécuter ; mesurer ses résultats ; surveiller son usage. Les notebooks traversent ces activités : ils permettent d'explorer et d'expliquer, sans remplacer à eux seuls une chaîne de production.

Le panorama LF AI & Data donne une entrée par familles. Son cycle de vie des projets distingue notamment expérimentation, incubation, graduation et archivage. Ce statut concerne la vie du projet ; il ne certifie pas le fonctionnement d'un produit particulier. Les palmarès annuels de laboratoires, de start-up ou de popularité ne remplacent pas une comparaison adaptée à son besoin.

Le cycle des attentes de Gartner propose cinq étapes : déclenchement, attentes exagérées, désillusion, compréhension progressive et adoption productive. Il aide à parler de l'écart entre promesses et usages ; sa courbe ne calcule ni une précision d'algorithme ni une date garantie de disponibilité.

Pour apprendre, on peut choisir un parcours selon la question qui bloque :

Besoin	Ressource en ligne
Déchiffrer un terme	Glossaire ML de Google
Refaire les bases statistiques et les algorithmes	CS229, Stanford
Approfondir l'apprentissage profond	CS230, Stanford et MIT 6.S191
Relier apprentissage et livraison d'un produit	Made With ML
Suivre l'innovation avec des dossiers de fond	Publications d'Olivier Ezratty
Refaire les mathématiques à son rythme	Guide des notations du livre

Les parcours internes d'une entreprise complètent ces ressources selon ses outils et ses responsabilités. Un diplôme donne des bases ; la pratique, la confrontation aux données et l'explication à d'autres permettent de continuer à apprendre.

11. Protocole de validation avant autorité

Fiche de décision avant apprentissage ou intégration

Avant l'entraînement d'un modèle ou l'intégration d'un composant IA, cette fiche réunit les quatre questions de cadrage, les dix contrats, l'échelle d'autorité et le niveau de preuve en une décision courte, relisible et auditable. Elle n'ajoute pas un onzième contrat d'ingénierie.

Champ de décision	Trace minimale exigée
Effet de service	Service visé, SLO mesurable, horizon temporel et coût d'une erreur.
Alternative sans IA	Règle, protocole ou solveur explicite de référence, avec le critère qui justifierait de ne pas employer l'IA.
Observation	Signaux, provenance, droits d'usage, granularité, fraîcheur, données manquantes et conditions hors distribution.
Décision et autorité	Décision produite, responsable, échelon maximal d'autorité, budget temporel et périmètre d'action.
Contraintes dures	Invariants appliqués hors du modèle et hors de sa récompense, avec le traitement de toute violation.
Repli indépendant	Politique déterministe non apprise, déclencheur, retour au nominal et essai indépendant du modèle et du lien de contrôle.
Preuve attendue	Niveau 1 à 5 visé, comparaison appariée au référentiel, incertitude et critères d'acceptation, y compris les résultats défavorables.
Exploitation et rejeu	Versions, données, graines, télémétrie, artefacts de rejeu, propriétaire du retour arrière et durée de conservation.

La sortie de cette fiche est une décision explicite :

- **sans IA** si le référentiel satisfait le besoin avec de meilleures garanties ;
- **mode ombre** si la valeur marginale ou la couverture hors distribution reste incertaine ;
- **recommandation** si un responsable conserve la décision et peut la refuser ;
- **exécution bornée** seulement si le gain est démontré au niveau de preuve visé et si contraintes et repli ont été vérifiés indépendamment.

Une case vide bloque toute promotion vers un niveau d'autorité supérieur. « Inconnu » constitue une observation recevable, jamais une autorisation.

1. **Formuler l'effet de service.** Débit utile, échéance, disponibilité, énergie, détectabilité ou coût — pas seulement la perte ou la récompense.
2. **Figer les contrats.** Même scénario, graine, vue de topologie, horloge, décision et réalisation pour tous les candidats comparés.
3. **Choisir plusieurs références.** Règle neutre, plus court chemin ou FIFO, heuristique métier et, lorsque possible, solveur explicite.
4. **Séparer les données.** Découpage par temps, site, trajectoire, type de panne et domaine administratif ; dédoublonnage avant la séparation des données.
5. **Tester la faisabilité.** Routes valides, demandes couvertes, capacités, politiques, unités, valeurs manquantes et contraintes impératives.
6. **Mesurer le coût de décision.** Préparation, inférence et réalisation sont chronométrées séparément ; une décision parfaite mais tardive échoue.

7. **Évaluer la stabilité.** Convergence, fréquence de changement des routes, réordonnancement, oscillations, reprise et temps passé en mode dégradé.
8. **Varier les conditions.** Plusieurs graines, charges, topologies, pannes, mesures périmées, absentes, empoisonnées et hors distribution.
9. **Quantifier l'incertitude.** Distribution des résultats, intervalles et quantiles de queue ; un écart sur une exécution ne prouve pas une supériorité.
10. **Changer de substrat.** Formule → simulation → émulation → matériel ou terrain selon le niveau de revendication.
11. **Déployer sans autorité d'abord.** Hors ligne, en observation sans action, en recommandation, puis sur un périmètre témoin, action bornée, puis extension si les critères restent satisfaits.
12. **Tester l'échec.** Modèle absent, trop lent, corrompu, hors domaine ; repli indisponible ; journal ou mesure partielle ; retour arrière interrompu.

Pour le routage, ajouter explicitement : validité du chemin dans la vue connue, couverture des demandes, respect de capacité, fraîcheur de la topologie, identité du nœud de décision, coût du trafic de contrôle, fréquence de changement des routes, convergence et effet paquet mesuré. Les métriques de récompense et les mesures de service restent deux colonnes distinctes.

---

12. Anti-patterns et reformulation correcte

Formulation trompeuse	Pourquoi elle échoue	Reformulation exploitable
« l'IA optimise le réseau »	aucun objectif, aucune autorité	« prédit X à horizon T et recommande Y parmi Z »
« meilleur que l'algorithme classique »	référence ou scénario non aligné	nommer la référence, les entrées, les graines, les métriques et l'incertitude
« validé en simulation »	le simulateur peut partager les hypothèses de la récompense	publier l'écart avec une mesure indépendante
« modèle hashé donc sûr »	intégrité des octets ≠ confiance ou provenance	vérifier origine, lignée, format de chargement et attestation
« aucune perte donc réseau sain »	les files peuvent seulement accumuler du délai	mesurer délai, gigue, échéances et blocage en arbre
« anomalie donc attaque »	panne et changement légitime peuvent être rares	conserver les événements et demander une preuve discriminante
« priorité 1 » sans contrat	sens numérique variable selon les systèmes	écrire l'ordre total et tester un cas réellement en contention
« protocole OLSRv2-like »	un modèle de diffusion n'est pas RFC 7181/7188	dire modélisé, puis exiger daemon, captures et convergence
« jumeau numérique » sans calibration	le modèle peut représenter un monde périmé	publier fraîcheur, hypothèses et erreur simulation–mesure
« agent autonome »	outils, droits et arrêt non définis	lister capacités, budget, approbations, journal et retour arrière

13. Fiche d'une page pour cadrer un cas d'usage

```
Nom et propriétaire :
Effet de service recherché :
Population, réseau et conditions couverts :
Entrées, unités, horloges, fraîcheur, données absentes :
Sortie exacte du modèle :
Décision finale et composant qui l'applique :
Actions interdites / contraintes dures :
```

```
Méthode(s) de référence et seuil minimal de gain :
Scénarios indépendants et métriques :
Latence, calcul, mémoire, énergie et SWaP-C :
Détection OOD, abstention et repli :
Déploiement témoin, retour arrière et reprise :
Artefacts, empreintes, vérificateur et attestation :
Limites de la revendication :
Décision de passage au niveau d'autorité suivant :
```

Si trois champs restent vagues — décision finale, repli, preuve — le cas d'usage n'est pas prêt à recevoir une autorité, même si le modèle obtient un excellent score hors ligne.

---

13 bis. Banc d'essai synthétique — six familles et leurs résultats négatifs

Les résultats suivants proviennent d'un banc **synthétique exécuté hors ligne** par l'auteur. Son code ne fait pas partie du dépôt compagnon. Ce banc confronte plusieurs méthodes de référence sur cinq graines fixes et conserve les données, la dispersion, les intervalles de confiance, les contraintes violées, la latence de référence et les résultats défavorables. Un manifeste SHA-256 relie le code, les schémas, les données et les sorties de cette expérience.

Famille	Résultat de référence	Résultat négatif conservé
prévision de trafic	persistance MAE `4,772` Mbit/s ; ridge à retards `6,149`	après rupture de régime, ridge est moins bon sur 5/5 graines
détection d'anomalies	F1 moyenne/écart-type `0,518` ; médiane/MAD `0,430`	les seuils statiques manquent 52/60 et 58/60 points de dérive lente
routage contraint	liens violés moyens `4,300 → 3,000`	la surcharge agrégée augmente `30,450 → 31,350` et le chemin s'allonge
dérive/OOD et repli	sous-provisionnement OOD `1,000 → 0,000`	la couverture d'automatisation tombe à `0,500`
télémétrie empoisonnée	erreur de moyenne attaquée `11,560` ; moyenne tronquée `1,233`	les attaques adaptatives ou majoritaires restent hors fixture
configuration générée	déploiements dangereux `5,000 → 0,000`	l'acceptation sûre tombe à `0,667` par rejet fermé d'un préfixe inconnu

Les six familles conservent leurs résultats défavorables et leurs limites. Les règles de vérification de cette édition sont réunies dans le dossier de référence.

La latence mesurée décrit une exécution Python complète sur la machine de référence ; ce n'est ni une latence par paquet, ni un WCET, ni une garantie de temps réel. Promouvoir l'un de ces exemples vers une autorité réseau exige encore données externes, méthode de référence opérationnelle, revue adversariale, observation sans action, déploiement témoin, mesure indépendante et retour arrière testé.

---

14. À retenir

1. **Partir du service, pas du modèle.** L'IA est une manière de produire une estimation ou une décision, jamais la définition du besoin.
2. **Nommer la sortie exacte.** Fraction, coefficient d'arête, score de chemin, route, configuration et paquet mesuré ne sont pas interchangeables.
3. **Séparer contrainte et récompense.** Ce qui est interdit reste impossible même si le modèle préfère l'enfreindre.
4. **Comparer à une référence évaluée dans les mêmes conditions.** Une heuristique simple peut être le meilleur résultat d'ingénierie.
5. **Changer de monde pour valider.** Une formule ne valide pas le simulateur ; une simulation ne valide pas le noyau ; un veth ne valide pas la radio.

6. **Proportionner l'autorité.** Observation, recommandation, simulation, action bornée et vérification sont des étapes, pas une formalité.
7. **Conserver le repli indépendant.** Il doit fonctionner lorsque le modèle, son service ou sa télémétrie sont indisponibles.
8. **Publier les limites avec les résultats.** Un statut `unsupported` explicite signale une limite qu'une métrique silencieusement absente peut masquer.

La révision référencée implémente niveaux analytiques, exécutions appariées, producteur d'émulation Linux et vérificateur ; chaque mesure exige cependant sa propre exécution privilégiée concluante. Les daemons avec captures de contrôle et les bancs RF restent hors contrat. **La maturité d'un cas d'usage est celle de sa preuve, de son autorité et de son repli — pas celle de son algorithme seul.**

Point de sortie. Si vous savez nommer la décision attendue, une référence et la limite du résultat, vous pouvez passer à un laboratoire ou reprendre le fil du livre. La carte de recherche, les normes et le dossier qui suivent sont des appuis à consulter lorsque vous voulez approfondir ou vérifier une affirmation.

15. Carte de la recherche IA et réseaux

Une architecture crédible ne peut pas reposer uniquement sur des concepts IA généraux et des standards. Les travaux ci-dessous donnent douze points d'appui propres aux réseaux. La sélection est **raisonnée mais non systématique** : elle ne constitue ni une méta-analyse, ni un classement, ni la preuve qu'une méthode est prête pour la production. Pour chaque contribution, le livre conserve la frontière de ce qu'elle ne démontre pas. Les liens ci-dessous conduisent aux publications ; la bibliographie propose d'autres points d'entrée. Le banc d'essai synthétique présente les comparaisons qui prolongent certaines de ces questions.

Prévision et anomalies

- Azzouni et Pujolle, *NeuTM*, 2018 formulent la prévision de matrices de trafic par réseau récurrent pour le SDN. Une seule trace historique GÉANT ne prouve cependant ni incertitude calibrée, ni résistance à la dérive, ni sûreté en boucle fermée.
- Mirsky et al., *Kitsune*, 2018 montrent un détecteur en ligne non supervisé pour passerelle contrainte. Les bancs d'essai et l'apprentissage supposé bénin ne couvrent pas un adversaire adaptatif, l'empoisonnement ou les régimes modernes de dérive.

Ces limites motivent les familles reproductibles `traffic_forecasting`, `anomaly_detection` et `telemetry_poisoning_robustness`, qui conservent explicitement rupture de régime, dérive lente, faux négatifs et télémétrie attaquée.

Routage, ingénierie de trafic et modèle de réseau

- Valadarsky et al., *Learning to Route*, 2017 posent tôt le routage adaptatif comme problème de RL profond. L'échelle d'un workshop et des mondes simplifiés laissent ouvertes généralisation de topologie, contraintes impératives et sûreté de déploiement.
- Rusek et al., *RouteNet*, 2020 emploient un GNN pour relier topologie, routage, trafic, délai, gigue et perte, puis optimiser. Une forte généralisation expérimentale ne garantit pas la robustesse à une télémétrie fautive, au décalage de distribution ou à l'exploitation du modèle.

La famille `constrained_traffic_engineering` garde donc la faisabilité hors du modèle et publie son résultat défavorable : réduire le nombre de liens violés peut augmenter la surcharge totale et la longueur des chemins.

Contrôle de congestion

- Jay et al., *A Deep Reinforcement Learning Perspective on Internet Congestion Control*, 2019 rendent le problème Aurora et son évaluation sur émulateur concrets. Un émulateur n'est pas Internet ; équité, sûreté et généralisation demeurent des obstacles de déploiement.

Le traité en déduit une règle : comparer l'effet de service et changer de substrat avant d'augmenter l'autorité, plutôt que promouvoir une récompense obtenue dans le même monde que l'entraînement.

RAN et allocation radio

- Bonati et al., *Intelligence and Learning in O-RAN*, 2021 relient ML, RIC/xApps et ordonnancement de slices sur Colosseum. Une intégration sur émulateur n'est pas une preuve opérateur, et les interfaces O-RAN ont évolué depuis.
- Liang, Ye et Li, *Spectrum Sharing in Vehicular Networks*, 2019 traitent spectre et puissance par MARL distribué sous canaux rapides. Le succès d'une récompense simulée ne démontre ni stabilité, ni coexistence sûre, ni robustesse aux décalages radio et trafic réels.

La cartographie inter-SDO sépare en conséquence les rôles SMO, RIC, A1, R1 et E2, tandis qu'un banc RF et des mesures indépendantes restent nécessaires pour toute revendication radio.

Automatisation et frontière de preuve

- Mestres et al., *Knowledge-Defined Networking*, 2017 joignent SDN, télémétrie, analyse de données et IA dans un plan de connaissance. Ce paradigme fondateur et sa faisabilité initiale ne constituent pas une assurance d'autonomie en production.
- Fu et al., *On the Use of ML for Blackbox System Performance Prediction*, 2021 fournissent un contre-exemple essentiel sur treize applications : les prédicteurs boîte noire ne sont pas automatiquement simples ni universels. Cette borne ne réfute toutefois pas toute inférence réseau ou toute boucle fermée.

L'ouvrage conserve donc les résultats négatifs, les méthodes de référence simples, l'abstention et la distinction entre score modèle et preuve de service.

Réseaux pour l'IA distribuée

- Li et al., *Scaling Distributed Machine Learning with the Parameter Server*, 2014 établissent une architecture de communication et de cohérence pour l'état ML distribué. Staleness et points chauds restent des compromis, et les charges modernes d'accélérateurs doivent être remesurées.
- Zhang et al., *Poseidon*, 2017 réduisent le coût de synchronisation par chevauchement sensible aux couches et stratégies hybrides. Les CNN, frameworks, 10 GbE et petites grappes de 2017 ne permettent pas d'extrapoler directement aux charges actuelles.
- Sapio et al., *SwitchML*, 2021 agrègent des mises à jour dans un commutateur programmable. Les ressources du fabric, l'arithmétique fixe, le synchronisme et le partage contraint ne conviennent pas à toute topologie d'entraînement.

Ces travaux donnent son contenu concret au domaine **Networks for AI** : placement, débit utile, cohérence, stragglers, agrégation in-network, limites numériques et repli hors fabric. Une citation soutient une proposition précise ; elle ne remplace ni référence locale, ni données indépendantes, ni contraintes de service, ni validation sur le substrat revendiqué.

---

16. Repères normatifs et sources de prolongement

Cinq livres pour approfondir l'IA

Ces cinq ouvrages, en anglais, prolongent les explications du livre. Pour commencer, les deux premiers donnent une vue d'ensemble et des méthodes accessibles ; les suivants permettent de pratiquer et d'approfondir. Les versions de lecture sont proposées en accès gratuit par les auteurs ou l'éditeur.

- **Artificial Intelligence: Foundations of Computational Agents** — David L. Poole et Alan K. Mackworth (3e édition, 2023). Une vue d'ensemble des agents, du raisonnement, des probabilités et de l'apprentissage. texte intégral (Poole et Mackworth 2023).
- **An Introduction to Statistical Learning: with Applications in Python** — Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani et Jonathan Taylor (2023). Pour comprendre régression, classification et validation, puis les essayer en Python. site des auteurs et PDF (James et al. 2023).
- **Dive into Deep Learning** — Aston Zhang, Zachary C. Lipton, Mu Li et Alexander J. Smola (2023). Des explications, des figures et des notebooks pour apprendre en exécutant le code. livre et notebooks (Zhang et al. 2023).
- **Deep Learning** — Ian Goodfellow, Yoshua Bengio et Aaron Courville (2016). Pour approfondir les réseaux de neurones, leurs bases mathématiques et leur entraînement. texte intégral (Goodfellow et al. 2016a).
- **Reinforcement Learning: An Introduction** — Richard S. Sutton et Andrew G. Barto (2e édition, 2018). Pour étudier comment un agent apprend de ses actions, avec récompenses, valeurs et politiques. édition et accès ouvert (Sutton et Barto 2018).

Normes et ressources réseau

Les liens officiels suivants ont été contrôlés le **30 juillet 2026**. Ils complètent le manuscrit ; ils ne remplacent ni les exigences applicables à un système précis, ni le gel des versions dans le manifeste d'un déploiement.

- ITU-T Y.3172 — Architectural framework for machine learning in future networks including IMT-2020 — chaîne ML, gestion et orchestration des fonctions apprises.
- ITU-T Y.3174 — Framework for data handling to enable machine learning in future networks including IMT-2020 — collecte, préparation, stockage et application des sorties ML.
- ITU-T Y.3181 — Architectural framework for machine learning sandbox — exigences, composants, points de référence et API de haut niveau du bac à sable.
- 3GPP TS 23.288 — Architecture enhancements for 5GS to support network data analytics services — fonctions et services d'analyse des données du cœur 5G ; vérifier la version visée.
- 3GPP TS 29.520 — 5G System; Network Data Analytics Services; Stage 3 — API `Nnwdaf` et réalisation de stade 3.
- 3GPP TS 28.104 — Management Data Analytics et TS 28.105 — AI/ML management — MDAF et gestion du cycle AI/ML dans le système de management.
- O-RAN ALLIANCE — spécifications et bulletin du 17 juillet 2026 — versions contrôlées de l'architecture, du SMO, de R1 et des E2SM KPM, RC et CCC ; bulletin du 27 février 2026 pour A1 et les versions non remplacées ; bulletin du 10 novembre 2025 pour E2AP et le train précédent.
- O-RAN NGRG — *Native AI Architecture Description* v1.2 et évolution du Near-RT RIC vers la 6G — rapports de recherche officiels, à distinguer des spécifications O-RAN.
- ETSI GS ZSM 009-1 — Closed-Loop Automation, Part 1: Enablers — gouvernance et coordination des boucles fermées.
- ETSI GS ZSM 002 — Reference Architecture et ETSI GR ZSM 015 — Network Digital Twin — distinguer la spécification de groupe `GS` du rapport de groupe `GR`.
- ETSI GS ENI 005 — ENI System Architecture et page officielle ISG ENI — architecture, politiques, connaissance et automatisation réseau.

- RFC 9315 — Intent-Based Networking: Concepts and Definitions — RFC IRTF informatif ; distinction entre intention, politique, réalisation et assurance.
- RFC 9232 — Network Telemetry Framework — RFC IETF informatif ; RFC 8639 et RFC 8641 — Proposed Standards pour abonnements YANG et YANG-Push.
- NIST AI Risk Management Framework 1.0 et NIST AI 100-2 E2025 — Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations — gouvernance du risque, évaluation, attaques et limites des mitigations.
- Network Digital Twin: Concepts and Reference Architecture, version 13 — Internet-Draft IRTF daté du 1er juillet 2026, travail en cours à ne pas citer comme RFC ni comme norme publiée.

Le cas de routage et son échelle de preuve montrent comment relier une décision d'allocation, une exécution et les limites de ce qu'elle permet d'affirmer.

---

16 bis. Dossier de référence, revue et correction

Pour contrôler une affirmation, partez du résultat et remontez à ses conditions :

- la carte de recherche relie douze travaux à leur contribution, à leurs limites et aux questions qu'ils laissent ouvertes ;
- les repères normatifs distinguent versions, statuts et portée des textes cités ;
- le banc synthétique expose les méthodes de référence, les résultats défavorables et les limites des six comparaisons ;
- le protocole de validation précise les preuves à réunir avant d'accorder une autorité d'action.

Une correction argumentée indique le passage concerné, ses hypothèses, le calcul ou l'expérience qui le contredit et la portée de la conclusion proposée. Une relecture indépendante confronte ces éléments ; un statut déclaré ne remplace pas l'examen de la preuve.

Discuter et prolonger les comparaisons

Les comparaisons précédentes reposent sur six précautions : dédoublonner avant de séparer les données, comparer à un solveur de référence, rejeter une décision tardive, éprouver des entrées périmées, empoisonnées ou hors distribution, commencer sans autorité d'action et essayer l'échec du repli. Les résultats défavorables et les limites de chaque méthode font partie de l'analyse.

Le tableau des six familles permet de discuter les résultats dans leurs conditions d'origine. Prolonger une comparaison demande de choisir les données, les hypothèses et les critères de vérification du nouveau montage. Sa généralisation à un réseau réel demande d'autres observations et une revue indépendante.

Étude de cas — Comprendre `sb3_trainer_project`

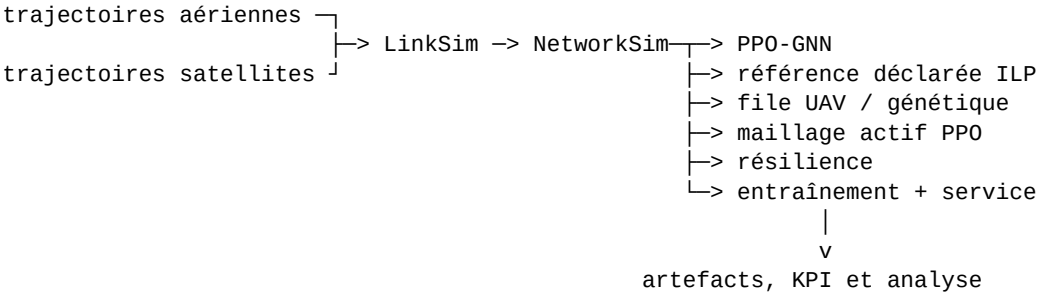
1. Statut et question directrice

> **Comment choisir et évaluer des chemins dans un réseau mobile, capacitaire, partiellement observable et soumis à des demandes concurrentes, sans confondre qualité du modèle, qualité des données et qualité du service ?**

Scénario de raisonnement. Imaginons deux chemins vers le même destinataire. Le premier a moins de relais, mais sa file est chargée. Le second est plus long et paraît disponible ; cette observation a toutefois quelques secondes d'âge. Lequel choisir, que faut-il vérifier et comment savoir si la décision a réellement amélioré la livraison ? Ce scénario est hypothétique : il introduit la question, sans annoncer un résultat du projet.

Le système ne demande pas simplement à une IA de « trouver une route ». Il construit une chaîne complète :

1. produire des trajectoires aériennes et satellitaires ;
2. calculer les possibilités de liaison ;
3. construire un graphe temporel et des demandes ;
4. entraîner ou exécuter plusieurs politiques ;
5. calculer une référence d'optimisation dont la provenance doit être vérifiée ;
6. exporter décisions, modèles, métriques et preuves ;
7. comparer les résultats dans des vues et notebooks.



Une trajectoire n'est pas une liaison ; une liaison n'est pas une route ; une route calculée n'est pas une garantie de livraison ; une récompense élevée n'est pas une preuve de sûreté.

Pour poursuivre la lecture. La section suivante présente les cinq applications du monde d'essai ; les deux encadrés ci-dessous précisent identifiants et responsabilités. Examiner une exécution détaillée suppose l'accès au dépôt privé. Les laboratoires L20 et L21 offrent une première expérience sur la décision et sa comparaison.

Repères de consultation — Identifiants et compatibilité

> `sb3_trainer_project` est l'identifiant d'installation historique d'une application privée AGILAB aujourd'hui présentée à l'opérateur comme **Routing Training Project**. Le nom historique reste dans les paquets, chemins et artefacts pour compatibilité. Cette étude décrit l'implémentation observée pour l'édition 1.0 ; elle n'en fait ni une API publique stable ni une preuve de performance opérationnelle.
>
> **Deux surfaces, une même implémentation.**
> `routing_training_project` est la surface opérateur canonique : elle emploie notamment le module `fcas_routing_ppo` et écrit par défaut sous `routing_training/pipeline`. `sb3_trainer_project` conserve l'implémentation sous-jacente, les identifiants historiques et le DAG multi-application de dix-sept étapes, avec des sorties sous `sb3_trainer/pipeline`. Le premier réutilise le module de calcul du second ; leurs noms de dossiers ne sont donc pas interchangeables. Le DAG décrit ci-dessous est celui de `apps/sb3_trainer_project/lab_stages.toml`.
>
> **Repère avant de commencer.**
> La Passerelle — Science des données et ingénierie réseau définit les notions communes. Dans cette étude,

SB3 signifie **Stable-Baselines3**, bibliothèque d'apprentissage par renforcement ; PPO est une méthode d'apprentissage ; GNN un réseau neuronal de graphe ; ILP une optimisation linéaire en nombres entiers ; DAG un graphe orienté sans cycle ; KPI un indicateur clé de performance ; MLflow un outil de suivi d'expériences ; MARL un apprentissage par renforcement à plusieurs agents.

Repère d'ingénierie — Agent de codage, Tokki et AGILAB

Un projet de cette taille produit deux volumes difficiles à suivre : beaucoup de code et de journaux techniques, puis beaucoup d'expériences et d'artefacts. Un agent peut accélérer le travail, mais cette vitesse ne vaut que si les responsabilités restent séparées.

Élément	Responsabilité principale	Ce qu'il ne prouve pas
agent de codage (*coding agent*)	lire le dépôt, raisonner, proposer ou appliquer un ensemble de différences (*diff*), écrire documentation et tests, lancer des outils autorisés	que son intention est correcte, que ses tests sont suffisants ou que le résultat peut être déployé
Tokki	encadrer le passage aux outils selon les règles configurées, exécuter les opérations déterministes admises et conserver un contexte compact relié aux preuves	que la commande choisie était la bonne, qu'un test réussi suffit ou que l'expérience est scientifiquement valide
AGILAB	structurer et exécuter le flux de travail, conserver paramètres, artefacts et manifeste d'exécution (*run manifest*), puis faciliter analyse, rejeu (*replay*) et transfert vers MLflow (*handoff*)	que le simulateur représente tout le réel, que la récompense est saine ou que le modèle est sûr
ingénieur ou relecteur	fixer but, contraintes et autorité ; vérifier différence, preuves, référence et comportement dégradé ; décider de la promotion	qu'une décision passée restera correcte après dérive du contexte

Une **session agentique** (*agentic session*) est une session de travail dans laquelle un agent logiciel lit du contexte, appelle des outils et propose ou applique des changements. Tokki encadre ici le passage aux outils et conserve un contexte relié aux preuves ; il n'est ni le modèle qui raisonne ni la source de vérité des validations AGILAB. Par exemple, `tokki run -- <commande>` peut exécuter un test long, en résumer la sortie et conserver un accès à la sortie complète. Ce reçu évite de recopier tout le journal ; le verdict appartient au test, aux contrôles locaux explicitement configurés par le dépôt — scripts déclenchés automatiquement (*hooks*), validateurs ou gardes de politique — et à la relecture de leurs résultats.

AGILAB couvre un autre niveau. Le DAG, les environnements contrôlés, les artefacts, `run_manifest.json` et l'intégration optionnelle à MLflow documentent et facilitent le rejeu et la comparaison. Ils ne capturent automatiquement ni toute dépendance externe ni tout état du monde. Lorsqu'une action d'agent doit laisser des éléments de preuve dédiés, `agilab agent-run` peut aussi produire un manifeste expurgé et une trace d'événements (*event trace*). Ce mécanisme documente l'exécution ; ce n'est ni un bac à sable de processus ni une garantie que l'action était autorisée ou correcte.

La boucle complète peut se lire ainsi :

```
intention, contraintes et métriques définies par l'humain
 -> agent : différences de code, documentation et tests
 -> Tokki : opérations encadrées et preuves récupérables
 -> contrôles du dépôt : tests, hooks et validateurs de politique
 -> AGILAB : flux de travail, données, graine, artefacts, KPI et manifeste
 -> comparaison à la référence et relecture indépendante
 -> décision humaine de validation, promotion ou déploiement
```

Deux preuves ne doivent donc pas se remplacer :

1. la **preuve logicielle** dit quel code a changé, quels tests ont été exécutés, quelle révision a été relue et avec quelle provenance ;

2. la **preuve expérimentale** dit quelles données, versions, graines, paramètres et étapes ont produit quels KPI et artefacts.

Un test réussi (*green test*) ne prouve pas que PPO améliore le service réel. Une récompense élevée ne prouve pas que le code est maintenable, ni que l'action respecte les contraintes opérationnelles. Les deux chaînes doivent se rejoindre sans se valider circulairement.

La **vélocité d'un agent** est donc le rythme de changements **utiles et validés**, pas sa vitesse d'écriture. L'agent peut coder, mais aussi auditer, planifier, reproduire une panne, documenter, construire un test ou analyser des preuves. Il ne reçoit pas pour autant l'autorité de fusionner ou déployer.

Généralisation. Cette séparation ne dépend pas de ces produits. Tokki illustre un contrôle local des opérations, avec un contexte et des preuves récupérables ; AGILAB, un atelier logiciel (*workbench*) d'expérimentation et de collecte de preuves ; l'agent peut être remplacé par un autre outil. Les commandes citées décrivent l'implémentation observée pour cette édition et doivent être revérifiées dans l'aide courante.

2. Les cinq applications de la chaîne

Flight Trajectory : les positions aériennes

Cette application produit les positions des aéronefs. Elles disent où se trouvent les nœuds, pas s'ils peuvent communiquer.

Satellite Trajectory : les positions satellitaires

Cette application produit les positions satellitaires et introduit une autre échelle de distance, de délai et de visibilité.

LinkSim : les hypothèses de liaison

Cette application combine les trajectoires avec des hypothèses de liaison. Ses sorties décrivent capacité, latence et éléments de provenance. Elles restent des résultats de modèle, pas des mesures radio ni des captures de paquets.

NetworkSim : le graphe et les demandes

Cette application transforme les liens en graphe temporel et en demandes. Il peut aussi produire la vue de topologie reçue par un nœud observateur. Il faut donc distinguer la topologie globale connue du simulateur et la topologie partiellement observée disponible au décideur.

Routing Training Project : décider et comparer

Cette application consomme graphe, demandes et preuves. Il entraîne une politique, exécute une inférence, calcule une référence ou produit une comparaison. Il est le lieu de décision et d'évaluation, pas le simulateur physique amont.

3. Le DAG réel : 17 étapes et 7 vagues

`lab_stages.toml` déclare dix-sept étapes. Leurs dépendances forment sept vagues :

Vague	Étapes	Sens
1	cinq installations	préparer les modules de calcul Flight, Satellite, Link, Network et Routing
2	`flight_pipeline`, `sat_pipeline`	produire les trajectoires en parallèle
3	`link_pipeline`	calculer les liens depuis les deux familles de trajectoires
4	`network_pipeline`	produire graphe, demandes et preuve d'observation

Vague	Étapes	Sens
5	six branches	PPO-GNN, génétique, maillage actif, ILP, MLflow et résilience
6	`relay_queue_service`	vérifier le contrat train-then-serve
7	`kpi_summary`	agréger les résultats et la preuve finale

Une dépendance exprime un ordre et un contrat d'artefact. Elle ne prouve pas que le contenu est scientifiquement valide : il faut aussi schéma, provenance, contrôles de cohérence et comparaison.

4. Où vivent les données

Les applications partagent une racine de travail puis possèdent chacune leur sous-répertoire :

```
<share>/
├── flight_trajectory/pipeline/
├── sat_trajectory/pipeline/Trajectory/
├── link_sim/pipeline/
├── network_sim/pipeline/
│   ├── ilp_topology.gml
│   ├── ilp_demands.json
│   ├── topology_observer_evidence.json
│   └── topology_summary.json
└── sb3_trainer/pipeline/
    ├── trainer_fcas_routing_ppo_gnn/
    ├── trainer_fcas_routing_ilp/
    ├── trainer_uav_relay_queue_genetic/
    ├── trainer_uav_active_mesh_ppo/
    ├── trainer_uav_relay_queue_ppo_mlflow/
    ├── trainer_uav_relay_queue_resilience/
    └── trainer_uav_relay_queue_service/
```

`data_in` désigne ce qui est lu et `data_out` ce qui est produit. `reset_target=true` impose de recréer la sortie courante afin qu'un ancien résultat ne soit pas confondu avec une nouvelle réussite. Une reproduction doit conserver configuration, graine aléatoire (*seed*), entrées, version du code et artefacts.

5. Transformer le routage en environnement Gymnasium

Gymnasium est une interface standard entre un simulateur et un algorithme. Elle formalise ce que l'agent observe, ce qu'il peut faire, comment le monde évolue, quelle récompense revient et quand un épisode se termine.

Un algorithme de renforcement exige sept contrats :

Contrat	Question réseau
état du simulateur	quel monde l'environnement fait-il évoluer, y compris ce que l'agent ne voit pas ?
observation	que sait réellement le décideur maintenant ?
action brute	quelle sortie la politique peut-elle proposer ?
réalisation	comment cette sortie devient-elle une route, une allocation ou une action appliquée ?
transition et pas de temps	comment et pendant quelle durée le réseau évolue-t-il ?
récompense et coûts	quel compromis guide l'apprentissage et quelles limites restent séparées ?
fin d'épisode	quelle fin est naturelle (*terminated*) et quelle limite externe tronque l'essai (*truncated*) ?

Dans la configuration canonique, le module `fcas_routing_ppo` utilise le mode d'action `edge` et l'architecture de politique `gnn`. L'observation sépare les demandes, les arêtes, les couples source–destination et le masque des demandes actives. Le GNN agrège ces éléments en un embedding global ; la tête de politique entraînée par PPO produit ensuite une valeur pour chaque position de la liste ordonnée d'arêtes. PPO est la méthode d'apprentissage, pas l'embedding. La politique ne conserve pas une représentation séparée de chaque arête jusqu'à l'action, ne choisit pas directement un chemin et ne crée aucune liaison.

À chaque pas, l'environnement :

1. construit le graphe visible à cet instant ;
2. énumère au plus `max_paths` chemins candidats par demande ;
3. les classe selon `capacity`, `wcett` ou `airtime` ;
4. applique les coefficients produits par la politique aux capacités effectives ;
5. réalise le partage de capacité et la contention ;
6. calcule débit livré, latence et récompense.

Une arête absente de la vue ne peut pas être inventée et certaines preuves manquantes provoquent un échec explicite. En revanche, dans le `TimeRoutingEnv` actuel, dépasser `max_latency` produit une pénalité de récompense : ce n'est pas un masque dur qui interdit automatiquement le chemin. Une contrainte de sûreté qui doit être inviolable doit donc être appliquée et testée séparément de la récompense.

> Stable-Baselines3 fournit les algorithmes d'apprentissage ; l'application fournit le monde, les observations, les actions, la récompense et les preuves.

Une récompense est une spécification compacte. Si elle oublie fraîcheur, équité ou classe critique, l'agent peut optimiser très efficacement la mauvaise propriété.

6. Trois familles d'environnements

`TimeRoutingEnv` traite des demandes variables sur un graphe mobile. Il sert au routage PPO-GNN et aux comparaisons de métriques, avec vue globale ou vue partielle d'un nœud de décision.

`UavQueueRoutingEnv` modélise une source, une destination et plusieurs relais dont les files, capacités, charges de fond et délais diffèrent. L'action choisit un relais ou une combinaison. Un relais géométriquement favorable peut être le mauvais choix s'il est saturé.

`UavActiveMeshEnv` autorise le déplacement des relais. Chaque relais peut rester immobile ou se déplacer selon quatre directions. La récompense combine livraison, latence, mouvement et séparation. Le problème devient conjoint : où placer le réseau et comment y router. L'implémentation emploie un contrôleur PPO centralisé ; elle ne prouve pas un apprentissage par renforcement multi-agent décentralisé général, souvent abrégé **MARL**.

Ici, « contrôleur PPO centralisé » désigne la politique de décision du simulateur, non un contrôleur SDN ni un orchestrateur opérationnel.

7. PPO, GNN, génétique et ILP

PPO (*Proximal Policy Optimization*) apprend ici, par interaction avec le simulateur, une politique de pondération des arêtes. Après entraînement, l'inférence est rapide, mais elle n'apporte ni preuve d'optimalité ni garantie hors du domaine évalué.

Le mot « proximal » décrit la **mise à jour de la politique**, pas la distance physique des nœuds. La boucle d'entraînement se lit ainsi :

```
1. exécuter la politique courante et collecter un lot d'interactions ;
2. faire estimer par le critique la valeur des observations ;
3. combiner récompenses et valeurs pour estimer les avantages, souvent par GAE ;
4. augmenter la probabilité des actions à avantage positif et diminuer les autres ;
```

```
5. écrêter dans l’objectif le ratio nouvelle / ancienne probabilité ;
6. réutiliser ce lot d’interactions quelques époques, puis collecter des données récentes.
```

L’écrêtage (*clipping*) décourage une modification trop brutale de la probabilité des actions dans l’objectif d’optimisation. Il ne borne ni la valeur de l’action, ni son effet sur le réseau, et ne garantit pas à lui seul une amélioration monotone. Masque d’actions, contraintes de capacité et barrières de sûreté restent dans l’environnement ou dans un mécanisme indépendant.

Approfondissement — Lire la configuration PPO. Pour une première lecture, retenir `gamma`, le volume d’interactions collecté avant une mise à jour et `clip_range`. Les autres paramètres deviennent nécessaires pour reproduire ou auditer précisément l’entraînement :

Paramètre PPO	Question à poser
`gamma`	jusqu’où le futur compte-t-il, compte tenu de la durée d’un pas ?
`gae_lambda`	quel compromis biais–variance utilise l’estimation d’avantage ?
`n_steps × n_envs`	combien d’interactions alimentent une collecte avant mise à jour ?
`batch_size`	quelle portion de cette collecte traite une mise à jour ?
`n_epochs`	combien de fois réutiliser les mêmes données récentes ?
`clip_range`	quelle variation du ratio de probabilité est décourageé ?
`ent_coef`	quelle pression conserve de la diversité dans les actions ?
`learning_rate`	à quelle vitesse les paramètres changent-ils ?

Augmenter `n_epochs` ne crée pas de nouvelles expériences ; cela réutilise les mêmes. Augmenter `gamma` ne donne pas davantage de mémoire à l’observation ; cela modifie le poids du futur dans le retour. Chaque valeur se lit donc avec la durée d’un pas, la taille du lot d’interactions et le domaine du scénario.

Dans la configuration `policy_arch="gnn"`, le **GNN** (*Graph Neural Network*, réseau neuronal de graphe) est l’**extracteur de caractéristiques** (*feature extractor*) de la politique PPO. L’implémentation encode les variables de nœud et d’arête, propage des messages, puis construit des représentations de demandes. Elle effectue ensuite un *pooling* moyen — une agrégation — des demandes et des nœuds concernés pour produire **un embedding global du graphe**. La tête de politique transforme ce résumé en valeurs associées à des positions d’arêtes ordonnées à l’avance.

```
observation du graphe
 -> GNN : embedding
 -> acteur : distribution d’actions
 -> environnement : coefficients réalisés en routage et service
 -> critique : valeur attendue pour aider l’entraînement
 -> PPO : mise à jour guidée par l’avantage et le clipping
```

À l’inférence, l’acteur produit l’action. Le critique sert principalement à l’apprentissage ; il ne valide ni la route ni ses contraintes.

Elle ne conserve donc pas jusqu’à l’action un embedding contextualisé distinct pour chaque arête. Le pooling peut fusionner des états différents. Cette architecture ne garantit à elle seule ni une explication par arête, ni l’invariance à une renumérotation, ni le transfert vers une taille ou une topologie différente. Ces propriétés doivent être testées. Le GNN n’est ni un second ordonnanceur ni un générateur autonome de routes.

La surface canonique nomme ce module `fcas_routing_ppo`. L’identifiant `fcas_routing_ppo_gnn` et le dossier `trainer_fcas_routing_ppo_gnn` subsistent dans la surface de compatibilité.

La branche génétique explore une population de solutions pour les scénarios de file UAV. L’absence de réseau neuronal n’en fait pas une vérité optimale.

La branche `fcas_routing_ilp` cherche à construire une référence MILP (*Mixed-Integer Linear Programming*, optimisation linéaire mixte en nombres entiers) avec PuLP/CBC et des contraintes explicites de capacité, conservation des flux et latence. Un résultat ne peut toutefois être appelé « optimum ILP » que si le solveur réel a traité la même instance et a confirmé le statut `Optimal`.

L'implémentation observée pour cette édition n'exporte pas dans `allocations_steps.json` le moteur d'exécution, le statut du solveur, la valeur de l'objectif ni un éventuel **écart d'optimalité** (*optimality gap*). De plus, un statut non optimal dans `ilp_worker`, ou certaines erreurs de préparation ou de résolution dans le module, peuvent déclencher un **allocateur glouton** (*greedy allocator*) de repli. L'absence du paquet `ilp_worker`, elle, provoque un échec explicite.

Le dossier `trainer_fcas_routing_ilp` ne suffit pas à établir une référence optimale. Sans provenance enregistrée du solveur, parler de **référence déclarée ILP, provenance à vérifier**. La comparaison exige ensuite les mêmes entrées, horizon, contraintes, unités et définitions de KPI.

8. Capacité, WCETT et airtime

Le mode `capacity` favorise les options de forte capacité effective. Deux chemins de capacité similaire peuvent pourtant consommer différemment une ressource radio partagée.

Trois métriques s'emboîtent :

- **ETX** (*Expected Transmission Count*) estime le nombre attendu de transmissions nécessaires pour livrer un paquet sur un lien ;
- **ETT** (*Expected Transmission Time*) convertit ce nombre en temps en tenant compte de la taille du paquet et de la capacité ;
- **WCETT** (*Weighted Cumulative Expected Transmission Time*) cumule ces temps et pénalise la réutilisation d'un même domaine radio le long du chemin.

Formellement :

$$ETT_i = ETX_i \frac{L \times 8}{10^6 C_i}$$

$$WCETT(P) = (1-\beta)\sum_{i \in P} ETT_i + \beta \max_d \sum_{i \in P, domain(i)=d} ETT_i$$

Dans ces formules :

- P est le chemin ;
- i désigne l'un de ses liens ;
- ETX_i est le nombre attendu de transmissions sur ce lien ;
- C_i est sa capacité en Mbit/s ;
- L est la taille du paquet en octets ;
- d désigne un domaine d'interférence ;
- $domain(i)$ indique le domaine auquel appartient le lien ;
- β, compris entre 0 et 1, règle l'importance donnée à la pénalité de réutilisation radio.

En langage courant : un chemin peut comporter des liens rapides et pourtant être mauvais si plusieurs de ces liens doivent parler chacun leur tour sur la même ressource radio. WCETT exige donc ETX, identité de lien et domaine d'interférence traçables.

Le mode `airtime` estime le temps de service par domaine puis le majore avec la fraction occupée au pas précédent. C'est une charge allouée modélisée, pas une mesure matérielle de channel busy time.

Les candidats sont bornés par `max_paths`. Le meilleur candidat retenu n'est donc pas nécessairement l'optimum parmi tous les chemins simples.

9. Topologie globale et observation partielle

Le mode `observed_partial` exige le chemin de la preuve, le nœud où la décision est prise et l'âge maximal admissible. Chaque annonce conserve origine, numéro de séquence, réception et expiration. Pour une origine, seule la séquence la plus récente reçue et encore valide peut être utilisée ; une ancienne séquence ne ressuscite pas après expiration de la plus récente.

Une observation partielle n'est pas seulement un graphe plus petit. Deux états réels différents peuvent produire la même observation locale : un lien absent de la vue peut être réellement coupé, jamais découvert, ou seulement décrit par une annonce retardée ou expirée. La politique ne peut donc pas supposer que « non observé » signifie « inexistant ».

Si une métrique avancée demande une capacité, un ETX, une identité de lien ou un domaine absent, l'exécution échoue explicitement. Elle ne complète pas la vue locale avec le graphe global et ne remplace pas WCETT par `capacity`.

Ce comportement applique trois idées du livre : une information possède un âge, l'absence de preuve est ambiguë, et une décision reproductible conserve la provenance de ce qu'elle a vu.

10. Contentions, services et débit réellement livré

Une demande peut porter service, priorité et latence maximale. Après sélection du chemin, la capacité n'est pas dupliquée. Lorsque plusieurs demandes partagent arêtes ou domaines, les allocations sont réduites. Le débit livré dépend du goulot du chemin et du facteur de contention le plus contraignant.

Il faut lire séparément chemin demandé, chemin réalisable, débit demandé, débit livré, latence, échéance et cause d'une livraison partielle.

11. Entraîner, suivre, servir et tester les défaillances

L'entraînement PPO produit un modèle et, selon la configuration, des **points de sauvegarde** (*checkpoints*), des métriques et une **trajectoire d'évaluation** (*rollout*). Une trajectoire exportée décrit une suite observation–action–résultat simulé ; elle ne constitue pas à elle seule un jeu de test indépendant. L'**horizon** est le nombre de pas futurs ou enregistrés pris en compte.

La branche `trainer_uav_relay_queue_ppo_mlflow` est une démonstration de suivi MLflow appliquée au scénario de files UAV, et non le suivi automatique de toute la chaîne FCAS PPO-GNN. Elle tente d'enregistrer :

- les paramètres scalaires, sauf quelques clés explicitement exclues (`mlflow_*`, `tracking_backend` et `openai_api_key`) ;
- les métriques numériques de `summary_metrics.json` ;
- tout fichier existant dont le chemin apparaît comme valeur textuelle dans le résultat, même hors du répertoire de sortie, ainsi que les fichiers `.json`, `.csv`, `.parquet`, `.gml`, `.zip` et `.pt` placés directement — sans recherche dans les sous-répertoires — dans ce répertoire.

Cette exclusion ciblée n'est **pas** un gestionnaire de secrets : d'autres mots de passe, jetons ou identifiants sensibles dans les arguments ou artefacts peuvent être envoyés au serveur de suivi. N'y transmettre aucun secret et contrôler les artefacts avant journalisation.

Elle écrit aussi `mlflow_tracking.json`, de statut `logged`, `missing_dependency` ou `error`. Une erreur de suivi n'entraîne pas nécessairement l'échec de l'entraînement : vérifier ce statut avant d'affirmer que l'exécution a été enregistrée.

MLflow améliore la recherche et la comparaison des exécutions. Il ne capture pas automatiquement la version du code, le verrou de dépendances, les empreintes de toutes les entrées, la séparation train/test ni la provenance du solveur ILP ; il ne prouve donc ni reproductibilité complète ni validité scientifique.

La branche `train-then-serve` vérifie :

`modèle entraîné → artefact identifiable → chargement → inférence → santé`

Charger un modèle franchit une frontière de confiance : origine, empreinte, format, versions et compatibilité doivent être contrôlés.

La branche de résilience injecte une panne de relais et compare routage fixe et adaptation. Elle doit observer détection, décision, perte, file, échéances, oscillations et nouvel état stable. Une politique peut être meilleure après convergence et pire pendant la transition.

12. Artefacts et preuve

Les sorties `allocations_steps.json`, `.parquet` ou `.csv` peuvent conserver chemin, liens, domaines d'interférence, coût, débits demandé et livré, latence, priorité, récompense, politique topologique et provenance.

Une comparaison attestable conserve ensemble :

```
code + configuration + dépendances + graine
+ entrées et leurs empreintes
+ modèle et point de sauvegarde
+ décisions détaillées
+ KPI agrégés
+ manifestes et provenance
```

Un graphique seul n'est pas une preuve. Un fichier de décisions sans les entrées qui les ont causées n'est pas un rejeu.

13. Lire les résultats

Le cockpit donne un accès rapide aux politiques. Les vues répondent à des questions différentes :

- comparaison : livraison, latence et chemin ;
- carte : trajectoires, graphe et allocations ;
- entraînement : courbes, stabilité et points de sauvegarde ;
- santé des files : comportement d'un scénario ;
- résilience : comparaison de plusieurs exécutions ;
- notebooks : analyse reproductible et exportable.

Une vue ne doit pas recalculer silencieusement la vérité. Elle lit des artefacts, affiche leur provenance et rend visibles les données manquantes.

14. Ce que le projet ne prouve pas

Une exécution réussie ne prouve pas qu'une politique est optimale **hors échantillon** (*out of sample*), que LinkSim reproduit exactement une radio, que la preuve simulée est une capture réseau, qu'un KPI moyen protège les extrêmes, ni que le contrôleur centralisé peut être déployé tel quel sur des nœuds réels.

Le projet est un banc de raisonnement, comparaison et preuve logicielle. Le terrain exige calibration, validation indépendante et essais de limites.

15. Protocole minimal d'une comparaison scientifique

Rejouer une exécution vérifie sa mécanique ; démontrer un gain exige davantage :

1. figer et hasher les scénarios avant l'analyse ;
2. séparer scénarios d'entraînement, de validation et de test ;
3. choisir hyperparamètres et point de sauvegarde sur la validation, puis n'ouvrir le test final qu'une fois ;

4. exécuter plusieurs **graines d’entraînement** indépendantes, et pas seulement plusieurs épisodes d’une même politique ;
5. comparer PPO, politique classique et référence optimisée sur les mêmes observations, espaces d’actions, contraintes et mécanismes de réalisation ;
6. tracer la performance en fonction des interactions avec l’environnement afin de mesurer l’efficacité en échantillons (*sample efficiency*) ;
7. publier moyenne, dispersion, quantiles de queue, différences appariées et violations de contraintes, pas seulement la récompense moyenne ;
8. distinguer explicitement évaluation stochastique et choix déterministe de l’action ;
9. mesurer aussi temps d’inférence, mémoire et énergie ;
10. réaliser des ablations, par exemple récompense, historique, masque d’actions, GNN contre MLP et politique apprise contre coefficients d’arêtes neutres ;
11. tester congestion, panne, mobilité, charges et topologies hors distribution ;
12. séparer clairement validité du solveur, validité du simulateur, performance de la politique et KPI réseau indépendants de la récompense.

Sans ce protocole, le résultat reste une démonstration logicielle ou une étude exploratoire. Il ne démontre ni supériorité statistique ni généralisation.

16. Grille de vérification d’une exécution disponible

1. Valider l’unicité des dix-sept identifiants et recalculer les sept vagues.
2. Vérifier que chaque entrée possède un producteur ou une donnée initiale.
3. Choisir une exécution PPO-GNN et une référence comparable ; si elle est étiquetée ILP, vérifier d’abord le moteur d’exécution, le statut optimal et l’absence de repli heuristique.
4. Comparer graines, demandes, topologie, horizon et métrique.
5. Sélectionner trois instants : normal, congestionné et dégradé.
6. Reconstruire pour chacun :

```
preuve observée
→ chemins candidats
→ action
→ chemin retenu
→ contention
→ débit livré / latence
→ récompense
→ ligne d’artefact
```

7. Invalider une preuve requise et vérifier l’échec explicite.
8. Rejouer avec la même graine et comparer les champs déterministes.
9. Documenter ce qui est exact, statistique, simulé ou non vérifié.

La vérification exige l’accès au dépôt privé et aux artefacts d’exécution ; sans eux, cette grille de revue ne constitue pas une reproduction. Elle est réussie lorsque le lecteur explique une décision au-delà du score global, retrouve ses entrées et identifie la première hypothèse qui empêcherait sa transposition au terrain.

17. Correspondance avec les treize idées

Idée	Manifestation
Information	schémas, unités, capacité, provenance
Temps	trajectoires, pas, échéances, fraîcheur
Partage	contention et domaines d’interférence
Identité	identifiants de nœuds, liens, demandes et artefacts
Découverte	annonces reçues, séquences et expiration

Idée	Manifestation
Graphe	chemins candidats et topologie temporelle
Contrôle	PPO, récompense et adaptation
Coopération	partage entre demandes et services
Confiance	refus en cas d’incertitude, provenance et modèles chargeables
Échelle	DAG parallèle, workers et réplication
Résilience	panne, adaptation et service conservé
Programmabilité	configuration déclarative et cockpit
Évolution	matrice de compatibilité, canari, télémétrie de repli et plan de retrait

Le projet devient compréhensible lorsque ces idées restent séparées. Les fusionner sous l’étiquette « IA de routage » masquerait précisément les contrats qu’il cherche à rendre observables.

Laboratoires

Choisissez une fiche selon les prérequis et les outils indiqués. Les premiers ateliers se trouvent dans le livre d'initiation.

L00 — Construire le banc

Pour les quatre premiers TP : une feuille ou Python 3.10 ou plus.

Pour les projets avancés, choisir les outils demandés par le montage :

- Linux ;
- Wireshark ;
- tcpdump ;
- iproute2 / `tc` ;
- iperf3 ;
- Python ;
- FRRouting ;
- containerlab ;
- GNS3 selon les images disponibles ;
- un dépôt Git.

Règles :

- employer des plages privées ou de documentation ;
- isoler les expériences ;
- ne pas annoncer de routes externes ;
- ne pas émettre de brouillage ;
- obtenir l'autorisation du réseau ;
- capturer uniquement le trafic du laboratoire ;
- documenter les hypothèses et limites.

L18 — Projet IA/réseau

TP guidé — 45 minutes. Environnement : Python, sans modèle d'IA ni GPU. Le banc calcule temps de barrière, volumes et limites de capacité de quatre workers synchrones.

```
python3 tp_reseaux_guides_fr/executer_tp_reseaux_guides_fr.py \
  --tp L18 --output /tmp/tp_L18_ia_reseau_fr
```

1. Prédire la barrière avec calculs 100/100/100/400 ms et 20 Mo par worker.
2. Calculer le transfert plat sur la racine à 0,1 Mo/ms.
3. Comparer deux groupes locaux à 0,2 Mo/ms avec réduction supposée exacte divisant le volume par deux ; modifier placement et taille de groupe.
4. Comparer deux et quatre émetteurs simultanés, puis expliquer une reprise depuis checkpoint. Le modèle interdit le recouvrement calcul/communication.

Questions et résultats attendus. Une hiérarchie supprime-t-elle le retardataire ? L'itération plate vaut 400 + 800 = 1200 ms ; la hiérarchie vaut 600 + 400 = 1000 ms avec 40 Mo à la racine. Deux émetteurs offrent encore 0,2 Mo/ms pour une capacité de 0,1, contre 0,4 pour quatre.

Corrigé raisonné. La réduction de volume est une hypothèse algorithmique ; le placement seul ne corrige pas un worker lent. Limiter la concurrence réduit l'incast sans augmenter le goulot. Le rejeu du worker lent coûte ici 400 ms. Aucun

framework IA, apprentissage distribué ou contrôle TCP réel n'est validé. Remise : budgets, hypothèses de réduction et variante de reprise ; corrigé détaillé.

L20 — Retour, Q, avantage et erreur TD

Public : première découverte du RL. **Durée :** 45 à 60 minutes. **Prérequis :** Python élémentaire ; aucune connaissance de PyTorch.

Ce TP rend calculables les notions du chapitre avant d'introduire un réseau de neurones. Deux relais produisent les récompenses suivantes :

- A : (1 ; 0,5 ; −4), gain immédiat puis débordement ;
- B : (0,2 ; 0,8 ; 1), gain initial modeste puis service stable.

Le notebook exécutable se trouve dans `machine_learning_labs/notebooks/tp_rl_retour_q_td.ipynb`.

Déroulé

1. calculer les deux retours pour γ = 0,9 ;
2. les interpréter comme Q(s, A) et Q(s, B) ;
3. calculer V(s) pour une politique équiprobable, puis les avantages ;
4. faire varier γ et repérer le changement de classement ;
5. comparer l'erreur TD d'une transition en cours à celle d'une vraie terminaison ;
6. expliquer pourquoi une troncature temporelle conserve le bootstrap.

Résultat attendu. Q(s, A) = −1,79, Q(s, B) = 1,73, V(s) = −0,03, avec des avantages de −1,76 et +1,76. Le lecteur doit surtout expliquer pourquoi la récompense immédiate classe mal les actions.

Critères de réussite

- les calculs passent les assertions du notebook ;
- terminaison et troncature ne sont pas confondues ;
- une pénalité de récompense n'est pas présentée comme une contrainte dure ;
- la conclusion contient une interprétation causale, pas seulement les nombres.

L21 — DQN pour un choix de relais

Public : première pratique du deep RL. **Durée :** 75 à 90 minutes. **Prérequis :** L20 et lecture de Python.

Le notebook `machine_learning_labs/notebooks/tp_rl_dqn_relais.ipynb` utilise un environnement Gymnasium synthétique : deux files, deux relais, une capacité variable et une limite externe de durée des épisodes. Il met volontairement le mécanisme à nu : réseau Q, rejeu d'expérience, réseau cible, exploration ε-greedy et arrêt du bootstrap lors d'une vraie terminaison seulement.

Le problème reste partiellement observable : le vecteur ne contient pas le budget de pertes restant, alors que son épuisement provoque une terminaison. Deux historiques peuvent donc présenter la même observation et la même action, mais des probabilités de terminaison différentes. Le DQN sans mémoire est ici une approximation sur les observations, pas une démonstration d'un état markovien complet. Cette limite est commune aux trois contrôleurs.

La limite de durée interrompt la collecte d'une tâche qui pourrait continuer : elle est traitée comme une troncature et conserve le bootstrap. Un horizon fini intrinsèque à la tâche demanderait une autre spécification, avec le temps restant dans l'état et une terminaison à son échéance.

Depuis `machine_learning_labs/` :

```
uv sync --python 3.12
uv run jupyter lab notebooks/
```

Déroulé

1. inspecter les valeurs renvoyées par `reset` et `step` ;
2. mesurer une politique aléatoire et l'heuristique `projected_queue` ;
3. entraîner le DQN sur CPU avec un seed fixé ;
4. évaluer les trois politiques sur les mêmes 40 seeds tenus à l'écart ;
5. comparer retour, paquets livrés et paquets perdus ;
6. transformer une indisponibilité de relais en masque d'action, et non en simple pénalité.

Critères de réussite

- le DQN et les baselines sont comparés sur des graines appariées, avec intervalles de confiance et publication explicite des compromis observés sur le retour, les pertes et le débit sur le banc fourni ;
- l'heuristique reste visible lorsqu'elle domine un KPI ;
- la courbe d'entraînement n'est pas utilisée comme résultat de test ;
- aucun résultat synthétique n'est extrapolé à un déploiement réel.

Le contrôle automatisé exécute les deux notebooks dans un répertoire temporaire et vérifie que les versions suivies restent sans sorties :

```
uv run python verify_notebooks.py --execute
uv run pytest -q
```

L22 — PPO-GNN, ILP et protocole de référence

Public : lecteur ayant terminé L21. **Durée :** environ 120 minutes hors temps de calcul. **Surface opérateur :** `apps/routing_training_project`.

Ce TP ne remplace pas le DQN par « un algorithme plus fort ». Il change d'échelle méthodologique : PPO-GNN est une politique acteur-critique apprise, l'ILP est une référence d'optimisation centralisée, et le contrôleur OLSRv2-like est une référence protocolaire. Ils ne partagent pas les mêmes hypothèses ; la comparaison n'a de sens que sur un contrat d'entrée et de preuve explicite.

1. Geler le contrat avant le calcul

Consigner dans le compte rendu :

- identifiant de scénario, seed et empreinte de topologie ;
- connaissance de topologie autorisée à chaque contrôleur ;
- classes de trafic, capacités, délais et contraintes impératives ;
- ensemble de scénarios d'entraînement et ensemble tenu à l'écart ;
- KPI : livraison, latence, pertes, violations et coût de décision.

Une comparaison est invalide si un contrôleur voit une topologie, une demande ou un budget différent.

2. Construire le plan guidé

Dans la page de planification, ajouter :

1. `Train learned route scorer • PPO-GNN` ;
2. `Compute optimization reference • ILP` ;
3. `Compare controllers • OLSRv2-like baseline vs PPO-GNN`.

La configuration prédéfinie ILP impose `allow_ilp_fallback=false` : une solution non optimale échoue au lieu d'être étiquetée « référence ». La comparaison de contrôleurs est optionnelle et désactivée par défaut ; l'activer explicitement.

Sauvegarder ou sélectionner une configuration prédéfinie **n'exécute pas** le calcul : lancer ensuite l'action WORKFLOW prévue par l'application.

Pour un exercice de bout en bout, utiliser un modèle de confiance produit par l'étape PPO-GNN. Quand ce modèle n'est pas disponible, travailler sur une paire d'instantanés déjà produite et déclarer cette limite ; ne jamais substituer un modèle factice à une preuve d'inférence.

3. Vérifier les artefacts

L'allocation ILP canonique est attendue sous :

```
routing_training/pipeline/trainer_fcas_routing_ilp/allocations_steps.json
```

La valider depuis la racine du dépôt :

```
python3 apps/routing_training_project/tools/verify_allocations.py \
  --alloc-file routing_training/pipeline/trainer_fcas_routing_ilp/allocations_steps.json
```

Conserver aussi les allocations candidate/référence, les résumés de routage, les manifests de modèle et les preuves du scénario. La provenance de topologie et la correspondance décision-réalisation sont nécessaires pour valider une sortie.

4. Interpréter sans faux classement universel

Présenter un tableau par scénario et un agrégat sur l'ensemble tenu à l'écart. Le minimum est : taux de livraison, latence, pertes, violations de contraintes et temps de décision. Toute amélioration doit être rapportée à sa référence et à son hypothèse d'information.

Critères de réussite

- mêmes scénario, seed, topologie et demandes pour chaque comparaison ;
- ILP strict confirmé, sans repli silencieux ;
- artefacts vérifiés et provenance conservée ;
- au moins un scénario tenu à l'écart ;
- conclusion séparant performance, faisabilité et coût de calcul ;
- aucune phrase « PPO bat l'ILP » fondée sur un scénario unique.

Sources pédagogiques et reproductibilité

Les notebooks sont des créations originales, inspirées par la progression de FIDLE mais modernisées pour Gymnasium. Les versions consultées, les liens et la prudence de licence sont consignés dans `machine_learning_labs/SOURCES.md` ; le verrou `machine_learning_labs/uv.lock` fige l'environnement reproductible.

Laboratoire L23 — Classification rare sans fuite de test

Question. Une exactitude élevée suffit-elle lorsque l'événement important est rare et que le coût d'une omission dépasse celui d'une fausse alerte ?

Le notebook `machine_learning_labs/notebooks/tp_ml_classification_rare.ipynb` construit trois partitions disjointes et identifiables : ajustement sur l'entraînement, comparaison des seuils exclusivement sur la validation avec un coût pondéré, puis évaluation du test selon le protocole final déclaré. Les empreintes SHA-256 identifient les partitions ; `test_open_count = 1` est une assertion de protocole, pas un compteur d'accès instrumenté. Ni le champ ni les empreintes ne prouvent l'absence de consultations ou de réglages antérieurs. L'ordre ajustement → sélection → évaluation reste à vérifier dans le code et à respecter par le lecteur.

Depuis `machine_learning_labs/`, exécuter :

```
uv sync --locked
uv run --locked jupyter nbconvert --to notebook --execute \
```

```
  --inplace notebooks/tp_ml_classification_rare.ipynb
uv run --locked python run_benchmarks.py
uv run --locked python build_manifest.py
uv run --locked python verify.py --execute-notebooks
```

Le compte rendu doit comparer le seuil par défaut au seuil choisi, publier la matrice de confusion, la précision, le rappel et le coût pondéré, puis expliquer pourquoi le test ne peut pas servir à sélectionner le seuil. Le protocole est recevable si les partitions sont disjointes, si le seuil est choisi sur la validation et si le test n'est ouvert qu'une fois. Une baisse du coût pondéré sur la validation indique un gain du réglage. Le seuil peut rester à `0,5` ; l'absence de gain reste un résultat valide.

Les résultats DQN du même paquet suivent une règle analogue : cinq graines d'entraînement, quarante épisodes appariés par politique, intervalles de confiance sur les écarts par rapport à la méthode de référence, filtrage des actions qui refuse d'agir en cas d'incertitude et manifeste d'empreintes. La méthode de référence de charge projetée ne lit que l'observation publiée ; elle n'accède plus à l'état privé de l'environnement.

Laboratoire L24 — DQN contraint par les pertes

Question. Une pénalité de perte plus forte à l'entraînement peut-elle supprimer la régression de paquets perdus observée en L21, tout en conservant un gain de retour et zéro action interdite ?

L'expérience `machine_learning_labs/run_constrained_dqn_experiment.py` balaie les pénalités d'entraînement (4, 6, 8, 12, 16). Chaque configuration réutilise les cinq graines d'entraînement canoniques et les quarante scénarios tenus à l'écart. L'évaluation conserve toujours la récompense canonique avec une pénalité de 4 : les retours restent donc comparables entre configurations.

La porte est fixée avant lecture du résultat. Par rapport à la politique aléatoire disponible, la borne supérieure à 95 % de la différence de pertes doit être inférieure ou égale à zéro, la borne inférieure à 95 % du gain de retour doit être strictement positive, et le filtre doit appliquer zéro action interdite. Une hypothèse rejetée reste un résultat valide du laboratoire.

Pénalité d'entraînement	Différence de retour vs aléatoire	Différence de pertes vs aléatoire	Porte
4	+8,02 ± 4,81	+3,65 ± 3,39	rejetée
6	+3,11 ± 1,25	-1,72 ± 0,24	franchie
8	+3,22 ± 1,28	-1,82 ± 0,07	franchie
12	+2,04 ± 2,05	-1,72 ± 0,21	rejetée
16	+2,81 ± 0,90	-1,81 ± 0,10	franchie

La sélection lexicographique retient la pénalité 8 : elle améliore le retour de 3,22 ± 1,28 et réduit les pertes de 1,82 ± 0,07 face à l'aléatoire. Elle ne domine toutefois pas robustement l'heuristique de charge projetée : +0,22 ± 1,28 de retour et +0,01 ± 0,07 paquet perdu. Cette heuristique, plus simple, reste donc la préférence pour ce scénario synthétique. La conclusion ne vaut ni preuve RF, ni performance terrain, ni autorité de déploiement.

Ces intervalles sont exploratoires : les mêmes observations servent à choisir la pénalité parmi cinq candidates et à décrire son résultat. Les intervalles à 95 % calculés candidate par candidate ne garantissent pas une couverture à 95 % après cette sélection. Une conclusion confirmatoire demanderait de figer la configuration, puis de l'évaluer sur de nouvelles graines ; une affirmation sur l'algorithme d'apprentissage demanderait aussi de nouveaux entraînements. La règle de sélection fixée à l'avance ne remplace pas cette confirmation.

Depuis `machine_learning_labs/` :

```
uv run python run_constrained_dqn_experiment.py
uv run python run_constrained_dqn_experiment.py --check
uv run python verify.py
```

Le JSON retient les exécutions, les différences appariées, les intervalles, la porte et la conclusion. Le CSV fournit les lignes auditables par graine ; le schéma et `manifest.json` lient ces artefacts au code qui les a produits.

Laboratoire L25 — Robustesse sous décalage

Question. Le DQN à pénalité 8 retenu en L24 domine-t-il robustement l'heuristique de charge projetée lorsque les conditions d'exploitation s'écartent du banc nominal ?

Le candidat est gelé avant l'expérience : cinq modèles sont réentraînés avec les mêmes graines et la configuration L24, uniquement sur le scénario nominal. Aucun seuil, poids ou hyperparamètre n'est ajusté après lecture des scénarios décalés. Chaque régime réutilise les quarante graines d'évaluation appariées :

- arrivées plus concentrées, à demande moyenne inchangée ;
- indisponibilités de relais prolongées ;
- files réduites de dix à six paquets ;
- asymétrie de service déplacée entre relais et phases.

Un défi composite combine rafales, petites files, service déplacé et indisponibilités alternées. Son empreinte est fixée avant l'évaluation ; il est ouvert une fois et n'intervient ni dans la sélection ni dans un nouvel entraînement.

La porte de pire cas reprend le niveau de confiance de L24, mais compare cette fois à la méthode de référence `projected_queue` : borne supérieure à 95 % de la différence de pertes inférieure ou égale à zéro, borne inférieure à 95 % du gain de retour strictement positive et zéro action interdite, dans chaque régime.

Régime	Différence de retour vs charge projetée	Différence de pertes vs charge projetée	Porte
contrôle nominal	+0,22 ± 1,28	+0,01 ± 0,07	rejetée
arrivées en rafales	+0,37 ± 0,32	-0,13 ± 0,25	rejetée
indisponibilités prolongées	0,00 ± 0,00	0,00 ± 0,00	rejetée
capacité réduite	+4,46 ± 5,47	+0,71 ± 0,94	rejetée
asymétrie de service déplacée	+1,36 ± 2,75	+0,22 ± 0,12	rejetée
défi composite scellé	-0,34 ± 0,89	+0,09 ± 0,23	rejetée

Toutes les exécutions appliquent zéro action interdite. Sous arrivées en rafales, le gain de retour est positif à 95 %, mais la réduction moyenne des pertes reste inconclusive. Avec les indisponibilités prolongées, le masque ne laisse qu'une action autorisée à chaque phase : les politiques deviennent identiques, ce qui sépare correctement autorité de décision et performance. Le défi composite ne révèle aucun avantage du DQN.

L25 ne prouve pas que l'heuristique est universellement supérieure ; il rejette la supériorité robuste du DQN gelé sur ce périmètre synthétique. En l'absence de gain robuste et compte tenu de sa simplicité, l'heuristique reste la préférence pour ces scénarios. Aucune extrapolation RF, terrain ou production n'est admise.

Depuis `machine_learning_labs/` :

```
uv run python run_l25_robustness_experiment.py
uv run python run_l25_robustness_experiment.py --check
uv run python verify.py
```

Le JSON conserve les cinq entraînements gelés, les résultats par régime, les différences appariées, les portes, l'empreinte et le compteur d'ouverture du défi. Le CSV, le schéma et `manifest.json` rendent l'ensemble auditable.

Laboratoire L26 — Abstention sur désaccord

Question. Le désaccord entre les cinq DQN de L25 permet-il de détecter les décisions fragiles et de se replier assez tôt sur l'heuristique de charge projetée pour éviter les régressions sous décalage ?

Le panel reste gelé : mêmes cinq graines, pénalité d'entraînement 8 et scénario nominal. À chaque décision, les cinq modèles votent parmi les actions autorisées. Le DQN majoritaire agit seulement si sa part des votes atteint le seuil ; sinon, le contrôleur s'abstient et applique l'heuristique de charge projetée. Trois seuils sont fixés avant exécution : 0,6 (majorité simple), 0,8 (quatre voix sur cinq) et 1,0 (unanimité).

Le taux d'accord n'est pas une probabilité calibrée de correction. Il mesure seulement le désaccord de cinq entraînements du même modèle synthétique. La porte demande, face au repli lui-même, zéro action interdite, une borne inférieure à 95 % de la différence de retour supérieure ou égale à zéro et une borne supérieure à 95 % de la différence de pertes inférieure ou égale à zéro. Les intervalles sont calculés sur les quarante graines d'évaluation appariées, pour le panel gelé.

La sélection utilise seulement le contrôle nominal et les quatre décalages déclarés. Elle minimise d'abord le nombre de régimes qui échouent, puis la pire borne supérieure de pertes, maximise la pire borne inférieure de retour et préfère enfin le seuil le plus élevé. Le défi final combine, sous une nouvelle empreinte, rafales décalées, files de sept paquets, service déplacé et indisponibilités alternées ; il est ouvert une fois après la sélection.

Seuil d'accord	Régimes déclarés en échec	Taux moyen de repli	Pire borne sup. de pertes	Pire borne inf. de retour
0,6	4 sur 5	0,00 %	+0,83	-0,24
0,8	4 sur 5	7,74 %	+0,48	-0,33
1,0	3 sur 5	20,22 %	+0,48	-3,72

L'unanimité est donc sélectionnée avant ouverture du défi, sans franchir la porte sur tous les régimes déclarés. Son détail reste nécessaire pour ne pas confondre moyenne favorable et non-régression démontrée :

Régime	Différence de retour vs repli	Différence de pertes vs repli	Taux de repli	Porte
contrôle nominal	0,00 ± 0,00	0,00 ± 0,00	18,33 %	franchie
arrivées en rafales	+0,34 ± 0,67	-0,05 ± 0,07	28,70 %	rejetée
indisponibilités prolongées	0,00 ± 0,00	0,00 ± 0,00	0,00 %	franchie
capacité réduite	+1,16 ± 0,86	-0,10 ± 0,12	27,57 %	rejetée
asymétrie de service déplacée	-2,41 ± 1,30	+0,25 ± 0,23	26,52 %	rejetée
défi final tenu à l'écart	+0,04 ± 0,05	0,00 ± 0,00	3,93 %	rejetée

Toutes les exécutions appliquent zéro action interdite. L'abstention réduit certaines régressions moyennes, mais ne certifie pas la non-régression : les bornes de pertes restent légèrement positives sous rafales et capacité réduite, l'asymétrie déplacée dégrade les deux objectifs, et le gain du défi final est inconclusif. L'heuristique de charge projetée reste donc la préférence sur ce périmètre. Ce résultat ne calibre pas l'incertitude, ne mesure pas le coût de l'ensemble et n'autorise aucune extrapolation RF, terrain ou production.

Depuis `machine_learning_labs/` :

```
uv run python run_l26_abstention_experiment.py
uv run python run_l26_abstention_experiment.py --check
uv run python verify.py
```

Le JSON conserve le panel, les votes agrégés, les décisions de repli, les différences appariées, la règle de sélection et le défi final. Le CSV fournit les lignes par graine ; le schéma et `manifest.json` lient les artefacts au code qui les a produits.

Laboratoire L27 — Garde de support nominal

Question. Une distance au support observable nominal, ajoutée à l'unanimité de L26, peut-elle détecter les décisions fragiles que le seul désaccord des DQN n'a pas interceptées ?

Le moniteur ne voit ni état privé ni résultat futur. Sa référence réunit les observations de quarante trajectoires nominales aléatoires et quarante trajectoires nominales de l'heuristique. Après déduplication, elle contient 288 vecteurs. Pour chaque observation, le score est la distance euclidienne au plus proche vecteur de référence, dans l'espace déjà normalisé de l'environnement.

Les graines de référence 100 à 139 réutilisent des conditions initiales d'entraînement : les cinq graines de modèle 7, 19, 31, 43 et 59 produisent, sur 180 épisodes par modèle, des graines d'environnement de 7 à 238 selon `graine_modèle + indice_épisode`. Cette réutilisation déclarée est admise pour construire le support nominal, sans constituer une évaluation indépendante. Le contrat vérifie donc les graines de chaque épisode, au-delà des germes des modèles. Des graines distinctes ne garantissent pas des observations distinctes dans ce simulateur fini.

Quarante nouvelles graines par politique, disjointes de la référence, des entraînements et de l'évaluation, calibrent le seuil au quantile supérieur 95 % du score maximal par épisode. Le seuil obtenu vaut 0,10 et signale 2,5 % des épisodes de calibration. Aucun régime décalé ni défi final n'intervient dans ce calcul.

La correction de cette déclaration est conservée dans `machine_learning_labs/results/l27_seed_contract_correction.json`, avec l'empreinte et la révision du reçu initial. Elle ne modifie ni les observations retenues ni les résultats numériques et ne correspond pas à un nouvel entraînement.

L'autorité suit une cascade fixée avant l'évaluation : hors support, repli immédiat vers l'heuristique ; dans le support, action DQN si les cinq modèles sont unanimes, sinon même repli. La porte reprend la non-régression à 95 % de L26 sur retour, pertes et action interdite, dans tous les régimes. Elle exige aussi un bénéfice de Pareto strict dans au moins un régime déclaré, pour ne pas compter le repli systématique comme un progrès utile.

Régime	Différence de retour vs heuristique	Différence de pertes vs heuristique	Repli hors support	Autorité DQN	Porte
contrôle nominal	0,000 ± 0,000	0,000 ± 0,000	0,00 %	81,67 %	franchie
arrivées en rafales	+0,340 ± 0,670	-0,050 ± 0,071	0,94 %	70,42 %	rejetée
indisponibilités prolongées	0,000 ± 0,000	0,000 ± 0,000	50,00 %	50,00 %	franchie
capacité réduite	0,000 ± 0,000	0,000 ± 0,000	100,00 %	0,00 %	franchie
asymétrie de service déplacée	-0,005 ± 0,009	0,000 ± 0,000	65,28 %	27,61 %	rejetée
défi final tenu à l'écart	0,000 ± 0,000	0,000 ± 0,000	100,00 %	0,00 %	franchie

Toutes les exécutions appliquent zéro action interdite. Par rapport à l'unanimité seule de L26, la garde récupère 2,41 ± 1,31 de retour et réduit les pertes de 0,25 ± 0,23 dans le régime d'asymétrie déplacée. Elle neutralise aussi le régime de capacité réduite et le nouveau défi en revenant entièrement à l'heuristique. Ces protections ne constituent toutefois aucun bénéfice propre face au repli qui les fournit.

Le régime en rafales révèle la limite centrale : les observations restent presque toutes proches du support nominal, mais leur séquence change. La garde ponctuelle ne franchit donc pas la porte dans tous les régimes et ne démontre aucun bénéfice de Pareto strict. L'heuristique demeure la préférence. La distance n'est ni une probabilité calibrée de correction, ni une preuve OOD opérationnelle ; le banc ne couvre ni temporalité longue, ni coût du moniteur, ni RF, terrain ou production.

Depuis `machine_learning_labs/` :

```
uv run python run_l27_support_guard_experiment.py
uv run python run_l27_support_guard_experiment.py --check
uv run python verify.py
```

Le JSON conserve les graines disjointes, l'empreinte de la référence, la calibration, les distances, les décisions d'autorité et de repli, les différences appariées et le défi final. Le CSV fournit les lignes par graine ; le schéma et `manifest.json` lient les artefacts au code qui les a produits.

Fiche de compte rendu commune

Chaque laboratoire répond aux sept questions :

1. problème ;
2. hypothèse ;
3. montage ;
4. mesure ;
5. résultat ;
6. explication ;
7. limite et généralisation.

Parcours IA pour les réseaux en 12 semaines

Ce parcours transforme le traité transversal en dossier d'ingénierie. Chaque semaine se termine par un artefact révisable, pas seulement par une présentation. Le banc d'essai synthétique donne des résultats à examiner et des limites à discuter. Pour expérimenter, il faut construire un montage adapté aux données et aux outils disponibles.

Semaine	Question centrale	Travail et preuve attendue
1	IA **pour** les réseaux, réseaux **pour** l'IA ou réseau natif IA ?	classifier trois cas, nommer observation, sortie, autorité et repli
2	Les données sont-elles exploitables ?	contrat de schéma, unité, horloge, fraîcheur, provenance, droits et rétention
3	Une prévision bat-elle la persistance après rupture de régime ?	banc d'essai de trafic, séparation temporelle, cinq graines, dispersion et résultat négatif
4	Une anomalie est-elle détectée sans être prise pour une cause ?	banc d'essai de pics et dérive, précision-rappel, faux négatifs et protocole de diagnostic
5	L'optimisation respecte-t-elle les contraintes du réseau ?	routage contraint, référence, violations, surcharge, latence et compromis de sauts
6	Le modèle sait-il s'abstenir ?	dérive/OOD, seuil documenté, couverture d'automatisation et repli sûr
7	La télémétrie résiste-t-elle à l'empoisonnement ?	menace, moyenne et estimateur robuste, limites face aux attaques adaptatives
8	Une configuration générée peut-elle être appliquée ?	schéma, liste d'autorisation, analyse sémantique, simulation, déploiement témoin, validation confirmée et retour arrière
9	Où se place la fonction dans les standards ?	ligne versionnée de la matrice UIT-T/3GPP/O-RAN/ETSI/IETF, sans fausse équivalence
10	Le jumeau ou bac à sable représente-t-il assez le réel ?	fidélité, scénarios, oracle, écart simulation–mesure et condition de promotion
11	Une autre équipe peut-elle réfuter le résultat ?	environnement figé, manifeste d'empreintes, commande unique, réplication et revue adversariale
12	Quelle autorité peut être accordée ?	soutenance « revendication → source → test → limite → repli », décision signée de promotion ou refus

Pour relire le dossier obtenu, confrontez données, métriques, hypothèses et empreintes. Leur cohérence ne constitue ni une certification de sûreté, ni une validation sur un réseau opérateur.

Repères pour vérifier une solution d’IA

Besoin de lecture ou de revue	Point d’entrée	Preuve ou artefact associé
taxonomie des trois domaines	Trois domaines à distinguer	schéma comparatif des rôles du réseau et de l’IA
données, horloges et provenance	Architecture inter-SDO et § 9.2	matrice de standards + manifeste de données
NWDAF, MDAF et gestion AI/ML	Architecture inter-SDO, bloc 3GPP	TS 23.288, 28.104, 28.105 et 29.520 versionnées
SMO, Non-RT RIC, Near-RT RIC, rApp et xApp	Architecture inter-SDO, bloc O-RAN	R1, A1, E2/O1 et E2SM explicitement profilés
ENI, ZSM et coordination de boucles	Architecture inter-SDO, bloc ETSI	GS ENI 005, GS ZSM 002 et GS ZSM 009-1
intention et télémétrie IP	Architecture inter-SDO, bloc IETF/IRTF	RFC 9315, 9232, 8639 et 8641 avec leur statut
prévision et anomalies	Traité §§ 3 et 5	familles `traffic_forecasting` et `anomaly_detection`
routage appris sous contraintes	Traité § 4	famille `constrained_traffic_engineering` et référence
dérive, OOD et abstention	Traité §§ 9 et 11	famille `drift_ood_fallback`
empoisonnement et menace	Traité §§ 5 et 9	famille `telemetry_poisoning_robustness`
configuration générée	Traité §§ 5.11, 5.14 et 11	famille `generated_configuration_safety`
jumeau, bac à sable, observation sans action et déploiement témoin	Traité §§ 4.2, 9.4 et 11	seuil de fidélité et décision de promotion
réseaux pour entraînement distribué	Traité § 8 et Idée X	L18, incast, retardataires et coût énergétique
recherche propre aux réseaux	Carte de recherche	douze publications, contributions et limites
sources et statut documentaire	Repères normatifs et bibliographie	textes cités, versions et statuts
revue et corrections	Dossier de référence	passage, hypothèses, preuve et limites d’une correction

Glossaire

5GC — *5G Core*, cœur du système 5G séparant notamment fonctions de contrôle et plan utilisateur.

ABR — Adaptive Bitrate Streaming, sélection dynamique de la représentation du prochain segment média.

ACK — accusé de réception.

Acteur–critique / actor–critic — famille de RL où l'acteur produit une politique d'action et le critique estime une valeur afin de guider l'apprentissage.

Adaptation au moment du test — ajustement de statistiques, représentations ou paramètres pendant l'inférence à partir des données rencontrées, avec risque de dérive ou d'effondrement.

Agent de codage / coding agent — agent logiciel capable d'examiner et modifier un dépôt, de lancer des outils et de produire tests ou documentation sous des permissions définies ; sa sortie reste un candidat à vérifier.

AGILAB — atelier logiciel (*workbench*) de reproductibilité qui structure carnets, scripts, flux de travail et actions d'agents en exécutions, artefacts et éléments facilitant le rejeu et la comparaison.

AIS — système d'identification automatique par lequel un navire diffuse notamment son identité, sa position et son mouvement déclarés.

Apprentissage actif — stratégie où le modèle choisit les exemples pour lesquels demander un label à un expert.

Apprentissage continu — mise à jour à travers de nouvelles tâches ou distributions en cherchant à conserver les compétences antérieures.

Apprentissage fédéré — entraînement distribué agrégeant des mises à jour de modèles sans centraliser systématiquement les données brutes.

AS — système autonome, domaine de politique BGP.

Avantage (RL) — différence $A = Q - V$ entre la valeur attendue d'une action et la valeur normalement attendue dans l'état ; il guide notamment les mises à jour de l'acteur.

Backhaul / liaison de collecte — transport reliant un réseau d'accès, un site ou une plateforme mobile au cœur de réseau et aux services distants.

Bagging — méthode d'ensemble entraînant plusieurs estimateurs sur des échantillons bootstrap puis agrégeant leurs sorties pour réduire surtout la variance.

Bagottement / flapping — alternance rapide d'un lien, voisin, route ou état entre disponible et indisponible, ou entre plusieurs chemins ou rôles, provoquant annonces, recalculs et instabilité sans exiger une coupure totale.

Bandes Ku / K / Ka — désignations de micro-ondes dont les bornes dépendent de la convention technique ; elles ne constituent ni une attribution réglementaire ni une autorisation d'émettre.

Bandit contextuel — décision répétée à effet principalement immédiat : observer un contexte, choisir une action et apprendre le gain reçu, sans modéliser une longue transition d'état.

BERT — *Bit Error Rate Tester*, équipement ou fonction qui émet une séquence connue et compte les bits reçus en erreur.

BGP — protocole de routage inter-domaine à vecteur de chemin.

BLER — taux de blocs reçus avec erreur avant réparation éventuelle.

Bouclier de sûreté / safety shield — filtre indépendant qui bloque ou remplace une action apprise lorsqu'elle sort de l'enveloppe de contraintes dures.

Cause racine / root cause — cause dont la correction empêche la répétition du mécanisme étudié ; elle se distingue du déclencheur, du premier symptôme et des facteurs qui ont amplifié l'incident.

Chiffrement homomorphe — chiffrement permettant certaines opérations sur les chiffrés, avec un résultat déchiffrable correspondant au calcul prévu ; les opérations, la précision et le modèle de menace dépendent du schéma.

Classification — sortie discrète associant une observation à une ou plusieurs classes, éventuellement avec des probabilités calibrées.

Cloud / informatique en nuage — accès à des ressources de calcul, de stockage et de service mutualisées, à la demande. Le terme anglais est conservé dans les noms officiels et lorsqu'il facilite le repérage technique.

Clustering — regroupement d'observations selon une structure découverte, sans supposer que chaque groupe possède d'emblée un sens causal ou métier.

Co-site / co-implantation — coexistence de plusieurs émetteurs, récepteurs et équipements proches sur une même plateforme ou un même site, avec risque de couplage et d'interférence interne.

Codec — contrat d'encodage et de décodage d'un média, avec ou sans pertes.

Cœur de réseau / core network — ensemble de fonctions communes reliant les accès aux services et interconnexions, notamment transport, identité, sessions, mobilité et politiques selon l'architecture.

Congestion — demande supérieure à la capacité d'une ressource.

CQI — indicateur quantifié de qualité de canal utilisé pour guider l'adaptation radio.

DNS — système distribué qui résout notamment des noms en données telles que des adresses ; sa disponibilité, son cache et sa délégation font partie du chemin d'accès à un service.

DSCP — champ de marquage DiffServ.

DTN — réseau tolérant délais et intermittence.

ECN — signal explicite de congestion sans rejet nécessaire.

Égalisation / equalization — traitement numérique qui compense une transformation estimée du canal avant la détection des symboles.

Embedding — représentation vectorielle, souvent apprise, qui place un token, un nœud, une arête ou un objet dans un espace numérique utile à une tâche ; cette projection peut compresser ou développer la dimension sans conserver nécessairement toutes les propriétés de l'objet.

ETX / ETT / WCETT — métriques estimant successivement le nombre de transmissions, leur durée et le coût cumulé avec pénalité de réutilisation radio.

EVM — erreur vectorielle mesurant l'écart entre symboles modulés reçus et idéaux.

FEC — redondance permettant de réparer certaines pertes sans retransmission.

GAE — *Generalized Advantage Estimation*, combinaison d'erreurs temporelles utilisée pour estimer l'avantage avec un compromis biais–variance.

Gestion de la connaissance / knowledge management (KM) — cycle qui extrait, relie, retrouve, valide, versionne et retire des connaissances en conservant provenance, portée et droits d'accès.

Gigue — variation du délai d'arrivée.

GNN — réseau neuronal représentant explicitement nœuds, liens et voisinages d'un graphe.

GNSS — ensemble des systèmes mondiaux de navigation par satellite, fournissant notamment position et temps.

Graphe de connaissances / knowledge graph — représentation d'entités et de relations dont les arêtes devraient porter provenance, validité, incertitude et statut de confirmation.

Handover — transfert de connexion entre points d'accès.

HARQ — retransmission automatique hybride combinant codage correcteur et nouvelles tentatives de transmission.

ILP / MILP — optimisation linéaire en nombres entiers, éventuellement mixte lorsque certaines variables restent continues, sous contraintes explicites.

Incast — nombreux émetteurs convergeant simultanément vers une destination.

Intent — résultat désiré exprimé à un niveau supérieur à la configuration.

KPI — indicateur clé de performance mesuré pour juger un système.

LLM — grand modèle de langage traitant des tokens au moyen de représentations vectorielles contextualisées, généralement entraîné à prédire ou reconstruire du texte selon son objectif.

LLR / soft bits — rapport de log-vraisemblance indiquant quel bit est favorisé et avec quelle confiance avant le décodage FEC.

LoRA — adaptation de faible rang qui entraîne de petites matrices ajoutées à un modèle principalement figé ; elle réduit le nombre de paramètres à adapter sans garantir la généralisation.

MAML — *Model-Agnostic Meta-Learning*, méta-apprentissage recherchant une initialisation adaptable à une nouvelle tâche en quelques mises à jour.

MANET — réseau mobile ad hoc.

MARL — apprentissage par renforcement multi-agent.

MCS — combinaison d'un ordre de modulation et d'un taux de codage choisie pour une qualité de canal donnée.

MDP / POMDP — processus de décision markovien, ou sa version partiellement observable lorsque l'agent ne voit pas l'état complet du monde.

MIMO — système utilisant plusieurs antennes d'émission et de réception pour diversité, formation de faisceaux ou couches spatiales.

Multicast — diffusion vers les membres d'un groupe.

NACK — acquittement négatif signalant une donnée absente ou non décodée.

NF / figure de bruit — dégradation du rapport signal/bruit introduite par le récepteur, exprimée en décibels ; le facteur de bruit correspondant est linéaire.

OLSRv2 — protocole proactif à état de liens pour MANET qui diffuse des messages de contrôle topologique sur ses interfaces configurées et réduit les relais d'inondation au moyen de MPR.

On-policy / off-policy — distinction entre un apprentissage utilisant surtout les trajectoires de la politique courante et un apprentissage pouvant réutiliser celles d'autres politiques.

OOD / hors distribution — observation suffisamment différente du domaine validé pour rendre la confiance du modèle incertaine.

Orchestration — coordination de plusieurs ressources, contrôleurs, dépendances et étapes de cycle de vie pour maintenir un résultat de service mesurable proche d'un état désiré.

PACE — chaîne Primary, Alternate, Contingency, Emergency.

POMDP — voir MDP / POMDP.

Pooling — agrégation de plusieurs représentations vectorielles en un résumé ; une moyenne, un maximum ou une somme peut rendre le calcul compact, mais fusionner des états distincts.

PPO — algorithme acteur–critique on-policy dont l'objectif écrête un ratio de probabilités pour décourager des mises à jour trop brutales ; cet écrêtage ne borne ni les actions ni leurs effets réseau.

Prévision — estimation d'une valeur, d'une trajectoire ou d'une distribution future pour un horizon explicitement défini.

QoE — qualité réellement perçue ou utile pour l'utilisateur et le service.

QoS — politique de traitement différencié selon les besoins de service.

QUIC — transport chiffré et multiplexé au-dessus d'UDP, avec des flux fiables et, via l'extension DATAGRAM, des datagrammes non fiables.

RAG — *Retrieval-Augmented Generation*, génération augmentée par recherche de sources externes ; elle facilite l'ancrage et la citation sans garantir que la réponse respecte ou interprète correctement les passages.

RAN — *Radio Access Network*, réseau d'accès radio reliant les terminaux au cœur mobile au moyen de stations de base et de transport associé.

Ranking / classement — sortie ordonnant des candidats sans attribuer nécessairement une distance absolue entre leurs rangs.

Récepteur neuronal / neural receiver — modèle appris remplaçant ou combinant des blocs numériques tels qu'estimation de canal, égalisation et détection afin de produire notamment des LLR.

Réduction de dimension — transformation d'une représentation vers moins de dimensions en cherchant à conserver l'information utile à un objectif.

Régression — sortie numérique estimant une quantité, un intervalle ou une distribution conditionnelle.

Retour (RL) / return — somme, généralement actualisée, des récompenses futures utilisée pour juger les conséquences d'une action.

Retour arrière (*rollback*) — action qui restaure une version ou configuration antérieure ; elle ne reconstitue pas automatiquement l'état externe supprimé ni ne garantit une reprise sans surcharge.

Retour d'expérience / post-incident review (PIR) / postmortem — analyse factuelle d'un incident reliant chronologie, impact, mécanisme causal, amplificateurs, reprise et actions vérifiables, sans chercher un coupable unique.

RL contraint — apprentissage par renforcement optimisant sous contraintes ; il constitue une approche du safe RL sans rendre seul le système sûr.

RL en ligne — apprentissage par renforcement qui collecte des conséquences et met encore sa politique à jour pendant l'exploitation ou une expérimentation connectée.

RL hors ligne — apprentissage par renforcement sur un journal fixe de trajectoires, sans exploration pendant l'entraînement ; les actions absentes du journal restent difficiles à évaluer.

Routeur — équipement ou fonction qui choisit une route et transmet un paquet IP vers une interface ou un prochain saut ; il peut aussi faire NAT ou pare-feu, mais ces rôles restent distincts.

RTT — délai aller-retour entre émission et réception du retour.

Scheduler / ordonnanceur — fonction choisissant quelles files ou quels utilisateurs reçoivent les ressources disponibles à chaque instant.

Scoring / rating — attribution d'une note de risque, de qualité ou de préférence, qui n'est pas nécessairement une probabilité ni une grandeur physique.

SDN — séparation et programmation du contrôle réseau.

Session agentique / agentic session — session de travail dans laquelle un agent logiciel lit du contexte, appelle des outils et propose ou applique des changements sous des permissions définies.

SINR — rapport du signal utile aux interférences plus bruit.

SLO — objectif mesurable de niveau de service.

SWaP — Size, Weight and Power : taille/encombrement, masse et puissance d'un système.

SWaP-C — SWaP complété par le coût total d'acquisition, d'intégration et de soutien.

TC / netem — outils Linux de gestion et d'émulation de trafic.

Télémétrie — production et transport de mesures distantes. La télémétrie d'exploitation observe un réseau ; la télémétrie d'essai relie en plus grandeur physique, métrologie, temps, enregistrement et preuve.

Tokki — dispositif local qui encadre les opérations des agents selon les règles configurées, permet l'exécution d'opérations déterministes admises sans nouvel appel au modèle et conserve un contexte relié à des preuves récupérables. Il n'est ni le modèle qui raisonne ni le validateur métier.

Valeur V / valeur d'action Q — retours attendus respectivement depuis un état et après une action donnée sous une politique.

YANG — langage de modélisation de données réseau.

Périmètre des cas présentés

Les passages consacrés aux communications de défense reposent uniquement sur des principes, normes et sources ouvertes. Ils portent sur l'architecture, la résilience, la qualité de service, la sûreté de fonctionnement et la compréhension défensive de l'environnement électromagnétique.

Ils ne donnent ni fréquences opérationnelles, paramètres de formes d'onde, procédures de brouillage, configurations réelles, ni tactiques de perturbation ou de localisation. L'accès multiple par répartition dans le temps (**TDMA**) et les interférences entre équipements co-installés (**co-site**) sont traités sous l'angle de la sûreté, de la compatibilité électromagnétique, de la qualité de service (**QoS**) et de la validation défensive : **concevoir un service qui continue à fonctionner dans un environnement mobile, congestionné, intermittent ou contesté**.

Pour aller plus loin — Bibliographie

Sources logicielles de l'étude de cas

- Farama Foundation, *Gymnasium documentation*, API des environnements, observations, actions, transitions et graines.
- Stable-Baselines3, documentation de PPO et guide des environnements personnalisés.
- Schulman et al., *Proximal Policy Optimization Algorithms*, arXiv:1707.06347.
- Sutton et Barto, *Reinforcement Learning: An Introduction*, deuxième édition, MIT Press, 2018.
- RFC 7181, *The Optimized Link State Routing Protocol Version 2*, pour le principe d'une métrique de lien extensible, sans lui attribuer une métrique ETX particulière.
- Documentation MLflow sur le suivi d'expériences, les modèles et les artefacts.
- Sources privées observées pour l'édition 1.0 : `lab_stages.toml`, `pipeline_view.dot`, `src/app_settings.toml`, `TimeRoutingEnv`, `UavQueueRoutingEnv`, `UavActiveMeshEnv` et leurs tests de contrat.

Gestion de la connaissance et IA générative

- W3C, *PROV-O: The PROV Ontology* — modèle interopérable reliant entités, activités, dérivations et agents responsables.
- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020 — combinaison d'une mémoire paramétrique et d'un index externe récupéré.
- NIST AI 600-1, *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile* — risques et actions de gouvernance pour les systèmes génératifs, notamment contenu non fiable, confidentialité et attaques par instructions.

Apprentissage automatique — repères primaires

- Leo Breiman, *Bagging Predictors* — agrégation bootstrap pour réduire la variance de prédicteurs instables.
- Chen et al., *A Simple Framework for Contrastive Learning of Visual Representations* — exemple d'apprentissage auto-supervisé contrastif.
- Ganin et al., *Domain-Adversarial Training of Neural Networks* — adaptation de domaine.
- Finn, Abbeel et Levine, *Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks* — méta-apprentissage.
- McMahan et al., *Communication-Efficient Learning of Deep Networks from Decentralized Data* — agrégation fédérée FedAvg.
- Ross, Gordon et Bagnell, *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning* — DAgger et décalage de distribution en imitation.
- Williams, *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning* — gradient de politique REINFORCE.
- Mnih et al., *Human-level Control through Deep Reinforcement Learning* — approximation de Q par réseau profond, DQN.
- Schulman et al., *High-Dimensional Continuous Control Using Generalized Advantage Estimation* — estimation d'avantage GAE.
- Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor* — acteur–critique off-policy et entropie.
- Fujimoto, van Hoof et Meger, *Addressing Function Approximation Error in Actor-Critic Methods* — TD3 et réduction des surestimations.
- Bellemare, Dabney et Munos, *A Distributional Perspective on Reinforcement Learning* — apprentissage de la distribution du retour.

- Sutton, Precup et Singh, *Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning* — options et RL hiérarchique.
- Kumar et al., *Conservative Q-Learning for Offline Reinforcement Learning* — estimation prudente en RL hors ligne.
- Kostrikov, Nair et Levine, *Offline Reinforcement Learning with Implicit Q-Learning* — famille IQL pour données fixes.
- Achiam et al., *Constrained Policy Optimization* — apprentissage par renforcement sous contraintes.
- Alshiekh et al., *Safe Reinforcement Learning via Shielding* — boucliers d'actions indépendants de la récompense.
- Lowe et al., *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments* — entraînement centralisé et exécution décentralisée en MARL.
- Kirkpatrick et al., *Overcoming Catastrophic Forgetting in Neural Networks* — apprentissage continu et oubli catastrophique.
- Wang et al., *Tent: Fully Test-Time Adaptation by Entropy Minimization* — adaptation non supervisée au moment de l'inférence.
- Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models* — adaptation économe en paramètres.

Index thématique

Sujet	Explication principale	Page imprimée
AI-RAN : IA pour la radio, calcul partagé et inférence proche	AI-RAN : trois usages complémentaires de l’IA et de la radio	108
AirLLM : chargement des poids par couche et mémoire GPU	5. Exploitation, sécurité et connaissance	124
Apprentissage distribué et confidentialité	7. Apprentissage distribué et confidentialité	154
Apprentissage : tâches et familles	2. Repères de science des données pour l’ingénieur réseau	6
Architecture IA inter-SDO	8 bis. Architecture opérationnelle inter-SDO	155
Bandits linéaires : estimation et exploration	2. Repères de science des données pour l’ingénieur réseau	6
Biais de représentation et de mesure : visages et production	Les données racontent-elles ce qui se passe ?	5
Chaîne de raisonnement : explication et vérification	Panorama des algorithmes et concepts d’apprentissage	57
Communications maritimes : navire, capteurs, bouée et terre	Repère — Pourquoi communiquer en mer reste difficile	113
Convention d’Einstein : indices et attention	Panorama des algorithmes et concepts d’apprentissage	57
Convolution : noyau, décalage et CNN	Panorama des algorithmes et concepts d’apprentissage	57
Corpus, vecteurs et TF-IDF	2. Repères de science des données pour l’ingénieur réseau	6
Dérivée et estimation du débit	Repères mathématiques — du résultat à l’usage	47
Données, évaluations comparatives et GPU	Panorama des algorithmes et concepts d’apprentissage	57
Fourier, harmoniques et signaux périodiques	Repères mathématiques — du résultat à l’usage	47
Gravimétrie marine : modèle, mesure et incertitude	Repère — Pourquoi communiquer en mer reste difficile	113
Hiérarchie d’agents : OpenCode, Tokki et AGILAB	5. Exploitation, sécurité et connaissance	124
IA : décision, autorité et preuve	11. Protocole de validation avant autorité	174
IA : essais reproductibles	13 bis. Banc d’essai synthétique — six familles et leurs résultats négatifs	176
IA et égalité des chances : accès, formation et liberté de choix	5. Exploitation, sécurité et connaissance	124
IA : exploitation et connaissance	5. Exploitation, sécurité et connaissance	124
IA pour la couche physique	2. Couche physique et radio	116
Jetons, contexte et génération	2. Repères de science des données pour l’ingénieur réseau	6
Laboratoires fondamentaux	L00 — Construire le banc	192
Mathématiques fondamentales et appliquées	Repères mathématiques — du résultat à l’usage	47
MLP et approximation universelle	Repères mathématiques — du résultat à l’usage	47
Modèle abstrait, réalisation concrète et validation	5. Exploitation, sécurité et connaissance	124
MoE et AirLLM : calcul actif et poids en mémoire	5. Exploitation, sécurité et connaissance	124

Sujet	Explication principale	Page imprimée
Petits modèles locaux : spécialisation et autonomie	5. Exploitation, sécurité et connaissance	124
PINN : physique et apprentissage	Panorama des algorithmes et concepts d’apprentissage	57
PPO, GNN et ILP	7. PPO, GNN, génétique et ILP	185
Qwen, GPT-6, Tokki et recherche documentaire	5. Exploitation, sécurité et connaissance	124
Retour, valeur et erreur TD	L20 — Retour, Q, avantage et erreur TD	193
Routage de flux réseau et routage de prompts	5. Exploitation, sécurité et connaissance	124
Taylor, approximation polynomiale et reste	Repères mathématiques — du résultat à l’usage	47
Transformers : de la traduction aux tableaux	Panorama des algorithmes et concepts d’apprentissage	57
Verrous de l’IA : XOR, rétropropagation, règles et séquences	Panorama des algorithmes et concepts d’apprentissage	57

Références citées

Adebayo, Julius et al. 2018. *Sanity Checks for Saliency Maps*. https://doi.org/10.48550/arXiv.1810.03292.

Agence nationale de la sécurité des systèmes d'information (ANSSI). 2026. *Le cadre de certification de cybersécurité européen*. https://cyber.gouv.fr/reglementation/reglementation-identite-confiance-numerique/cadre-de-certification-de-cybersecurite-europeen/.

Agostinelli, Forest, Matthew Hoffman, Peter Sadowski, et Pierre Baldi. 2014. *Learning Activation Functions to Improve Deep Neural Networks*. https://arxiv.org/abs/1412.6830.

Ainslie, Joshua, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, et Sumit Sanghai. 2023. « GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints ». *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895-901. https://doi.org/10.18653/v1/2023.emnlp-main.298.

AI-RAN Alliance. 2026. *AI-RAN Architecture Overview and Component Definitions*. Rapport d'architecture Version 1.2. AI-RAN Alliance. https://ai-ran.org/documents/AI-RAN-Architecture-v1.2.pdf.

Apache Software Foundation. 2026a. *Cluster Mode Overview*. Documentation Apache Spark 4.2.0. https://spark.apache.org/docs/4.2.0/cluster-overview.html.

Apache Software Foundation. 2026b. *Collaborative Filtering*. Documentation Apache Spark 4.2.0. https://spark.apache.org/docs/4.2.0/ml-collaborative-filtering.html.

Apache Software Foundation. 2026c. *GraphX Programming Guide*. Documentation Apache Spark 4.2.0. https://spark.apache.org/docs/4.2.0/graphx-programming-guide.html.

Apache Software Foundation. 2026d. *Machine Learning Library (MLlib) Guide*. Documentation Apache Spark 4.2.0. https://spark.apache.org/docs/4.2.0/ml-guide.html.

Apache Software Foundation. 2026e. *Spark SQL, DataFrames and Datasets Guide*. Documentation Apache Spark 4.2.0. https://spark.apache.org/docs/4.2.0/sql-programming-guide.html.

Apache Software Foundation. 2026f. *Structured Streaming Programming Guide*. Documentation Apache Spark 4.2.0. https://spark.apache.org/docs/4.2.0/streaming/index.html.

Apple. 2020. « Apple unleashes M1 ». novembre 10. https://www.apple.com/newsroom/2020/11/apple-unleashes-m1/.

Apple. 2025. « Get started with MLX for Apple silicon ». https://developer.apple.com/videos/play/wwdc2025/315/.

Apple Machine Learning Research. 2026. « MLX-LM ». https://github.com/ml-explore/mlx-lm.

Arazo, Eric, Diego Ortego, Paul Albert, Noel E. O'Connor, et Kevin McGuinness. 2019. *Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning*. https://arxiv.org/abs/1908.02983v5.

ARTE. 2026. « Voyages au pays des maths ». https://www.arte.tv/fr/videos/RC-021426/voyages-au-pays-des-maths/.

Ba, Jimmy Lei, Jamie Ryan Kiros, et Geoffrey E. Hinton. 2016. *Layer Normalization*. https://doi.org/10.48550/arXiv.1607.06450.

Bahdanau, Dzmitry, Kyunghyun Cho, et Yoshua Bengio. 2014. *Neural Machine Translation by Jointly Learning to Align and Translate*. https://arxiv.org/abs/1409.0473.

Baker, Fred. 1995. *Requirements for IP Version 4 Routers*. Request for Comments RFC 1812. IETF. https://www.rfc-editor.org/rfc/rfc1812.

Baydin, Atilim Gunes, Barak A. Pearlmutter, Alexey Andreyevich Radul, et Jeffrey Mark Siskind. 2018. « Automatic Differentiation in Machine Learning: a Survey ». *Journal of Machine Learning Research* 18 (153): 1-43. https://www.jmlr.org/papers/v18/17-468.html.

Bengio, Yoshua, Pascal Lamblin, Dan Popovici, et Hugo Larochelle. 2007. « Greedy Layer-Wise Training of Deep Networks ». *Advances in Neural Information Processing Systems 19*. https://proceedings.neurips.cc/paper/2006/hash/5da713a690c067105aeb2fae32403405-Abstract.html.

Blake, Steven, David Black, Mark Carlson, Elwyn Davies, Zheng Wang, et Walter Weiss. 1998. *An Architecture for Differentiated Services*. Request for Comments RFC 2475. IETF. https://www.rfc-editor.org/rfc/rfc2475.

Blanche, Vincent, Sophie Edouard, Colin Jourde, Sébastien Pac, et Louis Philbert. 2026. *Sciences en fin de collège : la baisse des résultats se poursuit en 2024*. Note d'Information 26.13. Direction de l'évaluation, de la prospective et de la performance (DEPP). https://www.education.gouv.fr/sites/default/files/document/Education%20nationale_DEPP%20NI-2026-13.pdf-515798.pdf.

Blei, David M., Andrew Y. Ng, et Michael I. Jordan. 2003. « Latent Dirichlet Allocation ». *Journal of Machine Learning Research* 3: 993-1022. https://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf.

Blundell, Charles, Julien Cornebise, Koray Kavukcuoglu, et Daan Wierstra. 2015. « Weight Uncertainty in Neural Network ». *Proceedings of the 32nd International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 37: 1613-22. https://proceedings.mlr.press/v37/blundell15.html.

Brown, Tom B. et al. 2020. *Language Models are Few-Shot Learners*. https://arxiv.org/abs/2005.14165.

ByteByteGo. 2026. *How to Build Models Without Language, with Logical Intelligence*. https://blog.bytebytego.com/p/how-to-build-models-without-language.

Chen, Ting, Simon Kornblith, Mohammad Norouzi, et Geoffrey Hinton. 2020. « A Simple Framework for Contrastive Learning of Visual Representations ». *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 119: 1597-607. https://proceedings.mlr.press/v119/chen20j.html.

Chicco, Davide, et Giuseppe Jurman. 2020. *The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation*. https://doi.org/10.1186/s12864-019-6413-7.

CLIPS. 2026. *CLIPS: A Tool for Building Expert Systems*. https://www.clipsrules.net/Documentation.html.

CNIL. 2025. *IA : finalisation des recommandations sur le développement des systèmes d'IA*. https://www.cnil.fr/fr/ia-finalisation-recommandations-developpement-des-systemes-ia.

CNRS. 2026. « FIDLE — Formation en intelligence artificielle et Deep Learning ». https://gricad-gitlab.univ-grenoble-alpes.fr/talks/fidle/-/wikis/home.

Cohen, Michael X. 2022. *A tutorial on generalized eigendecomposition for denoising, contrast enhancement, and dimension reduction in multichannel electrophysiology*. https://doi.org/10.1016/j.neuroimage.2021.118809.

Comaniciu, Dorin, et Peter Meer. 2002. « Mean Shift: A Robust Approach Toward Feature Space Analysis ». *IEEE Transactions on Pattern Analysis and Machine Intelligence* 24 (5): 603-19. https://doi.org/10.1109/34.1000236.

Commission européenne. 2026a. *AI Act Service Desk: Annex III*. https://ai-act-service-desk.ec.europa.eu/en/ai-act/annex-3.

Commission européenne. 2026b. *Navigating the AI Act*. https://digital-strategy.ec.europa.eu/en/faqs/navigating-ai-act.

Conceiçao, Pierre, Xuân Nghiem, et Louis Philbert. 2025. *Cedre histoire-géographie et enseignement moral et civique : en fin d'école, stabilité des résultats depuis 2006, mais hausse des écarts entre les élèves selon le profil social des écoles*. Note d'Information 25.06. Direction de l'évaluation, de la prospective et de la performance (DEPP). https://www.education.gouv.fr/media/199876/download.

Constantine, Barry, Gilles Forget, Rüdiger Geib, et Reinhard Schrage. 2011. « Framework for TCP Throughput Testing ». IETF, août. https://www.rfc-editor.org/rfc/rfc6349.html.

Cour des comptes. 2020. *L'École polytechnique : une stratégie incertaine, une gestion à redresser*. Rapport public annuel 2020, tome I. Cour des comptes. https://www.ccomptes.fr/sites/default/files/2023-10/20200225-12-TomeI-ecole-polytechnique.pdf.

Cybenko, George. 1989. « Approximation by Superpositions of a Sigmoidal Function ». *Mathematics of Control, Signals and Systems* 2: 303-14. https://doi.org/10.1007/BF02551274.

ddalcu. 2026. *Qwen3.8-27B-MLX-Serve-4bit: model card and conversion*. https://huggingface.co/ddalcu/Qwen3.8-27B-MLX-Serve-4bit.

Defferrard, Michaël, Xavier Bresson, et Pierre Vandergheynst. 2016. *Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering*. https://doi.org/10.48550/arXiv.1606.09375.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, et Kristina Toutanova. 2019. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. https://arxiv.org/abs/1810.04805.

Dosovitskiy, Alexey et al. 2020. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. https://arxiv.org/abs/2010.11929.

Du, Yilun. s. d. *Energy-Based Models: Representation, Inference, and Composition*. https://energy-based-model.github.io/.

Du, Yilun, et Igor Mordatch. 2019. *Implicit Generation and Generalization in Energy-Based Models*. https://arxiv.org/abs/1903.08689.

École polytechnique. 2024. *Bilan et statistiques du concours d'admission 2024*. Bilan statistique. École polytechnique. https://www.polytechnique.edu/admission-cycle-ingenieur/sites/admission/files/content/Livret_Stats%202024%20-%202.pdf.

École polytechnique. 2025. *Bilan et statistiques du concours d'admission 2025*. Bilan statistique. École polytechnique. https://www.polytechnique.edu/admission-cycle-ingenieur/sites/admission/files/content/Livret_Stats_2025.pdf.

Équipe aGrUM. 2026. « pyAgrum — Documentation et tutoriels ». https://pyagrum.readthedocs.io/en/latest/.

EUROCAE. 2021. *ER-022: Artificial Intelligence in Aeronautical Safety-Related Systems – Statement of Concerns*. https://www.eurocae.net/product/er-022-artificial-intelligence-in-aeronautical-safety-related-systems-statement-of-concerns/.

EUROCAE. 2026a. *Technical Work Programme 2026*. https://www.eurocae.net/wp-content/uploads/2025/12/TWP-2026.pdf.

EUROCAE. 2026b. *WG-114 Artificial Intelligence*. https://www.eurocae.net/working-group/wg-114/.

EUROCAE, et SAE International. 2024. *ER-027 / SAE AIR6987: Artificial Intelligence in Aeronautical Safety-Related Systems Taxonomy*. https://www.eurocae.net/eurocae-and-sae-international-publish-landmark-ai-taxonomy-document-for-the-global-aeronautical-industry/.

European Commission. 2024. *Commission Implementing Regulation (EU) 2024/482 adopting the European Common Criteria-based cybersecurity certification scheme (EUCC)*. https://eur-lex.europa.eu/eli/reg_impl/2024/482/oj.

European Parliament, et Council of the European Union. 2019. *Regulation (EU) 2019/881 on ENISA and on information and communications technology cybersecurity certification (Cybersecurity Act)*. https://eur-lex.europa.eu/eli/reg/2019/881/oj.

European Union Agency for Cybersecurity (ENISA). 2020. *Candidate EUCS Scheme v1.0*. https://certification.enisa.europa.eu/publications/candidate-eucs-scheme-v10_en.

Fernandez, Adrien, Franck Salles, Vincent Bernigole, et Hélène de Monestrol. 2026. *PISA 2025 : les acquis des élèves de 15 ans en compréhension de l'écrit et en culture mathématique en baisse en France et dans l'OCDE*. Note d'Information 26.40. Direction de l'évaluation, de la prospective et de la performance (DEPP). https://www.education.gouv.fr/sites/default/files/document/depp-ni-2026-40pisa-mathspdf-520231.pdf.

Finn, Chelsea, Pieter Abbeel, et Sergey Levine. 2017. « Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks ». *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 70: 1126-35. https://proceedings.mlr.press/v70/finn17a.html.

Gers, Felix A., Jürgen Schmidhuber, et Fred Cummins. 2000. « Learning to Forget: Continual Prediction with LSTM ». *Neural Computation* 12 (10): 2451-71. https://doi.org/10.1162/089976600300015015.

Goodfellow, Ian J. et al. 2014. *Generative Adversarial Networks*. https://doi.org/10.48550/arXiv.1406.2661.

Goodfellow, Ian, Yoshua Bengio, et Aaron Courville. 2016a. *Deep Learning*. MIT Press. https://www.deeplearningbook.org/.

Goodfellow, Ian, Yoshua Bengio, et Aaron Courville. 2016b. *Deep Learning : Linear Algebra*. https://www.deeplearningbook.org/contents/linear_algebra.html.

Goodfellow, Ian, Yoshua Bengio, et Aaron Courville. 2016c. *Deep Learning, Chapter 14: Autoencoders*. https://www.deeplearningbook.org/contents/autoencoders.html.

Goodfellow, Ian, Yoshua Bengio, et Aaron Courville. 2016d. *Deep Learning, Chapter 8: Optimization for Training Deep Models*. https://www.deeplearningbook.org/contents/optimization.html.

Google Cloud. 2024. *MLOps: Continuous delivery and automation pipelines in machine learning*. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning.

Gorishniy, Yury, Ivan Rubachev, Valentin Khrulkov, et Artem Babenko. 2021. *Revisiting Deep Learning Models for Tabular Data*. https://arxiv.org/abs/2106.11959.

Grover, Aditya, et Jure Leskovec. 2016. *node2vec: Scalable Feature Learning for Networks*. https://doi.org/10.48550/arXiv.1607.00653.

Guo, Chuan, Geoff Pleiss, Yu Sun, et Kilian Q. Weinberger. 2017. « On Calibration of Modern Neural Networks ». *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 70: 1321-30. https://proceedings.mlr.press/v70/guo17a.html.

Hamilton, William L., Rex Ying, et Jure Leskovec. 2017. *Inductive Representation Learning on Large Graphs*. https://doi.org/10.48550/arXiv.1706.02216.

Hammami, Seif Eddine. 2018. « Dynamic Network Resources Optimization based on Machine Learning and Cellular Data Mining ». Thèse de doctorat, Télécom SudParis. https://theses.fr/2018TELE0015.

Han, Song, Huizi Mao, et William J. Dally. 2015. « Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding ». *arXiv*, publication en ligne anticipée. https://doi.org/10.48550/arXiv.1510.00149.

Han, Song, Jeff Pool, Sharan Narang, et al. 2016. *DSD: Dense-Sparse-Dense Training for Deep Neural Networks*. https://doi.org/10.48550/arXiv.1607.04381.

Hariri, Mohsen et al. 2026. *Test-Time Scaling in Reasoning LLMs: Inference Regimes, Evaluation, and Reproducibility*. https://arxiv.org/abs/2608.04001.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, et Jian Sun. 2015. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. https://arxiv.org/abs/1502.01852.

Hinton, Geoffrey E., Simon Osindero, et Yee-Whye Teh. 2006. « A Fast Learning Algorithm for Deep Belief Nets ». *Neural Computation* 18 (7): 1527-54. https://doi.org/10.1162/neco.2006.18.7.1527.

Hinton, Geoffrey E., et Ruslan R. Salakhutdinov. 2006. « Reducing the Dimensionality of Data with Neural Networks ». *Science* 313 (5786): 504-7. https://doi.org/10.1126/science.1127647.

Hinton, Geoffrey, Oriol Vinyals, et Jeff Dean. 2015. « Distilling the Knowledge in a Neural Network ». *arXiv preprint arXiv:1503.02531*. https://arxiv.org/abs/1503.02531.

hmmlearn developers. 2026. *Tutorial: Hidden Markov Models*. https://hmmlearn.readthedocs.io/en/stable/tutorial.html.

Hollmann, Noah et al. 2025. *Accurate predictions on small data with a tabular foundation model*. https://doi.org/10.1038/s41586-024-08328-6.

Huang, Xin et al. 2020. *TabTransformer: Tabular Data Modeling Using Contextual Embeddings*. https://arxiv.org/abs/2012.06678.

Hugging Face. 2025a. *Caching*. https://huggingface.co/docs/transformers/v4.57.1/cache_explanation.

Hugging Face. 2025b. *KV cache strategies*. https://huggingface.co/docs/transformers/v4.57.1/kv_cache.

Hugging Face. 2026a. *Accelerate: Loading big models into memory*. https://huggingface.co/docs/accelerate/concept_guides/big_model_inference.

Hugging Face. 2026b. *PEFT: Quantization, version 0.21.0*. https://huggingface.co/docs/peft/v0.21.0/en/developer_guides/quantization.

Hugging Face. 2026c. *Transformers: Generation*. https://huggingface.co/docs/transformers/v5.12.0/en/main_classes/text_generation.

ImageNet project. 2026. *About ImageNet*. https://image-net.org/about.php.

International Maritime Organization. 2026. *Radiocommunications*. https://www.imo.org/en/ourwork/safety/pages/radiocommunications-default.aspx.

International Telecommunication Union. 2026. *Maritime communications and safety*. https://www.itu.int/en/mediacentre/backgrounders/Pages/Radiocommunications-for-keeping-ships-and-people-safe-at-sea.aspx.

Ioffe, Sergey, et Christian Szegedy. 2015. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. https://doi.org/10.48550/arXiv.1502.03167.

Jacob, Benoit, Skirmantas Kligys, Bo Chen, et al. 2018. « Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference ». *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2704-13. https://doi.org/10.48550/arXiv.1712.05877.

James, Gareth, Daniela Witten, Trevor Hastie, Robert Tibshirani, et Jonathan Taylor. 2023. *An Introduction to Statistical Learning: with Applications in Python*. Springer. https://www.statlearning.com/.

Jerison, David. 2010. *18.01SC Single Variable Calculus : Definition of the Definite Integral and First Fundamental Theorem*. MIT OpenCourseWare. https://ocw.mit.edu/courses/18-01sc-single-variable-calculus-fall-2010/pages/unit-3-the-definite-integral-and-its-applications/part-a-definition-of-the-definite-integral-and-first-fundamental-theorem/.

Jiang, Albert Q., Alexandre Sablayrolles, Antoine Roux, et al. 2024. « Mixtral of Experts ». *arXiv preprint arXiv:2401.04088*. https://arxiv.org/abs/2401.04088.

Johansson, David K. 2026. *Reasoning as Energy Minimization over Structured Latent Trajectories*. https://doi.org/10.48550/arXiv.2603.28248.

Joint Committee for Guides in Metrology. 2012. *Vocabulaire international de métrologie — Concepts fondamentaux et généraux et termes associés (VIM), JCGM 200:2012*. Troisième édition. https://jcgm.bipm.org/vim/fr/.

Katz, Guy, Clark Barrett, David L. Dill, Kyle Julian, et Mykel J. Kochenderfer. 2017. *Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks*. https://doi.org/10.48550/arXiv.1702.01135.

Keras developers. 2026a. *Image augmentation layers*. https://keras.io/api/layers/preprocessing_layers/image_augmentation/.

Keras developers. 2026b. *Introducing Keras 3*. https://keras.io/keras_3/.

Keras developers. 2026c. *Keras Applications: models and performance measurement protocol*. https://keras.io/api/applications/.

Kingma, Diederik P., et Max Welling. 2013. *Auto-Encoding Variational Bayes*. https://arxiv.org/abs/1312.6114.

Kipf, Thomas N., et Max Welling. 2017. *Semi-Supervised Classification with Graph Convolutional Networks*. https://doi.org/10.48550/arXiv.1609.02907.

Kirkpatrick, James, Razvan Pascanu, Neil Rabinowitz, et al. 2017. « Overcoming Catastrophic Forgetting in Neural Networks ». *Proceedings of the National Academy of Sciences* 114 (13): 3521-26. https://doi.org/10.1073/pnas.1611835114.

Knuth, Donald E., et Ronald W. Moore. 1975. « An analysis of alpha-beta pruning ». *Artificial Intelligence* 6 (4): 293-326. https://doi.org/10.1016/0004-3702(75)90019-3.

Kohonen, Teuvo. 1982. « Self-organized formation of topologically correct feature maps ». *Biological Cybernetics* 43: 59-69. https://doi.org/10.1007/BF00337288.

LeCun, Yann, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, et Fu Jie Huang. 2006. *A Tutorial on Energy-Based Learning*. https://cs.nyu.edu/~yann/research/ebm/.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». *Advances in Neural Information Processing Systems 33* 33: 9459-74. https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html.

Li, Gavin. 2023. *AirLLM: scaling large language models on low-end commodity computers*. Released. https://github.com/lyogavin/airllm.

Li, Lihong, Wei Chu, John Langford, et Robert E. Schapire. 2010. « A Contextual-Bandit Approach to Personalized News Article Recommendation ». *Proceedings of the 19th International Conference on World Wide Web*. https://doi.org/10.1145/1772690.1772758.

Li, Qimai, Zhichao Han, et Xiao-Ming Wu. 2018. *Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning*. https://doi.org/10.48550/arXiv.1801.07606.

LightGBM developers. 2026. *Features*. https://lightgbm.readthedocs.io/en/stable/Features.html.

Liu, Fei Tony, Kai Ming Ting, et Zhi-Hua Zhou. 2008. *Isolation Forest*. https://doi.org/10.1109/ICDM.2008.17.

Liu, Ziming, Yixuan Wang, Sachin Vaidya, et al. 2024. *KAN: Kolmogorov-Arnold Networks*. https://arxiv.org/abs/2404.19756.

Logical Intelligence. 2026. *Kona 1.0*. https://logicalintelligence.com/kona.

Machine Learnia. 2026. « Python Machine Learning — Notebooks de la série vidéo ». https://github.com/MachineLearnia/Python-Machine-Learning.

Maslow, Abraham H. 1943. « A Theory of Human Motivation ». *Psychological Review* 50 (4): 370-96. https://doi.org/10.1037/h0054346.

Masset, Nicolas. s. d. « ParaMaths — vidéos de mathématiques ». Consulté le 9 octobre 2026. https://www.youtube.com/@Paramaths_off.

McClenny, Levi, et Ulisses Braga-Neto. 2020. « Self-Adaptive Physics-Informed Neural Networks using a Soft Attention Mechanism ». *arXiv*, publication en ligne anticipée. https://doi.org/10.48550/arXiv.2009.04544.

McMahan, H. Brendan, Eider Moore, Daniel Ramage, Seth Hampson, et Blaise Agüera y Arcas. 2017. « Communication-Efficient Learning of Deep Networks from Decentralized Data ». *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, Proceedings of Machine Learning Research, vol. 54: 1273-82. https://proceedings.mlr.press/v54/mcmahan17a.html.

Meinshausen, Nicolai. 2006. « Quantile Regression Forests ». *Journal of Machine Learning Research* 7 (35): 983-99. https://www.jmlr.org/papers/v7/meinshausen06a.html.

Mikolov, Tomas, Kai Chen, Greg Corrado, et Jeffrey Dean. 2013. *Efficient Estimation of Word Representations in Vector Space*. https://arxiv.org/abs/1301.3781.

Ministère de l'Éducation nationale. 1985. *Recrutement des classes préparatoires technologiques TA : réponse à la question 68968 de Raymond Marcellin*. Journal officiel de l'Assemblée nationale, questions et réponses. https://archives.assemblee-nationale.fr/7/qst/7-qst-1985-09-02.pdf.

Ministère de l'Éducation nationale. 2024. *TIMSS 2023 : résultats en mathématiques et en sciences des élèves de CM1 et quatrième*. https://www.education.gouv.fr/timss-2023-resultats-en-mathematiques-et-en-sciences-des-eleves-de-cm1-et-4eme-462435.

MLCommons. 2026. *MLPerf Inference: Tiny*. https://mlcommons.org/benchmarks/inference-tiny/.

Morard, Jean-Pierre. 2023. « Tous Compétents IA ». Manuscrit.

National Institute of Standards and Technology. 2019. « NIST Study Evaluates Effects of Race, Age, Sex on Face Recognition Software ». décembre 19. https://www.nist.gov/news-events/news/2019/12/nist-study-evaluates-effects-race-age-sex-face-recognition-software.

National Institute of Standards and Technology. 2023. « AI RMF Playbook — Measure ». https://airc.nist.gov/airmf-resources/playbook/measure/.

Ng, Andrew Y., Michael I. Jordan, et Yair Weiss. 2001. « On Spectral Clustering: Analysis and an algorithm ». *Advances in Neural Information Processing Systems* 14. https://papers.nips.cc/paper/2001/hash/801272ee79cfde7fa5960571fee36b9b-Abstract.html.

Nickel, Maximilian, et Douwe Kiela. 2017. *Poincaré Embeddings for Learning Hierarchical Representations*. https://doi.org/10.48550/arXiv.1705.08039.

NIST. 2025. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. https://doi.org/10.6028/NIST.AI.100-2e2025.

NIST/SEMATECH. 2026a. *e-Handbook of Statistical Methods: Box-Jenkins Models*. https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc445.htm.

NIST/SEMATECH. 2026b. *e-Handbook of Statistical Methods: Normal Distribution*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda3661.htm.

NIST/SEMATECH. 2026c. *e-Handbook of Statistical Methods: Uniform Distribution*. https://www.itl.nist.gov/div898/handbook/eda/section3/eda3662.htm.

NVIDIA. 2026. *CUDA Programming Guide*. https://docs.nvidia.com/cuda/cuda-programming-guide/.

OECD. 2026. *AI principles*. https://www.oecd.org/en/topics/ai-principles.html.

ONNX Runtime developers. 2026. *Execution Providers*. https://onnxruntime.ai/docs/execution-providers/.

OpenAI. 2026. *GPT-6 Astra: model documentation*. https://developers.openai.com/api/docs/models/gpt-6-astra.

OpenCode. 2026. *Agents: configure and use specialized agents*. https://opencode.ai/docs/agents/.

Organisation de coopération et de développement économiques (OCDE). 2026. *PISA 2025 Results (Volume I): France*. Country note. Organisation de coopération et de développement économiques (OCDE). https://www.oecd.org/en/publications/pisa-2025-results-volume-i-country-notes_2d4ff9ea-en/france_6d229453-en.html.

Ouyang, Long et al. 2022. *Training language models to follow instructions with human feedback*. https://arxiv.org/abs/2203.02155.

Perozzi, Bryan, Rami Al-Rfou, et Steven Skiena. 2014. *DeepWalk: Online Learning of Social Representations*. https://doi.org/10.48550/arXiv.1403.6652.

pgmpy developers. 2026. *DiscreteBayesianNetwork*. https://pgmpy.org/api/generated/models/pgmpy.models.DiscreteBayesianNetwork.html.

Poole, David L., et Alan K. Mackworth. 2023. *Artificial Intelligence: Foundations of Computational Agents*. 3e éd. Cambridge University Press. https://artint.info/3e/html/ArtInt3e.html.

Prior Labs. 2026. *TabPFN-3.5: Technical Report*. https://priorlabs.ai/technical-reports/tabpfn-3-5.

Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, et Andrey Gulin. 2018. *CatBoost: unbiased boosting with categorical features*. https://arxiv.org/abs/1706.09516.

PyTorch contributors. 2026a. *Conv1d*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv1d.html.

PyTorch contributors. 2026b. *Dropout*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.Dropout.html.

PyTorch contributors. 2026c. *LSTM*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.LSTM.html.

PyTorch contributors. 2026d. *RNN*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.RNN.html.

PyTorch developers. 2026. *TorchAO: Quantized Inference*. https://docs.pytorch.org/ao/stable/workflows/inference.html.

Radford, Alec, Karthik Narasimhan, Tim Salimans, et Ilya Sutskever. 2018. *Improving Language Understanding by Generative Pre-Training*. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

Raffel, Colin et al. 2019. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. https://arxiv.org/abs/1910.10683.

Raissi, Maziar, Paris Perdikaris, et George Em Karniadakis. 2019. *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. https://doi.org/10.1016/j.jcp.2018.10.045.

Rippel, Oren, et Lubomir Bourdev. 2017. « Real-Time Adaptive Image Compression ». *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 70: 2922-30. https://proceedings.mlr.press/v70/rippel17a.html.

Rouxel, Didier, Olivier Sarzeaud, et Marie-Françoise Lequentrec-Lalancette. 2023. « Using Marine Data to Estimate the Accuracy of Free Air Gravity Anomaly Models Derived From Satellite Altimetry ». *Earth and Space Science* 10 (5): e2022EA002672. https://doi.org/10.1029/2022EA002672.

Rumelhart, David E., Geoffrey E. Hinton, et Ronald J. Williams. 1986. « Learning representations by back-propagating errors ». *Nature* 323: 533-36. https://doi.org/10.1038/323533a0.

Sainath, Tara N., et Carolina Parada. 2015. « Convolutional Neural Networks for Small-Footprint Keyword Spotting ». *Interspeech 2015*. https://www.isca-archive.org/interspeech_2015/sainath15b_interspeech.html.

Saito, Takaya, et Marc Rehmsmeier. 2015. *The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets*. https://doi.org/10.1371/journal.pone.0118432.

Scarfone, Karen, Murugiah Souppaya, Amanda Cody, et Angela Orebaugh. 2008. *Technical Guide to Information Security Testing and Assessment*. NIST Special Publication SP 800-115. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.SP.800-115.

scikit-learn contributors. 2026a. *HashingVectorizer*. Documentation scikit-learn 1.9.1. https://scikit-learn.org/1.9/modules/generated/sklearn.feature_extraction.text.HashingVectorizer.html.

scikit-learn contributors. 2026b. *Novelty and Outlier Detection*. Documentation scikit-learn 1.9.1. https://scikit-learn.org/1.9/modules/outlier_detection.html.

scikit-learn contributors. 2026c. *PoissonRegressor*. Documentation scikit-learn 1.9.1. https://scikit-learn.org/1.9/modules/generated/sklearn.linear_model.PoissonRegressor.html.

scikit-learn developers. 2026a. *Clustering*. https://scikit-learn.org/stable/modules/clustering.html.

scikit-learn developers. 2026b. *Cross-validation: evaluating estimator performance*. https://scikit-learn.org/stable/modules/cross_validation.html.

scikit-learn developers. 2026c. *Decision Trees*. https://scikit-learn.org/stable/modules/tree.html.

scikit-learn developers. 2026d. *Decomposing signals in components*. https://scikit-learn.org/stable/modules/decomposition.html.

scikit-learn developers. 2026e. *Decomposing signals in components: independent component analysis*. https://scikit-learn.org/stable/modules/decomposition.html#independent-component-analysis-ica.

scikit-learn developers. 2026f. *Ensembles: Gradient boosting, random forests, bagging, voting, stacking*. https://scikit-learn.org/stable/modules/ensemble.html.

scikit-learn developers. 2026g. *Feature extraction: text features and TF-IDF term weighting*. https://scikit-learn.org/stable/modules/feature_extraction.html.

scikit-learn developers. 2026h. *Gaussian mixture models*. https://scikit-learn.org/stable/modules/mixture.html.

scikit-learn developers. 2026i. *KMeans: objective and algorithm*. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html.

scikit-learn developers. 2026j. *Linear and Quadratic Discriminant Analysis*. https://scikit-learn.org/stable/modules/lda_qda.html.

scikit-learn developers. 2026k. *Linear Models*. https://scikit-learn.org/stable/modules/linear_model.html.

scikit-learn developers. 2026l. *LogisticRegression*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

scikit-learn developers. 2026m. *Manifold learning*. https://scikit-learn.org/stable/modules/manifold.html.

scikit-learn developers. 2026n. *Metrics and scoring: quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html.

scikit-learn developers. 2026o. *Naive Bayes*. https://scikit-learn.org/stable/modules/naive_bayes.html.

scikit-learn developers. 2026p. *Nearest Neighbors*. https://scikit-learn.org/stable/modules/neighbors.html.

scikit-learn developers. 2026q. *Neural network models (supervised)*. https://scikit-learn.org/stable/modules/neural_networks_supervised.html.

scikit-learn developers. 2026r. *Semi-supervised learning*. https://scikit-learn.org/stable/modules/semi_supervised.html.

Scikit-learn developers. 2026. *StandardScaler*. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

scikit-learn developers. 2026s. *Support Vector Machines*. https://scikit-learn.org/stable/modules/svm.html.

Settles, Burr. 2009. *Active Learning Literature Survey*. https://burrsettles.com/pub/settles.activelearning.pdf.

Simeoni, Oriane et al. 2025. *DINOv3*. https://arxiv.org/abs/2508.10104.

Sous-direction des systèmes d'information et des études statistiques (SIES). 2026. *Les effectifs en classes préparatoires aux grandes écoles à la rentrée 2025-2026*. Note Flash Nos. 2026-02. Ministère de l'Enseignement supérieur, de la Recherche et de l'Espace. https://www.enseignementsup-recherche.gouv.fr/sites/default/files/2026-02/nf-sies-2026-02--39443.pdf.

Stanford Institute for Human-Centered Artificial Intelligence. 2026. *AI Index Report 2026: Economy*. https://hai.stanford.edu/ai-index/2026-ai-index-report/economy.

Starmer, Josh. 2026. « StatQuest — Video Index ». https://statquest.org/video_index.html.

Sutskever, Ilya, Oriol Vinyals, et Quoc V. Le. 2014. *Sequence to Sequence Learning with Neural Networks*. https://arxiv.org/abs/1409.3215.

Sutton, Richard S., et Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2e éd. MIT Press. https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

Tan, Mingxing, Bo Chen, Ruoming Pang, et al. 2019. « MnasNet: Platform-Aware Neural Architecture Search for Mobile ». *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2820-28. https://openaccess.thecvf.com/content_CVPR_2019/papers/Tan_MnasNet_Platform-Aware_Neural_Architecture_Search_for_Mobile_CVPR_2019_paper.pdf.

ThalesGroup. 2026. *AGILAB: reproducible AI/ML workflows, distributed workers and services*. https://github.com/ThalesGroup/agilab.

The scikit-learn developers. 2026. *Comparing Different Clustering Algorithms on Toy Datasets*. Documentation scikit-learn 1.7.2. https://scikit-learn.org/1.7/auto_examples/cluster/plot_cluster_comparison.html.

The SciPy community. 2026. *Short-Time Fourier Transform: scipy.signal.stft*. SciPy documentation. https://docs.scipy.org/doc/scipy-1.15.2/reference/generated/scipy.signal.stft.html.

Tokki. 2026. *Tokki: compact, recoverable AI-agent context*. https://github.com/jpmorard/tokki-public.

Trekhleb, Oleksii. 2026. « Homemade Machine Learning ». https://github.com/trekhleb/homemade-machine-learning.

Turpin, Miles, Julian Michael, Ethan Perez, et Samuel R. Bowman. 2023. *Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting*. https://arxiv.org/abs/2305.04388.

UMAP developers. 2026. *Frequently Asked Questions*. https://umap-learn.readthedocs.io/en/latest/faq.html.

UNESCO. 2021. *Recommendation on the Ethics of Artificial Intelligence*. https://www.unesco.org/en/legal-affairs/recommendation-ethics-artificial-intelligence.

Union européenne. 2026. *Règlement (UE) 2026/1744 du 8 juillet 2026*. https://eur-lex.europa.eu/eli/reg/2026/1744/oj.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. *Attention Is All You Need*. https://arxiv.org/abs/1706.03762.

Veličković, Petar et al. 2018. *Graph Attention Networks*. https://doi.org/10.48550/arXiv.1710.10903.

Wang, Sifan, Yujun Teng, et Paris Perdikaris. 2020. *Understanding and mitigating gradient pathologies in physics-informed neural networks*. https://arxiv.org/abs/2001.04536.

Wang, Xuezhi et al. 2022. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. https://arxiv.org/abs/2203.11171.

Wei, Jason et al. 2022. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. https://arxiv.org/abs/2201.11903.

Wen, Wei, Chunpeng Wu, Yandan Wang, Yiran Chen, et Hai Li. 2016. *Learning Structured Sparsity in Deep Neural Networks*. https://doi.org/10.48550/arXiv.1608.03665.

Woods Hole Oceanographic Institution. 2026. *Nootka Buoy*. https://www.whoi.edu/what-we-do/explore/instruments/instruments-moorings-buoys/mooring-buoy-systems/nootka-buoy/.

XGBoost developers. 2026. *Introduction to Boosted Trees*. https://xgboost.readthedocs.io/en/stable/tutorials/model.html.

Xu, Eric. 2025. « Groundbreaking SuperPoD Interconnect: Leading a New Paradigm for AI Infrastructure ». Huawei, septembre 18. https://www.huawei.com/en/news/2025/9/hc-xu-keynote-speech.

Yarowsky, David. 1995. « Unsupervised Word Sense Disambiguation Rivaling Supervised Methods ». *33rd Annual Meeting of the Association for Computational Linguistics*, 189-96. https://doi.org/10.3115/981658.981684.

Yu, Runpeng, Weihao Yu, et Xinchao Wang. 2024. *KAN or MLP: A Fairer Comparison*. https://arxiv.org/abs/2407.16674.

Zhang, Aston, Zachary C. Lipton, Mu Li, et Alexander J. Smola. 2023. *Dive into Deep Learning*. Cambridge University Press. https://d2l.ai/.

Zoph, Barret, et Quoc V. Le. 2016. *Neural Architecture Search with Reinforcement Learning*. https://doi.org/10.48550/arXiv.1611.01578.